

HAETAE EasyCrypt Symbolic Correspondence Guide

Generated from the provable-security EasyCrypt manifest

May 2026

This guide indexes the mathematical objects written in the checked HAETAE EasyCrypt proof surface. It covers each manifest file and records every top-level `type`, `op`, `pred`, `lemma`, and `equiv` declaration, plus explicit `hoare`. proof steps. The line-by-line companion guides preserve every source line; this document instead gives the symbolic mathematical reading of each named declaration and why it appears in the proof.

Source manifest: `provable-security/proof-files.txt`. Declaration count: 1854. equivalences: 24, hoare proof steps: 12, lemmas: 1088, operations: 675, types: 55.

Contents

1	Symbolic Reading Rules	2
2	Manifest Order	2
3	HAETAE_Security.ec	4
4	Sig_ROM.ec	19
5	HAETAE_HopGames.ec	21
6	HAETAE_Reductions.ec	111
7	HAETAE_ROM.ec	122
8	HAETAE_ROM_Programming.ec	126
9	HAETAE_Scheme.ec	163
10	HAETAE_Transcript.ec	164
11	HAETAE_Events.ec	183
12	HAETAE_Assumptions.ec	185
13	HAETAE_Distributions.ec	190

14 HAETAE_Rejection.ec	214
15 HAETAE_Algebra.ec	270
16 HAETAE_Params.ec	359
17 HAETAE_FIPS202.ec	363
18 HAETAE_Keccak1600.ec	367
19 Regenerating This Guide	382

1 Symbolic Reading Rules

EasyCrypt construct	Mathematical correspondence
type T .	An abstract carrier set T . The proof may quantify over T , but it does not expose an implementation representation.
type $T = A * B$.	A definitional product $T \equiv A \times B$. Tuple projections in later code are projections from this product.
op $f(x:A) : B = e$.	A total mathematical function $f : A \rightarrow B$ with defining equation $f(x) = \text{eval}(e)$.
op [lossless] $d : A \text{ distr}$.	A probability distribution $d \in \text{Distr}(A)$ whose mass is 1.
pred $P(x:A)$. or Boolean op	A predicate $P \subseteq A$, equivalently $P : A \rightarrow \text{Bool}$.
lemma $L \text{ xs} : P$.	A theorem $\vdash \forall xs. P$. The proof script after it is the machine-checked derivation.
$\text{Pr}[G.\text{main}() @ \&m : E] \leq B$	A game probability bound $\text{Pr}[G : E] \leq B$. These are the numeric game-hop and final security claims.
equiv $\dots : R \implies S$.	A relational judgment $c_1 \sim c_2 [R \Rightarrow S]$ showing that two games preserve relation S from relation R .
hoare.	A proof step that changes the active goal to a Hoare triple $\{P\} c \{Q\}$.

2 Manifest Order

1. HAETAE_Security.ec – top-level theorem composition and concrete EUF-CMA bound.
2. Sig_ROM.ec – generic signature security games over a random oracle.
3. HAETAE_HopGames.ec – hybrid games, invariants, equivalences, and game-hop losses.
4. HAETAE_Reductions.ec – arithmetic assembly of concrete loss terms.
5. HAETAE_ROM.ec – typed random-oracle query and response model.
6. HAETAE_ROM_Programming.ec – random-oracle programming events and counted bad-event bounds.
7. HAETAE_Scheme.ec – HAETAE instance of the generic signature interface.
8. HAETAE_Transcript.ec – transcripts, forking relations, and extraction obligations.

9. `HAETAE_Events.ec` – named rejection-failure events.
10. `HAETAE_Assumptions.ec` – abstract MLWE, Module-SIS, and bimodal self-target games.
11. `HAETAE_Distributions.ec` – sampling distributions and support/cardinality facts.
12. `HAETAE_Rejection.ec` – rejection predicates and rejection-sampling probability bounds.
13. `HAETAE_Algebra.ec` – deterministic polynomial/vector/matrix algebra used by the model.
14. `HAETAE_Params.ec` – mode-dependent parameter constants.
15. `HAETAE_FIPS202.ec` – abstracted FIPS202/SHAKE support.
16. `HAETAE_Keccak1600.ec` – Keccak state-level support used by the hash model.

3 HAETAE_Security.ec

top-level theorem composition and concrete EUF-CMA bound

	Line	Construct	What was written	Symbolic corre
	21	operation		
		<pre> correctness_obligation op correctness_obligation = forall (sd : seed) (coins : random_coins) (m : message) (ctx : context), verify_internal haetae_mode (keygen_internal haetae_mode sd). '1 m ctx (sign_internal haetae_mode (keygen_internal haetae_mode sd). '2 m ctx coins). </pre>	<pre> correctness_obligation : seed × random_coins × message × context → Bool </pre>	Defines a total m by the EasyCryp
	28	lemma		
		<pre> correctness_obligation_holds lemma correctness_obligation_holds : correctness_obligation. </pre>	<pre> correctness_obligation_holds : ∀x̄. P(x̄) is a universally quantified fact. </pre>	Proves a structur justify later redu
	40	lemma		
		<pre> correctness_perfect lemma correctness_perfect &m : Pr[SIG.Correctness(H, HAETAE, A).main() @ &m : res] = 0%r. </pre>	<pre> correctness_perfect : Pr[c : E] = B is a probability bound or exact game probability. </pre>	Proves a structur justify later redu
	44	hoare proof step		
		<pre> hoare_step_for_correctness_perfect hoare. </pre>	Within correctness_perfect, the proof reduces the probabilistic goal to a Hoare triple $\{P\} c \{Q\}$.	Introduces a prog for the surroundi
	81	lemma		
		<pre> euf_cma_security_no_signing lemma euf_cma_security_no_signing &m : Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] <= Pr[MLWE_Game(Bmlwe).main() @ &m : res] + Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m : res] => Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m : res] <= Pr[ModuleSIS_Game(Bsis).main() @ &m : res] + bimodal_to_msis_reduction_loss_term => Pr[MLWE_Game(Bmlwe)... </pre>	<pre> euf_cma_security_no_signing : Pr[c : E] ≤ B is a probability bound or exact game probability. </pre>	Proves a bound t game-hop or fina
	136	lemma		

Line	Construct	What was written	Symbolic corre
euf_cma_security	<pre> lemma euf_cma_security &m : Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m : res] <= Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] + (fs_with_aborts_reprogramming_term + fs_with_aborts_min_entropy_term) => Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] <= Pr[MLWE_Game(Bmlwe).main() @ &m : res] + Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m : res] => Pr[...] </pre>	euf_cma_security : $\Pr[c : E] \leq B$ is a probability bound or exact game probability.	Proves a bound t game-hop or fina
188	lemma		
euf_cma_security_with_hop_interfaces	<pre> lemma euf_cma_security_with_hop_interfaces &m : fs_with_aborts_interfaces_ready haetae_mode => Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m : res] <= Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] + (fs_with_aborts_reprogramming_term + fs_with_aborts_min_entropy_term) => Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] <= Pr[MLWE_Game(Bmlwe).main() @ &m : res]... </pre>	euf_cma_security_with_hop_interfaces : $\Pr[c : E] \leq B$ is a probability bound or exact game probability.	Proves a bound t game-hop or fina
223	lemma		

Line	Construct	What was written	Symbolic corre
	<pre> euf_cma_security_from_simulated_hop lemma euf_cma_security_from_simulated_hop &m : fs_with_aborts_interfaces_ready haetae_mode => Pr[EUF_CMA_SimulatedSign(H, HAETAE, A, RealSigningSimulator(HAETAE)).main() @ &m : res] <= Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] + (fs_with_aborts_reprogramming_term + fs_with_aborts_min_entropy_term) => Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] <=... </pre>	<pre> euf_cma_security_from_simulated_hop : Pr[$c : E$] $\leq B$ is a probability bound or exact game probability. </pre>	Proves a distribu sampling or prob
278	<pre> lemma euf_cma_security_from_explicit_boundaries lemma euf_cma_security_from_explicit_boundaries &m : Pr[EUF_CMA_SimulatedSign(H, HAETAE, A, RealSigningSimulator(HAETAE)).main() @ &m : res] <= Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] + (fs_with_aborts_reprogramming_term + fs_with_aborts_min_entropy_term) => Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] <= Pr[MLWE_Game(Bmlwe).main() @ &m : res] +... </pre>	<pre> euf_cma_security_from_explicit_boundaries Pr[$\epsilon : E$] $\leq B$ is a probability bound or exact game probability. </pre>	Proves a bound t game-hop or fina
319	<pre> lemma </pre>		

Line	Construct	What was written	Symbolic corre
	euf_cma_security_from_rom_internal_transcript_hop	<pre>lemma euf_cma_security_from_rom_internal_transcript_hop &m : Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m : res] <= Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] + (fs_with_aborts_reprogramming_term + fs_with_aborts_min_entropy_term) => Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] <= Pr[MLWE_Game(Bmlwe).main() @ &m : res] + Pr[BimodalSelfTarg...</pre>	euf_cma_security_from_rom_internal_transcript_hop $\Pr[E] \leq B$ is a probability bound or exact game probability.
344	rom_internal_transcript_hop_from_clear_site_reduction	<pre>lemma rom_internal_transcript_hop_from_clear_site_reduction &m : Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m : res /\ transcript_log_signature_programming_sites_clear haetae_mode EUF_CMA_ROMInternalTranscriptSign.transcripts] <= Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] => Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m : res] <= Pr...</pre>	rom_internal_transcript_hop_from_clear_site_reduction $\Pr[E] \leq B$ is a probability bound or exact game probability.
375	rom_internal_transcript_hop_from_clear_site_counted_reduction	<pre>lemma rom_internal_transcript_hop_from_clear_site_counted_reduction &m : Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m : res /\ transcript_log_signature_programming_sites_clear haetae_mode EUF_CMA_ROMInternalTranscriptSign.transcripts] <= Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] => Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m : re...</pre>	rom_internal_transcript_hop_from_clear_site_counted_reduction $\Pr[E] \leq B$ is a probability bound or exact game probability.
418	lemma	lemma	

Line	Construct	What was written	Symbolic corre
	bimodal_to_module_sis_reduction_bound	lemma bimodal_to_module_sis_reduction_bound : &m : Pr[BimodalSelfTargetMSIS_Game(Bbimodal)).main() @ &m : res] <= Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(Bbimodal))).main() @ &m : res] + bimodal_to_msis_reduction_loss_term.	bimodal_to_module_sis_reduction_bound : Proves a bound t Pr[$c : E \leq B$ is a probability bound or exact game probability. game-hop or fina
434	lemma		
	euf_cma_security_from_mechanized_boundaries	lemma euf_cma_security_from_mechanized_boundaries : &m : Pr[EUF_CMA_SimulatedSign(H, HAETAE, A, RealSigningSimulator(HAETAE)).main() @ &m : res] <= Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] + (fs_with_aborts_reprogramming_term + fs_with_aborts_min_entropy_term) => Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] <= Pr[MLWE_Game(Bmlwe).main() @ &m : res]...	euf_cma_security_from_mechanized_boundaries : Proves a bound t Pr[$c : E \leq B$ is a probability bound or exact game probability. game-hop or fina
473	lemma		
	euf_cma_security_from_rom_transcript_and_mechanized_bimodal	lemma euf_cma_security_from_rom_transcript_and_mechanized_bimodal : &m : Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m : res] <= Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] + (fs_with_aborts_reprogramming_term + fs_with_aborts_min_entropy_term) => Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] <= Pr[MLWE_Game(Bmlwe).main() @ &m : res] + Pr[Bimod...	euf_cma_security_from_rom_transcript_and_mechanized_bimodal : Proves a bound t Pr[$c : E \leq B$ is a probability bound or exact game probability. game-hop or fina
496	lemma		

Line	Construct	What was written	Symbolic corre
euf_cma_security_from_rom_transcript_and_counted_bimodal	<pre> lemma euf_cma_security_from_rom_transcript_and_counted_bimodal &m : Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m : res] <= Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] + counted_rom_programming_loss_term => Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] <= Pr[MLWE_Game(Bmlwe).main() @ &m : res] + Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main()...</pre>	euf_cma_security_from_rom_transcript_and_counted_bimodal $\Pr[E] \leq B$ is a probability bound or exact game probability.	Proven to bind game-hop or final
547	lemma		
euf_cma_security_from_rom_clear_site_and_mechanized_bimodal	<pre> lemma euf_cma_security_from_rom_clear_site_and_mechanized_bimodal &m : Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m : res /\ transcript_log_signature_programming_sites_clear haetae_mode EUF_CMA_ROMInternalTranscriptSign.transcripts] <= Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] => Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] <= Pr[MLWE_Gam...</pre>	euf_cma_security_from_rom_clear_site_and_mechanized_bimodal $\Pr[E] \leq B$ is a probability bound or exact game probability.	Proven to bind game-hop or final
571	lemma		
euf_cma_security_from_rom_clear_site_and_counted_bimodal	<pre> lemma euf_cma_security_from_rom_clear_site_and_counted_bimodal &m : Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m : res /\ transcript_log_signature_programming_sites_clear haetae_mode EUF_CMA_ROMInternalTranscriptSign.transcripts] <= Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] => Pr[SIG.UF_NMA(H, HAETAE, N).main() @ &m : res] <= Pr[MLWE_Game(B...</pre>	euf_cma_security_from_rom_clear_site_and_counted_bimodal $\Pr[E] \leq B$ is a probability bound or exact game probability.	Proven to bind game-hop or final

Line	Construct	What was written	Symbolic corre
633	lemma	euF_CMA_Security_from_structural_nma_and_counted_bimodal	Provable bound
	<pre> lemma euF_CMA_Security_from_structural_nma_and_counted_bimodal &m : Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptAsNMA(A)).main() @ &m : res] <= Pr[MLWE_Game(Bmlwe).main() @ &m : res] + Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m : res] => Pr[MLWE_Game(Bmlwe).main() @ &m : res] <= mlwe_hardness_term => Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(B...</pre>	$\text{Pr}[E] \leq B$ is a probability bound or exact game probability.	game-hop or final
653	lemma	euF_CMA_Security_from_public_sim_nma_and_counted_bimodal	Provable bound
	<pre> lemma euF_CMA_Security_from_public_sim_nma_and_counted_bimodal &m : Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPublicSimAsNMA(A)).main() @ &m : res] <= Pr[MLWE_Game(Bmlwe).main() @ &m : res] + Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m : res] => Pr[MLWE_Game(Bmlwe).main() @ &m : res] <= mlwe_hardness_term => Pr[ModuleSIS_Game(BimodalMSIS_As_Mo...</pre>	$\text{Pr}[E] \leq B$ is a probability bound or exact game probability.	game-hop or final
673	lemma	euF_CMA_Security_from_paper_sim_nma_and_counted_bimodal	Provable bound
	<pre> lemma euF_CMA_Security_from_paper_sim_nma_and_counted_bimodal &m : Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPublicSimAsNMA(A)).main() @ &m : res] <= Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA(A, Samp)).main() @ &m : res] + rejection_sampling_loss_term => Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA(A, Samp)).main() @ &...</pre>	$\text{Pr}[E] \leq B$ is a probability bound or exact game probability.	game-hop or final
704	lemma		

Line	Construct	What was written	Symbolic corre
euf_cma_security_from_rom_paper_sim_nma_and_counted_bimodal	lemma	euf_cma_security_from_rom_paper_sim_nma_and_counted_bimodal	Proves and bounded
	euf_cma_security_from_rom_paper_sim_nma_and_counted_bimodal	$\Pr[E] \leq B$ is a probability bound or exact game probability.	game-hop or fina
	&m : Pr[SIG.UF_NMA(H, HAETAETAE,		
	ROMInternalTranscriptPaperSimAsNMA		
	(A,		
	ROMPaperSimSigningSampler(H))) .main()		
	@ &m : res] <=		
	Pr[MLWE_Game(Bmlwe).main() @ &m		
	: res] +		
	Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main()		
	@ &m : res] =>		
	Pr[MLWE_Game(Bmlwe).main() @ &m		
	: res] <= mlwe_hardness_term =>		
	Pr...		
725	lemma	euf_cma_security_from_real_signing_paper_sim_nma_and_counted_bimodal	Proves and bounded
euf_cma_security_from_real_signing_paper_sim_nma_and_counted_bimodal	lemma	euf_cma_security_from_real_signing_paper_sim_nma_and_counted_bimodal	Proves and bounded
	euf_cma_security_from_real_signing_paper_sim_nma_and_counted_bimodal	$\Pr[E] \leq B$ is a probability bound or exact game probability.	game-hop or fina
	&m : Pr[SIG.UF_NMA(H, HAETAETAE,		
	ROMInternalTranscriptPaperSimAsNMA		
	(A,		
	RealSigningPaperSimSampler(H))) .main()		
	@ &m : res] <=		
	Pr[MLWE_Game(Bmlwe).main() @ &m		
	: res] +		
	Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main()		
	@ &m : res] =>		
	Pr[MLWE_Game(Bmlwe).main() @ &m		
	: res] <= mlwe_hardness...		
746	lemma	euf_cma_security_from_haetae_rejection_paper_sim_nma_and_counted_bimodal	Proves and bounded
euf_cma_security_from_haetae_rejection_paper_sim_nma_and_counted_bimodal	lemma	euf_cma_security_from_haetae_rejection_paper_sim_nma_and_counted_bimodal	Proves and bounded
	euf_cma_security_from_haetae_rejection_paper_sim_nma_and_counted_bimodal	$\Pr[E] \leq B$ is a probability bound or exact game probability.	game-hop or fina
	&m : Pr[SIG.UF_NMA(H, HAETAETAE,		
	ROMInternalTranscriptPaperSimAsNMA		
	(A,		
	HAETAERejectionPaperSimSampler(H))) .main()		
	@ &m : res] <=		
	Pr[MLWE_Game(Bmlwe).main() @ &m		
	: res] +		
	Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main()		
	@ &m : res] =>		
	Pr[MLWE_Game(Bmlwe).main() @ &m		
	: res] <= mlwe_h...		

	Line	Construct	What was written	Symbolic correction
	768	lemma		
exact_hyperball_haetae_rejection_nma_bound_from_paper_sim		<pre> lemma exact_hyperball_haetae_rejection_nma_bound_from_paper_sim &m : Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, ExactHyperballPaperSimSampler(H)))].main() @ &m : res] <= Pr[MLWE_Game(Bmlwe).main() @ &m : res] + Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m : res] => Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A,... </pre>	<pre> exact_hyperball_haetae_rejection_nma_bound_from_paper_sim Pr[ϵ, E] < B is a probability bound or exact game probability. </pre>	<pre> exact_hyperball_haetae_rejection_nma_bound_from_paper_sim Pr[ϵ, E] < B is a probability bound or game-hop or final </pre>
	787	lemma		
haetae_rejection_nma_bound_from_exact_hyperball_sampling_hop		<pre> lemma haetae_rejection_nma_bound_from_exact_hyperball_sampling_hop &m : Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, HAETAERejectionPaperSimSampler(H)))].main() @ &m : res] <= Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, ExactHyperballHAETAERejectionPaperSimSampler(H)))].main() @ &m : res] + rejection_sampling_loss_term... </pre>	<pre> haetae_rejection_nma_bound_from_exact_hyperball_sampling_hop Pr[ϵ, E] < B is a probability bound or exact game probability. </pre>	<pre> haetae_rejection_nma_bound_from_exact_hyperball_sampling_hop Pr[ϵ, E] < B is a probability bound or game-hop or final </pre>
	819	lemma		

Line	Construct	What was written	Symbolic corre
	<pre> lemma haetae_rejection_exact_hyperball_sampling_hop_from_real_signing_exact_hyperball_hop &m : Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, RealSigningPaperSimSampler(H)))].main() @ &m : res] <= Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, ExactHyperballPaperSimSampler(H)))].main() @ &m : res] + rejection_sampling_loss...</pre>	haetae_rejection_exact_hyperball_sampling_hop_from_real_signing_exact_hyperball_hop $\Pr[E] \leq B$ is a probability bound or exact game probability.	Proves from exact simulator transit
843	<pre> lemma haetae_rejection_ro_exact_hyperball_sampling_hop_from_real_signing_ro_exact_hyperball_hop &m : Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, RealSigningPaperSimSampler(H)))].main() @ &m : res] <= Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, ROExactHyperballPaperSimSampler(H)))].main() @ &m : res] + rejection_sampli...</pre>	haetae_rejection_ro_exact_hyperball_sampling_hop_from_real_signing_ro_exact_hyperball_hop $\Pr[E] \leq B$ is a probability bound or exact game probability.	Proves from exact simulator transit
865	<pre> lemma real_signing_ro_exact_hyperball_hop_from_ro_signing_attempt_hop &m : Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, ROSigningAttemptPaperSimSampler(H)))].main() @ &m : res] <= Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, ROExactHyperballPaperSimSampler(H)))].main() @ &m : res] + rejection_sampling_loss_term => Pr[SI...</pre>	real_signing_ro_exact_hyperball_hop_from_ro_signing_attempt_hop $\Pr[E] \leq B$ is a probability bound or exact game probability.	Proves from exact simulator transit

	Line	Construct	What was written	Symbolic corre
	886	lemma		
real_signing_ro_exact_hyperball_hop_from_ro_signing_attempt_hop_loss		<pre> lemma real_signing_ro_exact_hyperball_hop_from_ro_signing_attempt_hop_loss &m (loss : real) : Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, ROSigningAttemptPaperSimSampler(H)))].main() @ &m : res] <= Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, ROExactHyperballPaperSimSampler(H)))].main() @ &m : res] + loss => Pr[SIG.UF_...</pre>	real_signing_ro_exact_hyperball_hop_from_ro_signing_attempt_hop_loss $\Pr[E] \leq B$ is a probability bound or exact game probability.	Proving quantifier game-hop or final
	908	lemma		
real_signing_ro_exact_hyperball_hop_from_budgeted_lifting		<pre> lemma real_signing_ro_exact_hyperball_hop_from_budgeted_lifting &m : Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, ROSigningAttemptPaperSimSampler(H)))].main() @ &m : res] <= Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptBudgetedPaperSimAsNMA (A, ROSigningAttemptPaperSimSampler(H)))].main() @ &m : res] => Pr[SIG.UF_NMA(H, HAETAE, ROMIntern...</pre>	real_signing_ro_exact_hyperball_hop_from_budgeted_lifting $\Pr[E] \leq B$ is a probability bound or exact game probability.	Budgeted lifting simulator transit
	943	lemma		

	Line	Construct	What was written	Symbolic corre
haetae_rejection_ro_exact_hyperball_sampling_hop_from_budgeted_lifting		lemma	haetae_rejection_ro_exact_hyperball_sampling_hop_from_budgeted_lifting	Proves an exact bound on simulator transition
		haetae_rejection_ro_exact_hyperball_sampling_hop_from_budgeted_lifting	$\Pr[s \in E] \leq B$ is a probability bound or exact game probability.	
		&m : Pr[SIG.UF_NMA(H, HAETAETAE,		
		ROMInternalTranscriptPaperSimAsNMA		
		(A,		
		ROSigningAttemptPaperSimSampler(H))) .main()		
		@ &m : res] <= Pr[SIG.UF_NMA(H,		
		HAETAETAE,		
		ROMInternalTranscriptBudgetedPaperSimAsNMA		
		(A,		
		ROSigningAttemptPaperSimSampler(H))) .main()		
		@ &m : res] => Pr[SIG.UF_NMA(H,		
		HAET...		
	974	lemma	euf_cma_security_from_exact_hyperball_paper_sim_bridge_and_counted_bimodal	Proves a bound on game-hop or final
euf_cma_security_from_exact_hyperball_paper_sim_bridge_and_counted_bimodal		lemma	euf_cma_security_from_exact_hyperball_paper_sim_bridge_and_counted_bimodal	
		euf_cma_security_from_exact_hyperball_paper_sim_bridge_and_counted_bimodal	$\Pr[s \in E] \leq B$ is a probability bound or exact game probability.	
		&m : Pr[SIG.UF_NMA(H, HAETAETAE,		
		ROMInternalTranscriptPaperSimAsNMA		
		(A,		
		HAETAERejectionPaperSimSampler(H))) .main()		
		@ &m : res] <= Pr[SIG.UF_NMA(H,		
		HAETAETAE,		
		ROMInternalTranscriptPaperSimAsNMA		
		(A,		
		ExactHyperballHAETAERejectionPaperSimSampler(H))) .main()		
		@ &m : res] => Pr[SIG.UF_NMA...		
	1007	lemma	euf_cma_security_from_exact_hyperball_sampling_hop_and_counted_bimodal	Proves a bound on game-hop and
euf_cma_security_from_exact_hyperball_sampling_hop_and_counted_bimodal		lemma	euf_cma_security_from_exact_hyperball_sampling_hop_and_counted_bimodal	
		euf_cma_security_from_exact_hyperball_sampling_hop_and_counted_bimodal	$\Pr[s \in E] \leq B$ is a probability bound or exact game probability.	
		&m : Pr[SIG.UF_NMA(H, HAETAETAE,		
		ROMInternalTranscriptPaperSimAsNMA		
		(A,		
		HAETAERejectionPaperSimSampler(H))) .main()		
		@ &m : res] <= Pr[SIG.UF_NMA(H,		
		HAETAETAE,		
		ROMInternalTranscriptPaperSimAsNMA		
		(A,		
		ExactHyperballHAETAERejectionPaperSimSampler(H))) .main()		
		@ &m : res] +		
		rejection_sampling...		
	1036	lemma		

Line	Construct	What was written	Symbolic corre
euf_cma_security_from_ro_exact_hyperball_sampling_hop_and_counted_bimodal	lemma euf_cma_security_from_ro_exact_hyperball_sampling_hop_and_counted_bimodal &m : Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, RealSigningPaperSimSampler(H)))].main() @ &m : res] <= Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, ROExactHyperballPaperSimSampler(H)))].main() @ &m : res] + rejection_sampling_loss_term =>...	euf_cma_security_from_ro_exact_hyperball_sampling_hop_and_counted_bimodal Pr[$e \leq E$] < B is a probability bound or exact game probability.	Pr[$e \leq E$] < B is a probability bound or game-hop or final
1072	lemma		
euf_cma_security_from_ro_signing_attempt_to_ro_exact_hyperball_hop_and_counted_bimodal	lemma euf_cma_security_from_ro_signing_attempt_to_ro_exact_hyperball_hop_and_counted_bimodal &m : Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, ROSigningAttemptPaperSimSampler(H)))].main() @ &m : res] <= Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, ROExactHyperballPaperSimSampler(H)))].main() @ &m : res] + rejection_samp...	euf_cma_security_from_ro_signing_attempt_to_ro_exact_hyperball_hop_and_counted_bimodal Pr[$e \leq E$] < B is a probability bound or exact game probability.	Pr[$e \leq E$] < B is a probability bound or game-hop or final
1102	lemma		
real_signing_ro_exact_hyperball_hop_checked_from_loss_budget	lemma real_signing_ro_exact_hyperball_hop_checked_from_loss_budget &m : Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, RealSigningPaperSimSampler(H)))].main() @ &m : res] <= Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, ROExactHyperballPaperSimSampler(H)))].main() @ &m : res] + rejection_sampling_loss_term.	real_signing_ro_exact_hyperball_hop_checked_from_loss_budget Pr[$e \leq E$] < B is a probability bound or exact game probability.	Pr[$e \leq E$] < B is a probability bound or game-hop or final

	Line	Construct	What was written	Symbolic corre
	1116	lemma		
real_signing_ro_exact_hyperball_hop_from_paper_sample_lifting		lemma real_signing_ro_exact_hyperball_hop_from_paper_sample_lifting &m : structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode => Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <= Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, ROExactHyperballPaperSi...	real_signing_ro_exact_hyperball_hop_from_paper_sample_lifting Pr[E] < B is a probability bound or exact game probability.	Paper sample t game-hop or fina
	1140	lemma		
euf_cma_security_from_paper_sample_lifting_and_counted_bimodal		lemma euf_cma_security_from_paper_sample_lifting_and_counted_bimodal &m : structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode => Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <= Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, ROExactHyperballPaperS...	euf_cma_security_from_paper_sample_lifting_and_counted_bimodal Pr[E] < B is a probability bound or exact game probability.	Paper sample t game-hop or fina
	1170	lemma		
euf_cma_security_from_budgeted_paper_sample_lifting_and_counted_bimodal		lemma euf_cma_security_from_budgeted_paper_sample_lifting_and_counted_bimodal &m : Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] <= Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptBudgetedPaperSimAsNMA (A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] => Pr[SIG.UF_NMA(H, HAE...	euf_cma_security_from_budgeted_paper_sample_lifting_and_counted_bimodal Pr[E] < B is a probability bound or exact game probability.	Paper sample t game-hop or fina

	Line	Construct	What was written	Symbolic corre	
euf_cma_security_from_checked_ro_sampling_hop_and_counted_bimodal	1217	lemma	lemma euf_cma_security_from_checked_ro_sampling_hop_and_counted_bimodal &m : Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, ROExactHyperballPaperSimSampler(H)))].main() @ &m : res] + rejection_sampling_loss_term <= Pr[MLWE_Game(Bmlwe).main() @ &m : res] + Pr[BimodalSelfTargetMSIS_Game(Bbimodal).main() @ &m : res] => Pr[MLWE_Game(Bmlwe).mai...	euf_cma_security_from_checked_ro_sampling_hop_and_counted_bimodal $\Pr[e_{\text{sig}} \leq B]$ is a probability bound or exact game probability.	Proves a bound on game-hop or final
euf_cma_security_from_real_signing_exact_hyperball_hop_and_counted_bimodal	1239	lemma	lemma euf_cma_security_from_real_signing_exact_hyperball_hop_and_counted_bimodal &m : Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, RealSigningPaperSimSampler(H)))].main() @ &m : res] <= Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, ExactHyperballPaperSimSampler(H)))].main() @ &m : res] + rejection_sampling_loss_term => P...	euf_cma_security_from_real_signing_exact_hyperball_hop_and_counted_bimodal $\Pr[e_{\text{sig}} \leq B]$ is a probability bound or exact game probability.	Hyperball hop to game-hop or final

4 Sig_ROM.ec

generic signature security games over a random oracle

Line	Construct	What was written	Symbolic correspondence	Why it is written this way
5	type pkey type pkey.	pkey is an abstract carrier set.	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.	
6	type skey type skey.	skey is an abstract carrier set.	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.	
7	type message type message.	message is an abstract carrier set.	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.	
8	type context type context.	context is an abstract carrier set.	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.	
9	type signature type signature.	signature is an abstract carrier set.	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.	
10	type ro_query type ro_query.	ro_query is an abstract carrier set.	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.	
11	type ro_output type ro_output.	ro_output is an abstract carrier set.	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.	
13	operation dro_output op [lossless] dro_output : ro_query -> ro_output distr.	dro_output : $1 \rightarrow \text{Distr}(\text{ro_query} \rightarrow \text{ro_output})$, with total probability mass 1.	Defines a sampler/distribution used by game procedures and probability bounds.	
15	type query type query = message * context.	query $\equiv$ message $\times$ context	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.	
16	type			

Line	Construct	What was written	Symbolic correspondence	Why it is written this way
signed_query	type signed_query = query * signature.	signed_query $\equiv$ query $\times$ signature	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.	
18	operation			
fresh_msg	op fresh_msg (qs : query list) (m : message) (ctx : context) : bool = ! ((m, ctx) \in qs).	fresh_msg $\subseteq$ List(query) $\times$ message $\times$ context is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.	
21	operation			
fresh_sig	op fresh_sig (qs : signed_query list) (m : message) (ctx : context) (sig : signature) : bool = ! (((m, ctx), sig) \in qs).	fresh_sig $\subseteq$ List(signed_query) $\times$ message $\times$ context $\times$ signature is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.	
25	lemma			
fresh_msg_nil	lemma fresh_msg_nil m ctx : fresh_msg [] m ctx.	fresh_msg_nil : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
28	lemma			
fresh_sig_nil	lemma fresh_sig_nil m ctx sig : fresh_sig [] m ctx sig.	fresh_sig_nil : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
32	type			
in_t	type in_t <- ro_query, type out_t <- ro_output, op dout <- dro_output.	in_t is an abstract carrier set.	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.	
149	lemma			
nma_as_cma_exact	lemma nma_as_cma_exact &m : Pr[EUF_CMA(H, S, NMA_As_CMA(A)).main() @ &m : res] = Pr[UF_NMA(H, S, A).main() @ &m : res].	nma_as_cma_exact : $\Pr[c : E] = B$ is a probability bound or exact game probability.	Proves an exact game equivalence or simulator transition.	

5 HAETAH_HopGames.ec

hybrid games, invariants, equivalences, and game-hop losses

	Line	Construct	What was written	Sym	
	81	operation			
		transcript_log_consistent	op transcript_log_consistent (md : mode) (pk : pkey) (qs : SIG.query list) (trs : transcript list) (rs : signing_transcript_record list) : bool = qs = signing_record_queries rs /\ trs = signing_record_transcripts rs /\ signing_record_log_sound md pk rs.	transcript_log_consistent : mode $\times$ pkey $\times$ List(SIG.query) $\times$ List(transcript) $\times$ List(signing_transcript_record) $\rightarrow$ SIG	Defin in th
	88	lemma			
		transcript_log_consistent_nil	lemma transcript_log_consistent_nil md pk : transcript_log_consistent md pk [] [] [].	transcript_log_consistent_nil : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Reco scrip
	95	lemma			
		transcript_log_consistent_cons	lemma transcript_log_consistent_cons md pk qs trs rs m ctx sig tr : transcript_log_consistent md pk qs trs rs => signing_record_relation md pk (m, ctx, sig, tr) => transcript_log_consistent md pk ((m, ctx) :: qs) (tr :: trs) ((m, ctx, sig, tr) :: rs).	transcript_log_consistent_cons : $P \Rightarrow Q$ is an implication used as a proof obligation.	Reco scrip
	114	lemma			
		transcript_log_consistent_no_min_entropy	lemma transcript_log_consistent_no_min_entropy md pk qs trs rs : transcript_log_consistent md pk qs trs rs => transcript_log_min_entropy_clear md trs.	transcript_log_consistent_no_min_entropy : $P \Rightarrow Q$ is an implication used as a proof obligation.	Reco scrip
	124	lemma			

	Line	Construct	What was written	Sym
transcript_log_consistent_valid		lemma transcript_log_consistent_valid md pk qs trs rs : transcript_log_consistent md pk qs trs rs => transcript_log_valid md trs.	transcript_log_consistent_valid : $P \Rightarrow Q$ is an implication used as a proof obligation.	Prove justif
transcript_log_ready	134	operation op transcript_log_ready (md : mode) (trs : transcript list) : bool = transcript_log_valid md trs /\ transcript_log_min_entropy_clear md trs.	transcript_log_ready $\subseteq$ mode $\times$ List(transcript) is a Boolean-valued predicate.	Nam used
budgeted_programmed_query_count_discipline	138	operation op budgeted_programmed_query_count_discipline (hash_count signing_count : int) : bool = 0 <= hash_count /\ hash_count <= hash_query_budget_count /\ 0 <= signing_count /\ signing_count <= signature_query_budget_count.	budgeted_programmed_query_count_discipline $\mathbb{Z} \times \mathbb{Z}$ is a Boolean-valued predicate.	Nam used
budgeted_paper_sim_signing_count_discipline	145	operation op budgeted_paper_sim_signing_count_discipline (signing_count : int) : bool = 0 <= signing_count /\ signing_count <= signature_query_budget_count.	budgeted_paper_sim_signing_count_discipline $\mathbb{Z}$ is a Boolean-valued predicate.	Nam used
budgeted_paper_sim_sampler_freshness_discipline	150	operation op budgeted_paper_sim_sampler_freshness_discipline (hash_count signing_count : int) (hash_qs sampler_qs : ro_query list) : bool = 0 <= hash_count /\ hash_count <= hash_query_budget_count /\ size hash_qs <= hash_count /\ 0 <= signing_count /\ signing_count <= signature_query_budget_count /\ size sampler_qs <= signing_count.	budgeted_paper_sim_sampler_freshness_discipline $\mathbb{Z} \times \mathbb{Z} \times \text{List(ro_query)} \times \text{List(ro_query)}$ is a Boolean-valued predicate.	Disciplin used

	Line	Construct	What was written	Sym
	160	operation		
sampler_rom_covered		<pre> op sampler_rom_covered (rom : (ro_query, ro_output * PROM.flag) fmap) (hash_qs sampler_qs : ro_query list) : bool = forall seed_coins, sampler_expand_query seed_coins \in rom => sampler_expand_query seed_coins \in hash_qs \/ sampler_expand_query seed_coins \in sampler_qs. </pre>	<pre> sampler_rom_covered : List(ro_query) × List(ro_query) → (ro_query, ro_output × PROM </pre>	Defin data
	168	lemma		
sampler_rom_covered_fresh_after_clean_seed		<pre> lemma sampler_rom_covered_fresh_after_clean_seed seed_coins rom hash_qs sampler_qs bad : sampler_rom_covered rom hash_qs sampler_qs => ! (bad \/ sampler_expand_query seed_coins \in hash_qs \/ sampler_expand_query seed_coins \in sampler_qs) => sampler_expand_query seed_coins \notin rom. </pre>	<pre> sampler_rom_covered_fresh_after_clean_seed </pre> $P \Rightarrow Q$ is an implication used as a proof obligation.	Prove game
	179	lemma		
sampler_rom_covered_self_log_preserves		<pre> lemma sampler_rom_covered_self_log_preserves seed_coins rom hash_qs sampler_qs : sampler_rom_covered rom hash_qs sampler_qs => sampler_rom_covered rom hash_qs (sampler_expand_query seed_coins :: sampler_qs). </pre>	<pre> sampler_rom_covered_self_log_preserves : </pre> $P \Rightarrow Q$ is an implication used as a proof obligation.	Prove game
	188	lemma		

	Line	Construct	What was written	Sym	
		sampler_rom_covered_old_log_clean_boundary	lemma sampler_rom_covered_old_log_clean_boundary. seed_coins rom hash_qs sampler_qs bad : sampler_rom_covered rom hash_qs sampler_qs => ! (bad /\ sampler_expand_query seed_coins \in hash_qs /\ sampler_expand_query seed_coins \in sampler_qs) => sampler_expand_query seed_coins \notin rom /\ sampler_rom_covered rom hash_qs (sampler_expand_query seed_coins :....	sampler_rom_covered_old_log_clean_boundary. $P \Rightarrow Q$ is an implication used as a proof obligation.	Prov game
	205	operation	op budgeted_programmed_reprogram_discipline. (pk : pkey) (sk : skey) (programmed : programming_site list) (bad_reprogram : bool) : bool = pk = public_key_of_secret haetae_mode sk /\ programming_site_log_conflict_free_for_honest haetae_mode sk programmed /\ ! bad_reprogram.	budgeted_programmed_reprogram_discipline. $pkey \times skey \times List(programming_site) \times Bool$ is a Boolean-valued predicate.	Name used
	212	operation	op budgeted_programmed_challenge_query_discipline. (hash_count : int) (queries : ro_query list) : bool = 0 <= hash_count /\ hash_count <= hash_query_budget_count /\ challenge_query_log_count queries <= hash_count + 1.	budgeted_programmed_challenge_query_discipline. $\mathbb{Z} \times List(ro_query)$ is a Boolean-valued predicate.	Line used
	218	lemma	lemma transcript_log_consistent_ready md pk qs trs rs : transcript_log_consistent md pk qs trs rs => transcript_log_ready md trs.	transcript_log_consistent_ready : $P \Rightarrow Q$ is an implication used as a proof obligation.	Reco scrip

	Line	Construct	What was written	Sym
	229	lemma transcript_log_valid_member	lemma transcript_log_valid_member md trs tr : transcript_log_valid md trs => tr \in trs => transcript_valid md tr.	transcript_log_valid_member : $P \Rightarrow Q$ is an implication used as a proof obligation. Proved by justification
	239	lemma transcript_log_min_entropy_clear_member	lemma transcript_log_min_entropy_clear_member md md trs tr : transcript_log_min_entropy_clear md trs => tr \in trs => ! min_entropy_failure md tr.	transcript_log_min_entropy_clear_member : Proved by $P \Rightarrow Q$ is an implication used as a proof game obligation.
	249	lemma transcript_log_ready_member_valid	lemma transcript_log_ready_member_valid md trs tr : transcript_log_ready md trs => tr \in trs => transcript_valid md tr.	transcript_log_ready_member_valid : $P \Rightarrow Q$ is an implication used as a proof obligation. Proved by justification
	259	lemma transcript_log_ready_member_no_min_entropy	lemma transcript_log_ready_member_no_min_entropy md trs tr : transcript_log_ready md trs => tr \in trs => ! min_entropy_failure md tr.	transcript_log_ready_member_no_min_entropy : Proved by $P \Rightarrow Q$ is an implication used as a proof obligation. script
	269	lemma transcript_log_ready_no_failure_for_matching_site	lemma transcript_log_ready_no_failure_for_matching_site md trs tr site : transcript_log_ready md trs => tr \in trs => programming_site_matches_transcript md tr site => ! fs_with_aborts_failure md tr site.	transcript_log_ready_no_failure_for_matching_site : Proved by $P \Rightarrow Q$ is an implication used as a proof obligation. script
	280	lemma	lemma	

	Line	Construct	What was written	Sym
transcript_log_ready_no_failure_for_programming_site		lemma	transcript_log_ready_no_failure_for_programming_site	Recall
		transcript_log_ready_no_failure_for_programming_site	$P \Rightarrow Q$ is an implication used as a proof obligation.	script
		md trs tr y :		
		transcript_log_ready md trs =>		
		tr \in trs =>		
		transcript_challenge_matches tr		
		y => ! fs_with_aborts_failure		
		md tr		
		(programming_site_of_transcript		
		md tr y).		
	293	lemma	transcript_log_ready_no_failure_for_signature_programming_site	Recall
transcript_log_ready_no_failure_for_signature_programming_site		lemma	$P \Rightarrow Q$ is an implication used as a proof obligation.	script
		transcript_log_ready_no_failure_for_signature_programming_site		
		md trs tr y :		
		transcript_log_ready md trs =>		
		tr \in trs =>		
		transcript_signature_challenge_matches		
		tr y => !		
		fs_with_aborts_failure md tr		
		(signature_programming_site_of_transcript		
		md tr y).		
	308	lemma	transcript_log_ready_actual_signature_site_clear_member	Recall
transcript_log_ready_actual_signature_site_clear_member		lemma	$P \Rightarrow Q$ is an implication used as a proof obligation.	game
		transcript_log_ready_actual_signature_site_clear_member		
		md trs tr :		
		transcript_log_ready md trs =>		
		tr \in trs =>		
		transcript_signature_programming_site_clear		
		md tr.		
	322	lemma	transcript_log_ready_signature_sites_clear	Prove
transcript_log_ready_signature_sites_clear		lemma	$P \Rightarrow Q$ is an implication used as a proof obligation.	game
		transcript_log_ready_signature_sites_clear		
		md trs : transcript_log_ready		
		md trs =>		
		transcript_log_signature_programming_sites_clear		
		md trs.		
	332	operation		

	Line	Construct	What was written	Sym	
		internal_transcript_state_sound	op internal_transcript_state_sound (md : mode) (pk : pkey) (sk : skey) (qs : SIG.query list) (trs : transcript list) (rs : signing_transcript_record list) : bool = pk = public_key_of_secret md sk /\ transcript_log_consistent md pk qs trs rs.	internal_transcript_state_sound : mode × pkey × skey × List(SIG.query) × List(transcript) × List(signing_transcript_record) → SIG	Defin in th
	338	lemma	internal_transcript_state_sound_nil : internal_transcript_state_sound_nil md sk : internal_transcript_state_sound md (public_key_of_secret md sk) sk [] [] [].	∀x̄. P(x̄) is a universally quantified fact.	Prove orack proce
	348	lemma	internal_transcript_state_sound_keygen : internal_transcript_state_sound_keygen md sd : internal_transcript_state_sound md (keygen_internal md sd). '1 (keygen_internal md sd). '2 [] [] [].	∀x̄. P(x̄) is a universally quantified fact.	Prove orack proce
	360	lemma	internal_transcript_state_sound_no_min_entropy : internal_transcript_state_sound_no_min_entropy md pk sk qs trs rs : internal_transcript_state_sound md pk sk qs trs rs => transcript_log_min_entropy_clear md trs.	P ⇒ Q is an implication used as a proof obligation.	Propo orack proce
	370	lemma	internal_transcript_state_sound_valid : internal_transcript_state_sound_valid md pk sk qs trs rs : internal_transcript_state_sound md pk sk qs trs rs => transcript_log_valid md trs.	P ⇒ Q is an implication used as a proof obligation.	Prove orack proce
	379	lemma			

Line	Construct	What was written	Sym
	internal_transcript_state_sound_log_ready	lemma internal_transcript_state_sound_log_ready md pk sk qs trs rs : internal_transcript_state_sound md pk sk qs trs rs => transcript_log_ready md trs.	internal_transcript_state_sound_log_ready : Prov $P \Rightarrow Q$ is an implication used as a proof obligation. oracle proce
388	lemma transcript_log_consistent_sign_internal_cons	lemma transcript_log_consistent_sign_internal_cons md sk qs trs rs m ctx coins : transcript_log_consistent md (public_key_of_secret md sk) qs trs rs => transcript_log_consistent md (public_key_of_secret md sk) ((m, ctx) :: qs) (transcript_of_signature md (public_key_of_secret md sk) m ctx (sign_internal md sk m ctx coins) :: trs) ((m, ctx, sign_internal m...	transcript_log_consistent_sign_internal_cons : Reco $P \Rightarrow Q$ is an implication used as a proof obligation. scrip
406	lemma internal_transcript_state_sound_sign_internal_cons	lemma internal_transcript_state_sound_sign_internal_cons md pk sk qs trs rs m ctx coins : internal_transcript_state_sound md pk sk qs trs rs => internal_transcript_state_sound md pk sk ((m, ctx) :: qs) (transcript_of_signature md pk m ctx (sign_internal md sk m ctx coins) :: trs) ((m, ctx, sign_internal md sk m ctx coins, transcript_of_signature md pk m c...	internal_transcript_state_sound_sign_internal_cons : Pro $P \Rightarrow Q$ is an implication used as a proof obligation. oracle proce
428	lemma	lemma	

	Line	Construct	What was written	Sym
		<pre> sign_internal_transcript_no_min_entropy lemma sign_internal_transcript_no_min_entropy md pk sk m ctx coins : pk = public_key_of_secret md sk => ! min_entropy_failure md (transcript_of_signature md pk m ctx (sign_internal md sk m ctx coins)). </pre>	<pre> sign_internal_transcript_no_min_entropy : $B \Rightarrow Q$ is an implication used as a proof obligation. </pre>	Reco scrip
	665	<pre> public_sim_source op public_sim_source (pk : pkey) (m : message) (ctx : context) (coins : random_coins) : byte list = coins ++ pk.'1 ++ ctx ++ m. </pre>	<pre> public_sim_source : pkey $\times$ message $\times$ context $\times$ random_coins $\rightarrow$ List(byte) </pre>	Defin by th
	670	<pre> public_sim_response_aux_vector op public_sim_response_aux_vector : mode -> pkey -> message -> context -> random_coins -> polyveck = fun md pk m ctx coins => deterministic_unit_polyveck md 29 (public_sim_source pk m ctx coins). </pre>	<pre> public_sim_response_aux_vector : 1 $\rightarrow$ mode -> pkey -> message -> context -> random_coins -> </pre>	Defin randd
	676	<pre> public_sim_response_vector op public_sim_response_vector : mode -> pkey -> message -> context -> random_coins -> polyvecl = fun md pk m ctx coins => deterministic_unit_polyvecl md 17 (public_sim_source pk m ctx coins). </pre>	<pre> public_sim_response_vector : 1 $\rightarrow$ mode -> pkey -> message -> context -> random_coins -> </pre>	Defin randd
	682	<pre> public_sim_hint_vector op public_sim_hint_vector : mode -> pkey -> message -> context -> random_coins -> hint_t = fun md pk m ctx coins => deterministic_unit_polyveck md 41 (public_sim_source pk m ctx coins). </pre>	<pre> public_sim_hint_vector : 1 $\rightarrow$ mode -> pkey -> message -> context -> random_coins -> </pre>	Defin randd
	688	<pre> operation </pre>	<pre> operation </pre>	

	Line	Construct	What was written	Sym
		<pre> public_sim_commitment_lowbits op public_sim_commitment_lowbits : mode -> pkey -> message -> context -> random_coins -> poly = fun md pk m ctx coins => let raw = deterministic_polyveck md (public_sim_source pk m ctx coins) in let row0 = nth poly_zero raw 0 in mkseq (fun i => if i = 0 then signing_entropy_token_of_coins coins else poly_coeff row0 i) n. </pre>	<pre> public_sim_commitment_lowbits : 1 → mode— > pkey— > message— > context— > random_coins— > poly </pre>	Defin
	700	operation		
		<pre> public_sim_commitment_challenge op public_sim_commitment_challenge : mode -> pkey -> message -> context -> random_coins -> challenge = fun md pk m ctx coins => challenge_hash md (public_sim_response_aux_vector md pk m ctx coins) (public_sim_commitment_lowbits md pk m ctx coins) (message_hash pk ctx m). </pre>	<pre> public_sim_commitment_challenge : 1 → mode— > pkey— > message— > context— > random_coins— > challenge </pre>	Defin
	708	operation		
		<pre> public_sim_commitment_highbits op public_sim_commitment_highbits : mode -> pkey -> message -> context -> random_coins -> polyveck = fun md pk m ctx coins => let ch = public_sim_commitment_challenge md pk m ctx coins in reconstructed_highbits md (public_sim_response_aux_vector md pk m ctx coins) (public_key_challenge_term md pk ch). </pre>	<pre> public_sim_commitment_highbits : 1 → mode— > pkey— > message— > context— > random_coins— > polyveck </pre>	Defin
	716	operation		

	Line	Construct	What was written	Sym
		<pre> public_sim_signature_token op public_sim_signature_token : mode -> pkey -> message -> context -> random_coins -> sig_token = fun md pk m ctx coins => (public_sim_response_vector md pk m ctx coins, public_sim_response_aux_vector md pk m ctx coins, public_sim_hint_vector md pk m ctx coins). </pre>	<pre> public_sim_signature_token : 1 → mode → pkey → message → context → random_coins → </pre>	Defin
	723	<pre> operation public_sim_signature op public_sim_signature : mode -> pkey -> message -> context -> random_coins -> signature = fun md pk m ctx coins => let highbits = public_sim_commitment_highbits md pk m ctx coins in let lowbits = public_sim_commitment_lowbits md pk m ctx coins in let mu = message_hash pk ctx m in let ch = public_sim_commitment_challenge md pk m ctx coins in (highbits, 1... </pre>	<pre> public_sim_signature : 1 → mode → pkey → message → context → random_coins → </pre>	Defin
	733	<pre> lemma public_sim_commitment_lowbitsE lemma public_sim_commitment_lowbitsE md sk m ctx coins : public_sim_commitment_lowbits md (public_key_of_secret md sk) m ctx coins = commitment_lowbits md sk m ctx coins. </pre>	<pre> public_sim_commitment_lowbitsE : L = R is an equality/rewriting fact. </pre>	Reco
	742	<pre> lemma </pre>		scrip

	Line	Construct	What was written	Sym
public_sim_response_aux_vectorE		lemma public_sim_response_aux_vectorE md sk m ctx coins : public_sim_response_aux_vector md (public_key_of_secret md sk) m ctx coins = response_aux_vector md sk m ctx coins.	public_sim_response_aux_vectorE : $L = R$ is an equality/rewriting fact.	Reco scrip
public_sim_commitment_challengeE	750	lemma lemma public_sim_commitment_challengeE md sk m ctx coins : public_sim_commitment_challenge md (public_key_of_secret md sk) m ctx coins = commitment_challenge md sk m ctx coins.	public_sim_commitment_challengeE : $L = R$ is an equality/rewriting fact.	Prove game
public_sim_commitment_highbitsE	758	lemma lemma public_sim_commitment_highbitsE md sk m ctx coins : public_sim_commitment_highbits md (public_key_of_secret md sk) m ctx coins = commitment_highbits md sk m ctx coins.	public_sim_commitment_highbitsE : $L = R$ is an equality/rewriting fact.	Reco scrip
public_sim_response_vectorE	767	lemma lemma public_sim_response_vectorE md sk m ctx coins : public_sim_response_vector md (public_key_of_secret md sk) m ctx coins = response_vector md sk m ctx coins.	public_sim_response_vectorE : $L = R$ is an equality/rewriting fact.	Reco scrip
public_sim_hint_vectorE	775	lemma lemma public_sim_hint_vectorE md sk m ctx coins : public_sim_hint_vector md (public_key_of_secret md sk) m ctx coins = hint_vector md sk m ctx coins.	public_sim_hint_vectorE : $L = R$ is an equality/rewriting fact.	Reco scrip

	Line	Construct	What was written	Sym
	783	lemma		
public_sim_signature_tokenE		<pre> lemma public_sim_signature_tokenE md sk m ctx coins : public_sim_signature_token md (public_key_of_secret md sk) m ctx coins = signature_token md sk m ctx coins. </pre>	public_sim_signature_tokenE : $L = R$ is an equality/rewriting fact.	Reco scrip
	793	lemma		
public_sim_signatureE		<pre> lemma public_sim_signatureE md sk m ctx coins : public_sim_signature md (public_key_of_secret md sk) m ctx coins = sign_internal md sk m ctx coins. </pre>	public_sim_signatureE : $L = R$ is an equality/rewriting fact.	Reco scrip
	855	type		
paper_sim_signature_sample		<pre> type paper_sim_signature_sample = sig_token * poly * int. </pre>	paper_sim_signature_sample $\equiv$ sig_token $\times$ poly $\times \mathbb{Z}$	Fixes later an im
	857	operation		
paper_sim_sample_token		<pre> op paper_sim_sample_token (s : paper_sim_signature_sample) : sig_token = s.'1. </pre>	paper_sim_sample_token : paper_sim_signature_sample $\rightarrow$ sig_token	Defin by th
	859	operation		
paper_sim_sample_lowbits_carrier		<pre> op paper_sim_sample_lowbits_carrier (s : paper_sim_signature_sample) : poly = s.'2. </pre>	paper_sim_sample_lowbits_carrier : paper_sim_signature_sample $\rightarrow$ poly	Defin by th
	862	operation		
paper_sim_sample_entropy		<pre> op paper_sim_sample_entropy (s : paper_sim_signature_sample) : int = s.'3. </pre>	paper_sim_sample_entropy : paper_sim_signature_sample $\rightarrow \mathbb{Z}$	Defin by th
	864	operation		
paper_sim_sample_response		<pre> op paper_sim_sample_response (s : paper_sim_signature_sample) : polyvecl = (paper_sim_sample_token s).'1. </pre>	paper_sim_sample_response : paper_sim_signature_sample $\rightarrow$ polyvecl	Defin by th
	866	operation		

	Line	Construct	What was written	Sym
		paper_sim_sample_response_aux	op paper_sim_sample_response_aux (s : paper_sim_signature_sample) : polyveck = (paper_sim_sample_token s).‘2.	paper_sim_sample_response_aux : paper_sim_signature_sample → polyveck
	869	operation		
		paper_sim_sample_hint	op paper_sim_sample_hint (s : paper_sim_signature_sample) : hint_t = (paper_sim_sample_token s).‘3.	paper_sim_sample_hint : paper_sim_signature_sample → hint_t
	872	operation		
		paper_sim_commitment_lowbits	op paper_sim_commitment_lowbits (md : mode) (s : paper_sim_signature_sample) : poly = mkseq (fun i => if i = 0 then paper_sim_sample_entropy s else poly_coeff (paper_sim_sample_lowbits_carrier s) i) n.	paper_sim_commitment_lowbits : mode × paper_sim_signature_sample → poly
	880	operation		
		paper_sim_commitment_challenge	op paper_sim_commitment_challenge (md : mode) (pk : pkey) (m : message) (ctx : context) (s : paper_sim_signature_sample) : challenge = challenge_hash md (paper_sim_sample_response_aux s) (paper_sim_commitment_lowbits md s) (message_hash pk ctx m).	paper_sim_commitment_challenge : mode × pkey × message × context × paper_sim_signature_sample → challenge
	888	operation		
		paper_sim_commitment_highbits	op paper_sim_commitment_highbits (md : mode) (pk : pkey) (m : message) (ctx : context) (s : paper_sim_signature_sample) : polyveck = let ch = paper_sim_commitment_challenge md pk m ctx s in reconstructed_highbits md (paper_sim_sample_response_aux s) (public_key_challenge_term md pk ch).	paper_sim_commitment_highbits : mode × pkey × message × context × paper_sim_signature_sample → polyveck

	Line	Construct	What was written	Sym
	896	operation		
paper_sim_signature		<pre> op paper_sim_signature (md : mode) (pk : pkey) (m : message) (ctx : context) (s : paper_sim_signature_sample) : signature = let highbits = paper_sim_commitment_highbits md pk m ctx s in let lowbits = paper_sim_commitment_lowbits md s in let mu = message_hash pk ctx m in let ch = paper_sim_commitment_challenge md pk m ctx s in (highbits, lowbits, mu, ch, p...</pre>	<pre> paper_sim_signature : mode × pkey × message × context × paper_sim_signature_sample → signature</pre>	Defin by th
	905	operation		
paper_sim_signature_sample_wf		<pre> op paper_sim_signature_sample_wf (md : mode) (s : paper_sim_signature_sample) : bool = polyvecl_wf md (paper_sim_sample_response s) /\ polyveck_wf md (paper_sim_sample_response_aux s) /\ polyveck_wf md (paper_sim_sample_hint s) /\ poly_wf (paper_sim_sample_lowbits_carrier s).</pre>	<pre> paper_sim_signature_sample_wf ⊆ mode × paper_sim_signature_sample is a Boolean-valued predicate.</pre>	Nam cond
	912	lemma		
paper_sim_commitment_lowbits_wf		<pre> lemma paper_sim_commitment_lowbits_wf md s : poly_wf (paper_sim_commitment_lowbits md s).</pre>	<pre> paper_sim_commitment_lowbits_wf : ∀x̄. P(x̄) is a universally quantified fact.</pre>	Prove justif
	918	lemma		
paper_sim_signature_valid		<pre> lemma paper_sim_signature_valid md pk m ctx s : paper_sim_signature_sample_wf md s => valid_signature md (paper_sim_signature md pk m ctx s).</pre>	<pre> paper_sim_signature_valid : P ⇒ Q is an implication used as a proof obligation.</pre>	Prove justif
	931	operation		

	Line	Construct	What was written	Sym
		<pre> public_sim_lowbits_carrier op public_sim_lowbits_carrier (md : mode) (pk : pkey) (m : message) (ctx : context) (coins : random_coins) : poly = nth poly_zero (deterministic_polyveck md (public_sim_source pk m ctx coins)) 0. </pre>	<pre> public_sim_lowbits_carrier : mode × pkey × message × context × random_coins → poly </pre>	Defin by th
	937	operation		
		<pre> paper_sim_sample_from_public_coins op paper_sim_sample_from_public_coins (md : mode) (pk : pkey) (m : message) (ctx : context) (coins : random_coins) : paper_sim_signature_sample = (public_sim_signature_token md pk m ctx coins, public_sim_lowbits_carrier md pk m ctx coins, signing_entropy_token_of_coins coins). </pre>	<pre> paper_sim_sample_from_public_coins : mode × pkey × message × context × random_coins → paper_sim_signature_sample </pre>	Defin data
	944	lemma		
		<pre> public_sim_lowbits_carrier_wf lemma public_sim_lowbits_carrier_wf md pk m ctx coins : poly_wf (public_sim_lowbits_carrier md pk m ctx coins). </pre>	<pre> public_sim_lowbits_carrier_wf : ∀x̄. P(x̄) is a universally quantified fact. </pre>	Prove justif
	955	lemma		
		<pre> paper_sim_sample_from_public_coins_wf lemma paper_sim_sample_from_public_coins_wf md pk m ctx coins : paper_sim_signature_sample_wf md (paper_sim_sample_from_public_coins md pk m ctx coins). </pre>	<pre> paper_sim_sample_from_public_coins_wf : Proved ∀x̄. P(x̄) is a universally quantified fact. </pre>	Prove game
	970	lemma		

	Line	Construct	What was written	Sym
paper_sim_sample_response_public_coinsE		lemma	paper_sim_sample_response_public_coinsE : $L = R$ is an equality/rewriting fact.	Prov game
		paper_sim_sample_response_public_coinsE		
		md pk m ctx coins :		
		paper_sim_sample_response		
		(paper_sim_sample_from_public_coins		
		md pk m ctx coins) =		
		public_sim_response_vector md pk		
		m ctx coins.		
	978	lemma		
paper_sim_sample_response_aux_public_coinsE		lemma	paper_sim_sample_response_aux_public_coinsE : $L = R$ is an equality/rewriting fact.	Prov game
		paper_sim_sample_response_aux_public_coinsE		
		md pk m ctx coins :		
		paper_sim_sample_response_aux		
		(paper_sim_sample_from_public_coins		
		md pk m ctx coins) =		
		public_sim_response_aux_vector		
		md pk m ctx coins.		
	986	lemma		
paper_sim_sample_hint_public_coinsE		lemma	paper_sim_sample_hint_public_coinsE : $L = R$ is an equality/rewriting fact.	Prov game
		paper_sim_sample_hint_public_coinsE		
		md pk m ctx coins :		
		paper_sim_sample_hint		
		(paper_sim_sample_from_public_coins		
		md pk m ctx coins) =		
		public_sim_hint_vector md pk m		
		ctx coins.		
	994	lemma		
paper_sim_sample_token_public_coinsE		lemma	paper_sim_sample_token_public_coinsE : $L = R$ is an equality/rewriting fact.	Prov game
		paper_sim_sample_token_public_coinsE		
		md pk m ctx coins :		
		paper_sim_sample_token		
		(paper_sim_sample_from_public_coins		
		md pk m ctx coins) =		
		public_sim_signature_token md pk		
		m ctx coins.		
	1001	lemma		

	Line	Construct	What was written	Sym
		paper_sim_commitment_lowbits_public_coinsE	lemma paper_sim_commitment_lowbits_public_coinsE is an equality/rewriting fact. md pk m ctx coins : paper_sim_commitment_lowbits md (paper_sim_sample_from_public_coins md pk m ctx coins) = public_sim_commitment_lowbits md pk m ctx coins.	Reco scrip
	1012	lemma		
		paper_sim_commitment_challenge_public_coinsE	lemma paper_sim_commitment_challenge_public_coinsE is an equality/rewriting fact. md pk m ctx coins : paper_sim_commitment_challenge md pk m ctx (paper_sim_sample_from_public_coins md pk m ctx coins) = public_sim_commitment_challenge md pk m ctx coins.	Reco game
	1022	lemma		
		paper_sim_commitment_highbits_public_coinsE	lemma paper_sim_commitment_highbits_public_coinsE is an equality/rewriting fact. md pk m ctx coins : paper_sim_commitment_highbits md pk m ctx (paper_sim_sample_from_public_coins md pk m ctx coins) = public_sim_commitment_highbits md pk m ctx coins.	Reco scrip
	1032	lemma		
		paper_sim_signature_public_coinsE	lemma paper_sim_signature_public_coinsE : $L = R$ is an equality/rewriting fact. md pk m ctx coins : paper_sim_signature md pk m ctx (paper_sim_sample_from_public_coins md pk m ctx coins) = public_sim_signature md pk m ctx coins.	Reco scrip
	1044	operation		

	Line	Construct	What was written	Sym	
		real_signing_lowbits_carrier	op real_signing_lowbits_carrier (md : mode) (sk : skey) (m : message) (ctx : context) (coins : random_coins) : poly = nth poly_zero (commitment_raw md sk m ctx coins) 0.	real_signing_lowbits_carrier : mode × skey × message × context × random_coins → poly	Defin by th
	1049	operation			
		paper_sim_sample_from_real_signing_coins	op paper_sim_sample_from_real_signing_coins (md : mode) (sk : skey) (m : message) (ctx : context) (coins : random_coins) : paper_sim_signature_sample = (signature_token md sk m ctx coins, real_signing_lowbits_carrier md sk m ctx coins, signing_entropy_token_of_coins coins).	paper_sim_sample_from_real_signing_coins mode × skey × message × context × random_coins → paper_sim_signature_sample	Defin data
	1056	lemma			
		real_signing_lowbits_carrier_wf	lemma real_signing_lowbits_carrier_wf md sk m ctx coins : poly_wf (real_signing_lowbits_carrier md sk m ctx coins).	real_signing_lowbits_carrier_wf : ∀x̄. P(x̄) is a universally quantified fact.	Prove justif
	1065	lemma			
		paper_sim_sample_from_real_signing_coins_wf	lemma paper_sim_sample_from_real_signing_coins_wf md sk m ctx coins : paper_sim_signature_sample_wf md (paper_sim_sample_from_real_signing_coins md sk m ctx coins).	paper_sim_sample_from_real_signing_coins_wf : ∀x̄. P(x̄) is a universally quantified fact.	Prove game
	1078	lemma			
		paper_sim_sample_response_real_signing_coinsE	lemma paper_sim_sample_response_real_signing_coinsE md sk m ctx coins : paper_sim_sample_response (paper_sim_sample_from_real_signing_coins md sk m ctx coins) = response_vector md sk m ctx coins.	paper_sim_sample_response_real_signing_coinsE : h = c is an equality/rewriting fact.	Defin game

	Line	Construct	What was written	Sym
	1086	lemma		
paper_sim_sample_response_aux_real_signing_coinsE		lemma	paper_sim_sample_response_aux_real_signing_coinsE is an equality/rewriting fact.	Prove game
		paper_sim_sample_response_aux_real_signing_coinsE		
		md sk m ctx coins :		
		paper_sim_sample_response_aux		
		(paper_sim_sample_from_real_signing_coins		
		md sk m ctx coins) =		
		response_aux_vector md sk m ctx		
		coins.		
	1094	lemma		
paper_sim_sample_hint_real_signing_coinsE		lemma	paper_sim_sample_hint_real_signing_coinsE is an equality/rewriting fact.	Prove game
		paper_sim_sample_hint_real_signing_coinsE		
		md sk m ctx coins :		
		paper_sim_sample_hint		
		(paper_sim_sample_from_real_signing_coins		
		md sk m ctx coins) = hint_vector		
		md sk m ctx coins.		
	1102	lemma		
paper_sim_sample_token_real_signing_coinsE		lemma	paper_sim_sample_token_real_signing_coinsE is an equality/rewriting fact.	Prove game
		paper_sim_sample_token_real_signing_coinsE		
		md sk m ctx coins :		
		paper_sim_sample_token		
		(paper_sim_sample_from_real_signing_coins		
		md sk m ctx coins) =		
		signature_token md sk m ctx		
		coins.		
	1109	lemma		
paper_sim_commitment_lowbits_real_signing_coinsE		lemma	paper_sim_commitment_lowbits_real_signing_coinsE is an equality/rewriting fact.	Prove game
		paper_sim_commitment_lowbits_real_signing_coinsE		
		md sk m ctx coins :		
		paper_sim_commitment_lowbits md		
		(paper_sim_sample_from_real_signing_coins		
		md sk m ctx coins) =		
		commitment_lowbits md sk m ctx		
		coins.		
	1120	lemma		

Line	Construct	What was written	Sym
	paper_sim_commitment_challenge_real_signing_coinsE	lemma paper_sim_commitment_challenge_real_signing_coinsE md sk m ctx coins : paper_sim_commitment_challenge md (public_key_of_secret md sk) m ctx (paper_sim_sample_from_real_signing_coins md sk m ctx coins) = commitment_challenge md sk m ctx coins.	paper_sim_commitment_challenge_real_signing_coinsE : Provable game
1131	lemma		
	paper_sim_commitment_highbits_real_signing_coinsE	lemma paper_sim_commitment_highbits_real_signing_coinsE md sk m ctx coins : paper_sim_commitment_highbits md (public_key_of_secret md sk) m ctx (paper_sim_sample_from_real_signing_coins md sk m ctx coins) = commitment_highbits md sk m ctx coins.	paper_sim_commitment_highbits_real_signing_coinsE : Record script
1142	lemma		
	paper_sim_signature_real_signing_coinsE	lemma paper_sim_signature_real_signing_coinsE md sk m ctx coins : paper_sim_signature md (public_key_of_secret md sk) m ctx (paper_sim_sample_from_real_signing_coins md sk m ctx coins) = sign_internal md sk m ctx coins.	paper_sim_signature_real_signing_coinsE : Record script
1154	operation		
	paper_sim_sample_from_rejection_attempt	op paper_sim_sample_from_rejection_attempt (st : signing_attempt_state) : paper_sim_signature_sample = (signing_attempt_state_token st, signing_attempt_state_lowbits_carrier st, signing_entropy_token_of_coins (signing_attempt_state_coins st)).	paper_sim_sample_from_rejection_attempt : Definition data

	Line	Construct	What was written	Sym
	1160	operation		
		paper_sim_abort_fallback_sample	op paper_sim_abort_fallback_sample (md : mode) : paper_sim_signature_sample = ((polyvecl_zero md, polyveck_zero md, polyveck_zero md), poly_zero, 0).	paper_sim_abort_fallback_sample : mode → paper_sim_signature_sample Defin by th
	1164	operation		
		haetae_rejection_sample_from_attempt	op haetae_rejection_sample_from_attempt (md : mode) (pk : pkey) (m : message) (ctx : context) (st : signing_attempt_state) : paper_sim_signature_sample = if signing_attempt_state_rejection_accepts md pk m ctx st then paper_sim_sample_from_rejection_attempt st else paper_sim_abort_fallback_sample md.	haetae_rejection_sample_from_attempt : mode × pkey × message × context × signing_attempt_state → paper_sim_signature_sample Defin data
	1171	lemma		
		paper_sim_abort_fallback_sample_wf	lemma paper_sim_abort_fallback_sample_wf md : paper_sim_signature_sample_wf md (paper_sim_abort_fallback_sample md).	paper_sim_abort_fallback_sample_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact. Prove game
	1182	lemma		
		paper_sim_sample_from_rejection_attempt_of_coinsE	lemma paper_sim_sample_from_rejection_attempt_of_coinsE md sk m ctx coins : paper_sim_sample_from_rejection_attempt (signing_attempt_state_of_coins md sk m ctx coins) = paper_sim_sample_from_real_signing_coins md sk m ctx coins.	paper_sim_sample_from_rejection_attempt_of_coinsE : $L = R$ is an equality/rewriting fact. Prove game
	1195	lemma		

Line	Construct	What was written	Sym
	paper_sim_commitment_lowbits_rejection_attemptE	lemma paper_sim_commitment_lowbits_rejection_attemptE is an equality/rewriting fact.	Atom script
1207	lemma	paper_sim_commitment_lowbits_rejection_attemptE	Atom
	paper_sim_commitment_challenge_rejection_attemptE	lemma paper_sim_commitment_challenge_rejection_attemptE is an implication used as a proof obligation.	Pattern game
1225	lemma	paper_sim_commitment_challenge_rejection_attemptE	Atom
	paper_sim_commitment_highbits_rejection_attemptE	lemma paper_sim_commitment_highbits_rejection_attemptE is an implication used as a proof obligation.	Atom script
1249	lemma	paper_sim_commitment_highbits_rejection_attemptE	Atom

	Line	Construct	What was written	Sym
		<pre> paper_sim_signature_rejection_attempt_fieldsE lemma paper_sim_signature_rejection_attempt_fieldsE md pk m ctx st : signing_attempt_state_field_accepts md pk m ctx st => paper_sim_signature md pk m ctx (paper_sim_sample_from_rejection_attempt st) = signing_attempt_state_signature_of_fields pk m ctx st. </pre>	<pre> paper_sim_signature_rejection_attempt_fieldsE P => Q is an implication used as a proof obligation. </pre>	Def script
	1277	<pre> lemma paper_sim_signature_rejection_attempt_matches_state lemma paper_sim_signature_rejection_attempt_matches_state md pk m ctx st : signing_attempt_state_field_accepts md pk m ctx st => paper_sim_signature md pk m ctx (paper_sim_sample_from_rejection_attempt st) = signing_attempt_state_signature st. </pre>	<pre> paper_sim_signature_rejection_attempt_matches_state P => Q is an implication used as a proof obligation. </pre>	Def oracle process
	1293	<pre> lemma paper_sim_signature_rejection_attempt_of_coinsE lemma paper_sim_signature_rejection_attempt_of_coinsE md sk m ctx coins : paper_sim_signature md (public_key_of_secret md sk) m ctx (paper_sim_sample_from_rejection_attempt (signing_attempt_state_of_coins md sk m ctx coins)) = sign_internal md sk m ctx coins. </pre>	<pre> paper_sim_signature_rejection_attempt_of_coinsE L of R is an equality/rewriting fact. </pre>	Def coin script
	1306	<pre> lemma </pre>	<pre> lemma </pre>	

	Line	Construct	What was written	Sym
haetae_rejection_sample_from_attempt_signatureE		lemma	haetae_rejection_sample_from_attempt_signatureE	Proof
		haetae_rejection_sample_from_attempt_signatureE	$P \Rightarrow Q$ is an implication used as a proof	game
		md pk m ctx st :	obligation.	
		signing_attempt_state_rejection_accepts		
		md pk m ctx st =>		
		paper_sim_signature md pk m ctx		
		(haetae_rejection_sample_from_attempt		
		md pk m ctx st) =		
		signing_attempt_state_signature		
		st.		
	1323	lemma		
haetae_rejection_sample_from_attempt_no_fallback		lemma	haetae_rejection_sample_from_attempt_no_fallback	Proof
		haetae_rejection_sample_from_attempt_no_fallback	$P \Rightarrow Q$ is an implication used as a proof	game
		md pk m ctx st :	obligation.	
		signing_attempt_state_rejection_accepts		
		md pk m ctx st =>		
		haetae_rejection_sample_from_attempt		
		md pk m ctx st =		
		paper_sim_sample_from_rejection_attempt		
		st.		
	1332	lemma		
haetae_rejection_sample_from_attempt_of_coinsE		lemma	haetae_rejection_sample_from_attempt_of_coinsE	Proof
		haetae_rejection_sample_from_attempt_of_coinsE	$L = R$ is an equality/rewriting fact.	game
		md sk m ctx coins :		
		haetae_rejection_sample_from_attempt		
		md (public_key_of_secret md sk)		
		m ctx		
		(signing_attempt_state_of_coins		
		md sk m ctx coins) =		
		paper_sim_sample_from_real_signing_coins		
		md sk m ctx coins.		
	1343	lemma		
haetae_rejection_sample_from_attempt_of_coins_wf		lemma	haetae_rejection_sample_from_attempt_of_coins_wf	Proof
		haetae_rejection_sample_from_attempt_of_coins_wf	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	game
		md sk m ctx coins :		
		paper_sim_signature_sample_wf md		
		(haetae_rejection_sample_from_attempt		
		md (public_key_of_secret md sk)		
		m ctx		
		(signing_attempt_state_of_coins		
		md sk m ctx coins)).		

	Line	Construct	What was written	Sym
	1774	lemma		
budgeted_paper_sim_sign_oracle_preserves_signing_count		lemma	budgeted_paper_sim_sign_oracle_preserves_signing_count	Sign
		budgeted_paper_sim_sign_oracle_preserves_signing_count	$P \Rightarrow Q$ is an implication used as a proof	game
		:	obligation.	
		hoare[ROMInternalTranscriptBudgetedPaperSimAsNMA(A,		
		Samp, H).0.sign :		
		budgeted_paper_sim_signing_count_discipline		
		ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count		
		==>		
		budgeted_paper_sim_signing_count_discipline		
		ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count].		
	1795	lemma		
adversary_budgeted_paper_sim_signing_count_preserved		lemma	adversary_budgeted_paper_sim_signing_count_preserved	Rec
		adversary_budgeted_paper_sim_signing_count_preserved	$P \Rightarrow Q$ is an implication used as a proof	scrip
		:	obligation.	
		hoare[ROMInternalTranscriptBudgetedPaperSimAsNMA(A,		
		Samp, H).A.forge :		
		budgeted_paper_sim_signing_count_discipline		
		ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count		
		==>		
		budgeted_paper_sim_signing_count_discipline		
		ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count].		
	1816	lemma		
budgeted_paper_sim_main_signing_count_preserved		lemma	budgeted_paper_sim_main_signing_count_preserved	Pres
		budgeted_paper_sim_main_signing_count_preserved	$P \Rightarrow Q$ is an implication used as a proof	scrip
		:	obligation.	
		hoare[ROMInternalTranscriptBudgetedPaperSimAsNMA(A,		
		Samp, H).forge : true ==>		
		budgeted_paper_sim_signing_count_discipline		
		ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count].		
	1836	lemma		
sampler_expand_query_point_bound		lemma	sampler_expand_query_point_bound :	Pro
		sampler_expand_query_point_bound	$X \leq Y$ is an arithmetic or probability	game
		q : mu drandom_coins (fun coins	upper bound.	
		=> sampler_expand_query coins =		
		q) <=		
		signing_randomness_point_bound.		
	1859	lemma		

	Line	Construct	What was written	Sym
sampler_expand_query_log_fset_bound		lemma sampler_expand_query_log_fset_bound (qs : ro_query list) : mu drandom_coins (fun coins => sampler_expand_query coins \in qs) <= (card (oflist qs))%r * signing_randomness_point_bound.	sampler_expand_query_log_fset_bound : $X \leq Y$ is an arithmetic or probability upper bound.	Prove game
	1877	lemma		
sampler_expand_query_log_budget_bound		lemma sampler_expand_query_log_budget_bound (qs : ro_query list) hc : 0 <= hc => hc <= hash_query_budget_count => size qs <= hc => mu drandom_coins (fun coins => sampler_expand_query coins \in qs) <= rom_hash_query_budget / challenge_support_cardinality_lower_bound.	sampler_expand_query_log_budget_bound : $X \leq Y$ is an arithmetic or probability upper bound.	Prove game
	1896	lemma		
sampler_expand_query_joint_log_budget_bound		lemma sampler_expand_query_joint_log_budget_bound (hash_qs sampler_qs : ro_query list) hc sc : 0 <= hc => hc <= hash_query_budget_count => 0 <= sc => sc <= signature_query_budget_count => size hash_qs <= hc => size sampler_qs <= sc => mu drandom_coins (fun coins => sampler_expand_query coins \in hash_qs \ / sampler_expand_query coins \in sampler_qs) <= (ro...	sampler_expand_query_joint_log_budget_bound : $X \leq Y$ is an arithmetic or probability upper bound.	Prove game
	1929	lemma		

	Line	Construct	What was written	Sym
		<pre> lemma concrete_lazy_rom_sampler_expand_query_fresh_le seed_coins (p : random_coins -> bool) &m : sampler_expand_query seed_coins \notin HAETAERO.FRO.m{m} => Pr[HAETAERO.FRO.get(sampler_expand_query seed_coins) @ &m : p (ro_signing_coins res)] <= mu drandom_coins p. </pre>	concrete_lazy_rom_sampler_expand_query_fresh_le $\Pr[c \in E] \leq B$ is a probability bound or exact game probability.	Fresh game
	1949	<pre> lemma concrete_lazy_rom_sampler_expand_query_fresh_eq seed_coins (p : random_coins -> bool) : phoare[HAETAERO.FRO.get : arg = sampler_expand_query seed_coins /\ sampler_expand_query seed_coins \notin HAETAERO.FRO.m ==> p (ro_signing_coins res)] = (mu drandom_coins p). </pre>	concrete_lazy_rom_sampler_expand_query_fresh_eq $P \leftrightarrow Q$ is an implication used as a proof obligation.	Fresh game
	1964	<pre> lemma concrete_lazy_rom_get_preserves_sampler_rom_covered q hash_qs sampler_qs : hoare[HAETAERO.FRO.get : arg = q /\ sampler_rom_covered HAETAERO.FRO.m hash_qs sampler_qs /\ (forall seed_coins, q = sampler_expand_query seed_coins => q \in hash_qs \/ q \in sampler_qs) ==> sampler_rom_covered HAETAERO.FRO.m hash_qs sampler_qs]. </pre>	concrete_lazy_rom_get_preserves_sampler_rom_covered $P \Rightarrow Q$ is an implication used as a proof obligation.	From game
	1982	<pre> lemma </pre>		

	Line	Construct	What was written	Sym
		<pre> concrete_lazy_rom_get_preserves_sampler_rom_covered_arg lemma concrete_lazy_rom_get_preserves_sampler_rom_covered_arg hash_qs sampler_qs : hoare[HAETAE_RO.FRO.get : sampler_rom_covered HAETAE_RO.FRO.m hash_qs sampler_qs /\ (forall seed_coins, arg = sampler_expand_query seed_coins => arg \in hash_qs /\ arg \in sampler_qs) ==> sampler_rom_covered HAETAE_RO.FRO.m hash_qs sampler_qs]. </pre>	concrete_lazy_rom_get_preserves_sampler_rom_covered_arg $P \Rightarrow Q$ is an implication used as a proof obligation.	Prove game
	1999	<pre> lemma concrete_lazy_rom_get_preserves_sampler_rom_covered_budgeted_logs : hoare[HAETAE_RO.FRO.get : sampler_rom_covered HAETAE_RO.FRO.m ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries /\ (forall seed_coins, arg = sampler_expand_query seed_coins => arg \in ROMInternalTransc... </pre>	concrete_lazy_rom_get_preserves_sampler_rom_covered_budgeted_logs $P \Rightarrow Q$ is an implication used as a proof obligation.	Prove game
	2024	<pre> lemma concrete_lazy_rom_get_preserves_sampler_rom_covered_hash_cons q hash_qs sampler_qs : hoare[HAETAE_RO.FRO.get : arg = q /\ sampler_rom_covered HAETAE_RO.FRO.m hash_qs sampler_qs ==> sampler_rom_covered HAETAE_RO.FRO.m (q :: hash_qs) sampler_qs]. </pre>	concrete_lazy_rom_get_preserves_sampler_rom_covered_hash_cons $P \Rightarrow Q$ is an implication used as a proof obligation.	Prove game
	2039	<pre> lemma </pre>	lemma	

	Line	Construct	What was written	Sym
concrete_lazy_rom_get_preserves_sampler_rom_covered_sampler_cons		lemma concrete_lazy_rom_get_preserves_sampler_rom_covered_sampler_cons q hash_qs sampler_qs : hoare[HAETAE_ROM.FROM.get : arg = q /\ sampler_rom_covered HAETAE_ROM.FROM.hash_qs sampler_qs ==> sampler_rom_covered HAETAE_ROM.FROM.hash_qs (q :: sampler_qs)].	concrete_lazy_rom_get_preserves_sampler_rom_covered_sampler_cons is an implication used as a proof obligation.	Prov game
sampler_bad_prequery_one_step_bound_nonnegative	2054	lemma sampler_bad_prequery_one_step_bound_nonnegative : 0%r <= rom_hash_query_budget / challenge_support_cardinality_lower_bound.	sampler_bad_prequery_one_step_bound_nonnegative is an arithmetic or probability upper bound.	Prov game
budgeted_paper_sim_sampler_bad_prequery_forge_fel_bound	2062	lemma budgeted_paper_sim_sampler_bad_prequery_forge_fel_bound pk &m : Pr[ROMInternalTranscriptBudgetedPaperSimAsNMA(A, Samp, H).forge(pk) @ &m : ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <= rom_signature_query_budget * (rom_hash_query_budget + rom_signature_query_budget) / challenge_support_cardinality_lower_bound).	budgeted_paper_sim_sampler_bad_prequery_forge_fel_bound is a probability bound or exact game probability.	Prov game
hoare_step_for_budgeted_paper_sim_sampler_bad_prequery_forge_fel_bound	2155	hoare proof step hoare.	Within budgeted_paper_sim_sampler_bad_prequery_forge_fel_bound the proof reduces the probabilistic goal to a Hoare triple $\{P\} c \{Q\}$.	Intro for
hoare_step_for_budgeted_paper_sim_sampler_bad_prequery_forge_fel_bound	2166	hoare proof step hoare.	Within budgeted_paper_sim_sampler_bad_prequery_forge_fel_bound the proof reduces the probabilistic goal to a Hoare triple $\{P\} c \{Q\}$.	Intro for
	2202	lemma		

Line	Construct	What was written	Sym
	budgeted_paper_sim_sampler_bad_prequery_forge_fel_phoare	lemma budgeted_paper_sim_sampler_bad_prequery_forge_fel_phoare : phoare[ROMInternalTranscriptBudgetedPaperSimAsNMA(A, Samp, H).forge : true ==> ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <= (rom_signature_query_budget * (rom_hash_query_budget + rom_signature_query_budget) / challenge_support_cardinality_lower_bound)).	budgeted_paper_sim_sampler_bad_prequery_Prom $X \leq Y$ is an arithmetic or probability game upper bound.
2214	lemma		
	budgeted_paper_sim_sampler_bad_prequery_nma_fel_bound	lemma budgeted_paper_sim_sampler_bad_prequery_nma_fel_bound : &m : Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptBudgetedPaperSimAsNMA(A, Samp)).main() @ &m : ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery] <= rom_signature_query_budget * (rom_hash_query_budget + rom_signature_query_budget) / challenge_support_cardinality_lower_bound).	budgeted_paper_sim_sampler_bad_prequery_Prom $\Pr[c \leq B]$ is a probability bound or exact game probability.
2239	lemma		
	concrete_lazy_rom_sampler_expand_query_fresh_phoare	lemma concrete_lazy_rom_sampler_expand_query_fresh_phoare : seed_coins (p : random_coins -> bool) : phoare[HAETAE_RO.FRO.get : arg = sampler_expand_query seed_coins /\ sampler_expand_query seed_coins \notin HAETAE_RO.FRO.m ==> p (ro_signing_coins res)] <= (mu drandom_coins p).	concrete_lazy_rom_sampler_expand_query_Prom $X \leq Y$ is an arithmetic or probability game upper bound.
2252	lemma		

	Line	Construct	What was written	Sym
		<pre> concrete_lazy_rom_sampler_expand_query_fresh_arg_phoare lemma concrete_lazy_rom_sampler_expand_query_fresh_arg_phoare (p : random_coins -> bool) : phoare[HAETAE_RO.FRO.get : (exists seed_coins, arg = sampler_expand_query seed_coins /\ sampler_expand_query seed_coins \notin HAETAE_RO.FRO.m) ==> p (ro_signing_coins res)] <= (mu drandom_coins p). </pre>	concrete_lazy_rom_sampler_expand_query_fresh_arg_phoare $X \leq Y$ is an arithmetic or probability upper bound.	Fresh game
	2266	<pre> lemma concrete_lazy_rom_sampler_expand_query_fresh_arg_clean_phoare (p : random_coins -> bool) : phoare[HAETAE_RO.FRO.get : (exists seed_coins, arg = sampler_expand_query seed_coins /\ sampler_expand_query seed_coins \notin HAETAE_RO.FRO.m) /\ ! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery ==> p (ro_signing_coins res) /\ ! ROMInternalTr... </pre>	concrete_lazy_rom_sampler_expand_query_fresh_arg_clean_phoare $P \Leftarrow Q$ is an implication used as a proof obligation.	Fresh game
	2285	<pre> lemma concrete_ro_signing_attempt_sample_with_seed_fresh_structural_le lemma concrete_ro_signing_attempt_sample_with_seed_fresh_structural_le seed_coins sk m ctx (p : paper_sim_signature_sample -> bool) : phoare[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed : arg = (seed_coins, m, ctx) /\ ROSigningAttemptPaperSimSampler.sk_current = sk /\ sampler_expand_query seed_coins \notin HAETAE_RO.FRO.m ==> p res] <=... </pre>	concrete_ro_signing_attempt_sample_with_seed_fresh_structural_le $X \leq Y$ is an arithmetic or probability upper bound.	Beed game
	2307	<pre> lemma </pre>	lemma	

	Line	Construct	What was written	Sym
concrete_ro_signing_attempt_sample_with_seed_fresh_structural_arg_le		lemma concrete_ro_signing_attempt_sample_with_seed_fresh_structural_arg_le sk m ctx (p : paper_sim_signature_sample -> bool) : phoare[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed : arg.'2 = m /\ arg.'3 = ctx /\ ROSigningAttemptPaperSimSampler.sk_current = sk /\ sampler_expand_query arg.'1 \notin HAETAE_RO.FRO.m => p res] <= (mu (dsigni...	concrete_ro_signing_attempt_sample_with_seed_fresh_structural_arg_le $X \leq Y$ is an arithmetic on probability upper bound.	Reed game
concrete_ro_signing_attempt_sample_with_seed_fresh_structural_arg_clean_le	2330	lemma concrete_ro_signing_attempt_sample_with_seed_fresh_structural_arg_clean_le sk m ctx (p : paper_sim_signature_sample -> bool) : phoare[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed : arg.'2 = m /\ arg.'3 = ctx /\ ROSigningAttemptPaperSimSampler.sk_current = sk /\ ! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery /\ s...	concrete_ro_signing_attempt_sample_with_seed_fresh_structural_arg_clean_le $L \leq R$ is an equality/rewriting fact.	Reed game
concrete_lazy_rom_sampler_expand_query_guarded_clean_phoare	2355	lemma concrete_lazy_rom_sampler_expand_query_guarded_clean_phoare (p : random_coins -> bool) : phoare[HAETAE_RO.FRO.get : (! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery => exists seed_coins, arg = sampler_expand_query seed_coins /\ sampler_expand_query seed_coins \notin HAETAE_RO.FRO.m) ==> p (ro_signing_coins res) /\ ! ROMInternalTran...	concrete_lazy_rom_sampler_expand_query_guarded_clean_phoare $P \Rightarrow Q$ is an implication used as a proof obligation.	Guar game
	2382	lemma		

	Line	Construct	What was written	Sym
concrete_ro_signing_attempt_sample_with_seed_guarded_clean_structural_arg_le		lemma concrete_ro_signing_attempt_sample_with_seed_guarded_clean_structural_arg_le sk m ctx (p : paper_sim_signature_sample -> bool) : phoare[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed : arg.'2 = m /\ arg.'3 = ctx /\ ROSigningAttemptPaperSimSampler.sk_current = sk /\ (! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery =... 2407 lemma	concrete_ro_signing_attempt_sample_with_seed_guarded_clean_structural_arg_le <i>P</i> is an equality/re-writing fact. concrete_ro_signing_attempt_sample_with_seed_fresh_structural_eq	Seed game
concrete_ro_signing_attempt_sample_with_seed_fresh_structural_eq		lemma concrete_ro_signing_attempt_sample_with_seed_fresh_structural_eq seed_coins sk m ctx (p : paper_sim_signature_sample -> bool) : phoare[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed : arg = (seed_coins, m, ctx) /\ ROSigningAttemptPaperSimSampler.sk_current = sk /\ sampler_expand_query seed_coins \notin HAETAE_RO.FRO.m ==> p res] = (... 2429 lemma	concrete_ro_signing_attempt_sample_with_seed_fresh_structural_eq <i>P</i> => <i>Q</i> is an implication used as a proof obligation.	Seed game
concrete_ro_exact_hyperball_sample_with_seed_exact		lemma concrete_ro_exact_hyperball_sample_with_seed_exact seed_coins sk m ctx (p : paper_sim_signature_sample -> bool) : phoare[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed : arg = (seed_coins, m, ctx) /\ ROExactHyperballPaperSimSampler.sk_current = sk ==> p res] = (mu (dexact_hyperball_signing_attempt_state haetae_mode sk m ctx) (fun st... 2448 lemma	concrete_ro_exact_hyperball_sample_with_seed_exact <i>P</i> => <i>Q</i> is an implication used as a proof obligation.	Seed game

Line	Construct	What was written	Sym
concrete_ro_exact_hyperball_sample_with_seed_exact_no_seed	lemma	concrete_ro_exact_hyperball_sample_with_seed_exact_no_seed is an implication used as a proof obligation.	Seed
2468	lemma	concrete_ro_exact_hyperball_sample_with_seed_exact_no_seed is a probability bound or exact game probability.	Seed
concrete_ro_exact_hyperball_sample_with_seed_exact_pr	lemma	concrete_ro_exact_hyperball_sample_with_seed_exact_pr is a probability bound or exact game probability.	Seed
2480	lemma	concrete_ro_signing_attempt_sample_with_seed_preserves_sampler_rom_covered is an implication used as a proof obligation.	Seed
concrete_ro_signing_attempt_sample_with_seed_preserves_sampler_rom_covered	lemma	concrete_ro_signing_attempt_sample_with_seed_preserves_sampler_rom_covered is an implication used as a proof obligation.	Seed
2494	lemma		

	Line	Construct	What was written	Sym
concrete_ro_exact_hyperball_sample_with_seed_preserves_sampler_rom_covered		lemma concrete_ro_exact_hyperball_sample_with_seed_preserves_sampler_rom_covered hash_qs sampler_qs : hoare[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed : sampler_rom_covered HAETAE_RO.FRO.m hash_qs sampler_qs /\n sampler_expand_query arg.'1 \nin sampler_qs ==> sampler_rom_covered HAETAE_RO.FRO.m hash_qs sampler_qs].	concrete_ro_exact_hyperball_sample_with_seed_preserves_sampler_rom_covered $P \Rightarrow Q$ is an implication used as a proof obligation.	Proof game
concrete_ro_signing_attempt_sample_with_seed_preserves_sampler_rom_covered_budgeted_logs	2510	lemma concrete_ro_signing_attempt_sample_with_seed_preserves_sampler_rom_covered_budgeted_logs : hoare[ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed : sampler_rom_covered HAETAE_RO.FRO.m ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries /\ sampler_expand_qu...	concrete_ro_signing_attempt_sample_with_seed_preserves_sampler_rom_covered_budgeted_logs $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact	Proof game
concrete_ro_exact_hyperball_sample_with_seed_preserves_sampler_rom_covered_budgeted_logs	2529	lemma concrete_ro_exact_hyperball_sample_with_seed_preserves_sampler_rom_covered_budgeted_logs : hoare[ROExactHyperballPaperSimSampler(HAETAE_RO.FRO).sample_with_seed : sampler_rom_covered HAETAE_RO.FRO.m ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries /\ sampler_expand_qu...	concrete_ro_exact_hyperball_sample_with_seed_preserves_sampler_rom_covered_budgeted_logs $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact	Proof game
	2550	lemma		

	Line	Construct	What was written	Sym
concrete_budgeted_ro_signing_attempt_o_sign_preserves_sampler_rom_covered		lemma	concrete_budgeted_ro_signing_attempt_o_sign_preserves_sampler_rom_covered	Prove game
			$\forall \vec{z}. P(\vec{z})$ is a universally quantified fact.	
		:		
		hoare[ROMInternalTranscriptBudgetedPaperSimAsNMA		
		(A,		
		ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO),		
		HAETAE_RO.FRO).O.sign :		
		sampler_rom_covered		
		HAETAE_RO.FRO.m		
		ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries		
		ROMInternalTranscriptBudgetedPaperSimAsNMA.sampl...		
	2577	lemma	concrete_budgeted_ro_exact_hyperball_o_sign_preserves_sampler_rom_covered	Prove game
concrete_budgeted_ro_exact_hyperball_o_sign_preserves_sampler_rom_covered		lemma	concrete_budgeted_ro_exact_hyperball_o_sign_preserves_sampler_rom_covered	Prove game
			$\forall \vec{z}. P(\vec{z})$ is a universally quantified fact.	
		:		
		hoare[ROMInternalTranscriptBudgetedPaperSimAsNMA		
		(A,		
		ROExactHyperballPaperSimSampler(HAETAE_RO.FRO),		
		HAETAE_RO.FRO).O.sign :		
		sampler_rom_covered		
		HAETAE_RO.FRO.m		
		ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries		
		ROMInternalTranscriptBudgetedPaperSimAsNMA.sampl...		
	2604	lemma	concrete_budgeted_ro_signing_attempt_o_sign_clean_sampler_fresh	Prove game
concrete_budgeted_ro_signing_attempt_o_sign_clean_sampler_fresh		lemma	concrete_budgeted_ro_signing_attempt_o_sign_clean_sampler_fresh	Prove game
			$\forall \vec{z}. P(\vec{z})$ is a universally quantified fact.	
		:		
		hoare[ROMInternalTranscriptBudgetedPaperSimAsNMA		
		(A,		
		ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO),		
		HAETAE_RO.FRO).O.sign :		
		sampler_rom_covered		
		HAETAE_RO.FRO.m		
		ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries		
		ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand...		
	2986	lemma		

	Line	Construct	What was written	Sym
internal_transcript_sign_oracle_preserves_state		lemma internal_transcript_sign_oracle_preserves_state : hoare[EUFCMA_InternalTranscriptSign(H, A).0.sign : internal_transcript_state_sound haetae_mode EUFCMA_InternalTranscriptSign.pk_current EUFCMA_InternalTranscriptSign.sk_current EUFCMA_InternalTranscriptSign.queries EUFCMA_InternalTranscriptSign.transcripts EUFCMA_InternalTranscriptSign.records...]	internal_transcript_sign_oracle_preserves_state $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Prov game
	3016	lemma		
adversary_internal_transcript_state_preserved		lemma adversary_internal_transcript_state_preserved : hoare[A(H, EUFCMA_InternalTranscriptSign(H, A).0).forge : internal_transcript_state_sound haetae_mode EUFCMA_InternalTranscriptSign.pk_current EUFCMA_InternalTranscriptSign.sk_current EUFCMA_InternalTranscriptSign.queries EUFCMA_InternalTranscriptSign.transcripts EUFCMA_InternalTranscriptSign.rec...]	adversary_internal_transcript_state_preserved $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Prov oracle proce
	3044	lemma		
internal_transcript_sign_main_state_sound		lemma internal_transcript_sign_main_state_sound : hoare[EUFCMA_InternalTranscriptSign(H, A).main : true ==> internal_transcript_state_sound haetae_mode EUFCMA_InternalTranscriptSign.pk_current EUFCMA_InternalTranscriptSign.sk_current EUFCMA_InternalTranscriptSign.queries EUFCMA_InternalTranscriptSign.transcripts EUFCMA_InternalTranscriptSign.records].	internal_transcript_sign_main_state_sound $B \Rightarrow Q$ is an implication used as a proof obligation.	Prov oracle proce
	3076	lemma		

Line	Construct	What was written	Sym
	<pre> internal_transcript_sign_main_transcripts_valid lemma internal_transcript_sign_main_transcripts_valid : hoare[EUF_CMA_InternalTranscriptSign(H, A).main : true ==> transcript_log_valid haetae_mode EUF_CMA_InternalTranscriptSign.transcripts]. </pre>	<pre> internal_transcript_sign_main_transcripts_valid $P \Rightarrow Q$ is an implication used as a proof obligation. </pre>	<pre> internal_transcript_sign_main_transcripts_valid </pre>
3118	<pre> internal_transcript_sign_main_log_ready lemma internal_transcript_sign_main_log_ready : hoare[EUF_CMA_InternalTranscriptSign(H, A).main : true ==> transcript_log_ready haetae_mode EUF_CMA_InternalTranscriptSign.transcripts]. </pre>	<pre> internal_transcript_sign_main_log_ready : $P \Rightarrow Q$ is an implication used as a proof obligation. </pre>	<pre> internal_transcript_sign_main_log_ready : </pre>
3160	<pre> internal_transcript_sign_main_no_min_entropy lemma internal_transcript_sign_main_no_min_entropy : hoare[EUF_CMA_InternalTranscriptSign(H, A).main : true ==> transcript_log_min_entropy_clear haetae_mode EUF_CMA_InternalTranscriptSign.transcripts]. </pre>	<pre> internal_transcript_sign_main_no_min_entropy $P \Rightarrow Q$ is an implication used as a proof obligation. </pre>	<pre> internal_transcript_sign_main_no_min_entropy </pre>
3202	<pre> rom_internal_transcript_sign_oracle_preserves_state lemma rom_internal_transcript_sign_oracle_preserves_state : hoare[EUF_CMA_ROMInternalTranscriptSign(H, A).O.sign : internal_transcript_state_sound haetae_mode EUF_CMA_ROMInternalTranscriptSign.pk_current EUF_CMA_ROMInternalTranscriptSign.sk_current EUF_CMA_ROMInternalTranscriptSign.queries EUF_CMA_ROMInternalTranscriptSign.transcripts EUF_CMA_ROMInternalT... </pre>	<pre> rom_internal_transcript_sign_oracle_preserves_state $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact. </pre>	<pre> rom_internal_transcript_sign_oracle_preserves_state </pre>
3238	<pre> lemma </pre>		

Line	Construct	What was written	Sym
	adversary_rom_internal_transcript_state_preserved	lemma adversary_rom_internal_transcript_state_preserved is a universally quantified fact. : hoare[A(H, EUF_CMA_ROMInternalTranscriptSign(H, A).0).forge : internal_transcript_state_sound haetae_mode EUF_CMA_ROMInternalTranscriptSign.pk_current EUF_CMA_ROMInternalTranscriptSign.sk_current EUF_CMA_ROMInternalTranscriptSign.queries EUF_CMA_ROMInternalTranscriptSign.transcripts EUF_CMA_ROMInte...	Reco orack proce
3266	rom_internal_transcript_sign_main_state_sound	lemma rom_internal_transcript_sign_main_state_sound is an implication used as a proof obligation. : hoare[EUF_CMA_ROMInternalTranscriptSign(H, A).main : true ==> internal_transcript_state_sound haetae_mode EUF_CMA_ROMInternalTranscriptSign.pk_current EUF_CMA_ROMInternalTranscriptSign.sk_current EUF_CMA_ROMInternalTranscriptSign.queries EUF_CMA_ROMInternalTranscriptSign.transcripts EUF_CMA_ROMInternal...	Reco orack proce
3300	rom_internal_transcript_sign_main_transcripts_valid	lemma rom_internal_transcript_sign_main_transcripts_valid is an implication used as a proof obligation. : hoare[EUF_CMA_ROMInternalTranscriptSign(H, A).main : true ==> transcript_log_valid haetae_mode EUF_CMA_ROMInternalTranscriptSign.transcripts].	Reco justif
3344	rom_internal_transcript_sign_main_log_ready	lemma rom_internal_transcript_sign_main_log_ready is an implication used as a proof obligation. : hoare[EUF_CMA_ROMInternalTranscriptSign(H, A).main : true ==> transcript_log_ready haetae_mode EUF_CMA_ROMInternalTranscriptSign.transcripts].	Reco scrip

	Line	Construct	What was written	Sym
	3388	lemma	rom_internal_transcript_sign_main_no_min_entropy	Record
rom_internal_transcript_sign_main_no_min_entropy		lemma	rom_internal_transcript_sign_main_no_min_entropy	
		rom_internal_transcript_sign_main_no_min_entropy	$P \Rightarrow Q$ is an implication used as a proof obligation.	scrip
		:		
		hoare[EUF_CMA_ROMInternalTranscriptSign(H,		
		A).main : true ==>		
		transcript_log_min_entropy_clear		
		haetae_mode		
		EUF_CMA_ROMInternalTranscriptSign.transcripts].		
	3432	lemma	rom_internal_transcript_sign_main_no_failure_for_signature_site	Record
rom_internal_transcript_sign_main_no_failure_for_signature_site		lemma	rom_internal_transcript_sign_main_no_failure_for_signature_site	
		rom_internal_transcript_sign_main_no_failure_for_signature_site	$P \Rightarrow Q$ is an implication used as a proof obligation.	scrip
		:		
		hoare[EUF_CMA_ROMInternalTranscriptSign(H,		
		A).main : true ==> forall tr y,		
		tr \in		
		EUF_CMA_ROMInternalTranscriptSign.transcripts		
		=>		
		transcript_signature_challenge_matches		
		tr y => !		
		fs_with_aborts_failure		
		haetae_mode tr		
		(signature_programming_site_of_transcript		
		haetae_mode tr y)].		
	3485	lemma	rom_internal_transcript_sign_main_signature_sites_clear	Provi
rom_internal_transcript_sign_main_signature_sites_clear		lemma	rom_internal_transcript_sign_main_signature_sites_clear	
		rom_internal_transcript_sign_main_signature_sites_clear	$P \Rightarrow Q$ is an implication used as a proof obligation.	game
		:		
		hoare[EUF_CMA_ROMInternalTranscriptSign(H,		
		A).main : true ==>		
		transcript_log_signature_programming_sites_clear		
		haetae_mode		
		EUF_CMA_ROMInternalTranscriptSign.transcripts].		
	3531	lemma		

	Line	Construct	What was written	Sym
		rom_internal_transcript_bad_forgery_zero	lemma rom_internal_transcript_bad_forgery_zero : &m : Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m : res /\ ! transcript_log_signature_programming_sites_clear haetae_mode EUF_CMA_ROMInternalTranscriptSign.transcripts] = 0%r.	rom_internal_transcript_bad_forgery_zero : Rec Pr[c : E] = B is a probability bound or exact game probability.
	3539	hoare_step_for_rom_internal_transcript_bad_forgery_zero	hoare.	Within rom_internal_transcript_bad_forgery_zero, for th the proof reduces the probabilistic goal to a Hoare triple {P} c {Q}.
	3548	rom_internal_transcript_bad_forgery_bound	lemma rom_internal_transcript_bad_forgery_bound : &m : Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m : res /\ ! transcript_log_signature_programming_sites_clear haetae_mode EUF_CMA_ROMInternalTranscriptSign.transcripts] <= fs_with_aborts_reprogramming_term + fs_with_aborts_min_entropy_term.	rom_internal_transcript_bad_forgery_boundProv Pr[c : E] ≤ B is a probability bound or exact game probability.
	3564	rom_internal_transcript_bad_forgery_counted_bound	lemma rom_internal_transcript_bad_forgery_counted_bound : &m : Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m : res /\ ! transcript_log_signature_programming_sites_clear haetae_mode EUF_CMA_ROMInternalTranscriptSign.transcripts] <= counted_rom_programming_loss_term.	rom_internal_transcript_bad_forgery_counted_bro Pr[c : E] ≤ B is a probability bound or exact game probability.
	3583		lemma	

	Line	Construct	What was written	Sym
counted_rom_internal_transcript_sign_oracle_preserves_budget_state		lemma	counted_rom_internal_transcript_sign_oracle_preserves_budget_state	Prov
		counted_rom_internal_transcript_sign_oracle_preserves_budget_state	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	game
		:		
		hoare[EUFCMA_ROMInternalTranscriptCountedSign(H,		
		A).O.sign :		
		internal_transcript_state_sound		
		haetae_mode		
		EUFCMA_ROMInternalTranscriptCountedSign.pk_current		
		EUFCMA_ROMInternalTranscriptCountedSign.sk_current		
		EUFCMA_ROMInternalTranscriptCountedSign.queries		
		EUFCMA_ROMInternalTran...		
	3653	lemma		
adversary_counted_rom_internal_transcript_budget_state_preserved		lemma	adversary_counted_rom_internal_transcript_budget_state_preserved	Budg
		adversary_counted_rom_internal_transcript_budget_state_preserved	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	oracle
		:		proce
		hoare[A(EUFCMA_ROMInternalTranscriptCountedSign(H,		
		A).C,		
		EUFCMA_ROMInternalTranscriptCountedSign(H,		
		A).O).forge :		
		internal_transcript_state_sound		
		haetae_mode		
		EUFCMA_ROMInternalTranscriptCountedSign.pk_current		
		EUFCMA_ROMInternalTranscriptCountedSign.sk_current		
		EUFCMA_ROMInternalT...		
	3694	lemma		
counted_rom_internal_transcript_main_bad_clear		lemma	counted_rom_internal_transcript_main_bad_clear	Prlea
		counted_rom_internal_transcript_main_bad_clear	$P \rightarrow Q$ is an implication used as a proof	game
		:	obligation.	
		hoare[EUFCMA_ROMInternalTranscriptCountedSign(H,		
		A).main : true ==> ! res].		
	3733	lemma		
counted_rom_internal_transcript_main_bad_false		lemma	counted_rom_internal_transcript_main_bad_false	Rfals
		counted_rom_internal_transcript_main_bad_false	$P \rightarrow Q$ is an implication used as a proof	scrip
		:	obligation.	
		hoare[EUFCMA_ROMInternalTranscriptCountedSign(H,		
		A).main : true ==> res =		
		false].		
	3740	lemma		

	Line	Construct	What was written	Sym
		counted_rom_internal_transcript_counted_bad_zero	lemma counted_rom_internal_transcript_counted_bad_zero &m : Pr[EUF_CMA_ROMInternalTranscriptCountedSign(H, A).main() @ &m : res] = 0%r.	counted_rom_internal_transcript_counted_bad_zero $\Pr[c \in E]$ B is a probability bound or exact game probability.
	3745	hoare_step_for_counted_rom_internal_transcript_counted_bad_zero	hoare proof step hoare.	Within counted_rom_internal_transcript_counted_bad_zero the proof reduces the probabilistic goal to a Hoare triple $\{P\} c \{Q\}$.
	3751	counted_rom_internal_transcript_budget_sound	lemma counted_rom_internal_transcript_budget_sound &m : counted_lazy_rom_bad_flags_budget_sound (Pr[EUF_CMA_ROMInternalTranscriptCountedSign(H, A).main() @ &m : res])).	counted_rom_internal_transcript_budget_sound $\Pr[c \in E] \bowtie B$ is a probability bound or exact game probability.
	3761	counted_rom_internal_transcript_counted_bad_subunit	lemma counted_rom_internal_transcript_counted_bad_subunit &m : Pr[EUF_CMA_ROMInternalTranscriptCountedSign(H, A).main() @ &m : res] < 1%r.	counted_rom_internal_transcript_counted_bad_subunit $\Pr[c \in E] \bowtie B$ is a probability bound or exact game probability.
	3778	programmed_counted_sign_oracle_preserves_min_entropy_clear	lemma programmed_counted_sign_oracle_preserves_min_entropy_clear : hoare[EUF_CMA_ROMProgrammedTranscriptCountedSign(H, A).0.sign : EUF_CMA_ROMProgrammedTranscriptCountedSign.pk_current = public_key_of_secret haetae_mode EUF_CMA_ROMProgrammedTranscriptCountedSign.sk_current /\ ! EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_min_entropy ==> EUF_CMA_ROMPr...	programmed_counted_sign_oracle_preserves_min_entropy_clear $P \Rightarrow Q$ is an implication used as a proof obligation.
	3818		lemma	

Line	Construct	What was written	Sym
	adversary_programmed_counted_min_entropy_preserved	lemma adversary_programmed_counted_min_entropy_preserved : hoare[A (EUF_CMA_ROMProgrammedTranscriptCountedSign(H, A).C, EUF_CMA_ROMProgrammedTranscriptCountedSign(H, A).O).forge : EUF_CMA_ROMProgrammedTranscriptCountedSign.pk_current = public_key_of_secret haetae_mode EUF_CMA_ROMProgrammedTranscriptCountedSign.sk_current /\ ! EUF_CMA_ROMProgrammedTranscrip...	adversary_programmed_counted_min_entropy_preserved Leq, B is an inequality/rewriting fact. Req scrip
3841	lemma programmed_counted_main_min_entropy_clear	lemma programmed_counted_main_min_entropy_clear : hoare[EUF_CMA_ROMProgrammedTranscriptCountedSign(H, A).main : true ==> ! EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_min_entropy].	programmed_counted_main_min_entropy_clear B is an implication used as a proof obligation. B game
3869	lemma programmed_counted_min_entropy_bad_zero	lemma programmed_counted_min_entropy_bad_zero : &m : Pr[EUF_CMA_ROMProgrammedTranscriptCountedSign(H, A).main() @ &m : EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_min_entropy] = 0%r.	programmed_counted_min_entropy_bad_zero B[c : E] = B is a probability bound or exact game probability. B scrip
3875	hoare_step_for_programmed_counted_min_entropy_bad_zero	hoare.	Within programmed_counted_min_entropy_bad_zero for the the proof reduces the probabilistic goal to a Hoare triple {P} c {Q}.
3879	lemma programmed_counted_min_entropy_bad_bound	lemma programmed_counted_min_entropy_bad_bound : &m : Pr[EUF_CMA_ROMProgrammedTranscriptCountedSign(H, A).main() @ &m : EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_min_entropy] <= counted_rom_min_entropy_term.	programmed_counted_min_entropy_bad_bound B[c : E] ≤ B is a probability bound or exact game probability. B game

	Line	Construct	What was written	Sym
	3888	lemma		
programmed_counted_prequery_reprogramming_bad_bound_from_budget_expr		lemma	programmed_counted_prequery_reprogramming_bad_bound_from_budget_expr	Reco
		programmed_counted_prequery_reprogramming_bad_bound_from_budget_expr	$\Pr[E] \leq B$ is a probability bound or exact game probability.	game
		&m :		
		Pr[EUF_CMA_ROMProgrammedTranscriptCountedSign(H,		
		A).main() @ &m :		
		EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_prequery		
		\/		
		EUF_CMA_ROMProgrammedTranscriptCountedSign.C.bad_reprogram]		
		<= ((rom_total_query_budget *		
		rom_total_query_budget) +		
		(rom_signature_query_budget *		
		(rom...		
	3915	lemma		
budgeted_counted_lazy_rom_get_true		lemma	budgeted_counted_lazy_rom_get_true :	Reco
		budgeted_counted_lazy_rom_get_true	$P \Rightarrow Q$ is an implication used as a proof obligation.	scrip
		:		
		hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H,		
		A).C.get : true ==> true].		
	3925	lemma		
budgeted_counted_lazy_rom_init_true		lemma	budgeted_counted_lazy_rom_init_true :	Reco
		budgeted_counted_lazy_rom_init_true	$P \Rightarrow Q$ is an implication used as a proof obligation.	scrip
		:		
		hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H,		
		A).C.init : true ==> true].		
	3935	lemma		
budgeted_counted_lazy_rom_bad_true		lemma	budgeted_counted_lazy_rom_bad_true :	Reco
		budgeted_counted_lazy_rom_bad_true	$P \Rightarrow Q$ is an implication used as a proof obligation.	scrip
		:		
		hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H,		
		A).C.bad : true ==> true].		
	3942	lemma		
budgeted_programmed_hash_get_preserves_challenge_query_discipline		lemma	budgeted_programmed_hash_get_preserves_challenge_query_discipline	Chall
		budgeted_programmed_hash_get_preserves_challenge_query_discipline	$P \Rightarrow Q$ is an implication used as a proof obligation.	game
		:		
		hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H,		
		A).AH.get :		
		budgeted_programmed_challenge_query_discipline		
		EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count		
		EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries		
		==> budgeted_programmed_ch...		

	Line	Construct	What was written	Sym
	3963	lemma		
budgeted_programmed_sign_oracle_preserves_challenge_query_discipline		<pre> lemma budgeted_programmed_sign_oracle_preserves_challenge_query_discipline : hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).O.sign : budgeted_programmed_challenge_query_discipline EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries ==> budgeted_programmed...</pre>	budgeted_programmed_sign_oracle_preserves_challenge_query_discipline is an implication used as a proof obligation.	Proof game
	3989	lemma		
adversary_budgeted_programmed_challenge_query_discipline_preserved		<pre> lemma adversary_budgeted_programmed_challenge_query_discipline_preserved : hoare[A(EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).AH, EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).O).forge : budgeted_programmed_challenge_query_discipline EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count EUF_CMA_ROMProgrammedTran...</pre>	adversary_budgeted_programmed_challenge_query_discipline is a universally quantified fact.	Quant game
	4011	lemma		
budgeted_programmed_main_challenge_query_discipline		<pre> lemma budgeted_programmed_main_challenge_query_discipline : hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main : true ==> budgeted_programmed_challenge_query_discipline EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries].</pre>	budgeted_programmed_main_challenge_query_discipline is an implication used as a proof obligation.	Proof game
	4042	lemma		
budgeted_programmed_main_challenge_query_count_bound		<pre> lemma budgeted_programmed_main_challenge_query_count_bound : hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main : true ==> challenge_query_log_count EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.hash_queries <= hash_query_budget_count + 1].</pre>	budgeted_programmed_main_challenge_query_count_bound is an arithmetic or probability upper bound.	Proof game
	4054	lemma		

	Line	Construct	What was written	Sym
		budgeted_programmed_adaptive_prequery_lift lemma budgeted_programmed_adaptive_prequery_lift md sk m ctx qs hc : budgeted_programmed_challenge_query_discipline hc qs => mu drandom_coins (fun coins => programming_site_prequeried qs (honest_signing_programming_site md sk m ctx coins)) <= (rom_hash_query_budget + 1%r) / challenge_support_cardinality_lower_bound.	budgeted_programmed_adaptive_prequery_lift $X \leq Y$ is an arithmetic or probability upper bound.	Record script
	4070	lemma budgeted_programmed_adaptive_prequery_lift_checked_31 md sk m ctx qs hc : budgeted_programmed_challenge_query_discipline hc qs => mu drandom_coins (fun coins => programming_site_prequeried qs (honest_signing_programming_site md sk m ctx coins)) <= 31%r / 288230376151711744%r.	budgeted_programmed_adaptive_prequery_lift $X \leq Y$ is an arithmetic or probability upper bound.	Record script
	4089	lemma budgeted_programmed_hash_get_preserves_discipline : hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign (H, A).AH.get : budgeted_programmed_query_count_discipline EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\br/> budgeted_programmed_reprogram_discipline EUF...	budgeted_programmed_hash_get_preserves_discipline $\forall \vec{r}. P(\vec{r})$ is a universally quantified fact.	Disci orack proce
	4119	lemma		

	Line	Construct	What was written	Sym
budgeted_programmed_reprogram_discipline_after_actual_signature		lemma budgeted_programmed_reprogram_discipline_after_actual_signature pk sk m ctx coins sites bad_reprogram : budgeted_programmed_reprogram_discipline pk sk sites bad_reprogram => pk = public_key_of_secret haetae_mode sk => budgeted_programmed_reprogram_discipline pk sk (actual_signature_programming_site haetae_mode (transcript_of_signature haetae_mode pk...	budgeted_programmed_reprogram_discipline $P \Rightarrow Q$ is an implication used as a proof obligation.	After oracle process
budgeted_programmed_sign_oracle_preserves_discipline	4157	lemma budgeted_programmed_sign_oracle_preserves_discipline : hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).O.sign : budgeted_programmed_query_count_discipline EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.signing_count /\n budgeted_programmed_reprogram_discipline...	budgeted_programmed_sign_oracle_preserves_discipline $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Preserved game
adversary_budgeted_programmed_discipline_preserved	4223	lemma adversary_budgeted_programmed_discipline_preserved : hoare[A(EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).AH, EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).O).forge : budgeted_programmed_query_count_discipline EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count EUF_CMA_ROMProgrammedTranscriptBudgetedCounte...	adversary_budgeted_programmed_discipline_preserved $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Preserved oracle process
	4260	lemma		

	Line	Construct	What was written	Sym
		budgeted_programmed_main_reprogram_clear	lemma budgeted_programmed_main_reprogram_clear : hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main : true ==> ! EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram].	Prove game
	4310	budgeted_programmed_bad_reprogram_zero	lemma budgeted_programmed_bad_reprogram_zero &m : Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &m : EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram] = 0%r.	Reco scrip
	4317	hoare_step_for_budgeted_programmed_bad_reprogram_zero	hoare proof step hoare.	Intro for th
	4321	budgeted_programmed_prequery_reprogram_bad_bound_from_prequery	lemma budgeted_programmed_prequery_reprogram_bad_bound_from_prequery &m : Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &m : EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery] <= ((rom_total_query_budget * rom_total_query_budget) + (rom_signature_query_budget * (rom_hash_query_budget + 1%r))) / challenge_support_card...	bad game
	4345		lemma	

Line	Construct	What was written	Sym
	budgeted_programmed_hash_get_preserves_query_budget lemma budgeted_programmed_hash_get_preserves_query_budget : hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).AH.get : 0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count /\n EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count <= hash_query_budget_count /\ 0 <= EUF_CMA_ROMProgrammedTranscriptBudgete...	budgeted_programmed_hash_get_preserves_query_budget $X \leq Y$ is an arithmetic or probability upper bound.	Query oracle process
4368	lemma budgeted_programmed_sign_oracle_preserves_query_budget lemma budgeted_programmed_sign_oracle_preserves_query_budget : hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).O.sign : 0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count /\n EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count <= hash_query_budget_count /\ 0 <= EUF_CMA_ROMProgrammedTranscriptBudg...	budgeted_programmed_sign_oracle_preserves_query_budget $X \leq Y$ is an arithmetic or probability upper bound.	Prov game
4399	lemma adversary_budgeted_programmed_query_budget_preserved lemma adversary_budgeted_programmed_query_budget_preserved : hoare[A(EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).AH, EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).O).forge : 0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count /\n EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count <= has...	adversary_budgeted_programmed_query_budget_preserved $X \leq Y$ is an arithmetic or probability upper bound.	Budget script
4430	lemma	lemma	

	Line	Construct	What was written	Sym
		budgeted_programmed_main_query_budget_preserved	lemma budgeted_programmed_main_query_budget_preserved is an arithmetic or probability : upper bound. hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main : true ==> 0 <= EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count /\n EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count <= hash_query_budget_count /\ 0 <= EUF_CMA_ROMProgrammedTranscriptBudg...	Proba scrip
	4467	lemma budgeted_programmed_query_budget_certifies_branch_bound	lemma budgeted_programmed_query_budget_certifies_branch_bound is a probability bound or &m : exact game probability. Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &m : EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery \\/ EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram] <= ((rom_total_query_budget * rom_total_query_budget) + (rom_signature_query_bu...	Prob game
	4483	lemma budgeted_programmed_query_budget_certifies_branch_bound_from_prequery	lemma budgeted_programmed_query_budget_certifies_branch_bound_from_prequery is a probability bound or &m : exact game probability. Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &m : EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery] <= ((rom_total_query_budget * rom_total_query_budget) + (rom_signature_query_budget * (rom_hash_query_budget + 1%r))) / challenge_suppo...	Prob game
	4500	lemma	lemma	

	Line	Construct	What was written	Sym
		<pre> budgeted_programmed_query_budget_certifies_checked_31_bound lemma budgeted_programmed_query_budget_certifies_checked_31_bound &m : Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &m : EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery] <= ((rom_total_query_budget * rom_total_query_budget) + (rom_signature_query_budget * (rom_hash_query_budget + 1%r))) / challenge_support_cardina...</pre>	$\text{Pr}\{E\} \leq B$ is a probability bound or exact game probability.	Prov
	4529	<pre> lemma budgeted_sampled_direct_counted_lazy_rom_get_true lemma budgeted_sampled_direct_counted_lazy_rom_get_true : hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign (H, A, DirectSigningCoinSampler).C.get : true ==> true].</pre>	$P \Rightarrow Q$ is an implication used as a proof obligation.	Prov
	4540	<pre> lemma budgeted_sampled_direct_counted_lazy_rom_init_true lemma budgeted_sampled_direct_counted_lazy_rom_init_true : hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign (H, A, DirectSigningCoinSampler).C.init : true ==> true].</pre>	$P \Rightarrow Q$ is an implication used as a proof obligation.	Prov
	4551	<pre> lemma budgeted_sampled_direct_counted_lazy_rom_bad_true lemma budgeted_sampled_direct_counted_lazy_rom_bad_true : hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign (H, A, DirectSigningCoinSampler).C.bad : true ==> true].</pre>	$P \Rightarrow Q$ is an implication used as a proof obligation.	Prov
	4559	<pre> lemma</pre>		

	Line	Construct	What was written	Sym
budgeted_sampled_direct_hash_get_preserves_challenge_query_discipline		lemma	budgeted_sampled_direct_hash_get_preserves_challenge_query_discipline	Pres
		budgeted_sampled_direct_hash_get_preserves_challenge_query_discipline	$\forall \vec{r}. P(\vec{r})$ is a universally quantified fact.	game
		:		
		hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign		
		(H, A,		
		DirectSigningCoinSampler).AH.get		
		:		
		budgeted_programmed_challenge_query_discipline		
		EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count		
		EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_que...		
	4582	lemma	budgeted_sampled_direct_sign_oracle_preserves_challenge_query_discipline	Pres
budgeted_sampled_direct_sign_oracle_preserves_challenge_query_discipline		lemma	budgeted_sampled_direct_sign_oracle_preserves_challenge_query_discipline	Pres
		budgeted_sampled_direct_sign_oracle_preserves_challenge_query_discipline	$\forall \vec{r}. P(\vec{r})$ is a universally quantified fact.	game
		:		
		hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign		
		(H, A,		
		DirectSigningCoinSampler).0.sign		
		:		
		budgeted_programmed_challenge_query_discipline		
		EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count		
		EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash...		
	4612	lemma	adversary_budgeted_sampled_direct_challenge_query_discipline_preserved	Pres
adversary_budgeted_sampled_direct_challenge_query_discipline_preserved		lemma	adversary_budgeted_sampled_direct_challenge_query_discipline_preserved	Pres
		adversary_budgeted_sampled_direct_challenge_query_discipline_preserved	$\forall \vec{r}. P(\vec{r})$ is a universally quantified fact.	game
		: hoare[		
		A(EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign		
		(H, A,		
		DirectSigningCoinSampler).AH,		
		EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign		
		(H, A,		
		DirectSigningCoinSampler).0).forge		
		:		
		budgeted_programmed_challenge_query_discipline		
		EUF_CMA_ROMProgrammedTranscriptBudgeted...		
	4636	lemma		

	Line	Construct	What was written	Sym
		<pre> lemma budgeted_sampled_direct_main_challenge_query_discipline : hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign (H, A, DirectSigningCoinSampler).main : true ==> budgeted_programmed_challenge_query_discipline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.hash_queries]. </pre>	<code>budgeted_sampled_direct_main_challenge_query_discipline</code> is an implication used as a proof obligation.	Query game
	4668	<pre> lemma budgeted_sampled_direct_hash_get_preserves_discipline : hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign (H, A, DirectSigningCoinSampler).AH.get : budgeted_programmed_query_count_discipline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count /\ budgeted_prog... </pre>	<code>budgeted_sampled_direct_hash_get_preserves_discipline</code> is a universally quantified fact.	Proof game
	4699	<pre> lemma budgeted_sampled_direct_sign_oracle_preserves_discipline : hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign (H, A, DirectSigningCoinSampler).O.sign : budgeted_programmed_query_count_discipline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.signing_count /\ budgeted_p... </pre>	<code>budgeted_sampled_direct_sign_oracle_preserves_discipline</code> is a universally quantified fact.	Proof game
	4769	<pre> lemma </pre>	<code>lemma</code>	

	Line	Construct	What was written	Sym
		adversary_budgeted_sampled_direct_discipline_preserved	lemma adversary_budgeted_sampled_direct_discipline_preserved is a universally quantified fact. : hoare[A(EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign (H, A, DirectSigningCoinSampler).AH, EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign (H, A, DirectSigningCoinSampler).0).forge : budgeted_programmed_query_count_discipline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversar...	Prop game
	4808	lemma		
		budgeted_sampled_direct_main_reprogram_clear	lemma budgeted_sampled_direct_main_reprogram_clear is an implication used as a proof obligation. : hoare[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign (H, A, DirectSigningCoinSampler).main : true ==> ! EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram].	Prov game
	4859	lemma		
		budgeted_sampled_direct_bad_reprogram_zero	lemma budgeted_sampled_direct_bad_reprogram_zero is a probability bound or exact game probability. &m : Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign (H, A, DirectSigningCoinSampler).main() @ &m : EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram] = 0%r.	Prov game
	4867	hoare proof step		
		hoare_step_for_budgeted_sampled_direct_bad_reprogram_zero	Within budgeted_sampled_direct_bad_reprogram_zero, the proof reduces the probabilistic goal to a Hoare triple $\{P\} c \{Q\}$.	Intro zero,tl
	4871	lemma		
		budgeted_sampled_direct_one_step_bound_nonnegative	lemma budgeted_sampled_direct_one_step_bound_nonnegative is an arithmetic or probability upper bound. : 0%r <= (rom_hash_query_budget + 1%r) / challenge_support_cardinality_lower_bound.	Prov game

	Line	Construct	What was written	Sym
	4881	lemma		
budgeted_sampled_direct_sampler_message_hash_prequery_bound		<pre> lemma budgeted_sampled_direct_sampler_message_hash_prequery_bound : pk sk qs m ctx : phoare[DirectSigningCoinSampler.sample : budgeted_programmed_challenge_query_discipline EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.adversary_hash_count qs /\ pk = public_key_of_secret haetae_mode sk /\ EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.pk_current =... </pre>	budgeted_sampled_direct_sampler_message_hash_prequery_bound This an equality/rewriting fact.	Prosh game
	4920	lemma		
budgeted_sampled_direct_bad_prequery_bound		<pre> lemma budgeted_sampled_direct_bad_prequery_bound : &m : islossless H.get => islossless H.set => Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign (H, A, DirectSigningCoinSampler).main() @ &m : EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery] <= ((rom_hash_query_budget + rom_signature_query_budget) * (rom_hash_query_budget + 1%r)) / cha... </pre>	budgeted_sampled_direct_bad_prequery_bound $\Pr[E] \leq B$ is a probability bound or exact game probability.	Indv game
	4993	hoare proof step		
hoare_step_for_budgeted_sampled_direct_bad_prequery_bound		hoare.	Within budgeted_sampled_direct_bad_prequery_bound the proof reduces the probabilistic goal to a Hoare triple $\{P\} c \{Q\}$.	Intro foundt
	4999	hoare proof step		
hoare_step_for_budgeted_sampled_direct_bad_prequery_bound		hoare.	Within budgeted_sampled_direct_bad_prequery_bound the proof reduces the probabilistic goal to a Hoare triple $\{P\} c \{Q\}$.	Intro foundt
	5130	hoare proof step		
hoare_step_for_budgeted_sampled_direct_bad_prequery_bound		hoare.	Within budgeted_sampled_direct_bad_prequery_bound the proof reduces the probabilistic goal to a Hoare triple $\{P\} c \{Q\}$.	Intro foundt

	Line	Construct	What was written	Sym
	5141	hoare proof step		
hoare_step_for_budgeted_sampled_direct_bad_prequery_bound		hoare.	Within budgeted_sampled_direct_bad_prequery_bound, the proof reduces the probabilistic goal to a Hoare triple $\{P\} c \{Q\}$.	Intro
	5182	lemma		
budgeted_sampled_direct_prequery_reprogram_bad_bound		lemma budgeted_sampled_direct_prequery_reprogram_bad_bound : &m : islossless H.get => islossless H.set => Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign (H, A, DirectSigningCoinSampler).main() @ &m : EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery \/ EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram] =< counted_rom...	budgeted_sampled_direct_prequery_reprogram_bad_bound : Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery &m : islossless H.get => islossless H.set => Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery \/ EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram] =< counted_rom...] =< counted_rom...]	Prob game
	5213	lemma		
budgeted_sampled_direct_prequery_reprogram_bad_checked_31		lemma budgeted_sampled_direct_prequery_reprogram_bad_checked_31 : &m : islossless H.get => islossless H.set => Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign (H, A, DirectSigningCoinSampler).main() @ &m : EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery \/ EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram] =< 31%r /...	budgeted_sampled_direct_prequery_reprogram_bad_checked_31 : Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery &m : islossless H.get => islossless H.set => Pr[EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_prequery \/ EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign.C.bad_reprogram] =< 31%r /...] =< 31%r /...]	Prob game
	5241	equivalence		

Line	Construct	What was written	Sym
	budgeted_programmed_counted_sampled_direct_hash_get_equiv	equiv budgeted_programmed_counted_sampled_direct_hash_get_equiv is a relational program : equivalence. EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).AH.get ~ EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign (H, A, DirectSigningCoinSampler).AH.get :={glob H, arg} /\ EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.adversary_hash_count{1} = EUF_CMA_ROMProgrammedTranscriptBudget...	disShow obser relati
5295	equivalence		
	budgeted_programmed_counted_sampled_direct_sign_equiv	equiv budgeted_programmed_counted_sampled_direct_sign_equiv is a relational program : equivalence. EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).O.sign ~ EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign (H, A, DirectSigningCoinSampler).O.sign :={glob H, arg} /\ EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign.pk_current{1} = EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign...	disShow obser relati
5380	equivalence		
	adversary_budgeted_programmed_counted_sampled_direct_equiv	equiv adversary_budgeted_programmed_counted_sampled_direct_equiv is a relational program : equivalence. A(EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).AH, EUF_CMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).O).forge ~ A(EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign (H, A, DirectSigningCoinSampler).AH, EUF_CMA_ROMProgrammedTranscriptBudgetedSampledSign (H, A, DirectSigni...	disShow obser relati
5480	equivalence		

	Line	Construct	What was written	Sym
		budgeted_programmed_counted_sampled_direct_main_equiv	equiv budgeted_programmed_counted_sampled_direct_main_equiv : EUFCMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main ~ EUFCMA_ROMProgrammedTranscriptBudgetedSampledSign (H, A, DirectSigningCoinSampler).main : ={glob H, glob A} ==> ={res} /\n EUFCMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery{1} = EUFCMA_ROMProgrammedTranscriptBudget...	budgeted_programmed_counted_sampled_direct_main_equiv is a relational program G = C [Res S] is a relational program equivalence. observed relation
	5572	lemma	lemma budgeted_programmed_counted_sampled_direct_prequery_reprogram_bad_exact &m : Pr[EUFCMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &m : EUFCMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery \\ EUFCMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram] = Pr[EUFCMA_ROMProgrammedTranscriptBudgetedSampledSign (H, A,...	budgeted_programmed_counted_sampled_direct_prequery_reprogram_bad_exact is a probability bound or Pr[E] ≤ B is a probability bound or exact game probability. game
	5585	lemma	lemma budgeted_programmed_prequery_reprogram_bad_checked_31_via_direct_sampler &m : islossless H.get => islossless H.set => Pr[EUFCMA_ROMProgrammedTranscriptBudgetedCountedSign(H, A).main() @ &m : EUFCMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_prequery \\ EUFCMA_ROMProgrammedTranscriptBudgetedCountedSign.C.bad_reprogram] <= 31%r / 288230376151...	budgeted_programmed_prequery_reprogram_bad_checked_31_via_direct_sampler is a probability bound or Pr[E] ≤ B is a probability bound or exact game probability. game
	5608	equivalence		

Line	Construct	What was written	Sym	
	haetae_rom_internal_oracle_erasure	equiv haetae_rom_internal_oracle_erasure : SIG.EUF_CMA(H, HAETAE, A).0.sign ~ EUF_CMA_ROMInternalTranscriptSign(H, A).0.sign : ={glob H, arg} /\ SIG.EUF_CMA.sk{1} = EUF_CMA_ROMInternalTranscriptSign.sk_current{2} /\ SIG.EUF_CMA.queries{1} = EUF_CMA_ROMInternalTranscriptSign.queries{2} ==> ={glob H, res} /\ SIG.EUF_CMA.sk{1} = EUF_CMA_ROMInternalTranscript...	haetae_rom_internal_oracle_erasure : $\mathcal{C}_1 \sim \mathcal{C}_2 [R \Rightarrow S]$ is a relational program equivalence.	Show observ relati
5637	adversary_haetae_rom_internal_erasure	equiv adversary_haetae_rom_internal_erasure : A(H, SIG.EUF_CMA(H, HAETAE, A).0).forge ~ A(H, EUF_CMA_ROMInternalTranscriptSign(H, A).0).forge : ={glob H, glob A, arg} /\ SIG.EUF_CMA.sk{1} = EUF_CMA_ROMInternalTranscriptSign.sk_current{2} /\ SIG.EUF_CMA.queries{1} = EUF_CMA_ROMInternalTranscriptSign.queries{2} ==> ={glob H, res} /\ SIG.EUF_CMA.sk{1} = EUF...	adversary_haetae_rom_internal_erasure : $\mathcal{C}_1 \sim \mathcal{C}_2 [R \Rightarrow S]$ is a relational program equivalence.	Show observ relati
5672	haetae_rom_internal_transcript_erasure_exact	lemma haetae_rom_internal_transcript_erasure_exact &m : Pr[SIG.EUF_CMA(H, HAETAE, A).main() @ &m : res] = Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m : res].	haetae_rom_internal_transcript_erasure_exact $\Pr[\text{exact}] = B$ is a probability bound or exact game probability.	Prove simul
5713		equiv		

	Line	Construct	What was written	Sym
rom_internal_oracle_structural_nma		equiv	rom_internal_oracle_structural_nma :	Show
		rom_internal_oracle_structural_nma	$\mathcal{C}_1 \sim \mathcal{C}_2 [R \Rightarrow S]$ is a relational program	observed
		:	equivalence.	relation
		EUF_CMA_ROMInternalTranscriptSign(H,		
		A).0.sign ~		
		ROMInternalTranscriptAsNMA(A,		
		H).0.sign : = {glob H, arg} /\		
		EUF_CMA_ROMInternalTranscriptSign.pk_current{1}		
		=		
		ROMInternalTranscriptAsNMA.pk_current{2}		
		/\		
		EUF_CMA_ROMInternalTranscriptSign.sk_current{1}		
		=		
		ROMInternalTranscriptAsNMA.sk_current{2}		
		/\ EUF_CMA_ROMIntern...		
	5755	equivalence		
adversary_rom_internal_structural_nma		equiv	adversary_rom_internal_structural_nma :	Show
		adversary_rom_internal_structural_nma	$\mathcal{C}_1 \sim \mathcal{C}_2 [R \Rightarrow S]$ is a relational program	observed
		:	equivalence.	relation
		A(H,		
		EUF_CMA_ROMInternalTranscriptSign(H,		
		A).0).forge ~ A(H,		
		ROMInternalTranscriptAsNMA(A,		
		H).0).forge : = {glob H, glob A,		
		arg} /\		
		EUF_CMA_ROMInternalTranscriptSign.pk_current{1}		
		=		
		ROMInternalTranscriptAsNMA.pk_current{2}		
		/\		
		EUF_CMA_ROMInternalTranscriptSign.sk_current{1}		
		=		
		ROMInternalTranscriptAsNMA.sk_curren...		
	5811	lemma		
rom_internal_transcript_structural_nma_bound		lemma	rom_internal_transcript_structural_nma_bound	Proof
		rom_internal_transcript_structural_nma_bound	$\Pr[E] \leq B$ is a probability bound or	game
		&m :	exact game probability.	
		Pr[EUFCMA_ROMInternalTranscriptSign(H,		
		A).main() @ &m : res] <=		
		Pr[SIG.UF_NMA(H, HAETAE,		
		ROMInternalTranscriptAsNMA(A)).main()		
		@ &m : res].		
	5858	lemma		

	Line	Construct	What was written	Sym
rom_internal_transcript_clear_site_structural_nma_bound		lemma	rom_internal_transcript_clear_site_structural_nma_bound : $\Pr[E] \leq B$ is a probability bound or exact game probability.	Prove game
		rom_internal_transcript_clear_site_structural_nma_bound : &m : Pr[EUF_CMA_ROMInternalTranscriptSign(H, A).main() @ &m : res /\ transcript_log_signature_programming_sites_clear haetae_mode EUF_CMA_ROMInternalTranscriptSign.transcripts] <= Pr[SIG.UF_NMA(H, HAETAETAE, ROMInternalTranscriptAsNMA(A)).main() @ &m : res].		
	5881	equivalence		
rom_internal_public_sim_nma_oracle		equiv	rom_internal_public_sim_nma_oracle : $\mathcal{C}_1 \sim \mathcal{C}_2 [R \Rightarrow S]$ is a relational program equivalence.	Show obser relati
		rom_internal_public_sim_nma_oracle : : ROMInternalTranscriptAsNMA(A, H).0.sign ~ ROMInternalTranscriptPublicSimAsNMA(A, H).0.sign : ={glob H, arg} /\ ROMInternalTranscriptAsNMA.pk_current{1} = ROMInternalTranscriptPublicSimAsNMA.pk_current{2} /\ public_key_of_secret haetae_mode ROMInternalTranscriptAsNMA.sk_current{1} = ROMInternalTranscriptPublicSimA...		
	5927	equivalence		
adversary_rom_internal_public_sim_nma		equiv	adversary_rom_internal_public_sim_nma : $\mathcal{C}_1 \sim \mathcal{C}_2 [R \Rightarrow S]$ is a relational program equivalence.	Show obser relati
		adversary_rom_internal_public_sim_nma : : A(H, ROMInternalTranscriptAsNMA(A, H).0).forge ~ A(H, ROMInternalTranscriptPublicSimAsNMA(A, H).0).forge : ={glob H, glob A, arg} /\ ROMInternalTranscriptAsNMA.pk_current{1} = ROMInternalTranscriptPublicSimAsNMA.pk_current{2} /\ public_key_of_secret haetae_mode ROMInternalTranscriptAsNMA.sk_current{1} = ROMInt...		

	Line	Construct	What was written	Sym
	5988	lemma		
rom_internal_nma_public_sim_exact		<pre> lemma rom_internal_nma_public_sim_exact &m : Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptAsNMA(A)).main() @ &m : res] = Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPublicSimAsNMA(A)).main() @ &m : res]. </pre>	<pre> rom_internal_nma_public_sim_exact : Pr[c : E] = B is a probability bound or exact game probability. </pre>	Prove simul
	6050	equivalence		
rom_internal_public_sim_rom_paper_sim_nma_oracle		<pre> equiv rom_internal_public_sim_rom_paper_sim_nma_oracle : ROMInternalTranscriptPublicSimAsNMA(A, H).0.sign ~ ROMInternalTranscriptPaperSimAsNMA (A, ROMPaperSimSigningSampler(H), H).0.sign : ={glob H, arg} /\ ROMInternalTranscriptPublicSimAsNMA.pk_current{1} = ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\ ROMPaperSimSigningSampler.pk_current{2} = ROMI... </pre>	<pre> rom_internal_public_sim_rom_paper_sim_nma_oracle : [R ⇒ S] is a relational program equivalence. </pre>	Show observ relati
	6097	equivalence		
adversary_rom_internal_public_sim_rom_paper_sim_nma		<pre> equiv adversary_rom_internal_public_sim_rom_paper_sim_nma : A(H, ROMInternalTranscriptPublicSimAsNMA(A, H).0).forge ~ A(H, ROMInternalTranscriptPaperSimAsNMA (A, ROMPaperSimSigningSampler(H), H).0).forge : ={glob H, glob A, arg} /\ ROMInternalTranscriptPublicSimAsNMA.pk_current{1} = ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\ ROMPaperSimSigningSam... </pre>	<pre> adversary_rom_internal_public_sim_rom_paper_sim_nma : [R ⇒ S] is a relational program equivalence. </pre>	Show observ relati
	6158	lemma		

	Line	Construct	What was written	Sym
		rom_internal_nma_public_sim_rom_paper_sim_exact	lemma rom_internal_nma_public_sim_rom_paper_sim_exact = &m : Pr[SIG.UF_NMA(H, HAETAETAE, exact game probability. ROMInternalTranscriptPublicSimAsNMA(A)).main() @ &m : res] = Pr[SIG.UF_NMA(H, HAETAETAE, ROMInternalTranscriptPaperSimAsNMA(A, ROMPaperSimSigningSampler(H)))).main() @ &m : res].	rom_internal_nma_public_sim_rom_paper_sim_exact Pr[$\mathcal{C}_m : E$] = B is a probability bound or exact game probability.
	6221	equivalence	rom_internal_real_signing_paper_sim_nma_oracle	rom_internal_real_signing_paper_sim_nma_oracle [$\mathcal{C}_m \sim \mathcal{C}_g$] $R \Rightarrow S$ is a relational program observed relative
		rom_internal_real_signing_paper_sim_nma_oracle	equiv rom_internal_real_signing_paper_sim_nma_oracle : ROMInternalTranscriptAsNMA(A, H).O.sign ~ ROMInternalTranscriptPaperSimAsNMA(A, RealSigningPaperSimSampler(H), H).O.sign :={glob H, arg} /\ ROMInternalTranscriptAsNMA.pk_current{1} = ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\ public_key_of_secret haetae_mode ROMInternalTranscriptAsNMA.sk_cu...	rom_internal_real_signing_paper_sim_nma_oracle [$\mathcal{C}_m \sim \mathcal{C}_g$] $R \Rightarrow S$ is a relational program observed relative
	6278	equivalence	adversary_rom_internal_real_signing_paper_sim_nma	adversary_rom_internal_real_signing_paper_sim_nma [$\mathcal{C}_m \sim \mathcal{C}_g$] $R \Rightarrow S$ is a relational program observed relative
		adversary_rom_internal_real_signing_paper_sim_nma	equiv adversary_rom_internal_real_signing_paper_sim_nma : A(H, ROMInternalTranscriptAsNMA(A, H).O).forge ~ A(H, ROMInternalTranscriptPaperSimAsNMA(A, RealSigningPaperSimSampler(H), H).O).forge :={glob H, glob A, arg} /\ ROMInternalTranscriptAsNMA.pk_current{1} = ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\ public_key_of_secret haetae_mode ROMInte...	adversary_rom_internal_real_signing_paper_sim_nma [$\mathcal{C}_m \sim \mathcal{C}_g$] $R \Rightarrow S$ is a relational program observed relative

	Line	Construct	What was written	Sym
	6354	lemma		
rom_internal_nma_real_signing_paper_sim_exact		<pre> lemma rom_internal_nma_real_signing_paper_sim_exact &m : Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptAsNMA(A)).main() @ &m : res] = Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, RealSigningPaperSimSampler(H))).main() @ &m : res]. </pre>	<pre> rom_internal_nma_real_signing_paper_sim_exact Pr[exact] = B is a probability bound or exact game probability. </pre>	Pr[exact]
	6427	equivalence		
real_signing_ro_attempt_paper_sim_nma_oracle		<pre> equiv real_signing_ro_attempt_paper_sim_nma_oracle : ROMInternalTranscriptPaperSimAsNMA (A, RealSigningPaperSimSampler(H), H).O.sign ~ ROMInternalTranscriptPaperSimAsNMA (A, ROSigningAttemptPaperSimSampler(H), H).O.sign : ={glob H, arg} /\ ROMInternalTranscriptPaperSimAsNMA.pk_current{1} = ROMInternalTranscriptPaperSimAsNMA.pk_current{2} /\ RealSigningPap... </pre>	<pre> real_signing_ro_attempt_paper_sim_nma_oracle forall C, S [R => S] is a relational program equivalence. </pre>	forall C, S [R => S] is a relational program equivalence.
	6485	equivalence		

Line	Construct	What was written	Sym	
	adversary_real_signing_ro_attempt_paper_sim_nma	equiv adversary_real_signing_ro_attempt_paper_sim_nma : A(H, ROMInternalTranscriptPaperSimAsNMA (A, RealSigningPaperSimSampler(H), H).0).forge ~ A(H, ROMInternalTranscriptPaperSimAsNMA (A, ROSigningAttemptPaperSimSampler(H), H).0).forge : ={glob H, glob A, arg} /\n ROMInternalTranscriptPaperSimAsNMA.pk_current{1} = ROMInternalTranscriptPaperSimAsNMA.pk_cur...	adversary_real_signing_ro_attempt_paper_sim_nma [$R \Rightarrow S$] is a relational program equivalence.	Shows observ relati
6548	rom_internal_nma_real_signing_ro_attempt_paper_sim_exact	lemma rom_internal_nma_real_signing_ro_attempt_paper_sim_exact &m : Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, RealSigningPaperSimSampler(H))).main() @ &m : res] = Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res].	rom_internal_nma_real_signing_ro_attempt_paper_sim_exact [E] is a probability bound or exact game probability.	Proba simul
6615		operation		

	Line	Construct	What was written	Sym	
		structural_to_exact_hyperball_paper_sample_loss_obligation	op structural_to_exact_hyperball_paper_sample_loss_obligation = mode × sk × message × context × (md : mode) : bool = forall (sk : skey) (m : message) (ctx : context) (p : paper_sim_signature_sample -> bool), mu (dsigning_attempt_state md sk m ctx) (fun st => p (paper_sim_sample_from_rejection_attempt st)) <= mu (dexact_hyperball_signing_attempt_state md sk m ctx) (fun st => p (paper_sim_s...	structural_to_exact_hyperball_paper_sample_loss_obligation mode × sk × message × context × paper_sim_signature_sample -> bool is a Boolean-valued predicate.	Nos used
	6625	lemma	concrete_ro_signing_attempt_to_exact_hyperball_paper_sample_loss_obligation concrete_ro_signing_attempt_to_exact_hyperball_paper_sample_loss_obligation seed_coins m ctx (p : paper_sim_signature_sample -> bool) &m : structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode => sampler_expand_query seed_coins \notin HAETAЕ_RO.FRO.m{m} => ROSigningAttemptPaperSimSampler.sk_current{m} = ROExactHyperballPaperSimSample...	concrete_ro_signing_attempt_to_exact_hyperball_paper_sample_loss_obligation P => Q is an implication used as a proof obligation.	Perba game
	6655	lemma	concrete_ro_signing_attempt_to_exact_hyperball_paper_sample_loss_obligation concrete_ro_signing_attempt_to_exact_hyperball_paper_sample_loss_obligation seed_coins m ctx (p : paper_sim_signature_sample -> bool) hash_qs sampler_qs bad &m : structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode => sampler_rom_covered HAETAЕ_RO.FRO.m{m} hash_qs sampler_qs => ! (bad \/ sampler_expand_query seed_coins \in hash_qs \...	concrete_ro_signing_attempt_to_exact_hyperball_paper_sample_loss_obligation P => Q is an implication used as a proof obligation.	Perba game

	Line	Construct	What was written	Sym
	6678	lemma	concrete_budgeted_o_sign_sample_with_seed_clean_loss_surface	Proof
concrete_budgeted_o_sign_sample_with_seed_clean_loss_surface		lemma	concrete_budgeted_o_sign_sample_with_seed_clean_loss_surface	game
		seed_coins m ctx (p :	obligation.	
		paper_sim_signature_sample ->		
		bool) &m :		
		structural_to_exact_hyperball_paper_sample_loss_obligation		
		haetae_mode =>		
		sampler_rom_covered		
		HAETAETAE_ROM.FROM.m{m}		
		ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m}		
		ROMInternalTranscriptBudgetedPaperSimAsNMA.sa...		
	6708	lemma	concrete_budgeted_o_sign_old_log_sample_with_seed_clean_loss_surface	Proof
concrete_budgeted_o_sign_old_log_sample_with_seed_clean_loss_surface		lemma	concrete_budgeted_o_sign_old_log_sample_with_seed_clean_loss_surface	game
		seed_coins m ctx (p :	obligation.	
		paper_sim_signature_sample ->		
		bool) old_hash_qs old_sampler_qs		
		old_bad &m :		
		structural_to_exact_hyperball_paper_sample_loss_obligation		
		haetae_mode => old_hash_qs =		
		ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m}		
		=> old_sampler_qs = ROMInte...		
	6740	lemma	concrete_budgeted_o_sign_old_log_signature_from_sample_clean_loss_surface	Proof
concrete_budgeted_o_sign_old_log_signature_from_sample_clean_loss_surface		lemma	concrete_budgeted_o_sign_old_log_signature_from_sample_clean_loss_surface	game
		seed_coins m ctx (p : signature	obligation.	
		-> bool) old_hash_qs		
		old_sampler_qs old_bad &m :		
		structural_to_exact_hyperball_paper_sample_loss_obligation		
		haetae_mode => old_hash_qs =		
		ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m}		
		=> old_sampler_qs =		
		ROMInternalTranscri...		
	6779	lemma		

	Line	Construct	What was written	Sym
concrete_budgeted_o_sign_old_log_signature_clean_event_loss_surface		<pre> lemma concrete_budgeted_o_sign_old_log_signature_clean_event_loss_surface seed_coins m ctx (p : signature -> bool) old_hash_qs old_sampler_qs old_bad &m : structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode => old_hash_qs = ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m} => old_sampler_qs = ROMInternalTranscriptBudg...</pre>	concrete_budgeted_o_sign_old_log_signature_clean_event_loss_surface $P \Rightarrow Q$ is an implication used as a proof obligation.	Protocol game
concrete_frozen_old_log_sample_clean_event_loss_surface	6818	<pre> lemma concrete_frozen_old_log_sample_clean_event_loss_surface seed_coins m ctx (p : paper_sim_signature_sample -> bool) old_hash_qs old_sampler_qs old_bad &m : structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode => sampler_rom_covered HAETAETAE_ROM.FRO.m{m} old_hash_qs old_sampler_qs => ! (old_bad \/ sampler_expand_query seed_coins \in old...</pre>	concrete_frozen_old_log_sample_clean_event_loss_surface $P \Rightarrow Q$ is an implication used as a proof obligation.	Protocol game
concrete_frozen_old_log_signature_clean_event_loss_surface	6845	<pre> lemma concrete_frozen_old_log_signature_clean_event_loss_surface seed_coins m ctx pk (p : signature -> bool) old_hash_qs old_sampler_qs old_bad &m : structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode => sampler_rom_covered HAETAETAE_ROM.FRO.m{m} old_hash_qs old_sampler_qs => ! (old_bad \/ sampler_expand_query seed_coins \in old_hash_qs \/\...</pre>	concrete_frozen_old_log_signature_clean_event_loss_surface $P \Rightarrow Q$ is an implication used as a proof obligation.	Protocol game
	6875	<pre> lemma</pre>		

	Line	Construct	What was written	Sym
concrete_frozen_old_log_self_logged_signature_clean_event_loss_surface		lemma concrete_frozen_old_log_self_logged_signature_clean_event_loss_surface seed_coins m ctx pk (p : signature -> bool) old_hash_qs old_sampler_qs old_bad &m : structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode => sampler_rom_covered HAETAЕ_RO.FRO.m{m} old_hash_qs old_sampler_qs => ! (old_bad \/ sampler_expand_query seed_coins \in old...	concrete_frozen_old_log_self_logged_signature_clean_event_loss_surface $P \Rightarrow Q$ is an implication used as a proof obligation.	Prove game
concrete_budgeted_self_logged_signature_clean_event_loss_surface	6913	lemma concrete_budgeted_self_logged_signature_clean_event_loss_surface seed_coins m ctx pk (p : signature -> bool) &m : structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode => sampler_rom_covered HAETAЕ_RO.FRO.m{m} ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m} ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_exp...	concrete_budgeted_self_logged_signature_clean_event_loss_surface $P \Rightarrow Q$ is an implication used as a proof obligation.	Prove game
concrete_lazy_rom_get_lossless	6956	lemma concrete_lazy_rom_get_lossless : islossless HAETAЕ_RO.FRO.get.	concrete_lazy_rom_get_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Prove sample
structural_attempt_exact_hyperball_paper_sample_one_step_loss	6965	lemma structural_attempt_exact_hyperball_paper_sample_one_step_loss sk m ctx (p : paper_sim_signature_sample -> bool) : structural_to_exact_hyperball_paper_sample_loss_obligation haetae_mode => phoare[StructuralAttemptPaperSample.sample : arg = (sk, m, ctx) ==> p res] <= (mu (dexact_hyperball_signing_attempt_state haetae_mode sk m ctx) (fun st => p (paper...	structural_attempt_exact_hyperball_paper_sample_one_step_loss $X \leq Y$ is an arithmetic or probability upper bound.	Prove game

	Line	Construct	What was written	Sym
	6986	lemma	exact_hyperball_paper_sample_one_step_upper	Pr
exact_hyperball_paper_sample_one_step_upper		lemma	exact_hyperball_paper_sample_one_step_upper is an arithmetic or probability upper bound.	game
		exact_hyperball_paper_sample_one_step_upper		
		sk m ctx (p :		
		paper_sim_signature_sample ->		
		bool) :		
		phoare[ExactHyperballPaperSample.sample		
		: arg = (sk, m, ctx) ==> p res]		
		<= (mu		
		(dexact_hyperball_signing_attempt_state		
		haetae_mode sk m ctx) (fun st =>		
		p		
		(paper_sim_sample_from_rejection_attempt		
		st))).		
	7016	lemma	rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting	Pr
rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting		lemma	rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting is a probability bound or exact game probability.	game
		rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting		
		&m :		
		structural_to_exact_hyperball_paper_sample_loss_obligation		
		haetae_mode => Pr[SIG.UF_NMA(H,		
		HAETAETAE,		
		ROMInternalTranscriptPaperSimAsNMA		
		(A,		
		ROSigningAttemptPaperSimSampler(H)))>.main()		
		@ &m : res] <= Pr[SIG.UF_NMA(H,		
		HAETAETAE,		
		ROMInternalTranscriptPaperSimAsNMA		
		(A,...		
	7036	lemma	rom_internal_budgeted_ro_signing_attempt_signing_call_loss_bound	Pr
rom_internal_budgeted_ro_signing_attempt_signing_call_loss_bound		lemma	rom_internal_budgeted_ro_signing_attempt_signing_call_loss_bound is a probability bound or exact game probability.	game
		rom_internal_budgeted_ro_signing_attempt_signing_call_loss_bound		
		&m : Pr[SIG.UF_NMA(H, HAETAETAE,		
		ROMInternalTranscriptBudgetedPaperSimAsNMA		
		(A,		
		ROSigningAttemptPaperSimSampler(H)))>.main()		
		@ &m : 0 <		
		ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count]		
		<= rom_signature_query_budget *		
		rejection_sampling_loss_term.		
	7057	lemma		

	Line	Construct	What was written	Sym
rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting		lemma	rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting	rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting
		rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting	$\Pr[c = E] \leq B$ is a probability bound on exact game probability.	
		&m :		
		structural_to_exact_hyperball_paper_sample_loss_obligation		
		haetae_mode => Pr[SIG.UF_NMA(H,		
		HAETAE,		
		ROMInternalTranscriptBudgetedPaperSimAsNMA		
		(A,		
		ROSigningAttemptPaperSimSampler(H)))		
		@ &m : res] <= Pr[SIG.UF_NMA(H,		
		HAETAE, ROMInternalTranscript...		
	7097	lemma	rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting	rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting
rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_clean_lifting		lemma	rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_clean_lifting	rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_clean_lifting
		rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_clean_lifting	$\Pr[c = E] \leq B$ is a probability bound on exact game probability.	
		&m : Pr[SIG.UF_NMA(H, HAETAE,		
		ROMInternalTranscriptBudgetedPaperSimAsNMA		
		(A,		
		ROSigningAttemptPaperSimSampler(H)))		
		@ &m : res /\ !		
		ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery]		
		<= Pr[SIG.UF_NMA(H, HAETAE,		
		ROMInternalTranscriptBudgetedPape...		
	7154	lemma	rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_clean_lifting	rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_clean_lifting
rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_clean_conditioned_lifting		lemma	rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_clean_conditioned_lifting	rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_clean_conditioned_lifting
		rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_clean_conditioned_lifting	$\Pr[c = E] \leq B$ is a probability bound on exact game probability.	
		&m :		
		structural_to_exact_hyperball_paper_sample_loss_obligation		
		haetae_mode => Pr[SIG.UF_NMA(H,		
		HAETAE,		
		ROMInternalTranscriptBudgetedPaperSimAsNMA		
		(A,		
		ROSigningAttemptPaperSimSampler(H)))		
		@ &m : res /\ !		
		ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler...		
	7183	lemma	rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_clean_conditioned_lifting	rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_clean_conditioned_lifting

	Line	Construct	What was written	Sym
rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_checked_clean_lifting		lemma	rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_checked_clean_lifting	Pr[$\epsilon \leq E$] $\leq B$ is a probability bound for exact game probability.
		rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_checked_clean_lifting		
		&m :		
		structural_to_exact_hyperball_paper_sample_loss_obligation		
		haetae_mode => Pr[SIG.UF_NMA(H,		
		HAETAETAE,		
		ROMInternalTranscriptBudgetedPaperSimAsNMA		
		(A,		
		ROSigningAttemptPaperSimSampler(H)))>.main()		
		@ &m : res] <= Pr[SIG.UF_NMA(H,		
		HAETAETAE, ROMInternalTranscrip...		
	7205	lemma	rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_budgeted_lifting	Pr[$\epsilon \leq E$] $\leq B$ is a probability bound for exact game probability.
rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_budgeted_lifting		lemma	rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_budgeted_lifting	Pr[$\epsilon \leq E$] $\leq B$ is a probability bound for exact game probability.
		rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_budgeted_lifting		
		&m : Pr[SIG.UF_NMA(H, HAETAETAE,		
		ROMInternalTranscriptPaperSimAsNMA		
		(A,		
		ROSigningAttemptPaperSimSampler(H)))>.main()		
		@ &m : res] <= Pr[SIG.UF_NMA(H,		
		HAETAETAE,		
		ROMInternalTranscriptBudgetedPaperSimAsNMA		
		(A,		
		ROSigningAttemptPaperSimSampler(H)))>.main()		
		@ &m : res] => Pr[SIG.UF_...		
	7243	lemma	rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_budgeted_checked_clean_lifting	Pr[$\epsilon \leq E$] $\leq B$ is a probability bound for exact game probability.
rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_budgeted_checked_clean_lifting		lemma	rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_budgeted_checked_clean_lifting	Pr[$\epsilon \leq E$] $\leq B$ is a probability bound for exact game probability.
		rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_budgeted_checked_clean_lifting		
		&m : Pr[SIG.UF_NMA(H, HAETAETAE,		
		ROMInternalTranscriptPaperSimAsNMA		
		(A,		
		ROSigningAttemptPaperSimSampler(H)))>.main()		
		@ &m : res] <= Pr[SIG.UF_NMA(H,		
		HAETAETAE,		
		ROMInternalTranscriptBudgetedPaperSimAsNMA		
		(A,		
		ROSigningAttemptPaperSimSampler(H)))>.main()		
		@ &m : res]...		
	7291	lemma		

	Line	Construct	What was written	Sym
rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_bound		lemma rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_bound : Pr[exact_hyperball_loss_bound] ≤ Pr[exact_hyperball_loss_bound] + rejection_sampling_loss_term. ROMInternalTranscriptPaperSimAsNMA (A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] ≤ Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] + rejection_sampling_loss_term.	rom_internal_nma_ro_signing_attempt_ro_exact_hyperball_loss_bound : Pr[exact_hyperball_loss_bound] ≤ Pr[exact_hyperball_loss_bound] + rejection_sampling_loss_term. ROMInternalTranscriptPaperSimAsNMA (A, ROSigningAttemptPaperSimSampler(H))).main() @ &m : res] ≤ Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA (A, ROExactHyperballPaperSimSampler(H))).main() @ &m : res] + rejection_sampling_loss_term.	Exact game probability.
concrete_lazy_rom_get_preserves_signature_event	7326	lemma concrete_lazy_rom_get_preserves_signature_event : (p : signature -> bool) : phoare[HAETAE_RO.FRO.get : p (paper_sim_signature haetae_mode pk m ctx smp) ==> p (paper_sim_signature haetae_mode pk m ctx smp)] = 1/r.	concrete_lazy_rom_get_preserves_signature_event : (p : signature -> bool) : phoare[HAETAE_RO.FRO.get : p (paper_sim_signature haetae_mode pk m ctx smp) ==> p (paper_sim_signature haetae_mode pk m ctx smp)] = 1/r.	Exact game probability.
concrete_lazy_rom_get_preserves_signature_event_with_pk	7338	lemma concrete_lazy_rom_get_preserves_signature_event_with_pk : (p : signature -> bool) : phoare[HAETAE_RO.FRO.get : ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current = pk /\ p (paper_sim_signature haetae_mode pk m ctx smp) ==> ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current = pk /\ p (paper_sim_signature haetae_mode pk m ctx smp)] = ...	concrete_lazy_rom_get_preserves_signature_event_with_pk : (p : signature -> bool) : phoare[HAETAE_RO.FRO.get : ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current = pk /\ p (paper_sim_signature haetae_mode pk m ctx smp) ==> ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current = pk /\ p (paper_sim_signature haetae_mode pk m ctx smp)] = ...	Exact game probability.
	7352	lemma		

	Line	Construct	What was written	Sym
concrete_lazy_rom_get_preserves_current_signature_event		lemma concrete_lazy_rom_get_preserves_current_signature_event m ctx smp (p : signature -> bool) : phoare[HAETAE_RO.FRO.get : p (paper_sim_signature haetae_mode ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current m ctx smp) ==> p (paper_sim_signature haetae_mode ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current m ctx smp))] = 1%r.	concrete_lazy_rom_get_preserves_current_signature_event $P \Rightarrow Q$ is an implication used as a proof obligation.	Prova orack proce
	7366	lemma concrete_lazy_rom_get_true	lemma concrete_lazy_rom_get_true : phoare[HAETAE_RO.FRO.get : true ==> true] = 1%r.	concrete_lazy_rom_get_true : $P \Rightarrow Q$ is an implication used as a proof obligation. Reo scrip
concrete_lazy_rom_get_preserves_budgeted_clean_signature_event	7372	lemma concrete_lazy_rom_get_preserves_budgeted_clean_signature_event pk m ctx smp (p : signature -> bool) : phoare[HAETAE_RO.FRO.get : ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current = pk /\ ! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery /\ p (paper_sim_signature haetae_mode pk m ctx smp) ==> ROMInternalTranscriptBudgetedPaperSimA...	concrete_lazy_rom_get_preserves_budgeted_clean_signature_event $P \Rightarrow Q$ is an implication used as a proof obligation. Proa game	
concrete_lazy_rom_get_preserves_budgeted_clean_signature_event_hoare	7388	lemma concrete_lazy_rom_get_preserves_budgeted_clean_signature_event_hoare pk m ctx smp (p : signature -> bool) : hoare[HAETAE_RO.FRO.get : ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current = pk /\ ! ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery /\ p (paper_sim_signature haetae_mode pk m ctx smp) ==> ROMInternalTranscriptBudgetedPape...	concrete_lazy_rom_get_preserves_budgeted_clean_signature_event_hoare $P \Rightarrow Q$ is an implication used as a proof obligation. Proa game	
	7404	lemma		

	Line	Construct	What was written	Sym
concrete_ro_signing_attempt_sample_with_seed_lossless		lemma	concrete_ro_signing_attempt_sample_with_seed_lossless	Reed
		concrete_ro_signing_attempt_sample_with_seed_lossless	$\forall \vec{x}. P(\vec{x})$ is an universally quantified fact.	sample
		: islossless		
		ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO).sample_with_seed.		
	7413	lemma	concrete_budgeted_ro_signing_attempt_o_sign_active_clean_signature_structural_lossless	Prove
concrete_budgeted_ro_signing_attempt_o_sign_active_clean_signature_structural_lossless		lemma	$L \models B$ is an equality/rewriting fact.	game
		concrete_budgeted_ro_signing_attempt_o_sign_active_clean_signature_structural_lossless		
		pk sk msg ctxt (p : signature		
		-> bool) :		
		phoare[ROMInternalTranscriptBudgetedPaperSimAsNMA		
		(A,		
		ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO),		
		HAETAE_RO.FRO).O.sign : arg =		
		(msg, ctxt) /\		
		ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count		
		< signature_query_budget...		
	7487	lemma	concrete_budgeted_ro_exact_hyperball_o_sign_active_signature_exact	Prove
concrete_budgeted_ro_exact_hyperball_o_sign_active_signature_exact		lemma	$L \models B$ is an equality/rewriting fact.	simul
		concrete_budgeted_ro_exact_hyperball_o_sign_active_signature_exact		
		pk sk msg ctxt (p : signature		
		-> bool) :		
		phoare[ROMInternalTranscriptBudgetedPaperSimAsNMA		
		(A,		
		ROExactHyperballPaperSimSampler(HAETAE_RO.FRO),		
		HAETAE_RO.FRO).O.sign : arg =		
		(msg, ctxt) /\		
		ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count		
		< signature_query_budget_count		
		/\ ROMIn...		
	7541	lemma	concrete_budgeted_ro_exact_hyperball_sample_with_seed_to_o_sign_active_projection	Prove
concrete_budgeted_ro_exact_hyperball_sample_with_seed_to_o_sign_active_projection		lemma	$P \Rightarrow Q$ is an implication used as a proof obligation.	game
		concrete_budgeted_ro_exact_hyperball_sample_with_seed_to_o_sign_active_projection		
		seed_coins pk sk msg ctxt (p : signature -> bool) &m :		
		ROMInternalTranscriptBudgetedPaperSimAsNMA.signing_count{m}		
		< signature_query_budget_count		
		=>		
		ROMInternalTranscriptBudgetedPaperSimAsNMA.pk_current{m}		
		= pk =>		
		ROExactHyperballPaperSimSampler.sk_current{m}		
		= sk => P...		

	Line	Construct	What was written	Sym
	7585	lemma	concrete_budgeted_ro_signing_attempt_o	Prove
concrete_budgeted_ro_signing_attempt_o_sign_to_self_logged_sample_active_projection		lemma	$P \Rightarrow Q$ is an implication used as a proof obligation.	game
		concrete_budgeted_ro_signing_attempt_o_sign_to_self_logged_sample_active_projection		
		seed_coins pk sk msg ctxt (p :		
		signature -> bool) &m :		
		sampler_rom_covered		
		HAETAE_RO.FRO.m{m}		
		ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m}		
		ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_expand_queries{m}		
		=> !		
		(ROMInternalTranscriptBudgeted...		
	7673	lemma	concrete_budgeted_o_sign_clean_one_call	Poss
concrete_budgeted_o_sign_clean_one_call_loss_from_self_logged_surface		lemma	$P \Rightarrow Q$ is an implication used as a proof obligation.	game
		concrete_budgeted_o_sign_clean_one_call_loss_from_self_logged_surface		
		seed_coins m ctx pk (p :		
		signature -> bool) &m :		
		structural_to_exact_hyperball_paper_sample_loss_obligation		
		haetae_mode =>		
		sampler_rom_covered		
		HAETAE_RO.FRO.m{m}		
		ROMInternalTranscriptBudgetedPaperSimAsNMA.adversary_hash_queries{m}		
		ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler...		
	7733	lemma	concrete_rom_internal_budgeted_nma_ro_signing_attempt_o	Prove
concrete_rom_internal_budgeted_nma_ro_signing_attempt_o_exact_hyperball_clean_conditioned_lifting		lemma	$\Pr[E] \leq B$ is a probability bound on exact game probability.	game
		concrete_rom_internal_budgeted_nma_ro_signing_attempt_o_exact_hyperball_clean_conditioned_lifting		
		&m :		
		structural_to_exact_hyperball_paper_sample_loss_obligation		
		haetae_mode =>		
		Pr[SIG.UF_NMA(HAETAE_RO.FRO,		
		HAETAE,		
		ROMInternalTranscriptBudgetedPaperSimAsNMA		
		(A,		
		ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO))) .main()		
		@ &m : res /\ !		
		ROMInternalTransc...		
	7767	lemma		

	Line	Construct	What was written	Sym
concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_clean_lifting		lemma	concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_clean_lifting	Sign
		concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_clean_lifting	$\Pr[E] \leq B$ is a probability bound from exact game probability.	
		&m :		
		Pr[SIG.UF_NMA(HAETAE_RO.FRO, HAETAE, ROMInternalTranscriptBudgetedPaperSimAsNMA (A, ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO)))].main()		
		@ &m : res /\ !		
		ROMInternalTranscriptBudgetedPaperSimAsNMA.sampler_bad_prequery]		
		<= Pr[SIG.UF_NMA(HAETAE_RO.F...		
	7795	lemma	concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting	Sign
concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting		lemma	concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting	Sign
		concrete_rom_internal_budgeted_nma_ro_signing_attempt_ro_exact_hyperball_loss_from_paper_sample_lifting	$\Pr[E] \leq B$ is a probability bound from exact game probability.	
		&m :		
		structural_to_exact_hyperball_paper_sample_loss_obligation		
		haetae_mode =>		
		Pr[SIG.UF_NMA(HAETAE_RO.FRO, HAETAE, ROMInternalTranscriptBudgetedPaperSimAsNMA (A, ROSigningAttemptPaperSimSampler(HAETAE_RO.FRO)))].main()		
		@ &m : res] <= Pr[SIG.UF_NMA...		
	7828	equivalence	real_signing_rejection_paper_sim_nma_oracle	Show
real_signing_rejection_paper_sim_nma_oracle		equiv	real_signing_rejection_paper_sim_nma_oracle	obs
		real_signing_rejection_paper_sim_nma_oracle	$\mathcal{C}_{\text{prac}} \mathcal{C}_2 [R \Rightarrow S]$ is a relational program equivalence.	relati
		:		
		ROMInternalTranscriptPaperSimAsNMA		
		(A,		
		RealSigningPaperSimSampler(H),		
		H).0.sign ~		
		ROMInternalTranscriptPaperSimAsNMA		
		(A,		
		HAETAERejectionPaperSimSampler(H),		
		H).0.sign : ={glob H, arg} /\		
		ROMInternalTranscriptPaperSimAsNMA.pk_current{1}		
		=		
		ROMInternalTranscriptPaperSimAsNMA.pk_current{2}		
		/\ RealSigningPaper...		
	7886	equivalence		

	Line	Construct	What was written	Sym
		adversary_real_signing_rejection_paper_sim_nma	equiv	adversary_real_signing_rejection_paper_sim
			adversary_real_signing_rejection_paper_sim_nma	adversary_real_signing_rejection_paper_sim
			: A(H,	$C \sim C_m[R \Rightarrow S]$ is a relational program
			ROMInternalTranscriptPaperSimAsNMA	equivalence.
			(A,	
			RealSigningPaperSimSampler(H),	
			H).0).forge ~ A(H,	
			ROMInternalTranscriptPaperSimAsNMA	
			(A,	
			HAETAERejectionPaperSimSampler(H),	
			H).0).forge : ={glob H, glob A,	
			arg} /\	
			ROMInternalTranscriptPaperSimAsNMA.pk_current{1}	
			=	
			ROMInternalTranscriptPaperSimAsNMA.pk_curre...	
	7963	lemma		rom_internal_nma_real_signing_haetae_rejection
		rom_internal_nma_real_signing_haetae_rejection_paper_sim_exact	lemma	rom_internal_nma_real_signing_haetae_rejection
			rom_internal_nma_real_signing_haetae_rejection_paper_sim_exact	Pr[$C \sim E$] = B is a probability bound or
			&m : Pr[SIG.UF_NMA(H, HAETAE,	exact game probability.
			ROMInternalTranscriptPaperSimAsNMA	
			(A,	
			RealSigningPaperSimSampler(H))).main()	
			@ &m : res] = Pr[SIG.UF_NMA(H,	
			HAETAE,	
			ROMInternalTranscriptPaperSimAsNMA	
			(A,	
			HAETAERejectionPaperSimSampler(H))).main()	
			@ &m : res].	
	8039	equivalence		

	Line	Construct	What was written	Sym	
		exact_hyperball_haetae_rejection_paper_sim_nma_oracle	equiv exact_hyperball_haetae_rejection_paper_sim_nma_oracle : ROMInternalTranscriptPaperSimAsNMA (A, ExactHyperballHAETAERejectionPaperSimSampler(H), H).0.sign ~ ROMInternalTranscriptPaperSimAsNMA (A, ExactHyperballPaperSimSampler(H), H).0.sign : ={glob H, arg} /\ ROMInternalTranscriptPaperSimAsNMA.pk_current{1} = ROMInternalTranscriptPaperSimAsNMA.pk_cur...	exact_hyperball_haetae_rejection_paper_sim_nma_oracle is a relational program equivalence.	Show observ relati
	8101	adversary_exact_hyperball_haetae_rejection_paper_sim_nma	equiv adversary_exact_hyperball_haetae_rejection_paper_sim_nma : A(H, ROMInternalTranscriptPaperSimAsNMA (A, ExactHyperballHAETAERejectionPaperSimSampler(H), H).0).forge ~ A(H, ROMInternalTranscriptPaperSimAsNMA (A, ExactHyperballPaperSimSampler(H), H).0).forge : ={glob H, glob A, arg} /\ ROMInternalTranscriptPaperSimAsNMA.pk_current{1} = ROMInternalTrans...	adversary_exact_hyperball_haetae_rejection_paper_sim_nma is a relational program equivalence.	Show observ relati
	8164	rom_internal_nma_exact_hyperball_rejection_paper_sim_no_fallback_exact	lemma rom_internal_nma_exact_hyperball_rejection_paper_sim_no_fallback_exact &m : Pr[SIG.UF_NMA(H, HAETAERejectionPaperSimAsNMA ROMInternalTranscriptPaperSimAsNMA (A, ExactHyperballHAETAERejectionPaperSimSampler(H)))].main() @ &m : res] = Pr[SIG.UF_NMA(H, HAETAERejectionPaperSimAsNMA ROMInternalTranscriptPaperSimAsNMA (A, ExactHyperballPaperSimSampler(H)))].main() @ &m : res].	rom_internal_nma_exact_hyperball_rejection_paper_sim_no_fallback_exact is a probability bound or exact game probability.	Prova simul

	Line	Construct	What was written	Sym
	8241	lemma		
		<pre> public_sim_to_paper_sim_nma_hop_from_sampling_hop lemma public_sim_to_paper_sim_nma_hop_from_sampling_hop &m : Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPublicSimAsNMA(A)).main() @ &m : res] <= Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPaperSimAsNMA(A, Samp)).main() @ &m : res] + rejection_sampling_loss_term => Pr[SIG.UF_NMA(H, HAETAE, ROMInternalTranscriptPublicSimAsNMA(A)).main() @ &m : res] <=...</pre>	<pre> public_sim_to_paper_sim_nma_hop_from_sampling_hop Pr[$\phi \wedge E$] $\leq$ B is a probability bound or exact game probability.</pre>	Recom scrip
	8268	equivalence		
		<pre> transcript_logging_oracle_erasure equiv transcript_logging_oracle_erasure : EUF_CMA_SimulatedSign(H, S, A, Sim).0.sign ~ EUF_CMA_TranscriptSimulatedSign(H, S, A, Sim).0.sign : ={glob H, glob S, glob Sim, arg} /\ EUF_CMA_SimulatedSign.queries{1} = EUF_CMA_TranscriptSimulatedSign.queries{2} ==> ={glob H, glob S, glob Sim, res} /\ EUF_CMA_SimulatedSign.queries{1} = EUF_CMA_TranscriptSimulate...</pre>	<pre> transcript_logging_oracle_erasure : $\mathcal{C}_1 \sim$ $\mathcal{C}_2 [R \Rightarrow S]$ is a relational program equivalence.</pre>	Show obser relati
	8286	equivalence		

	Line	Construct	What was written	Sym
adversary_transcript_logging_erasure		equiv adversary_transcript_logging_erasure : A(H, EUF_CMA_SimulatedSign(H, S, A, Sim).O).forge ~ A(H, EUF_CMA_TranscriptSimulatedSign(H, S, A, Sim).O).forge : ={glob H, glob S, glob A, glob Sim, arg} /\n EUF_CMA_SimulatedSign.queries{1} = EUF_CMA_TranscriptSimulatedSign.queries{2} ==> ={glob H, glob S, glob Sim, res} /\n EUF_CMA_SimulatedSign.queries{1} = E...	adversary_transcript_logging_erasure : $\mathcal{C}_1 \sim \mathcal{C}_2 [R \Rightarrow S]$ is a relational program equivalence.	Show obs relati
transcript_logging_erasure_exact	8309	lemma transcript_logging_erasure_exact &m : Pr[EUF_CMA_SimulatedSign(H, S, A, Sim).main() @ &m : res] = Pr[EUF_CMA_TranscriptSimulatedSign(H, S, A, Sim).main() @ &m : res].	transcript_logging_erasure_exact : Pr[c : E] = B is a probability bound or exact game probability.	Prove simul
real_signing_oracle_equiv	8350	equiv real_signing_oracle_equiv : SIG.EUF_CMA(H, S, A).O.sign ~ EUF_CMA_SimulatedSign(H, S, A, RealSigningSimulator(S)).O.sign : ={glob H, glob S, arg} /\n SIG.EUF_CMA.sk{1} = RealSigningSimulator.sk{2} /\n SIG.EUF_CMA.queries{1} = EUF_CMA_SimulatedSign.queries{2} ==> ={glob H, glob S, res} /\n SIG.EUF_CMA.sk{1} = RealSigningSimulator.sk{2} /\n SIG.EUF_CMA.qu...	real_signing_oracle_equiv : $\mathcal{C}_1 \sim$ $\mathcal{C}_2 [R \Rightarrow S]$ is a relational program equivalence.	Show obs relati
	8369	equiv		

	Line	Construct	What was written	Sym	
		adversary_real_signing_equiv	equiv adversary_real_signing_equiv : A(H, SIG.EUF_CMA(H, S, A).0).forge ~ A(H, EUF_CMA_SimulatedSign(H, S, A, RealSigningSimulator(S)).0).forge :={glob H, glob S, glob A, arg} /\ SIG.EUF_CMA.sk{1} = RealSigningSimulator.sk{2} /\ SIG.EUF_CMA.queries{1} = EUF_CMA_SimulatedSign.queries{2} ==>={glob H, glob S, res} /\ SIG.EUF_CMA.sk{1} = RealSigningSimulato...	adversary_real_signing_equiv : $\mathcal{C}_1 \sim \mathcal{C}_2 [R \Rightarrow S]$ is a relational program equivalence.	Show observed relation
	8393	lemma			
		real_signing_simulator_exact	lemma real_signing_simulator_exact &m : Pr[SIG.EUF_CMA(H, S, A).main() @ &m : res] = Pr[EUF_CMA_SimulatedSign(H, S, A, RealSigningSimulator(S)).main() @ &m : res].	real_signing_simulator_exact : $\Pr[c : E] = B$ is a probability bound or exact game probability.	Prove sampling
	8507	lemma			
		bimodal_special_soundness_lift_exact	lemma bimodal_special_soundness_lift_exact &m md : Pr[BimodalSpecialSoundness_Game(H, S, F, X).main(md) @ &m : res] = Pr[SpecialSoundness_Game(H, S, F, BimodalTranscriptExtractor_As_ModuleSIS(X)).main(md) @ &m : res].	bimodal_special_soundness_lift_exact : $\Pr[c : E] = B$ is a probability bound or exact game probability.	Prove simulation
	8541	operation			

	Line	Construct	What was written	Sym	
		special_soundness_interfaces_ready	op special_soundness_interfaces_ready (md : mode) : bool = fork_public_reconstruction_obligation md /\ forking_extraction_obligation md /\ bimodal_to_module_sis_lift_obligation md.	special_soundness_interfaces_ready ⊆ mode is a Boolean-valued predicate.	Nam used
	8546	lemma	lemma special_soundness_interfaces_ready_from_forking md : forking_extraction_obligation md => special_soundness_interfaces_ready md.	special_soundness_interfaces_ready_from_forking P ⇒ Q is an implication used as a proof obligation.	forking justif
	8559	lemma	lemma special_soundness_interfaces_public_reconstruction md : special_soundness_interfaces_ready md => fork_public_reconstruction_obligation md.	special_soundness_interfaces_public_reconstruction P ⇒ Q is an implication used as a proof obligation.	Public justif
	8568	lemma	lemma special_soundness_interfaces_ready_sound md : special_soundness_interfaces_ready md => special_soundness_obligation md.	special_soundness_interfaces_ready_sound : Prove P ⇒ Q is an implication used as a proof obligation.	Prove justif
	8577	lemma	lemma special_soundness_interfaces_ready_sound_with_public_reconstruction md : special_soundness_interfaces_ready md => special_soundness_with_public_reconstruction_obligation md.	special_soundness_interfaces_ready_sound_with_public_reconstruction P ⇒ Q is an implication used as a proof obligation.	With justif
	8587	operation			

	Line	Construct	What was written	Sym
		<pre> signing_simulator_sound op signing_simulator_sound (md : mode) : bool = forall (pk : pkey) (m : message) (ctx : context) (sig : signature), signing_simulation_relation md pk m ctx sig => verify_internal md pk m ctx sig. </pre>	<pre> signing_simulator_sound $\subseteq$ mode $\times$ pkey $\times$ message $\times$ context $\times$ signature is a Boolean-valued predicate. </pre>	<pre> Nam used </pre>
	8592	<pre> lemma signing_simulator_sound_current md : signing_simulator_sound md. </pre>	<pre> signing_simulator_sound_current : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact. </pre>	<pre> Prove samp </pre>
	8600	<pre> lemma rejection_sampling_bound_from_fs_bound rejection_sampling_bound_from_fs_bound : fs_with_aborts_bound_obligation => rejection_sampling_bound_obligation. </pre>	<pre> rejection_sampling_bound_from_fs_bound : Prove $P \Rightarrow Q$ is an implication used as a proof obligation. </pre>	<pre> Prove game </pre>
	8615	<pre> operation fs_with_aborts_interfaces_ready op fs_with_aborts_interfaces_ready (md : mode) : bool = signing_simulator_sound md /\ special_soundness_interfaces_ready md /\ fs_with_aborts_bound_obligation. </pre>	<pre> fs_with_aborts_interfaces_ready $\subseteq$ mode is a Boolean-valued predicate. </pre>	<pre> Nam used </pre>
	8620	<pre> lemma fs_with_aborts_interfaces_ready_from_parts fs_with_aborts_interfaces_ready_from_parts md : special_soundness_interfaces_ready md => fs_with_aborts_bound_obligation => fs_with_aborts_interfaces_ready md. </pre>	<pre> fs_with_aborts_interfaces_ready_from_parts : Prove $P \Rightarrow Q$ is an implication used as a proof obligation. </pre>	<pre> Reco scrip </pre>
	8634	<pre> lemma </pre>		

	Line	Construct	What was written	Sym
fs_with_aborts_interfaces_ready_from_forking		lemma	fs_with_aborts_interfaces_ready_from_forking	Reco
		fs_with_aborts_interfaces_ready_from_forking	$P \Rightarrow Q$ is an implication used as a proof	scrip
		md :	obligation.	
		forking_extraction_obligation md		
		=>		
		fs_with_aborts_interfaces_ready		
		md.		
	8644	lemma		
fs_with_aborts_interfaces_ready_holds		lemma	fs_with_aborts_interfaces_ready_holds :	Reco
		fs_with_aborts_interfaces_ready_holds	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	scrip
		md :		
		fs_with_aborts_interfaces_ready		
		md.		
	8651	lemma		
structural_to_exact_hyperball_paper_sample_loss_from_obligation		lemma	structural_to_exact_hyperball_paper_sample_loss	Prolos
		structural_to_exact_hyperball_paper_sample_loss	$X \leq Y$ is an arithmetic or probability	game
		(md : mode) :	upper bound.	
		structural_to_exact_hyperball_paper_sample_loss_obligation		
		md => forall (sk : skey) (m :		
		message) (ctx : context) (p :		
		paper_sim_signature_sample ->		
		bool), mu		
		(dsigning_attempt_state md sk m		
		ctx) (fun st => p		
		(paper_sim_sample_from_rejection_attempt		
		st)) <= mu (dexact_hype...		
	8667	lemma		
structural_to_exact_hyperball_paper_sample_loss_from_attempt_loss		lemma	structural_to_exact_hyperball_paper_sample_loss	Prolos
		structural_to_exact_hyperball_paper_sample_loss	$X \leq Y$ is an arithmetic or probability	game
		(md : mode) :	upper bound.	
		structural_to_exact_hyperball_attempt_loss_obligation		
		=> forall (sk : skey) (m :		
		message) (ctx : context) (p :		
		paper_sim_signature_sample ->		
		bool), mu		
		(dsigning_attempt_state md sk m		
		ctx) (fun st => p		
		(paper_sim_sample_from_rejection_attempt		
		st)) <= mu (dexact_hyperball_...		
	8684	lemma		

	Line	Construct	What was written	Sym
structural_to_exact_hyperball_paper_sample_loss_obligation_from_attempt_loss		lemma	structural_to_exact_hyperball_paper_sample_loss_obligation_from_attempt_loss	Prlos
		structural_to_exact_hyperball_paper_sample_loss_obligation_from_attempt_loss	$B \Rightarrow Q$ is an implication used as a proof	game
		(md : mode) :	obligation.	
		structural_to_exact_hyperball_attempt_loss_obligation		
		=>		
		structural_to_exact_hyperball_paper_sample_loss_obligation		
		md.		
	8696	lemma		
structural_to_exact_hyperball_paper_sample_loss_from_coin_sample_pair_loss		lemma	structural_to_exact_hyperball_paper_sample_loss_from_coin_sample_pair_loss	Prlos
		structural_to_exact_hyperball_paper_sample_loss_from_coin_sample_pair_loss	$X \leq Y$ is an arithmetic or probability	game
		(md : mode) :	upper bound.	
		structural_to_exact_hyperball_coin_sample_pair_loss_obligation		
		=> forall (sk : skey) (m :		
		message) (ctx : context) (p :		
		paper_sim_signature_sample ->		
		bool), mu		
		(dsigning_attempt_state md sk m		
		ctx) (fun st => p		
		(paper_sim_sample_from_rejection_attempt		
		st)) <= mu...		
	8712	lemma		
structural_to_exact_hyperball_paper_sample_loss_obligation_from_coin_sample_pair_loss		lemma	structural_to_exact_hyperball_paper_sample_loss_obligation_from_coin_sample_pair_loss	Prlos
		structural_to_exact_hyperball_paper_sample_loss_obligation_from_coin_sample_pair_loss	$B \Rightarrow Q$ is an implication used as a proof	game
		(md : mode) :	obligation.	
		structural_to_exact_hyperball_coin_sample_pair_loss_obligation		
		=>		
		structural_to_exact_hyperball_paper_sample_loss_obligation		
		md.		
	8724	lemma		
dexact_hyperball_reference_paper_sim_attempt_boundary_ready		lemma	dexact_hyperball_reference_paper_sim_attempt_boundary_ready	Prlos
		dexact_hyperball_reference_paper_sim_attempt_boundary_ready	$L \leq R$ is an inequality / rewriting fact.	game
		(md : mode) (sk : skey) (m :		
		message) (ctx : context) : mu		
		(dexact_hyperball_signing_attempt_state		
		md sk m ctx)		
		(signing_attempt_state_reference_rejection_aborts		
		md) = 0%r.		
	8732	lemma		

	Line	Construct	What was written	Sym
dexact_hyperball_full_rejection_paper_sim_attempt_boundary_ready		lemma	dexact_hyperball_full_rejection_paper_sim_attempt_boundary_ready	Att
		dexact_hyperball_full_rejection_paper_sim_attempt_boundary_ready	$I \equiv R$ is an equality/re-writing fact.	game
		(md : mode) (sk : skey) (m : message) (ctx : context) : mu		
		(dexact_hyperball_signing_attempt_state md sk m ctx)		
		(signing_attempt_state_rejection_aborts md (public_key_of_secret md sk) m ctx) = 0%r.		
	8741	lemma	dexact_hyperball_haetae_rejection_sample_no_fallback_mu	Prov
dexact_hyperball_haetae_rejection_sample_no_fallback_mu		lemma	$B \Rightarrow Q$ is an implication used as a proof obligation.	sample
		dexact_hyperball_haetae_rejection_sample_no_fallback_mu		
		(md : mode) (sk : skey) (m : message) (ctx : context) (p : paper_sim_signature_sample -> bool) : mu		
		(dexact_hyperball_signing_attempt_state md sk m ctx) (fun st => p (haetae_rejection_sample_from_attempt md (public_key_of_secret md sk) m ctx st)) = mu		
		(dexact_hyperball_signing_attempt_state md...		
	8758	lemma	dexact_hyperball_haetae_rejection_sample_signature_no_fallback_mu	Prov
dexact_hyperball_haetae_rejection_sample_signature_no_fallback_mu		lemma	$B \Rightarrow Q$ is an implication used as a proof obligation.	sample
		dexact_hyperball_haetae_rejection_sample_signature_no_fallback_mu		
		(md : mode) (sk : skey) (m : message) (ctx : context) (p : signature -> bool) : mu		
		(dexact_hyperball_signing_attempt_state md sk m ctx) (fun st => p (paper_sim_signature md (public_key_of_secret md sk) m ctx (haetae_rejection_sample_from_attempt md (public_key_of_secret md sk) m ctx s...		
	8777	lemma		

Line	Construct	What was written	Sym
dexact_hyperball_paper_sim_rejection_attempt_signature_mu	<pre> lemma dexact_hyperball_paper_sim_rejection_attempt_signature_mu (md : mode) (sk : skey) (m : message) (ctx : context) (p : signature -> bool) : mu (dexact_hyperball_signing_attempt_state md sk m ctx) (fun st => p (paper_sim_signature md (public_key_of_secret md sk) m ctx (paper_sim_sample_from_rejection_attempt st))) = mu (dexact_hyperball_signing_attempt...</pre>	dexact_hyperball_paper_sim_rejection_attempt_signature_mu $P \Rightarrow Q$ is an implication used as a proof obligation.	Provable sample
8800	lemma		
dexact_hyperball_haetae_rejection_sample_signature_mu	<pre> lemma dexact_hyperball_haetae_rejection_sample_signature_mu (md : mode) (sk : skey) (m : message) (ctx : context) (p : signature -> bool) : mu (dexact_hyperball_signing_attempt_state md sk m ctx) (fun st => p (paper_sim_signature md (public_key_of_secret md sk) m ctx (haetae_rejection_sample_from_attempt md (public_key_of_secret md sk) m ctx st))) = mu (d...</pre>	dexact_hyperball_haetae_rejection_sample_signature_mu $P \Rightarrow Q$ is an implication used as a proof obligation.	Provable sample

6 HAETAE_Reductions.ec

arithmetic assembly of concrete loss terms

	Line	Construct	What was written	Symbolic correspondence
	9	operation		
mlwe_hardness_term		op mlwe_hardness_term : real.	mlwe_hardness_term : $1 \rightarrow \mathbb{R}$	Defines a total mathematical function by the EasyCrypt model.
	10	operation		
module_sis_hardness_term		op module_sis_hardness_term : real.	module_sis_hardness_term : $1 \rightarrow \mathbb{R}$	Defines a total mathematical function by the EasyCrypt model.
	11	operation		
bimodal_to_msis_reduction_loss_term		op bimodal_to_msis_reduction_loss_term : real = 0%r.	bimodal_to_msis_reduction_loss_term : $1 \rightarrow \mathbb{R}$	Defines a concrete numeric loss term in the final security inequality.
	13	operation		
bimodal_selftarget_msis_hardness_term		op bimodal_selftarget_msis_hardness_term : Bool = module_sis_hardness_term + bimodal_to_msis_reduction_loss_term.	bimodal_selftarget_msis_hardness_term : $1 \rightarrow \text{Bool}$	Defines a total mathematical function by the EasyCrypt model.
	16	operation		
signature_query_count		op signature_query_count : real = 2%r.	signature_query_count : $1 \rightarrow \mathbb{R}$	Defines the random-oracle/hash interaction data used by the games.
	17	operation		
hash_query_count		op hash_query_count : real = 2%r.	hash_query_count : $1 \rightarrow \mathbb{R}$	Defines the random-oracle/hash interaction data used by the games.
	18	operation		
signature_query_budget_count		op signature_query_budget_count : int = 2.	signature_query_budget_count : $1 \rightarrow \mathbb{Z}$	Defines the random-oracle/hash interaction data used by the games.
	19	operation		
hash_query_budget_count		op hash_query_budget_count : int = 2.	hash_query_budget_count : $1 \rightarrow \mathbb{Z}$	Defines the random-oracle/hash interaction data used by the games.
	20	operation		
abort_probability		op abort_probability : real = 0%r.	abort_probability : $1 \rightarrow \mathbb{R}$	Defines a total mathematical function by the EasyCrypt model.
	21	operation		
min_entropy_loss_factor		op min_entropy_loss_factor : real = 2%r.	min_entropy_loss_factor : $1 \rightarrow \mathbb{R}$	Defines a concrete numeric loss term in the final security inequality.
	22	operation		
challenge_support_bits_lower_bound		op challenge_support_bits_lower_bound : int = 58.	challenge_support_bits_lower_bound : $1 \rightarrow \mathbb{Z}$	Defines a concrete numeric loss term in the final security inequality.
	23	operation		

	Line	Construct	What was written	Symbolic correspondence
challenge_support_cardinality_lower_bound		op challenge_support_cardinality_lower_bound : real = 288230376151711744%r.	challenge_support_cardinality_lower_bound : $\mathbb{R}$	Defines a concrete numeric loss term in the final security inequality.
	26	operation		
rom_hash_query_budget		op rom_hash_query_budget : real = hash_query_count.	rom_hash_query_budget : $1 \rightarrow \mathbb{R}$	Defines the random-oracle/hash inter data used by the games.
	27	operation		
rom_signature_query_budget		op rom_signature_query_budget : real = signature_query_count.	rom_signature_query_budget : $1 \rightarrow \mathbb{R}$	Defines the random-oracle/hash inter data used by the games.
	28	operation		
rom_total_query_budget		op rom_total_query_budget = rom_hash_query_budget + rom_signature_query_budget + 1%r.	rom_total_query_budget : $1 \rightarrow \text{Bool}$	Defines the random-oracle/hash inter data used by the games.
	31	operation		
counted_rom_collision_term		op counted_rom_collision_term = (rom_total_query_budget * rom_total_query_budget) / challenge_support_cardinality_lower_bound.	counted_rom_collision_term : $1 \rightarrow \text{Bool}$	Defines the random-oracle/hash inter data used by the games.
	35	operation		
counted_rom_prequery_term		op counted_rom_prequery_term = (rom_signature_query_budget * (rom_hash_query_budget + 1%r)) / challenge_support_cardinality_lower_bound.	counted_rom_prequery_term : $1 \rightarrow \text{Bool}$	Defines the random-oracle/hash inter data used by the games.
	39	operation		
counted_rom_min_entropy_term		op counted_rom_min_entropy_term = (rom_signature_query_budget * (rom_hash_query_budget + 1%r)) / challenge_support_cardinality_lower_bound.	counted_rom_min_entropy_term : $1 \rightarrow \text{Bool}$	Defines the random-oracle/hash inter data used by the games.
	43	operation		
counted_rom_prequery_reprogramming_term		op counted_rom_prequery_reprogramming_term = counted_rom_collision_term + counted_rom_prequery_term.	counted_rom_prequery_reprogramming_term : $1 \rightarrow \text{Bool}$	Defines the random-oracle/hash inter data used by the games.
	46	operation		
counted_rom_programming_loss_term		op counted_rom_programming_loss_term = counted_rom_collision_term + counted_rom_prequery_term + counted_rom_min_entropy_term.	counted_rom_programming_loss_term : $1 \rightarrow \text{Bool}$	Defines a concrete numeric loss term in the final security inequality.
	51	operation		

	Line	Construct	What was written	Symbolic correspondence
		signing_query_scale op signing_query_scale = signature_query_count / (1%r - abort_probability).	signing_query_scale : 1 → Bool	Defines the random-oracle/hash interaction data used by the games.
	54	operation		
		fs_with_aborts_reprogramming_term op fs_with_aborts_reprogramming_term = sqrt signing_query_scale * (hash_query_count + 1%r + sqrt signing_query_scale).	fs_with_aborts_reprogramming_term : 1 → Bool	Defines a total mathematical function by the EasyCrypt model.
	58	operation		
		fs_with_aborts_min_entropy_term op fs_with_aborts_min_entropy_term = min_entropy_loss_factor * (signing_query_scale * (hash_query_count + 1%r)).	fs_with_aborts_min_entropy_term : 1 → Bool	Defines a total mathematical function by the EasyCrypt model.
	62	operation		
		rejection_sampling_loss_term op rejection_sampling_loss_term = fs_with_aborts_reprogramming_term + fs_with_aborts_min_entropy_term.	rejection_sampling_loss_term : 1 → Bool	Defines a concrete numeric loss term in the final security inequality.
	65	operation		
		haetae_euf_bound op haetae_euf_bound = mlwe_hardness_term + bimodal_selftarget_msis_hardness_term + rejection_sampling_loss_term.	haetae_euf_bound : 1 → Bool	Defines a concrete numeric loss term in the final security inequality.
	70	operation		
		haetae_euf_counted_rom_bound op haetae_euf_counted_rom_bound = mlwe_hardness_term + bimodal_selftarget_msis_hardness_term + counted_rom_programming_loss_term.	haetae_euf_counted_rom_bound : 1 → Bool	Defines a concrete numeric loss term in the final security inequality.
	75	lemma		
		haetae_euf_boundE lemma haetae_euf_boundE : haetae_euf_bound = mlwe_hardness_term + module_sis_hardness_term + bimodal_to_msis_reduction_loss_term + fs_with_aborts_reprogramming_term + fs_with_aborts_min_entropy_term.	haetae_euf_boundE : $L = R$ is an equality/rewriting fact.	Proves a bound that contributes to a game-hop or final security inequality.

	Line	Construct	What was written	Symbolic correspondence
	87	lemma		
haetae_euf_bound_groupedE		<pre> lemma haetae_euf_bound_groupedE : haetae_euf_bound = (mlwe_hardness_term + (module_sis_hardness_term + bimodal_to_msis_reduction_loss_term)) + (fs_with_aborts_reprogramming_term + fs_with_aborts_min_entropy_term).</pre>	haetae_euf_bound_groupedE : $L = R$ is an equality/rewriting fact.	Proves a bound that contributes to a game-hop or final security inequality.
	98	lemma		
haetae_euf_bound_rejection_groupedE		<pre> lemma haetae_euf_bound_rejection_groupedE : haetae_euf_bound = (mlwe_hardness_term + (module_sis_hardness_term + bimodal_to_msis_reduction_loss_term)) + rejection_sampling_loss_term.</pre>	haetae_euf_bound_rejection_groupedE : $L = R$ is an equality/rewriting fact.	Proves a bound that contributes to a game-hop or final security inequality.
	107	lemma		
haetae_euf_counted_rom_bound_groupedE		<pre> lemma haetae_euf_counted_rom_bound_groupedE : haetae_euf_counted_rom_bound = (mlwe_hardness_term + (module_sis_hardness_term + bimodal_to_msis_reduction_loss_term)) + counted_rom_programming_loss_term.</pre>	haetae_euf_counted_rom_bound_groupedE : $L = R$ is an equality/rewriting fact.	Proves a bound that contributes to a game-hop or final security inequality.
	117	lemma		
signing_query_scaleE		<pre> lemma signing_query_scaleE : signing_query_scale = signature_query_count.</pre>	signing_query_scaleE : $L = R$ is an equality/rewriting fact.	Proves a bound that contributes to a game-hop or final security inequality.
	123	lemma		
signing_query_scale_nonnegative		<pre> lemma signing_query_scale_nonnegative : 0%r <= signing_query_scale.</pre>	signing_query_scale_nonnegative : $X \leq Y$ is an arithmetic or probability upper bound.	Proves a bound that contributes to a game-hop or final security inequality.
	129	lemma		
fs_with_aborts_reprogramming_term_nonnegative		<pre> lemma fs_with_aborts_reprogramming_term_nonnegative : 0%r <= fs_with_aborts_reprogramming_term.</pre>	fs_with_aborts_reprogramming_term_nonnegative : $X \leq Y$ is an arithmetic or probability upper bound.	Proves a reusable theorem so later proofs can rewrite, apply, or compose
	140	lemma		

	Line	Construct	What was written	Symbolic correspondence
fs_with_aborts_min_entropy_term_nonnegative		lemma fs_with_aborts_min_entropy_term_nonnegative : 0%r <= fs_with_aborts_min_entropy_term. 151 lemma	fs_with_aborts_min_entropy_term_nonnegative : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
rom_hash_query_budget_nonnegative		lemma rom_hash_query_budget_nonnegative : 0%r <= rom_hash_query_budget. 155 lemma	rom_hash_query_budget_nonnegative : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
rom_signature_query_budget_nonnegative		lemma rom_signature_query_budget_nonnegative : 0%r <= rom_signature_query_budget. 159 lemma	rom_signature_query_budget_nonnegative : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
hash_query_budget_countE		lemma hash_query_budget_countE : hash_query_count = (hash_query_budget_count)%r. 163 lemma	hash_query_budget_countE : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
signature_query_budget_countE		lemma signature_query_budget_countE : signature_query_count = (signature_query_budget_count)%r. 167 lemma	signature_query_budget_countE : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
hash_query_budget_count_nonnegative		lemma hash_query_budget_count_nonnegative : 0 <= hash_query_budget_count. 171 lemma	hash_query_budget_count_nonnegative : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
signature_query_budget_count_nonnegative		lemma signature_query_budget_count_nonnegative : 0 <= signature_query_budget_count. 175 lemma	signature_query_budget_count_nonnegative : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
min_entropy_loss_factor_nonnegative		lemma min_entropy_loss_factor_nonnegative : 0%r <= min_entropy_loss_factor. 179 lemma	min_entropy_loss_factor_nonnegative : $X \leq Y$ is an arithmetic or probability upper bound.	Proves a bound that contributes to a game-hop or final security inequality.
challenge_support_cardinality_lower_bound_positive		lemma challenge_support_cardinality_lower_bound_positive : 0%r < challenge_support_cardinality_lower_bound. 183 lemma	challenge_support_cardinality_lower_bound : $\forall x. P(x)$ is a universally quantified fact.	Positive: distribution fact needed for sampling or probability mass reasoning

	Line	Construct	What was written	Symbolic correspondence
challenge_support_cardinality_lower_bound_nonnegative	183	lemma lemma challenge_support_cardinality_lower_bound_nonnegative : 0%r <= challenge_support_cardinality_lower_bound.	challenge_support_cardinality_lower_bound_nonnegative : $X \leq Y$ is an arithmetic or probability upper bound.	Nonnegative distribution fact needed for sampling or probability mass reasoning
rom_total_query_budget_nonnegative	189	lemma lemma rom_total_query_budget_nonnegative : 0%r <= rom_total_query_budget.	rom_total_query_budget_nonnegative : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later proofs can rewrite, apply, or compose
counted_rom_collision_term_nonnegative	200	lemma lemma counted_rom_collision_term_nonnegative : 0%r <= counted_rom_collision_term.	counted_rom_collision_term_nonnegative : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later proofs can rewrite, apply, or compose
counted_rom_prequery_term_nonnegative	209	lemma lemma counted_rom_prequery_term_nonnegative : 0%r <= counted_rom_prequery_term.	counted_rom_prequery_term_nonnegative : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later proofs can rewrite, apply, or compose
counted_rom_min_entropy_term_nonnegative	220	lemma lemma counted_rom_min_entropy_term_nonnegative : 0%r <= counted_rom_min_entropy_term.	counted_rom_min_entropy_term_nonnegative : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later proofs can rewrite, apply, or compose
counted_rom_prequery_reprogramming_term_nonnegative	231	lemma lemma counted_rom_prequery_reprogramming_term_nonnegative : 0%r <= counted_rom_prequery_reprogramming_term.	counted_rom_prequery_reprogramming_term_nonnegative : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later proofs can rewrite, apply, or compose
counted_rom_programming_loss_term_nonnegative	240	lemma lemma counted_rom_programming_loss_term_nonnegative : 0%r <= counted_rom_programming_loss_term.	counted_rom_programming_loss_term_nonnegative : $X \leq Y$ is an arithmetic or probability upper bound.	Negative a bound that contributes to a game-hop or final security inequality.
counted_rom_collision_termE	251	lemma lemma counted_rom_collision_termE : counted_rom_collision_term = 25%r / challenge_support_cardinality_lower_bound.	counted_rom_collision_termE : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later proofs can rewrite, apply, or compose

	Line	Construct	What was written	Symbolic correspondence
	263	lemma		
counted_rom_prequery_termE		lemma counted_rom_prequery_termE : counted_rom_prequery_term = 6%r / challenge_support_cardinality_lower_bound.	counted_rom_prequery_termE : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	274	lemma		
counted_rom_min_entropy_termE		lemma counted_rom_min_entropy_termE : counted_rom_min_entropy_term = 6%r / challenge_support_cardinality_lower_bound.	counted_rom_min_entropy_termE : $L =$ R is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	285	lemma		
counted_rom_prequery_reprogramming_termE		lemma counted_rom_prequery_reprogramming_termE : counted_rom_prequery_reprogramming_term = 31%r / challenge_support_cardinality_lower_bound.	counted_rom_prequery_reprogramming_termE : $L =$ R is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	294	lemma		
counted_rom_programming_loss_termE		lemma counted_rom_programming_loss_termE : counted_rom_programming_loss_term = 37%r / challenge_support_cardinality_lower_bound.	counted_rom_programming_loss_termE : $L = R$ is an equality/rewriting fact.	Proves a bound that contributes to a game-hop or final security inequality.
	304	lemma		
counted_rom_programming_loss_term_subunit		lemma counted_rom_programming_loss_term_subunit : counted_rom_programming_loss_term < 1%r.	counted_rom_programming_loss_term_subunit : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.
	313	lemma		
counted_rom_programming_loss_term_positive		lemma counted_rom_programming_loss_term_positive : 0%r < counted_rom_programming_loss_term.	counted_rom_programming_loss_term_positive : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.
	322	lemma		
counted_rom_programming_loss_term_le1		lemma counted_rom_programming_loss_term_le1 : 1%r >= counted_rom_programming_loss_term.	counted_rom_programming_loss_term_le1 : $L = R$ is an equality/rewriting fact.	Proves a bound that contributes to a game-hop or final security inequality.

	Line	Construct	What was written	Symbolic correspondence
	328	lemma		
counted_rom_prequery_reprogramming_budget_numeratorE		lemma counted_rom_prequery_reprogramming_budget_numeratorE : (rom_total_query_budget * rom_total_query_budget) + (rom_signature_query_budget * (rom_hash_query_budget + 1%r)) = 31%r.	counted_rom_prequery_reprogramming_budget_numeratorE : L = R is an equality/rewriting fact.	Record a reusable theorem so later p- scripts can rewrite, apply, or compose
	340	lemma		
counted_rom_budget_numeratorE		lemma counted_rom_budget_numeratorE : (rom_total_query_budget * rom_total_query_budget) + (rom_signature_query_budget * (rom_hash_query_budget + 1%r)) + (rom_signature_query_budget * (rom_hash_query_budget + 1%r)) = 37%r.	counted_rom_budget_numeratorE : L = R is an equality/rewriting fact.	Records a reusable theorem so later p- scripts can rewrite, apply, or compose
	353	lemma		
counted_rom_programming_loss_term_cardinalityE		lemma counted_rom_programming_loss_term_cardinalityE : counted_rom_programming_loss_term = ((rom_total_query_budget * rom_total_query_budget) + (rom_signature_query_budget * (rom_hash_query_budget + 1%r)) + (rom_signature_query_budget * (rom_hash_query_budget + 1%r))) / challenge_support_cardinality_lower_bound.	counted_rom_programming_loss_term_cardinalityE : L = R is an equality/rewriting fact.	Record a bound that contributes to a game-hop or final security inequality.
	364	lemma		
counted_rom_prequery_reprogramming_term_cardinalityE		lemma counted_rom_prequery_reprogramming_term_cardinalityE : counted_rom_prequery_reprogramming_term = ((rom_total_query_budget * rom_total_query_budget) + (rom_signature_query_budget * (rom_hash_query_budget + 1%r))) / challenge_support_cardinality_lower_bound.	counted_rom_prequery_reprogramming_term_cardinalityE : L = R is an equality/rewriting fact.	Record a reusable theorem so later p- scripts can rewrite, apply, or compose

	Line	Construct	What was written	Symbolic correspondence
	374	lemma		
counted_rom_programming_loss_term_concreteE		lemma	counted_rom_programming_loss_term_concreteE	Proves a bound that contributes to a game-hop or final security inequality.
		counted_rom_programming_loss_term_concreteE	$L = R$ is an equality/rewriting fact.	
		:		
		counted_rom_programming_loss_term		
		= 37%r / 288230376151711744%r.		
	382	lemma		
counted_rom_prequery_reprogramming_term_concreteE		lemma	counted_rom_prequery_reprogramming_term_concreteE	Reusable theorem so later p
		counted_rom_prequery_reprogramming_term_concreteE	$L = R$ is an equality/rewriting fact.	scripts can rewrite, apply, or compose
		:		
		counted_rom_prequery_reprogramming_term		
		= 31%r / 288230376151711744%r.		
	390	lemma		
counted_rom_programming_loss_term_checked_subunit		lemma	counted_rom_programming_loss_term_checked_subunit	Proves a bound that contributes to a game-hop or final security inequality.
		counted_rom_programming_loss_term_checked_subunit	$L = R$ is an equality/rewriting fact.	
		: 0%r <		
		counted_rom_programming_loss_term		
		/\		
		counted_rom_programming_loss_term		
		< 1%r /\		
		counted_rom_programming_loss_term		
		= 37%r / 288230376151711744%r.		
	403	lemma		
counted_rom_support_lower_bound_summary		lemma	counted_rom_support_lower_bound_summary	Proves a distribution fact needed for sampling or probability mass reasoning
		counted_rom_support_lower_bound_summary	$L = R$ is an equality/rewriting fact.	
		:		
		challenge_support_bits_lower_bound		
		= 58 /\		
		challenge_support_cardinality_lower_bound		
		= 288230376151711744%r.		
	413	lemma		
counted_rom_support_dominates_budget		lemma	counted_rom_support_dominates_budget	Proves a distribution fact needed for sampling or probability mass reasoning
		counted_rom_support_dominates_budget	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	
		: 37%r <		
		challenge_support_cardinality_lower_bound.		
	417	lemma		
counted_rom_prequery_reprogramming_support_dominates_budget		lemma	counted_rom_prequery_reprogramming_support_dominates_budget	Proves a distribution fact needed for sampling or probability mass reasoning
		counted_rom_prequery_reprogramming_support_dominates_budget	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	
		: 31%r <		
		challenge_support_cardinality_lower_bound.		
	421	lemma		

	Line	Construct	What was written	Symbolic correspondence
counted_rom_prequery_reprogramming_term_subunit		lemma	counted_rom_prequery_reprogramming_term	Proves a bound that contributes to a
		counted_rom_prequery_reprogramming_term_subunit	$\forall \vec{r}. B(\vec{r})$ is a universally quantified fact.	game-hop or final security inequality.
		:		
		counted_rom_prequery_reprogramming_term		
		< 1%r.		
	430	lemma		
counted_rom_prequery_reprogramming_from_query_budgets_and_support		lemma	counted_rom_prequery_reprogramming_from_query_budgets_and_support	Proves a distribution fact needed for
		counted_rom_prequery_reprogramming_from_query_budgets_and_support	$L = R$ is an equality/rewriting fact.	sampling or probability mass reasoning
		:		
		counted_rom_prequery_reprogramming_term		
		= ((rom_total_query_budget *		
		rom_total_query_budget) +		
		(rom_signature_query_budget *		
		(rom_hash_query_budget + 1%r)))		
		/		
		challenge_support_cardinality_lower_bound		
		/\ ((rom_total_query_budget *		
		rom_total_query_budget) +		
		(rom_signature_query_budge...		
	445	lemma		
counted_rom_programming_loss_splitE		lemma	counted_rom_programming_loss_splitE :	Proves a bound that contributes to a
		counted_rom_programming_loss_splitE	$L = R$ is an equality/rewriting fact.	game-hop or final security inequality.
		:		
		counted_rom_programming_loss_term		
		=		
		counted_rom_prequery_reprogramming_term		
		+ counted_rom_min_entropy_term.		
	454	lemma		
counted_rom_loss_from_query_budgets_and_support		lemma	counted_rom_loss_from_query_budgets_and_support	Proves a distribution fact needed for
		counted_rom_loss_from_query_budgets_and_support	$L = R$ is an equality/rewriting fact.	sampling or probability mass reasoning
		:		
		counted_rom_programming_loss_term		
		= ((rom_total_query_budget *		
		rom_total_query_budget) +		
		(rom_signature_query_budget *		
		(rom_hash_query_budget + 1%r)) +		
		(rom_signature_query_budget *		
		(rom_hash_query_budget + 1%r)))		
		/		
		challenge_support_cardinality_lower_bound		
		/\ ((rom_total_query_budget *		
		rom_total_que...		

	Line	Construct	What was written	Symbolic correspondence
rejection_sampling_loss_term_nonnegative	471	lemma lemma rejection_sampling_loss_term_nonnegative : 0%r <= rejection_sampling_loss_term.	rejection_sampling_loss_term_nonnegative : $X \leq Y$ is an arithmetic or probability upper bound.	Proves a bound that contributes to a game-hop or final security inequality.
fs_with_aborts_min_entropy_termE	479	lemma lemma fs_with_aborts_min_entropy_termE : fs_with_aborts_min_entropy_term = 12%r.	fs_with_aborts_min_entropy_termE : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
fs_with_aborts_min_entropy_term_ge1	486	lemma lemma fs_with_aborts_min_entropy_term_ge1 : 1%r <= fs_with_aborts_min_entropy_term.	fs_with_aborts_min_entropy_term_ge1 : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
rejection_sampling_loss_term_ge1	492	lemma lemma rejection_sampling_loss_term_ge1 : 1%r <= rejection_sampling_loss_term.	rejection_sampling_loss_term_ge1 : $X \leq Y$ is an arithmetic or probability upper bound.	Proves a bound that contributes to a game-hop or final security inequality.
bimodal_to_msis_reduction_loss_termE	500	lemma lemma bimodal_to_msis_reduction_loss_termE : bimodal_to_msis_reduction_loss_term = 0%r.	bimodal_to_msis_reduction_loss_termE : $L = R$ is an equality/rewriting fact.	Proves a bound that contributes to a game-hop or final security inequality.

7 HAETAETAE_ROM.ec

typed random-oracle query and response model

	Line	Construct	What was written	Symbolic correspondence	Why
	10	type			
	ro_query	<pre> type ro_query = [MessageHashQuery of (pkey * context * message) ChallengeHashQuery of (mode * polyveck * poly * crh) MatrixExpandQuery of (mode * seed) SamplerExpandQuery of random_coins]. </pre>	$\text{ro_query} \equiv$ $[\text{MessageHashQueryof}(\text{pkey} \times \text{context} \times \text{message}) \text{ChallengeHashQueryof}(\text{mode} \times \text{polyveck} \times \text{poly} \times \text{crh}) \text{MatrixExpandQueryof}(\text{mode} \times \text{seed}) \text{SamplerExpandQueryof}(\text{random_coins})]$	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.	
	15	type			
	ro_output	<pre> type ro_output = crh * challenge * matrix * random_coins. </pre>	$\text{ro_output} \equiv$ $\text{crh} \times \text{challenge} \times \text{matrix} \times \text{random_coins}$	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.	
	17	operation			
	message_hash_query	<pre> op message_hash_query (pk : pkey) (ctx : context) (m : message) : ro_query = MessageHashQuery (pk, ctx, m). </pre>	$\text{message_hash_query} :$ $\text{pkey} \times \text{context} \times \text{message} \rightarrow \text{ro_query}$	Defines the random-oracle/hash interface data used by the games.	
	19	operation			
	challenge_hash_query	<pre> op challenge_hash_query (md : mode) (w1 : polyveck) (w0 : poly) (mu : crh) : ro_query = ChallengeHashQuery (md, w1, w0, mu). </pre>	$\text{challenge_hash_query} :$ $\text{mode} \times \text{polyveck} \times \text{poly} \times \text{crh} \rightarrow \text{ro_query}$	Defines the random-oracle/hash interface data used by the games.	
	22	operation			
	matrix_expand_query	<pre> op matrix_expand_query (md : mode) (sd : seed) : ro_query = MatrixExpandQuery (md, sd). </pre>	$\text{matrix_expand_query} : \text{mode} \times \text{seed} \rightarrow$ ro_query	Defines the random-oracle/hash interface data used by the games.	
	24	operation			
	sampler_expand_query	<pre> op sampler_expand_query (coins : random_coins) : ro_query = SamplerExpandQuery coins. </pre>	$\text{sampler_expand_query} : \text{random_coins} \rightarrow$ ro_query	Defines the random-oracle/hash interface data used by the games.	
	27	operation			
	ro_crh_zero	<pre> op ro_crh_zero : crh = nseq crhbytes 0. </pre>	$\text{ro_crh_zero} : 1 \rightarrow \text{crh}$	Defines a total mathematical function used by the EasyCrypt model.	
	28	operation			
	ro_challenge_zero	<pre> op ro_challenge_zero : challenge = poly_zero. </pre>	$\text{ro_challenge_zero} : 1 \rightarrow \text{challenge}$	Defines a total mathematical function used by the EasyCrypt model.	

Line	Construct	What was written	Symbolic correspondence	Why
29	operation ro_matrix_zero	op ro_matrix_zero : matrix = [].	ro_matrix_zero : 1 → matrix	Defines a total mathematical function used by the EasyCrypt model.
30	operation ro_random_coins_zero	op ro_random_coins_zero : random_coins = nseq seedbytes 0.	ro_random_coins_zero : 1 → random_coins	Defines a total mathematical function used by the EasyCrypt model.
31	operation ro_output_zero	op ro_output_zero : ro_output = (ro_crh_zero, ro_challenge_zero, ro_matrix_zero, ro_random_coins_zero).	ro_output_zero : 1 → ro_output	Defines a total mathematical function used by the EasyCrypt model.
34	operation ro_output_to_crh	op ro_output_to_crh (y : ro_output) : crh = y.'1.	ro_output_to_crh : ro_output → crh	Defines a total mathematical function used by the EasyCrypt model.
35	operation ro_output_to_challenge	op ro_output_to_challenge (y : ro_output) : challenge = y.'2.	ro_output_to_challenge : ro_output → challenge	Defines a total mathematical function used by the EasyCrypt model.
36	operation ro_output_to_matrix	op ro_output_to_matrix (y : ro_output) : matrix = y.'3.	ro_output_to_matrix : ro_output → matrix	Defines a total mathematical function used by the EasyCrypt model.
37	operation ro_output_to_random_coins	op ro_output_to_random_coins (y : ro_output) : random_coins = y.'4.	ro_output_to_random_coins : ro_output → random_coins	Defines a total mathematical function used by the EasyCrypt model.
39	operation ro_output_of_crh	op ro_output_of_crh (mu : crh) : ro_output = (mu, ro_challenge_zero, ro_matrix_zero, ro_random_coins_zero).	ro_output_of_crh : crh → ro_output	Defines a total mathematical function used by the EasyCrypt model.
41	operation ro_output_of_challenge	op ro_output_of_challenge (ch : challenge) : ro_output = (ro_crh_zero, ch, ro_matrix_zero, ro_random_coins_zero).	ro_output_of_challenge : challenge → ro_output	Defines a total mathematical function used by the EasyCrypt model.
43	operation ro_output_of_matrix	op ro_output_of_matrix (a : matrix) : ro_output = (ro_crh_zero, ro_challenge_zero, a, ro_random_coins_zero).	ro_output_of_matrix : matrix → ro_output	Defines a total mathematical function used by the EasyCrypt model.

	Line	Construct	What was written	Symbolic correspondence	Why
	45	operation			
ro_output_of_random_coins		<pre> op ro_output_of_random_coins (coins : random_coins) : ro_output = (ro_crh_zero, ro_challenge_zero, ro_matrix_zero, coins).</pre>	<pre> ro_output_of_random_coins : random_coins → ro_output</pre>	Defines a total mathematical function used by the EasyCrypt model.	
	48	operation			
dmatrix_for_query		<pre> op dmatrix_for_query (x : mode * seed) : matrix distr = dmatrix x.'1.</pre>	<pre> dmatrix_for_query : mode × seed → Distr(matrix)</pre>	Defines a sampler/distribution used by game procedures and probability bounds.	
	50	operation			
dro_output		<pre> op dro_output (q : ro_query) : ro_output distr = with q = MessageHashQuery _ => dmap dcrh ro_output_of_crh with q = ChallengeHashQuery x => dmap (dchallenge x.'1) ro_output_of_challenge with q = MatrixExpandQuery x => dmap (dmatrix_for_query x) ro_output_of_matrix with q = SamplerExpandQuery _ => dmap drandom_coins ro_output_of_random_coins.</pre>	<pre> dro_output : ro_query → Distr(ro_output)</pre>	Defines a sampler/distribution used by game procedures and probability bounds.	
	59	lemma			
matrix_query_distribution_lossless		<pre> lemma matrix_query_distribution_lossless x : is_lossless (dmatrix_for_query x).</pre>	<pre> matrix_query_distribution_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.</pre>	Proves a distribution fact needed for sampling or probability mass reasoning.	
	67	lemma			
ro_output_distribution_lossless		<pre> lemma ro_output_distribution_lossless q : is_lossless (dro_output q).</pre>	<pre> ro_output_distribution_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.</pre>	Proves a distribution fact needed for sampling or probability mass reasoning.	
	81	type			
in_t		<pre> type in_t <- ro_query, type out_t <- ro_output, op dout <- dro_output.</pre>	in_t is an abstract carrier set.	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.	
	93	operation			
ro_message_hash		<pre> op ro_message_hash (y : ro_output) : crh = ro_output_to_crh y.</pre>	<pre> ro_message_hash : ro_output → crh</pre>	Defines the random-oracle/hash interface data used by the games.	

	Line	Construct	What was written	Symbolic correspondence	Why
	94	operation			
	ro_challenge_hash	op ro_challenge_hash (y : ro_output) : challenge = ro_output_to_challenge y.	ro_challenge_hash : ro_output → challenge	Defines the random-oracle/hash interface data used by the games.	
	95	operation			
	ro_matrix	op ro_matrix (y : ro_output) : matrix = ro_output_to_matrix y.	ro_matrix : ro_output → matrix	Defines a total mathematical function used by the EasyCrypt model.	
	96	operation			
	ro_signing_coins	op ro_signing_coins (y : ro_output) : random_coins = ro_output_to_random_coins y.	ro_signing_coins : ro_output → random_coins	Defines a total mathematical function used by the EasyCrypt model.	
	99	lemma			
sampler_expand_query_dro_output_ro_signing_coins		lemma sampler_expand_query_dro_output_ro_signing_coins : seed_coins (p : random_coins → bool) : mu (dro_output (sampler_expand_query seed_coins)) (fun y => p (ro_signing_coins y)) = mu drandom_coins p.	sampler_expand_query_dro_output_ro_signing_coins : $P \Rightarrow Q$ is an implication used as a proof obligation.	Prove coins bound that contributes to a game-hop or final security inequality.	

8 HAETAE_ROM_Programming.ec

random-oracle programming events and counted bad-event bounds

	Line	Construct	What was written	Symbolic correspondence
	18	type programming_site	type programming_site = ro_query * ro_output. programming_site $\equiv$ ro_query $\times$ ro_output	Fixes the mathematical carrier used later declarations without committing an implementation representation.
	20	operation programming_site_query	op programming_site_query (site : programming_site) : ro_query = site.'1. programming_site $\rightarrow$ ro_query	Defines the random-oracle/hash into data used by the games.
	21	operation programming_site_output	op programming_site_output (site : programming_site) : ro_output = site.'2. programming_site $\rightarrow$ ro_output	Defines a total mathematical function by the EasyCrypt model.
	23	operation programming_site_of_transcript	op programming_site_of_transcript (md : mode) (tr : transcript) (y : ro_output) : programming_site = (transcript_challenge_query md tr, y). programming_site_of_transcript : mode $\times$ transcript $\times$ ro_output $\rightarrow$ programming_site	Defines transcript or extraction data in the forking/special-soundness argument.
	27	operation signature_programming_site_of_transcript	op signature_programming_site_of_transcript (md : mode) (tr : transcript) (y : ro_output) : programming_site = (transcript_signature_challenge_query md tr, y). signature_programming_site_of_transcript : mode $\times$ transcript $\times$ ro_output $\rightarrow$ programming_site	Defines transcript or extraction data in the forking/special-soundness argument.
	31	operation transcript_signature_challenge_output	op transcript_signature_challenge_output (tr : transcript) : ro_output = ro_output_of_challenge (sig_challenge (transcript_signature tr)). transcript_signature_challenge_output : transcript $\rightarrow$ ro_output	Defines transcript or extraction data in the forking/special-soundness argument.
	34	operation		

	Line	Construct	What was written	Symbolic correspondence
		<pre> actual_signature_programming_site op actual_signature_programming_site (md : mode) (tr : transcript) : programming_site = signature_programming_site_of_transcript md tr (transcript_signature_challenge_output tr). </pre>	<pre> actual_signature_programming_site : mode × transcript → programming_site </pre>	Defines a total mathematical function by the EasyCrypt model.
	39	operation		
		<pre> honest_signing_programming_site op honest_signing_programming_site (md : mode) (sk : skey) (m : message) (ctx : context) (coins : random_coins) : programming_site = actual_signature_programming_site md (transcript_from_honest_signing md sk m ctx coins). </pre>	<pre> honest_signing_programming_site : mode × skey × message × context × random_coins → programming_site </pre>	Defines a total mathematical function by the EasyCrypt model.
	45	operation		
		<pre> honest_signing_programming_site_query op honest_signing_programming_site_query (md : mode) (sk : skey) (m : message) (ctx : context) (coins : random_coins) : ro_query = programming_site_query (honest_signing_programming_site md sk m ctx coins). </pre>	<pre> honest_signing_programming_site_query : mode × skey × message × context × random_coins → ro_query </pre>	Defines the random-oracle/hash interface data used by the games.
	51	operation		
		<pre> programmed_site_query_min_entropy_bound_for op programmed_site_query_min_entropy_bound_for (d : random_coins distr) (md : mode) (sk : skey) (m : message) (ctx : context) (qs : ro_query list) (bd : real) : bool = forall q, q \in qs => mu d (fun coins => honest_signing_programming_site_query md sk m ctx coins = q) <= bd. </pre>	<pre> programmed_site_query_min_entropy_bound_for : Distr(random_coins) × mode × skey × message × context × List(ro_query) × ℝ is a Boolean-valued predicate. </pre>	Defines a Boolean mathematical construct used in statements and invariants.
	60	operation		

	Line	Construct	What was written	Symbolic correspondence
		<pre> programmed_site_query_min_entropy_bound op programmed_site_query_min_entropy_bound_for (md : mode) (sk : skey) (m : message) (ctx : context) (qs : ro_query list) (bd : real) : bool = programmed_site_query_min_entropy_bound_for drandom_coins md sk m ctx qs bd. </pre>	<pre> programmed_site_query_min_entropy_bound mode $\times$ skey $\times$ message $\times$ context $\times$ List(ro_query) $\times \mathbb{R}$ is a Boolean-valued predicate. </pre>	Names a Boolean mathematical condition used in statements and invariants.
	65	<pre> programming_site_matches_transcript op programming_site_matches_transcript (md : mode) (tr : transcript) (site : programming_site) : bool = programming_site_query site = transcript_challenge_query md tr /\ transcript_challenge_matches tr (programming_site_output site). </pre>	<pre> programming_site_matches_transcript $\subseteq$ mode $\times$ transcript $\times$ programming_site is a Boolean-valued predicate. </pre>	Names a Boolean mathematical condition used in statements and invariants.
	70	<pre> reprogramming_failure op reprogramming_failure (md : mode) (tr : transcript) (site : programming_site) : bool = ! programming_site_matches_transcript md tr site. </pre>	<pre> reprogramming_failure $\subseteq$ mode $\times$ transcript $\times$ programming_site is a Boolean-valued predicate. </pre>	Names a bad event whose probability is later shown to be zero or bounded.
	74	<pre> challenge_entropy_support_ok op challenge_entropy_support_ok (md : mode) (tr : transcript) : bool = challenge_sparse md (transcript_challenge tr). </pre>	<pre> challenge_entropy_support_ok $\subseteq$ mode $\times$ transcript is a Boolean-valued predicate. </pre>	Names a well-formedness or support condition so it can be reused in lemmas.
	77	<pre> min_entropy_failure op min_entropy_failure (md : mode) (tr : transcript) : bool = ! challenge_entropy_support_ok md tr. </pre>	<pre> min_entropy_failure $\subseteq$ mode $\times$ transcript is a Boolean-valued predicate. </pre>	Names a bad event whose probability is later shown to be zero or bounded.
	80	<pre> transcript_log_min_entropy_clear op transcript_log_min_entropy_clear (md : mode) (trs : transcript list) : bool = all (fun tr => ! min_entropy_failure md tr) trs. </pre>	<pre> transcript_log_min_entropy_clear $\subseteq$ mode $\times$ List(transcript) is a Boolean-valued predicate. </pre>	Names a Boolean mathematical condition used in statements and invariants.

	Line	Construct	What was written	Symbolic correspondence
	83	operation		
		signing_record_log_min_entropy_clear	op $\text{signing_record_log_min_entropy_clear} \subseteq \text{mode} \times \text{List}(\text{signing_transcript_record})$ $\text{is a Boolean-valued predicate.}$	Names a Boolean mathematical construct used in statements and invariants.
			$(\text{md} : \text{mode}) (\text{rs} : \text{signing_transcript_record list})$ $: \text{bool} = \text{all } (\text{fun } r \Rightarrow !$ $\text{min_entropy_failure md}$ $(\text{signing_record_transcript } r))$ rs.	
	87	operation		
		fs_with_aborts_failure	$\text{fs_with_aborts_failure} \subseteq \text{mode} \times \text{transcript} \times \text{programming_site}$ $\text{is a Boolean-valued predicate.}$	Names a bad event whose probability is later shown to be zero or bounded.
			$\text{op fs_with_aborts_failure (md : mode) (tr : transcript) (site : programming_site) : bool =}$ $\text{reprogramming_failure md tr site}$ $\vee \text{ min_entropy_failure md tr.}$	
	91	operation		
		transcript_signature_programming_site_clear	$\text{transcript_signature_programming_site_clear} \subseteq \text{mode} \times \text{transcript}$ $\text{is a Boolean-valued predicate.}$	Names a Boolean mathematical construct used in statements and invariants.
			op $\text{transcript_signature_programming_site_clear (md : mode) (tr : transcript) : bool = !}$ $\text{fs_with_aborts_failure md tr}$ $(\text{actual_signature_programming_site md tr}).$	
	95	operation		
		transcript_log_signature_programming_sites_clear	$\text{transcript_log_signature_programming_sites_clear} \subseteq \text{mode} \times \text{List}(\text{transcript})$ $\text{is a Boolean-valued predicate.}$	Names a Boolean mathematical construct used in statements and invariants.
			op $\text{transcript_log_signature_programming_sites_clear (md : mode) (trs : transcript list) : bool = all}$ $(\text{transcript_signature_programming_site_clear md}) \text{ trs.}$	
	99	operation		
		fs_with_aborts_bound_obligation	$\text{fs_with_aborts_bound_obligation} \subseteq 1$ $\text{is a Boolean-valued predicate.}$	Names a Boolean mathematical construct used in statements and invariants.
			op $\text{fs_with_aborts_bound_obligation : bool = 0\%r <=}$ $\text{fs_with_aborts_reprogramming_term}$ $\wedge \text{ 0\%r <=}$ $\text{fs_with_aborts_min_entropy_term.}$	
	103	type		

	Line	Construct	What was written	Symbolic correspondence
		<pre>counted_rom_event type counted_rom_event = [CountedHashQuery of ro_query CountedSignatureSite of programming_site CountedProgrammedSite of programming_site CountedMinEntropyCheck of transcript].</pre>	<pre>counted_rom_event ≡ [CountedHashQueryofro_query CountedSignatureSiteofprogramming_site CountedProgrammedSiteofprogramming_site CountedMinEntropyCheckoftranscript].</pre>	Fixes the mathematical carrier used in the implementation representation.
	109	operation		
		<pre>counted_event_is_hash_query op counted_event_is_hash_query (ev : counted_rom_event) : bool = with ev = CountedHashQuery _ => true with ev = CountedSignatureSite _ => false with ev = CountedProgrammedSite _ => false with ev = CountedMinEntropyCheck _ => false.</pre>	<pre>counted_event_is_hash_query ⊆ counted_rom_event is a Boolean-valued predicate.</pre>	Names a Boolean mathematical carrier used in statements and invariants.
	115	operation		
		<pre>counted_event_is_signature_site op counted_event_is_signature_site (ev : counted_rom_event) : bool = with ev = CountedHashQuery _ => false with ev = CountedSignatureSite _ => true with ev = CountedProgrammedSite _ => false with ev = CountedMinEntropyCheck _ => false.</pre>	<pre>counted_event_is_signature_site ⊆ counted_rom_event is a Boolean-valued predicate.</pre>	Names a Boolean mathematical carrier used in statements and invariants.
	121	operation		
		<pre>counted_event_is_programmed_site op counted_event_is_programmed_site (ev : counted_rom_event) : bool = with ev = CountedHashQuery _ => false with ev = CountedSignatureSite _ => false with ev = CountedProgrammedSite _ => true with ev = CountedMinEntropyCheck _ => false.</pre>	<pre>counted_event_is_programmed_site ⊆ counted_rom_event is a Boolean-valued predicate.</pre>	Names a Boolean mathematical carrier used in statements and invariants.
	127	operation		

	Line	Construct	What was written	Symbolic correspondence	
		counted_event_is_min_entropy_check	op counted_event_is_min_entropy_check (ev : counted_rom_event) : bool = with ev = CountedHashQuery _ => false with ev = CountedSignatureSite _ => false with ev = CountedProgrammedSite _ => false with ev = CountedMinEntropyCheck _ => true.	counted_event_is_min_entropy_check $\subseteq$ counted_rom_event is a Boolean-valued predicate.	Names a Boolean mathematical construct used in statements and invariants.
	133	operation			
		ro_query_is_challenge	op ro_query_is_challenge (q : ro_query) : bool = with q = MessageHashQuery _ => false with q = ChallengeHashQuery _ => true with q = MatrixExpandQuery _ => false with q = SamplerExpandQuery _ => false.	ro_query_is_challenge $\subseteq$ ro_query is a Boolean-valued predicate.	Names a Boolean mathematical construct used in statements and invariants.
	139	operation			
		challenge_query_log_count	op challenge_query_log_count (qs : ro_query list) : int = card (oflist (filter ro_query_is_challenge qs)).	challenge_query_log_count : List(ro_query) $\rightarrow \mathbb{Z}$	Defines the random-oracle/hash input data used by the games.
	142	lemma			
		challenge_query_log_count_cons_le	lemma challenge_query_log_count_cons_le q qs : challenge_query_log_count (q :: qs) <= challenge_query_log_count qs + 1.	challenge_query_log_count_cons_le : $X \leq Y$ is an arithmetic or probability upper bound.	Proves a bound that contributes to game-hop or final security inequality.
	156	lemma			
		challenge_query_log_count_nonchallenge_cons	lemma challenge_query_log_count_nonchallenge_cons q qs : ! ro_query_is_challenge q => challenge_query_log_count (q :: qs) = challenge_query_log_count qs.	challenge_query_log_count_nonchallenge_cons : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a bound that contributes to game-hop or final security inequality.
	169	lemma			

	Line	Construct	What was written	Symbolic correspondence
		challenge_query_log_count_message_cons	lemma challenge_query_log_count_message_cons = R is an equality/rewriting fact. pk ctx m qs : challenge_query_log_count (message_hash_query pk ctx m :: qs) = challenge_query_log_count qs.	challenge_query_log_count_message_cons : Proves a bound that contributes to game-hop or final security inequality.
	177	lemma		
		challenge_query_log_count_matrix_cons	lemma challenge_query_log_count_matrix_cons = R is an equality/rewriting fact. md sd qs : challenge_query_log_count (matrix_expand_query md sd :: qs) = challenge_query_log_count qs.	challenge_query_log_count_matrix_cons : Proves a bound that contributes to game-hop or final security inequality.
	185	lemma		
		challenge_query_log_count_sampler_cons	lemma challenge_query_log_count_sampler_cons = R is an equality/rewriting fact. coins qs : challenge_query_log_count (sampler_expand_query coins :: qs) = challenge_query_log_count qs.	challenge_query_log_count_sampler_cons : Proves a bound that contributes to game-hop or final security inequality.
	193	operation		
		counted_trace_hash_queries	op counted_trace_hash_queries (evs : counted_rom_event list) : int = size (filter counted_event_is_hash_query evs).	counted_trace_hash_queries : List(counted_rom_event) $\rightarrow \mathbb{Z}$ Defines the random-oracle/hash internal data used by the games.
	196	operation		
		counted_trace_signature_sites	op counted_trace_signature_sites (evs : counted_rom_event list) : int = size (filter counted_event_is_signature_site evs).	counted_trace_signature_sites : List(counted_rom_event) $\rightarrow \mathbb{Z}$ Defines a total mathematical function by the EasyCrypt model.
	199	operation		
		counted_trace_programmed_sites	op counted_trace_programmed_sites (evs : counted_rom_event list) : int = size (filter counted_event_is_programmed_site evs).	counted_trace_programmed_sites : List(counted_rom_event) $\rightarrow \mathbb{Z}$ Defines a total mathematical function by the EasyCrypt model.

	Line	Construct	What was written	Symbolic correspondence
	202	operation		
		counted_trace_min_entropy_checks op counted_trace_min_entropy_checks (evs : counted_rom_event list) : int = size (filter counted_event_is_min_entropy_check evs).	counted_trace_min_entropy_checks : List(counted_rom_event) $\rightarrow \mathbb{Z}$	Defines a total mathematical function by the EasyCrypt model.
	205	operation		
		programming_sites_same_query op programming_sites_same_query (left right : programming_site) : bool = programming_site_query left = programming_site_query right.	programming_sites_same_query $\subseteq$ programming_site $\times$ programming_site is a Boolean-valued predicate.	Names a Boolean mathematical construct used in statements and invariants.
	209	operation		
		programming_sites_conflict op programming_sites_conflict (left right : programming_site) : bool = programming_sites_same_query left right /\n programming_site_output left <> programming_site_output right.	programming_sites_conflict $\subseteq$ programming_site $\times$ programming_site is a Boolean-valued predicate.	Names a Boolean mathematical construct used in statements and invariants.
	214	operation		
		programming_site_prequeried op programming_site_prequeried (queries : ro_query list) (site : programming_site) : bool = programming_site_query site \in queries.	programming_site_prequeried $\subseteq$ List(ro_query) $\times$ programming_site is a Boolean-valued predicate.	Names a Boolean mathematical construct used in statements and invariants.
	218	operation		
		programming_site_reprograms op programming_site_reprograms (programmed : programming_site list) (site : programming_site) : bool = has (fun old => programming_sites_conflict site old) programmed.	programming_site_reprograms $\subseteq$ List(programming_site) $\times$ programming_site is a Boolean-valued predicate.	Names a Boolean mathematical construct used in statements and invariants.
	222	operation		

	Line	Construct	What was written	Symbolic correspondence
		<pre> programming_site_fresh op programming_site_fresh (queries : ro_query list) (programmed : programming_site list) (site : programming_site) : bool = ! programming_site_prequeried queries site /\ ! programming_site_reprograms programmed site. </pre>	<pre> programming_site_fresh ⊆ List(ro_query) × List(programming_site) × programming_site is a Boolean-valued predicate. </pre>	Names a Boolean mathematical construct used in statements and invariants.
	228	<pre> operation programming_transcript_fresh op programming_transcript_fresh (md : mode) (queries : ro_query list) (programmed : programming_site list) (tr : transcript) : bool = programming_site_fresh queries programmed (actual_signature_programming_site md tr) /\ ! min_entropy_failure md tr. </pre>	<pre> programming_transcript_fresh ⊆ mode × List(ro_query) × List(programming_site) × transcript is a Boolean-valued predicate. </pre>	Names a Boolean mathematical construct used in statements and invariants.
	235	<pre> operation rom_counted_programming_bad op rom_counted_programming_bad (queries : ro_query list) (programmed : programming_site list) (entropy_bad : bool) : bool = entropy_bad \/ has (fun site => programming_site_prequeried queries site) programmed \/ has (fun site => programming_site_reprograms programmed site) programmed. </pre>	<pre> rom_counted_programming_bad ⊆ List(ro_query) × List(programming_site) × Bool is a Boolean-valued predicate. </pre>	Names a bad event whose probability is later shown to be zero or bounded.
	242	<pre> operation rom_counted_state_bad op rom_counted_state_bad (queries : ro_query list) (programmed : programming_site list) (bad_min_entropy : bool) : bool = rom_counted_programming_bad queries programmed bad_min_entropy. </pre>	<pre> rom_counted_state_bad ⊆ List(ro_query) × List(programming_site) × Bool is a Boolean-valued predicate. </pre>	Names a bad event whose probability is later shown to be zero or bounded.

	Line	Construct	What was written	Symbolic correspondence
	247	operation		
		counted_rom_programming_bound_obligation	op counted_rom_programming_bound_obligation : bool = 0%r <= counted_rom_programming_loss_term.	counted_rom_programming_bound_obligation is a Boolean mathematical construct and is a Boolean-valued predicate. Names a Boolean mathematical construct used in statements and invariants.
	319	lemma		
		counted_lazy_rom_init_clears	lemma counted_lazy_rom_init_clears : hoare[CountedLazyROM(0).init : true ==> ! CountedLazyROM.bad_prequery /\ ! CountedLazyROM.bad_reprogram /\ ! CountedLazyROM.bad_min_entropy /\ CountedLazyROM.programmed_sites = []].	counted_lazy_rom_init_clears : $P \Rightarrow Q$ is an implication used as a proof obligation. Proves a bound that contributes to game-hop or final security inequality.
	333	lemma		
		counted_lazy_rom_init_clears_hash_queries	lemma counted_lazy_rom_init_clears_hash_queries : : hoare[CountedLazyROM(0).init : : true ==> CountedLazyROM.hash_queries = []].	counted_lazy_rom_init_clears_hash_queries : $P \Rightarrow Q$ is an implication used as a proof obligation. Proves a bound that contributes to game-hop or final security inequality.
	343	lemma		

	Line	Construct	What was written	Symbolic correspondence
counted_lazy_rom_get_preserves_clear		<pre> lemma counted_lazy_rom_get_preserves_clear : : hoare[CountedLazyROM(0).get : ! CountedLazyROM.bad_prequery /\ ! CountedLazyROM.bad_reprogram /\ ! CountedLazyROM.bad_min_entropy /\ CountedLazyROM.programmed_sites = [] ==> ! CountedLazyROM.bad_prequery /\ ! CountedLazyROM.bad_reprogram /\ ! CountedLazyROM.bad_min_entropy /\ CountedLazyROM.programmed_sites = []]. </pre>	counted_lazy_rom_get_preserves_clear : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a bound that contributes to game-hop or final security inequality
counted_lazy_rom_get_nonchallenge_preserves_challenge_query_count	360	<pre> lemma counted_lazy_rom_get_nonchallenge_preserves_challenge_query_count : n : hoare[CountedLazyROM(0).get : ! ro_query_is_challenge q /\ challenge_query_log_count CountedLazyROM.hash_queries <= n ==> challenge_query_log_count CountedLazyROM.hash_queries <= n]. </pre>	counted_lazy_rom_get_nonchallenge_preserves_challenge_query_count : $X \leq Y$ is an arithmetic function or probability upper bound.	Proves challenge query count contributes to game-hop or final security inequality
counted_lazy_rom_get_preserves_bad_clear	372	<pre> lemma counted_lazy_rom_get_preserves_bad_clear : : hoare[CountedLazyROM(0).get : ! CountedLazyROM.bad_prequery /\ ! CountedLazyROM.bad_reprogram /\ ! CountedLazyROM.bad_min_entropy ==> ! CountedLazyROM.bad_prequery /\ ! CountedLazyROM.bad_reprogram /\ ! CountedLazyROM.bad_min_entropy]. </pre>	counted_lazy_rom_get_preserves_bad_clear : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a bound that contributes to game-hop or final security inequality

	Line	Construct	What was written	Symbolic correspondence
	387	lemma		
counted_lazy_rom_get_preserves_min_entropy_clear		lemma counted_lazy_rom_get_preserves_min_entropy_clear : hoare[CountedLazyROM(0).get : ! CountedLazyROM.bad_min_entropy ==> ! CountedLazyROM.bad_min_entropy].	counted_lazy_rom_get_preserves_min_entropy_clear $B \Rightarrow Q$ is an implication used as a proof obligation.	Prove a bound that contributes to game-hop or final security inequality
	398	lemma		
counted_lazy_rom_observe_transcript_preserves_clear		lemma counted_lazy_rom_observe_transcript_preserves_clear : hoare[CountedLazyROM(0).observe_transcript : ! min_entropy_failure md tr /\ ! CountedLazyROM.bad_prequery /\ ! CountedLazyROM.bad_reprogram /\ ! CountedLazyROM.bad_min_entropy /\ CountedLazyROM.programmed_sites = [] ==> ! CountedLazyROM.bad_prequery /\ ! CountedLazyROM.bad_reprogram /\ ! CountedL...	counted_lazy_rom_observe_transcript_preserves_clear $B \Rightarrow Q$ is an implication used as a proof obligation.	Prove a bound that contributes to game-hop or final security inequality
	417	lemma		
counted_lazy_rom_observe_transcript_preserves_min_entropy_clear		lemma counted_lazy_rom_observe_transcript_preserves_min_entropy_clear : hoare[CountedLazyROM(0).observe_transcript : ! min_entropy_failure md tr /\ ! CountedLazyROM.bad_min_entropy ==> ! CountedLazyROM.bad_min_entropy].	counted_lazy_rom_observe_transcript_preserves_min_entropy_clear $B \Rightarrow Q$ is an implication used as a proof obligation.	Prove a bound that contributes to game-hop or final security inequality
	430	lemma		

	Line	Construct	What was written	Symbolic correspondence
		<pre> counted_lazy_rom_observe_transcript_preserves_bad_clear lemma counted_lazy_rom_observe_transcript_preserves_bad_clear : hoare[CountedLazyROM(0).observe_transcript : ! min_entropy_failure md tr /\ ! CountedLazyROM.bad_prequery /\ ! CountedLazyROM.bad_reprogram /\ ! CountedLazyROM.bad_min_entropy ==> ! CountedLazyROM.bad_prequery /\ ! CountedLazyROM.bad_reprogram /\ ! CountedLazyROM.bad_min_entropy]. </pre>	<pre> counted_lazy_rom_observe_transcript_preserves_bad_clear B ⇒ Q is an implication used as a proof obligation. </pre>	Preserves bad clear that contributes to game-hop or final security inequality
	447	<pre> lemma counted_lazy_rom_program_preserves_bad_clear_if_fresh lemma counted_lazy_rom_program_preserves_bad_clear_if_fresh : hoare[CountedLazyROM(0).program : ! CountedLazyROM.bad_prequery /\ ! CountedLazyROM.bad_reprogram /\ ! CountedLazyROM.bad_min_entropy /\ programming_site_fresh CountedLazyROM.hash_queries CountedLazyROM.programmed_sites site ==> ! CountedLazyROM.bad_prequery /\ ! CountedLazyROM.bad_reprogram /\... </pre>	<pre> counted_lazy_rom_program_preserves_bad_clear_if_fresh B ⇒ Q is an implication used as a proof obligation. </pre>	Preserves if fresh that contributes to game-hop or final security inequality
	465	<pre> lemma counted_lazy_rom_program_preserves_min_entropy_clear lemma counted_lazy_rom_program_preserves_min_entropy_clear : hoare[CountedLazyROM(0).program : ! CountedLazyROM.bad_min_entropy ==> ! CountedLazyROM.bad_min_entropy]. </pre>	<pre> counted_lazy_rom_program_preserves_min_entropy_clear B ⇒ Q is an implication used as a proof obligation. </pre>	Entropy clear that contributes to game-hop or final security inequality

	Line	Construct	What was written	Symbolic correspondence
	476	lemma counted_lazy_rom_bad_preserves_min_entropy_clear lemma counted_lazy_rom_bad_preserves_min_entropy_clear : hoare[CountedLazyROM(0).bad : ! CountedLazyROM.bad_min_entropy ==> ! CountedLazyROM.bad_min_entropy].	counted_lazy_rom_bad_preserves_min_entropy_clear : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a bound that contributes to game-hop or final security inequality
	486	lemma counted_lazy_rom_bad_false_when_clear lemma counted_lazy_rom_bad_false_when_clear : hoare[CountedLazyROM(0).bad : ! CountedLazyROM.bad_prequery /\ ! CountedLazyROM.bad_reprogram /\ ! CountedLazyROM.bad_min_entropy ==> ! res].	counted_lazy_rom_bad_false_when_clear : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a bound that contributes to game-hop or final security inequality
	550	lemma programming_site_self_matches lemma programming_site_self_matches md tr y : transcript_challenge_matches tr y => programming_site_matches_transcript md tr (programming_site_of_transcript md tr y).	programming_site_self_matches : $P \Rightarrow Q$ is an implication used as a proof obligation.	Records a reusable theorem so later scripts can rewrite, apply, or compute
	560	lemma programming_site_self_no_reprogramming_failure lemma programming_site_self_no_reprogramming_failure md tr y : transcript_challenge_matches tr y => ! reprogramming_failure md tr (programming_site_of_transcript md tr y).	programming_site_self_no_reprogramming_failure : $P \Rightarrow Q$ is an implication used as a proof obligation.	Records a reusable theorem so later scripts can rewrite, apply, or compute
	569	lemma		

	Line	Construct	What was written	Symbolic correspondence
fs_with_aborts_failureE		lemma fs_with_aborts_failureE md tr site : fs_with_aborts_failure md tr site = (reprogramming_failure md tr site \/ min_entropy_failure md tr).	fs_with_aborts_failureE : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later scripts can rewrite, apply, or compute
programming_site_prequeried_cons	574	lemma programming_site_prequeried_cons qs site : programming_site_prequeried (programming_site_query site :: qs) site.	programming_site_prequeried_cons : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later scripts can rewrite, apply, or compute
programming_site_prequeried_cons_preserve	578	lemma programming_site_prequeried_cons_preserve q qs site : programming_site_prequeried qs site => programming_site_prequeried (q :: qs) site.	programming_site_prequeried_cons_preserve : $P \Rightarrow Q$ is an implication used as a proof obligation.	Records a reusable theorem so later scripts can rewrite, apply, or compute
programming_site_prequeried_output_irrelevant	583	lemma programming_site_prequeried_output_irrelevant q y1 y2 : programming_site_prequeried qs (q, y1) = programming_site_prequeried qs (q, y2).	programming_site_prequeried_output_irrelevant : P is an equality/rewriting fact.	Proves a bound that contributes to game-hop or final security inequality
programming_site_prequeried_witness	588	lemma programming_site_prequeried_witness qs q y : q \in qs => programming_site_prequeried qs (q, y).	programming_site_prequeried_witness : $P \Rightarrow Q$ is an implication used as a proof obligation.	Records a reusable theorem so later scripts can rewrite, apply, or compute
	593	lemma		

	Line	Construct	What was written	Symbolic correspondence
honest_signing_programming_site_queryE		lemma	honest_signing_programming_site_queryE	Records a reusable theorem so later
		honest_signing_programming_site_queryE	$L \equiv R$ is an equality/rewriting fact.	scripts can rewrite, apply, or compo
		md sk m ctx coins :		
		honest_signing_programming_site_query		
		md sk m ctx coins =		
		challenge_hash_query md		
		(commitment_highbits md sk m ctx		
		coins) (commitment_lowbits md sk		
		m ctx coins) (message_hash		
		(public_key_of_secret md sk) ctx		
		m).		
	611	operation		
programmed_site_entropy_token_from_query		op	programmed_site_entropy_token_from_query	Defines the random-oracle/hash int
		programmed_site_entropy_token_from_query	$ro_query \rightarrow \mathbb{Z}$	data used by the games.
		(q : ro_query) : int = with q		
		= ChallengeHashQuery x => nth 0		
		x.'3 0 with q = MessageHashQuery		
		_ => 0 with q =		
		MatrixExpandQuery _ => 0 with q		
		= SamplerExpandQuery _ => 0.		
	617	lemma		
honest_signing_programming_site_query_entropy_token		lemma	honest_signing_programming_site_query_entropy_token	Records a reusable theorem so later
		honest_signing_programming_site_query_entropy_token	$L \equiv R$ is an equality/rewriting fact.	scripts can rewrite, apply, or compo
		md sk m ctx coins :		
		programmed_site_entropy_token_from_query		
		(honest_signing_programming_site_query		
		md sk m ctx coins) =		
		signing_entropy_token_of_coins		
		coins.		
	629	lemma		
honest_signing_programming_site_query_token_injective		lemma	honest_signing_programming_site_query_token_injective	Records a reusable theorem so later
		honest_signing_programming_site_query_token_injective	$P \rightarrow Q$ is an implication used as a proof	scripts can rewrite, apply, or compo
		md sk m ctx x q :	obligation.	
		signing_entropy_token_in_range x		
		=>		
		honest_signing_programming_site_query		
		md sk m ctx		
		(signing_entropy_token_to_coins		
		x) = q => x =		
		programmed_site_entropy_token_from_query		
		q.		

	Line	Construct	What was written	Symbolic correspondence
	644	lemma		
signing_distribution_programming_site_query_spread		lemma	signing_distribution_programming_site_query_spread	Represents a reusable theorem so later scripts can rewrite, apply, or compose
		signing_distribution_programming_site_query_spread	$X \leq Y$ is an arithmetic or probability upper bound.	
		(d : random_coins distr) md sk		
		m ctx q :		
		signing_randomness_token_spread		
		d => mu d (fun coins =>		
		honest_signing_programming_site_query		
		md sk m ctx coins = q) <= 1%r /		
		challenge_support_cardinality_lower_bound.		
	673	lemma		
honest_signing_programming_site_query_spread		lemma	honest_signing_programming_site_query_spread	Represents a reusable theorem so later scripts can rewrite, apply, or compose
		honest_signing_programming_site_query_spread	$X \leq Y$ is an arithmetic or probability upper bound.	
		md sk m ctx q : mu		
		drandom_coins (fun coins =>		
		honest_signing_programming_site_query		
		md sk m ctx coins = q) <= 1%r /		
		challenge_support_cardinality_lower_bound.		
	684	lemma		
honest_signing_programming_site_query_spread_bound		lemma	honest_signing_programming_site_query_spread_bound	Represents a bound that contributes to game-hop or final security inequality
		honest_signing_programming_site_query_spread_bound	$\forall \vec{r}. P(\vec{r})$ is a universally quantified fact.	
		md sk m ctx qs :		
		programmed_site_query_min_entropy_bound		
		md sk m ctx qs (1%r /		
		challenge_support_cardinality_lower_bound).		
	694	lemma		
honest_signing_programming_site_query_spread_bound_for		lemma	honest_signing_programming_site_query_spread_bound_for	Represents a bound that contributes to game-hop or final security inequality
		honest_signing_programming_site_query_spread_bound_for	$P \Rightarrow Q$ is an implication used as a proof obligation.	
		(d : random_coins distr) md sk		
		m ctx qs :		
		signing_randomness_token_spread		
		d =>		
		programmed_site_query_min_entropy_bound_for		
		d md sk m ctx qs (1%r /		
		challenge_support_cardinality_lower_bound).		
	704	lemma		
honest_signing_programming_site_query_is_challenge		lemma	honest_signing_programming_site_query_is_challenge	Represents a bound that contributes to game-hop or final security inequality
		honest_signing_programming_site_query_is_challenge	$\forall \vec{r}. P(\vec{r})$ is a universally quantified fact.	
		md sk m ctx coins :		
		ro_query_is_challenge		
		(honest_signing_programming_site_query		
		md sk m ctx coins).		

	Line	Construct	What was written	Symbolic correspondence
	713	lemma		
programming_site_prequeried_honest_challenge_filter		lemma programming_site_prequeried_honest_challenge_filter qs md sk m ctx coins : programming_site_prequeried qs (honest_signing_programming_site md sk m ctx coins) = programming_site_prequeried (filter ro_query_is_challenge qs) (honest_signing_programming_site md sk m ctx coins).	programming_site_prequeried_honest_challenge_filter <i>Let R</i> is an equality/rewriting fact.	Deviater bound that contributes to game-hop or final security inequality
	726	lemma		
programming_site_prequeried_honest_message_consE		lemma programming_site_prequeried_honest_message_consE pk ctx m qs md sk sm sctx coins : programming_site_prequeried (message_hash_query pk ctx m :: qs) (honest_signing_programming_site md sk sm sctx coins) = programming_site_prequeried qs (honest_signing_programming_site md sk sm sctx coins).	programming_site_prequeried_honest_message_consE <i>Let R</i> is an equality/rewriting fact.	Records a reusable theorem so later scripts can rewrite, apply, or compo
	745	lemma		
honest_signing_programming_site_outputE		lemma honest_signing_programming_site_outputE md sk m ctx coins : programming_site_output (honest_signing_programming_site md sk m ctx coins) = ro_output_of_challenge (challenge_hash md (commitment_highbits md sk m ctx coins) (commitment_lowbits md sk m ctx coins) (message_hash (public_key_of_secret md sk) ctx m)).	honest_signing_programming_site_outputE <i>Let R</i> is an equality/rewriting fact.	Records a reusable theorem so later scripts can rewrite, apply, or compo
	764	lemma		

	Line	Construct	What was written	Symbolic correspondence
		<honest_signing_programming_site_functional_output< h=""> <pre> lemma honest_signing_programming_site_functional_output md sk1 m1 ctx1 coins1 sk2 m2 ctx2 coins2 : honest_signing_programming_site_query md sk1 m1 ctx1 coins1 = honest_signing_programming_site_query md sk2 m2 ctx2 coins2 => programming_site_output (honest_signing_programming_site md sk1 m1 ctx1 coins1) = programming_site_output (honest_signing_programming...</pre> </honest_signing_programming_site_functional_output<>	<honest_signing_programming_site_functional_output< h=""> $P \Rightarrow Q$ is an implication used as a proof obligation. </honest_signing_programming_site_functional_output<>	Records a reusable theorem so later scripts can rewrite, apply, or compute
	778	lemma <pre> honest_signing_programming_sites_no_conflict honest_signing_programming_sites_no_conflict md sk1 m1 ctx1 coins1 sk2 m2 ctx2 coins2 : ! programming_sites_conflict (honest_signing_programming_site md sk1 m1 ctx1 coins1) (honest_signing_programming_site md sk2 m2 ctx2 coins2).</pre>	<honest_signing_programming_sites_no_conflict< h=""> $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact. </honest_signing_programming_sites_no_conflict<>	Records a reusable theorem so later scripts can rewrite, apply, or compute
	795	lemma <pre> actual_signature_programming_site_sign_internalE actual_signature_programming_site_sign_internalE md pk sk m ctx coins : pk = public_key_of_secret md sk => actual_signature_programming_site md (transcript_of_signature md pk m ctx (sign_internal md sk m ctx coins)) = honest_signing_programming_site md sk m ctx coins.</pre>	<honest_signing_programming_site_sign_internale< h=""> $P \Rightarrow Q$ is an implication used as a proof obligation. </honest_signing_programming_site_sign_internale<>	Records a bound that contributes to game-hop or final security inequality
	808	lemma		

	Line	Construct	What was written	Symbolic correspondence
		<pre> actual_signature_programming_site_sign_internal_any_pkE lemma actual_signature_programming_site_sign_internal_any_pkE md pk sk m ctx coins : actual_signature_programming_site md (transcript_of_signature md pk m ctx (sign_internal md sk m ctx coins)) = honest_signing_programming_site md sk m ctx coins.</pre>	<pre> actual_signature_programming_site_sign_internal_any_pkE md pk sk m ctx coins : actual_signature_programming_site md (transcript_of_signature md pk m ctx (sign_internal md sk m ctx coins)) = honest_signing_programming_site md sk m ctx coins.</pre>	<pre> actual_signature_programming_site_sign_internal_any_pkE md pk sk m ctx coins : actual_signature_programming_site md (transcript_of_signature md pk m ctx (sign_internal md sk m ctx coins)) = honest_signing_programming_site md sk m ctx coins.</pre>
	823	<pre> lemma actual_signature_programming_site_prequeried_after_cons md pk sk m ctx coins q qs : pk = public_key_of_secret md sk => programming_site_prequeried qs (honest_signing_programming_site md sk m ctx coins) => programming_site_prequeried (q :: qs) (actual_signature_programming_site md (transcript_of_signature md pk m ctx (sign_internal md sk m ctx coins))).</pre>	<pre> actual_signature_programming_site_prequeried_after_cons md pk sk m ctx coins q qs : pk = public_key_of_secret md sk => programming_site_prequeried qs (honest_signing_programming_site md sk m ctx coins) => programming_site_prequeried (q :: qs) (actual_signature_programming_site md (transcript_of_signature md pk m ctx (sign_internal md sk m ctx coins))).</pre>	<pre> actual_signature_programming_site_prequeried_after_cons md pk sk m ctx coins q qs : pk = public_key_of_secret md sk => programming_site_prequeried qs (honest_signing_programming_site md sk m ctx coins) => programming_site_prequeried (q :: qs) (actual_signature_programming_site md (transcript_of_signature md pk m ctx (sign_internal md sk m ctx coins))).</pre>
	839	<pre> lemma actual_signature_programming_site_prequeried_after_message_hash md pk sk m ctx coins qpk qctx qm qs : pk = public_key_of_secret md sk => programming_site_prequeried qs (honest_signing_programming_site md sk m ctx coins) => programming_site_prequeried (message_hash_query qpk qctx qm :: qs) (actual_signature_programming_site md (transcript_of_signatur...</pre>	<pre> actual_signature_programming_site_prequeried_after_message_hash md pk sk m ctx coins qpk qctx qm qs : pk = public_key_of_secret md sk => programming_site_prequeried qs (honest_signing_programming_site md sk m ctx coins) => programming_site_prequeried (message_hash_query qpk qctx qm :: qs) (actual_signature_programming_site md (transcript_of_signatur...</pre>	<pre> actual_signature_programming_site_prequeried_after_message_hash md pk sk m ctx coins qpk qctx qm qs : pk = public_key_of_secret md sk => programming_site_prequeried qs (honest_signing_programming_site md sk m ctx coins) => programming_site_prequeried (message_hash_query qpk qctx qm :: qs) (actual_signature_programming_site md (transcript_of_signatur...</pre>
	854	<pre> lemma</pre>	<pre> lemma</pre>	<pre> lemma</pre>

	Line	Construct	What was written	Symbolic correspondence
actual_signature_programming_site_prequeried_message_hashE		<pre> lemma actual_signature_programming_site_prequeried_message_hashE md pk sk m ctx coins qpk qctx qm qs : pk = public_key_of_secret md sk => programming_site_prequeried (message_hash_query qpk qctx qm :: qs) (actual_signature_programming_site md (transcript_of_signature md pk m ctx (sign_internal md sk m ctx coins))) = programming_site_prequeried qs (honest...</pre>	actual_signature_programming_site_prequeried_message_hashE is an implication used as a proof obligation.	Re_message_hashE theorem so later scripts can rewrite, apply, or compo
actual_signature_programming_site_prequeried_message_hash_any_pkE	870	<pre> lemma actual_signature_programming_site_prequeried_message_hash_any_pkE md pk sk m ctx coins qpk qctx qm qs : programming_site_prequeried (message_hash_query qpk qctx qm :: qs) (actual_signature_programming_site md (transcript_of_signature md pk m ctx (sign_internal md sk m ctx coins))) = programming_site_prequeried qs (honest_signing_programming_site md...</pre>	actual_signature_programming_site_prequeried_message_hash_any_pkE is an equality/rewriting fact.	Re_message_hashE theorem so later scripts can rewrite, apply, or compo
programming_site_conflict_free	884	<pre> op programming_site_conflict_free (site : programming_site) (sites : programming_site list) : bool = all (fun old => ! programming_sites_conflict site old) sites.</pre>	programming_site_conflict_free $\subseteq$ programming_site $\times$ List(programming_site) is a Boolean-valued predicate.	Names a Boolean mathematical con used in statements and invariants.
	888	operation		

Line	Construct	What was written	Symbolic correspondence
	<pre> programming_site_log_conflict_free_for_honest op programming_site_log_conflict_free_for_honest (md : mode) (sk : skey) (sites : programming_site list) : bool = forall m ctx coins, programming_site_conflict_free (honest_signing_programming_site md sk m ctx coins) sites. </pre>	<pre> programming_site_log_conflict_free_for_honest mode × skey × List(programming_site) Boolean-valued predicate. </pre>	Not is a Boolean mathematical connective used in statements and invariants.
894	<pre> lemma programming_site_log_conflict_free_for_honest_nil md sk : programming_site_log_conflict_free_for_honest md sk []. </pre>	<pre> programming_site_log_conflict_free_for_honest_nil $\forall \vec{x}. B(\vec{x})$ </pre>	Restoril is a reusable theorem so later scripts can rewrite, apply, or compose.
900	<pre> lemma programming_site_log_conflict_free_for_honest_cons md sk m ctx coins sites : programming_site_log_conflict_free_for_honest md sk sites => programming_site_log_conflict_free_for_honest md sk (honest_signing_programming_site md sk m ctx coins :: sites). </pre>	<pre> programming_site_log_conflict_free_for_honest_cons $B \Rightarrow Q$ obligation. </pre>	Restoril is a reusable theorem so later scripts can rewrite, apply, or compose.
914	<pre> lemma programming_site_conflict_free_no_reprograms site sites : programming_site_conflict_free site sites => ! programming_site_reprograms sites site. </pre>	<pre> programming_site_conflict_free_no_reprograms $B \Rightarrow Q$ obligation. </pre>	Restoril is a reusable theorem so later scripts can rewrite, apply, or compose.
924	<pre> lemma </pre>		

	Line	Construct	What was written	Symbolic correspondence
		<pre> actual_signature_programming_site_sign_internal_conflict_step lemma actual_signature_programming_site_sign_internal_conflict_step md pk sk m ctx coins sites : pk = public_key_of_secret md sk => programming_site_log_conflict_free_for_honest md sk sites => ! programming_site_reprograms sites (actual_signature_programming_site md (transcript_of_signature md pk m ctx (sign_internal md sk m ctx coins))) /\ programming_si...</pre>	<pre> actual_signature_programming_site_sign_internal_conflict_step B -> Q is an implication used as a proof obligation.</pre>	Records conflict as a theorem so later scripts can rewrite, apply, or compute
	949	<pre> lemma programmed_site_reprogram_one_step_conflict_free_zero lemma programmed_site_reprogram_one_step_conflict_free_zero md sk m ctx sites : (forall coins, programming_site_conflict_free (honest_signing_programming_site md sk m ctx coins) sites) => mu drandom_coins (fun coins => programming_site_reprograms sites (honest_signing_programming_site md sk m ctx coins)) = 0%r.</pre>	<pre> programmed_site_reprogram_one_step_conflict_free_zero B -> Q is an implication used as a proof obligation.</pre>	Records conflict as a theorem so later scripts can rewrite, apply, or compute
	969	<pre> lemma programmed_site_prequery_one_step_fset_bound_for lemma programmed_site_prequery_one_step_fset_bound_for (d : random_coins distr) md sk m ctx qs bd : programmed_site_query_min_entropy_bound_for d md sk m ctx qs bd => mu d (fun coins => programming_site_prequeried qs (honest_signing_programming_site md sk m ctx coins)) <= (card (oflist qs))%r * bd.</pre>	<pre> programmed_site_prequery_one_step_fset_bound_for X <= Y is an arithmetic or probability upper bound.</pre>	Bounds fobound that contributes to game-hop or final security inequality
	999	<pre> lemma</pre>		

	Line	Construct	What was written	Symbolic correspondence
		<pre> programmed_site_prequery_one_step_fset_bound lemma programmed_site_prequery_one_step_fset_bound md sk m ctx qs bd : programmed_site_query_min_entropy_bound md sk m ctx qs bd => mu drandom_coins (fun coins => programming_site_prequeried qs (honest_signing_programming_site md sk m ctx coins)) <= (card (oflist qs))%r * bd. </pre>	<pre> programmed_site_prequery_one_step_fset_bound X ≤ Y is an arithmetic or probability upper bound. </pre>	<pre> Bound Bounds: a bound that contributes to game-hop or final security inequality </pre>
	1012	<pre> lemma programmed_site_prequery_one_step_budget_bound_for (d : random_coins distr) md sk m ctx qs : (card (oflist qs))%r <= rom_hash_query_budget + 1%r => programmed_site_query_min_entropy_bound_for d md sk m ctx qs (1%r / challenge_support_cardinality_lower_bound) => mu d (fun coins => programming_site_prequeried qs (honest_signing_programming_site md sk... </pre>	<pre> programmed_site_prequery_one_step_budget_bound_for X ≤ Y is an arithmetic or probability upper bound. </pre>	<pre> Bound Bounds: a bound that contributes to game-hop or final security inequality </pre>
	1044	<pre> lemma programmed_site_prequery_one_step_budget_bound md sk m ctx qs : (card (oflist qs))%r <= rom_hash_query_budget + 1%r => programmed_site_query_min_entropy_bound md sk m ctx qs (1%r / challenge_support_cardinality_lower_bound) => mu drandom_coins (fun coins => programming_site_prequeried qs (honest_signing_programming_site md sk m ctx coins)) <= (rom_h... </pre>	<pre> programmed_site_prequery_one_step_budget_bound X ≤ Y is an arithmetic or probability upper bound. </pre>	<pre> Bound Bounds: a bound that contributes to game-hop or final security inequality </pre>
	1061	<pre> lemma </pre>		

Line	Construct	What was written	Symbolic correspondence
	<pre> programmed_site_prequery_one_step_challenge_fset_bound_for lemma programmed_site_prequery_one_step_challenge_fset_bound_for (d : random_coins distr) md sk m ctx qs bd : programmed_site_query_min_entropy_bound_for d md sk m ctx (filter ro_query_is_challenge qs) bd => mu d (fun coins => programming_site_prequeried qs (honest_signing_programming_site md sk m ctx coins)) <= (card (oflist (filter ro_query_is_challenge... </pre>	<pre> programmed_site_prequery_one_step_challenge_fset_bound_for X ≤ Y is an arithmetic or probability upper bound. </pre>	Proves fset_bound that contributes to game-hop or final security inequality
1081	<pre> lemma programmed_site_prequery_one_step_challenge_fset_bound lemma programmed_site_prequery_one_step_challenge_fset_bound md sk m ctx qs bd : programmed_site_query_min_entropy_bound md sk m ctx (filter ro_query_is_challenge qs) bd => mu drandom_coins (fun coins => programming_site_prequeried qs (honest_signing_programming_site md sk m ctx coins)) <= (card (oflist (filter ro_query_is_challenge qs)))%r * bd. </pre>	<pre> programmed_site_prequery_one_step_challenge_fset_bound X ≤ Y is an arithmetic or probability upper bound. </pre>	Proves fset_bound that contributes to game-hop or final security inequality
1095	<pre> lemma programmed_site_prequery_one_step_challenge_budget_bound_for programmed_site_prequery_one_step_challenge_budget_bound_for (d : random_coins distr) md sk m ctx qs : (card (oflist (filter ro_query_is_challenge qs)))%r <= rom_hash_query_budget + 1%r => programmed_site_query_min_entropy_bound_for d md sk m ctx (filter ro_query_is_challenge qs) (1%r / challenge_support_cardinality_lower_bound) => mu d (fun coins =>... </pre>	<pre> programmed_site_prequery_one_step_challenge_budget_bound_for X ≤ Y is an arithmetic or probability upper bound. </pre>	Proves budget_bound that contributes to game-hop or final security inequality
1129	<pre> lemma </pre>		

Line	Construct	What was written	Symbolic correspondence
	<pre> programmed_site_prequery_one_step_challenge_budget_bound lemma programmed_site_prequery_one_step_challenge_budget_bound md sk m ctx qs : (card (oflist (filter ro_query_is_challenge qs)))%r <= rom_hash_query_budget + 1%r => programmed_site_query_min_entropy_bound md sk m ctx (filter ro_query_is_challenge qs) (1%r / challenge_support_cardinality_lower_bound) => mu drandom_coins (fun coins => programming_site_prequery...</pre>	<pre> programmed_site_prequery_one_step_challenge_budget_bound X ≤ Y is an arithmetic or probability upper bound.</pre>	Proves budget bound that contributes to game-hop or final security inequality
1148	<pre> lemma programmed_site_prequery_one_step_challenge_concrete_bound lemma programmed_site_prequery_one_step_challenge_concrete_bound md sk m ctx qs : (card (oflist (filter ro_query_is_challenge qs)))%r <= rom_hash_query_budget + 1%r => mu drandom_coins (fun coins => programming_site_prequeried qs (honest_signing_programming_site md sk m ctx coins)) <= (rom_hash_query_budget + 1%r) / challenge_support_cardinality_lower_bound.</pre>	<pre> programmed_site_prequery_one_step_challenge_concrete_bound X ≤ Y is an arithmetic or probability upper bound.</pre>	Proves concrete bound that contributes to game-hop or final security inequality
1165	<pre> lemma programmed_site_prequery_one_step_challenge_concrete_bound_for lemma programmed_site_prequery_one_step_challenge_concrete_bound_for (d : random_coins distr) md sk m ctx qs : signing_randomness_token_spread d => (card (oflist (filter ro_query_is_challenge qs)))%r <= rom_hash_query_budget + 1%r => mu d (fun coins => programming_site_prequeried qs (honest_signing_programming_site md sk m ctx coins)) <= (rom_hash_query_b...</pre>	<pre> programmed_site_prequery_one_step_challenge_concrete_bound_for X ≤ Y is an arithmetic or probability upper bound.</pre>	Proves concrete bound for that contributes to game-hop or final security inequality
1183	<pre> lemma</pre>		

	Line	Construct	What was written	Symbolic correspondence	
		challenge_query_log_count_budget_card_bound	lemma challenge_query_log_count_budget_card_bound hc qs : 0 <= hc => hc <= hash_query_budget_count => challenge_query_log_count qs <= hc + 1 => (card (oflist (filter ro_query_is_challenge qs)))%r <= rom_hash_query_budget + 1%r.	challenge_query_log_count_budget_card_bound $X \leq Y$ is an arithmetic or probability upper bound.	Proves a bound that contributes to game-hop or final security inequality
	1195	lemma			
		programmed_site_prequery_one_step_challenge_counter_bound	lemma programmed_site_prequery_one_step_challenge_counter_bound md sk m ctx qs hc : 0 <= hc => hc <= hash_query_budget_count => challenge_query_log_count qs <= hc + 1 => mu drandom_coins (fun coins => programming_site_prequeried qs (honest_signing_programming_site md sk m ctx coins)) <= (rom_hash_query_budget + 1%r) / challenge_support_cardinality_lower_b...	programmed_site_prequery_one_step_challenge_counter_bound $X \leq Y$ is an arithmetic or probability upper bound.	Proves counter bound game-hop or final security inequality
	1212	lemma			
		programmed_site_prequery_one_step_challenge_counter_bound_for	lemma programmed_site_prequery_one_step_challenge_counter_bound_for (d : random_coins distr) md sk m ctx qs hc : signing_randomness_token_spread d => 0 <= hc => hc <= hash_query_budget_count => challenge_query_log_count qs <= hc + 1 => mu d (fun coins => programming_site_prequeried qs (honest_signing_programming_site md sk m ctx coins)) <= (rom_hash_query...	programmed_site_prequery_one_step_challenge_counter_bound_for $X \leq Y$ is an arithmetic or probability upper bound.	Proves counter bound game-hop or final security inequality
	1231	lemma			

Line	Construct	What was written	Symbolic correspondence
	<pre> programmed_site_prequery_one_step_message_hash_counter_bound lemma programmed_site_prequery_one_step_message_hash_counter_bound md pk sk m ctx qpk qctx qm qs hc : 0 <= hc => hc <= hash_query_budget_count => challenge_query_log_count qs <= hc + 1 => mu drandom_coins (fun coins => programming_site_prequeried (message_hash_query qpk qctx qm :: qs) (actual_signature_programming_site md (transcript_of_signature md pk m... </pre>	<pre> programmed_site_prequery_one_step_message_hash_counter_bound $X \leq Y$ is an arithmetic or probability upper bound. </pre>	Proves a bound that contributes to game-hop or final security inequality
1257	<pre> lemma direct_signing_coin_sampler_prequery_bound md sk m ctx qs hc : 0 <= hc => hc <= hash_query_budget_count => challenge_query_log_count qs <= hc + 1 => phoare[DirectSigningCoinSampler.sample : true ==> programming_site_prequeried qs (honest_signing_programming_site md sk m ctx res)] <= ((rom_hash_query_budget + 1%r) / challenge_support_cardinality_lower_bound... </pre>	<pre> direct_signing_coin_sampler_prequery_bound $X \leq Y$ is an arithmetic or probability upper bound. </pre>	Proves a bound that contributes to game-hop or final security inequality
1283	<pre> lemma direct_signing_coin_sampler_budgeted_prequery_bound md sk m ctx qs : challenge_query_log_count qs <= hash_query_budget_count + 1 => phoare[DirectSigningCoinSampler.sample : true ==> programming_site_prequeried qs (honest_signing_programming_site md sk m ctx res)] <= ((rom_hash_query_budget + 1%r) / challenge_support_cardinality_lower_bound). </pre>	<pre> direct_signing_coin_sampler_budgeted_prequery_bound $X \leq Y$ is an arithmetic or probability upper bound. </pre>	Proves a bound that contributes to game-hop or final security inequality

	Line	Construct	What was written	Symbolic correspondence
	1301	lemma		
direct_signing_coin_sampler_message_hash_prequery_bound		<pre> lemma direct_signing_coin_sampler_message_hash_prequery_bound md pk sk m ctx qpk qctx qm qs hc : 0 <= hc => hc <= hash_query_budget_count => challenge_query_log_count qs <= hc + 1 => phoare[DirectSigningCoinSampler.sample : true ==> programming_site_prequeried (message_hash_query qpk qctx qm :: qs) (actual_signature_programming_site md (transcript_of_sign...</pre>	<pre> direct_signing_coin_sampler_message_hash_prequery_bound X <= Y is an arithmetic or probability upper bound.</pre>	Prequery bound that contributes to game-hop or final security inequality
	1333	lemma		
direct_signing_coin_sampler_message_hash_budgeted_prequery_bound		<pre> lemma direct_signing_coin_sampler_message_hash_budgeted_prequery_bound md pk sk m ctx qpk qctx qm qs : challenge_query_log_count qs <= hash_query_budget_count + 1 => phoare[DirectSigningCoinSampler.sample : true ==> programming_site_prequeried (message_hash_query qpk qctx qm :: qs) (actual_signature_programming_site md (transcript_of_signature md pk m ctx...</pre>	<pre> direct_signing_coin_sampler_message_hash_budgeted_prequery_bound X <= Y is an arithmetic or probability upper bound.</pre>	Budgeted prequery bound that contributes to game-hop or final security inequality
	1353	lemma		
programmed_site_prequery_one_step_concrete_bound		<pre> lemma programmed_site_prequery_one_step_concrete_bound md sk m ctx qs : (card (oflist qs))%r <= rom_hash_query_budget + 1%r => mu drandom_coins (fun coins => programming_site_prequeried qs (honest_signing_programming_site md sk m ctx coins)) <= (rom_hash_query_budget + 1%r) / challenge_support_cardinality_lower_bound.</pre>	<pre> programmed_site_prequery_one_step_concrete_bound X <= Y is an arithmetic or probability upper bound.</pre>	One-step bound that contributes to game-hop or final security inequality

	Line	Construct	What was written	Symbolic correspondence
	1369	lemma		
programmed_site_prequery_reprogram_one_step_concrete_bound		lemma programmed_site_prequery_reprogram_one_step_concrete_bound md sk m ctx qs sites : (forall coins, programming_site_conflict_free (honest_signing_programming_site md sk m ctx coins) sites) => (card (oflist qs))%r <= rom_hash_query_budget + 1%r => mu drandom_coins (fun coins => programming_site_prequeried qs (honest_signing_programming_site md sk m ctx...)	programmed_site_prequery_reprogram_one_step_concrete_bound $X \leq Y$ is an arithmetic or probability upper bound.	Step can be used to contribute to game-hop or final security inequality
	1404	lemma		
programmed_site_query_min_entropy_bound_constant_impossible		lemma programmed_site_query_min_entropy_bound_constant_impossible md sk m ctx qs bd q : q \in qs => (forall coins, honest_signing_programming_site_query md sk m ctx coins = q) => bd < 1%r => programmed_site_query_min_entropy_bound md sk m ctx qs bd => false.	programmed_site_query_min_entropy_bound_constant_impossible $P \Rightarrow Q$ is an implication used as a proof obligation.	Pro constant impossible game-hop or final security inequality
	1426	lemma		
programming_site_reprograms_conflict_cons		lemma programming_site_reprograms_conflict_cons site old programmed : programming_sites_conflict site old => programming_site_reprograms (old :: programmed) site.	programming_site_reprograms_conflict_cons $P \Rightarrow Q$ is an implication used as a proof obligation.	Records a reusable theorem so later scripts can rewrite, apply, or compo
	1434	lemma		
rom_counted_programming_bad_min_entropy		lemma rom_counted_programming_bad_min_entropy queries programmed : rom_counted_programming_bad queries programmed true.	rom_counted_programming_bad_min_entropy $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later scripts can rewrite, apply, or compo
	1438	lemma		

	Line	Construct	What was written	Symbolic correspondence
rom_counted_programming_bad_prequery_cons		lemma	rom_counted_programming_bad_prequery_cons	Records a reusable theorem so later
		rom_counted_programming_bad_prequery_cons	$P \Rightarrow Q$ is an implication used as a proof	scripts can rewrite, apply, or compo
		queries programmed site	obligation.	
		entropy_bad :		
		programming_site_prequeried		
		queries site =>		
		rom_counted_programming_bad		
		queries (site :: programmed)		
		entropy_bad.		
	1448	lemma		
counted_rom_programming_bound_obligation_holds		lemma	counted_rom_programming_bound_obligation_holds	Proves a bound that contributes to
		counted_rom_programming_bound_obligation_holds	$\forall \vec{x}. B(\vec{x})$ is a universally quantified fact.	game-hop or final security inequality
		:		
		counted_rom_programming_bound_obligation.		
	1460	operation		
counted_lazy_rom_bad_flags_budget_sound		op	counted_lazy_rom_bad_flags_budget_sound	Names a bad event whose probabili
		counted_lazy_rom_bad_flags_budget_sound	B is a Boolean-valued predicate.	later shown to be zero or bounded.
		(p : real) : bool = p <=		
		counted_rom_programming_loss_term.		
	1463	lemma		
counted_lazy_rom_bad_flags_universal_bound		lemma	counted_lazy_rom_bad_flags_universal_bound	Proves a bound that contributes to
		counted_lazy_rom_bad_flags_universal_bound	$\Pr[E] = B$ is a probability bound or	game-hop or final security inequality
		&m : 1%r >=	exact game probability.	
		Pr[CountedLazyROMBadGame(0,		
		A).main() @ &m : res].		
	1469	lemma		
counted_lazy_rom_bad_flags_query_budget_bound		lemma	counted_lazy_rom_bad_flags_query_budget_bound	Proves a bound that contributes to
		counted_lazy_rom_bad_flags_query_budget_bound	$\Pr[E] \leq B$ is a probability bound or	game-hop or final security inequality
		&m :	exact game probability.	
		counted_lazy_rom_bad_flags_budget_sound		
		(Pr[CountedLazyROMBadGame(0,		
		A).main() @ &m : res]) =>		
		Pr[CountedLazyROMBadGame(0,		
		A).main() @ &m : res] <=		
		counted_rom_programming_loss_term.		
	1478	lemma		

	Line	Construct	What was written	Symbolic correspondence
		counted_lazy_rom_bad_flags_query_budget_bound_subunit	lemma counted_lazy_rom_bad_flags_query_budget_bound_subunit : &m : counted_lazy_rom_bad_flags_budget_sound (Pr [CountedLazyROMBadGame (0, A).main() @ &m : res]) => Pr [CountedLazyROMBadGame (0, A).main() @ &m : res] < 1%r.	counted_lazy_rom_bad_flags_query_budget_bound_subunit : Proves a bound that contributes to game-hop or final security inequality.
	1491	lemma fs_with_aborts_no_programming_failure	lemma fs_with_aborts_no_programming_failure : md tr y : transcript_challenge_matches tr y => fs_with_aborts_failure md tr (programming_site_of_transcript md tr y) = min_entropy_failure md tr.	fs_with_aborts_no_programming_failure : Records a reusable theorem so later scripts can rewrite, apply, or compute obligations.
	1501	lemma transcript_signature_challenge_output_matches	lemma transcript_signature_challenge_output_matches : tr : transcript_signature_challenge_matches tr (transcript_signature_challenge_output tr).	transcript_signature_challenge_output_matches : Proves a bound that contributes to game-hop or final security inequality.
	1511	lemma honest_transcript_challenge_entropy_support	lemma honest_transcript_challenge_entropy_support : md sk m ctx coins : challenge_entropy_support_ok md (transcript_from_honest_signing md sk m ctx coins).	honest_transcript_challenge_entropy_support : Proves a distribution fact needed for sampling or probability mass reasoning.
	1521	lemma honest_transcript_no_min_entropy_failure	lemma honest_transcript_no_min_entropy_failure : md sk m ctx coins : ! min_entropy_failure md (transcript_from_honest_signing md sk m ctx coins).	honest_transcript_no_min_entropy_failure : Records a reusable theorem so later scripts can rewrite, apply, or compute obligations.
	1529	lemma	lemma	

	Line	Construct	What was written	Symbolic correspondence
signing_transcript_relation_challenge_entropy_support		lemma	signing_transcript_relation_challenge_entropy_support	Proves a distribution fact needed for
		signing_transcript_relation_challenge_entropy_support	$P \Rightarrow Q$ is an implication used as a proof obligation.	sampling or probability mass reason
		md pk m ctx sig tr :		
		signing_transcript_relation md		
		pk m ctx sig tr =>		
		challenge_entropy_support_ok md		
		tr.		
	1549	lemma	signing_transcript_relation_no_min_entropy_failure	Records a reusable theorem so later
signing_transcript_relation_no_min_entropy_failure		lemma	$P \Rightarrow Q$ is an implication used as a proof obligation.	scripts can rewrite, apply, or compo
		signing_transcript_relation_no_min_entropy_failure		
		md pk m ctx sig tr :		
		signing_transcript_relation md		
		pk m ctx sig tr => !		
		min_entropy_failure md tr.		
	1560	lemma	signing_record_relation_no_min_entropy_failure	Records a reusable theorem so later
signing_record_relation_no_min_entropy_failure		lemma	$P \Rightarrow Q$ is an implication used as a proof obligation.	scripts can rewrite, apply, or compo
		signing_record_relation_no_min_entropy_failure		
		md pk r :		
		signing_record_relation md pk r		
		=> ! min_entropy_failure md		
		(signing_record_transcript r).		
	1572	lemma	signing_record_log_sound_no_min_entropy	Proves a structural correctness fact
signing_record_log_sound_no_min_entropy		lemma	$P \Rightarrow Q$ is an implication used as a proof obligation.	justify later reductions.
		signing_record_log_sound_no_min_entropy		
		md pk rs :		
		signing_record_log_sound md pk		
		rs =>		
		signing_record_log_min_entropy_clear		
		md rs.		
	1584	lemma	signing_record_log_sound_transcripts_no_min_entropy	Proves a structural correctness fact
signing_record_log_sound_transcripts_no_min_entropy		lemma	$P \Rightarrow Q$ is an implication used as a proof obligation.	justify later reductions.
		signing_record_log_sound_transcripts_no_min_entropy		
		md pk rs :		
		signing_record_log_sound md pk		
		rs =>		
		transcript_log_min_entropy_clear		
		md (signing_record_transcripts		
		rs).		
	1597	lemma		

	Line	Construct	What was written	Symbolic correspondence
fs_with_aborts_no_failure_for_signing_relation		lemma	fs_with_aborts_no_failure_for_signing_relation	Records a reusable theorem so later
		fs_with_aborts_no_failure_for_signing_relation	$P \Rightarrow Q$ is an implication used as a proof	scripts can rewrite, apply, or compo
		md pk m ctx sig tr site :	obligation.	
		signing_transcript_relation md		
		pk m ctx sig tr =>		
		programming_site_matches_transcript		
		md tr site => !		
		fs_with_aborts_failure md tr		
		site.		
	1609	lemma		
fs_with_aborts_no_failure_for_signing_record		lemma	fs_with_aborts_no_failure_for_signing_record	Records a reusable theorem so later
		fs_with_aborts_no_failure_for_signing_record	$P \Rightarrow Q$ is an implication used as a proof	scripts can rewrite, apply, or compo
		md pk r site :	obligation.	
		signing_record_relation md pk r		
		=>		
		programming_site_matches_transcript		
		md (signing_record_transcript r)		
		site => !		
		fs_with_aborts_failure md		
		(signing_record_transcript r)		
		site.		
	1624	lemma		
fs_with_aborts_no_failure_for_honest_programming		lemma	fs_with_aborts_no_failure_for_honest_programming	Records a reusable theorem so later
		fs_with_aborts_no_failure_for_honest_programming	$P \Rightarrow Q$ is an implication used as a proof	scripts can rewrite, apply, or compo
		md sk m ctx coins y :	obligation.	
		transcript_challenge_matches		
		(transcript_from_honest_signing		
		md sk m ctx coins) y => !		
		fs_with_aborts_failure md		
		(transcript_from_honest_signing		
		md sk m ctx coins)		
		(programming_site_of_transcript		
		md		
		(transcript_from_honest_signing		
		md sk m ctx coins) y).		
	1639	lemma		

	Line	Construct	What was written	Symbolic correspondence
		<pre> transcript_valid_programming_site_signature_query lemma transcript_valid_programming_site_signature_query md tr y : transcript_valid md tr => programming_site_query (programming_site_of_transcript md tr y) = programming_site_query (signature_programming_site_of_transcript md tr y). </pre>	<pre> transcript_valid_programming_site_signature_query R => Q is an implication used as a proof obligation. </pre>	Proves a structural correctness fact justify later reductions.
	1650	<pre> lemma transcript_valid_signature_programming_site_matches md tr y : transcript_valid md tr => transcript_signature_challenge_matches tr y => programming_site_matches_transcript md tr (signature_programming_site_of_transcript md tr y). </pre>	<pre> transcript_valid_signature_programming_site_matches R => Q is an implication used as a proof obligation. </pre>	Proves a structural correctness fact justify later reductions.
	1666	<pre> lemma transcript_valid_signature_programming_site_no_reprogramming md tr y : transcript_valid md tr => transcript_signature_challenge_matches tr y => ! reprogramming_failure md tr (signature_programming_site_of_transcript md tr y). </pre>	<pre> transcript_valid_signature_programming_site_no_reprogramming R => Q is an implication used as a proof obligation. </pre>	Proves a structural correctness fact justify later reductions.
	1679	<pre> lemma transcript_valid_actual_signature_programming_site_matches md tr : transcript_valid md tr => programming_site_matches_transcript md tr (actual_signature_programming_site md tr). </pre>	<pre> transcript_valid_actual_signature_programming_site_matches R => Q is an implication used as a proof obligation. </pre>	Proves a structural correctness fact justify later reductions.
	1691	<pre> lemma </pre>		

	Line	Construct	What was written	Symbolic correspondence
transcript_valid_actual_signature_programming_site_no_reprogramming		lemma	transcript_valid_actual_signature_programming_site_no_reprogramming	Proves a structural correctness fact
		transcript_valid_actual_signature_programming_site_no_reprogramming	$P \Rightarrow Q$ is an implication used as a proof obligation.	justify later reductions.
		md tr : transcript_valid md tr		
		=> ! reprogramming_failure md		
		tr		
		(actual_signature_programming_site		
		md tr).		
	1702	lemma	valid_forking_pair_signature_programming_site_query	Proves a structural correctness fact
valid_forking_pair_signature_programming_site_query		lemma	valid_forking_pair_signature_programming_site_query	Proves a structural correctness fact
		valid_forking_pair_signature_programming_site_query	$P \Rightarrow Q$ is an implication used as a proof obligation.	justify later reductions.
		md tr1 tr2 y1 y2 :		
		valid_forking_pair md tr1 tr2 =>		
		programming_site_query		
		(signature_programming_site_of_transcript		
		md tr1 y1) =		
		programming_site_query		
		(signature_programming_site_of_transcript		
		md tr2 y2).		
	1713	lemma	valid_forking_pair_programming_site_query	Proves a structural correctness fact
valid_forking_pair_programming_site_query		lemma	valid_forking_pair_programming_site_query	Proves a structural correctness fact
		valid_forking_pair_programming_site_query	$P \Rightarrow Q$ is an implication used as a proof obligation.	justify later reductions.
		md tr1 tr2 y1 y2 :		
		valid_forking_pair md tr1 tr2 =>		
		programming_site_query		
		(programming_site_of_transcript		
		md tr1 y1) =		
		programming_site_query		
		(programming_site_of_transcript		
		md tr2 y2).		
	1723	lemma		

	Line	Construct	What was written	Symbolic correspondence
		<code>valid_forking_pair_self_programming_sites_no_reprogramming</code> <code>lemma</code> <code>valid_forking_pair_self_programming_sites_no_reprogramming</code> <code>md tr1 tr2 y1 y2 :</code> <code>valid_forking_pair md tr1 tr2 =></code> <code>transcript_challenge_matches tr1</code> <code>y1 =></code> <code>transcript_challenge_matches tr2</code> <code>y2 => ! reprogramming_failure</code> <code>md tr1</code> <code>(programming_site_of_transcript</code> <code>md tr1 y1) /\ !</code> <code>reprogramming_failure md tr2</code> <code>(programming_site_of_transcript</code> <code>md tr2 y2).</code>	<code>valid_forking_pair_self_programming_sites_no_reprogramming</code> $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves reprogramming correctness fact justify later reductions.
	1739	<code>lemma</code> <code>fs_bound_parts_nonnegative</code> <code>lemma fs_bound_parts_nonnegative</code> <code>:</code> <code>fs_with_aborts_bound_obligation</code> <code>=> 0%r <=</code> <code>fs_with_aborts_reprogramming_term</code> <code>/\ 0%r <=</code> <code>fs_with_aborts_min_entropy_term.</code>	<code>fs_bound_parts_nonnegative : $X \leq Y$</code> is an arithmetic or probability upper bound.	Proves a bound that contributes to game-hop or final security inequality.
	1745	<code>lemma</code> <code>fs_with_aborts_bound_obligation_holds</code> <code>lemma</code> <code>fs_with_aborts_bound_obligation_holds</code> <code>:</code> <code>fs_with_aborts_bound_obligation.</code>	<code>fs_with_aborts_bound_obligation_holds :</code> $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to game-hop or final security inequality.

9 HAETAE_Scheme.ec

HAETAE instance of the generic signature interface

Line	Construct	What was written	Symbolic correspondence	Why it is written this way
13	type			
pkey	<pre> type pkey <- pkey, type skey <- skey, type message <- message, type context <- context, type signature <- signature, type ro_query <- ro_query, type ro_output <- ro_output, op dro_output <- dro_output. </pre>	pkey is an abstract carrier set.	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.	
22	operation			
haetae_mode	<pre> op haetae_mode : mode. </pre>	haetae_mode : $1 \rightarrow \text{mode}$	Defines a total mathematical function used by the EasyCrypt model.	

10 HAETAETranscript.ec

transcripts, forking relations, and extraction obligations

	Line	Construct	What was written	Symbolic correspondence
	11	type		
	transcript	type transcript = pkey * message * context * polyveck * poly * crh * challenge * signature.	$\text{transcript} \equiv \text{pkey} \times \text{message} \times \text{context} \times \text{polyveck} \times \text{poly} \times \text{crh} \times \text{challenge} \times \text{signature}$	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.
	14	operation		
	signature_commitment_highbits	op signature_commitment_highbits : mode -> pkey -> message -> context -> signature -> polyveck = fun (_ : mode) (_ : pkey) (_ : message) (_ : context) sig => sig_commitment_highbits sig.	signature_commitment_highbits : $\text{mode} \times \text{pkey} \times \text{message} \times \text{context} \rightarrow \text{mode-} > \text{pkey-} > \text{message-} > \text{context-} > \text{signature-} > \text{polyveck}$	Defines a total mathematical function used by the EasyCrypt model.
	18	operation		
	signature_commitment_lowbits	op signature_commitment_lowbits : mode -> pkey -> message -> context -> signature -> poly = fun (_ : mode) (_ : pkey) (_ : message) (_ : context) sig => sig_commitment_lowbits sig.	signature_commitment_lowbits : $\text{mode} \times \text{pkey} \times \text{message} \times \text{context} \rightarrow \text{mode-} > \text{pkey-} > \text{message-} > \text{context-} > \text{signature-} > \text{poly}$	Defines a total mathematical function used by the EasyCrypt model.
	22	operation		
	signature_message_hash	op signature_message_hash : mode -> pkey -> message -> context -> signature -> crh = fun (_ : mode) (_ : pkey) (_ : message) (_ : context) sig => sig_message_hash sig.	signature_message_hash : $\text{mode} \times \text{pkey} \times \text{message} \times \text{context} \rightarrow \text{mode-} > \text{pkey-} > \text{message-} > \text{context-} > \text{signature-} > \text{crh}$	Defines the random-oracle/hash interface data used by the games.
	26	operation		
	signature_challenge	op signature_challenge : mode -> pkey -> message -> context -> signature -> challenge = fun (_ : mode) (_ : pkey) (_ : message) (_ : context) sig => sig_challenge sig.	signature_challenge : $\text{mode} \times \text{pkey} \times \text{message} \times \text{context} \rightarrow \text{mode-} > \text{pkey-} > \text{message-} > \text{context-} > \text{signature-} > \text{challenge}$	Defines a total mathematical function used by the EasyCrypt model.
	31	operation		
	transcript_pk	op transcript_pk (tr : transcript) : pkey = tr.'1.	transcript_pk : transcript $\rightarrow$ pkey	Defines transcript or extraction data used in the forking/special-soundness arguments
	32	operation		
	transcript_message	op transcript_message (tr : transcript) : message = tr.'2.	transcript_message : transcript $\rightarrow$ message	Defines transcript or extraction data used in the forking/special-soundness arguments

	Line	Construct	What was written	Symbolic correspondence
	33	operation		
		transcript_context op transcript_context (tr : transcript) : context = tr.'3.	transcript_context : transcript → context	Defines transcript or extraction data used in the forking/special-soundness arguments
	34	operation		
		transcript_commitment_highbits op transcript_commitment_highbits (tr : transcript) : polyveck = tr.'4.	transcript_commitment_highbits : transcript → polyveck	Defines transcript or extraction data used in the forking/special-soundness arguments
	35	operation		
		transcript_commitment_lowbits op transcript_commitment_lowbits (tr : transcript) : poly = tr.'5.	transcript_commitment_lowbits : transcript → poly	Defines transcript or extraction data used in the forking/special-soundness arguments
	36	operation		
		transcript_message_hash op transcript_message_hash (tr : transcript) : crh = tr.'6.	transcript_message_hash : transcript → crh	Defines the random-oracle/hash interface data used by the games.
	37	operation		
		transcript_challenge op transcript_challenge (tr : transcript) : challenge = tr.'7.	transcript_challenge : transcript → challenge	Defines transcript or extraction data used in the forking/special-soundness arguments
	38	operation		
		transcript_signature op transcript_signature (tr : transcript) : signature = tr.'8.	transcript_signature : transcript → signature	Defines transcript or extraction data used in the forking/special-soundness arguments
	40	operation		
		transcript_of_signature op transcript_of_signature : mode → pkey → message → context → signature → transcript = fun md pk m ctx sig => (pk, m, ctx, signature_commitment_highbits md pk m ctx sig, signature_commitment_lowbits md pk m ctx sig, signature_message_hash md pk m ctx sig, signature_challenge md pk m ctx sig, sig).	transcript_of_signature : 1 → mode → pkey → message → context → signature → transcript	Defines transcript or extraction data used in the forking/special-soundness arguments
	50	operation		

	Line	Construct	What was written	Symbolic correspondence
transcript_challenge_query		<pre> op transcript_challenge_query (md : mode) (tr : transcript) : ro_query = challenge_hash_query md (transcript_commitment_highbits tr) (transcript_commitment_lowbits tr) (transcript_message_hash tr). </pre>	<pre> transcript_challenge_query : mode × transcript → ro_query </pre>	Defines the random-oracle/hash interface data used by the games.
	57	operation		
transcript_signature_challenge_query		<pre> op transcript_signature_challenge_query (md : mode) (tr : transcript) : ro_query = challenge_hash_query md (sig_commitment_highbits (transcript_signature tr)) (sig_commitment_lowbits (transcript_signature tr)) (sig_message_hash (transcript_signature tr)). </pre>	<pre> transcript_signature_challenge_query : mode × transcript → ro_query </pre>	Defines the random-oracle/hash interface data used by the games.
	65	operation		
transcript_verifies		<pre> op transcript_verifies (md : mode) (tr : transcript) : bool = verify_internal md (transcript_pk tr) (transcript_message tr) (transcript_context tr) (transcript_signature tr). </pre>	<pre> transcript_verifies ⊆ mode × transcript is a Boolean-valued predicate. </pre>	Names a Boolean mathematical condition used in statements and invariants.
	72	operation		
transcript_challenge_matches		<pre> op transcript_challenge_matches (tr : transcript) (y : ro_output) : bool = ro_challenge_hash y = transcript_challenge tr. </pre>	<pre> transcript_challenge_matches ⊆ transcript × ro_output is a Boolean-valued predicate. </pre>	Names a Boolean mathematical condition used in statements and invariants.
	75	operation		

	Line	Construct	What was written	Symbolic correspondence
transcript_signature_challenge_matches		<pre> op transcript_signature_challenge_matches (tr : transcript) (y : ro_output) : bool = ro_challenge_hash y = sig_challenge (transcript_signature tr). </pre>	$\text{transcript_signature_challenge_matches} \subseteq \text{transcript} \times \text{ro_output}$ is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.
transcript_fields_match_signature	79	operation <pre> op transcript_fields_match_signature (tr : transcript) : bool = transcript_commitment_highbits tr = sig_commitment_highbits (transcript_signature tr) /\ transcript_commitment_lowbits tr = sig_commitment_lowbits (transcript_signature tr) /\ transcript_message_hash tr = sig_message_hash (transcript_signature tr) /\ transcript_challenge tr = sig_challenge (t... </pre>	$\text{transcript_fields_match_signature} \subseteq \text{transcript}$ is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.
transcript_valid	89	operation <pre> op transcript_valid (md : mode) (tr : transcript) : bool = transcript_verifies md tr /\ valid_signature md (transcript_signature tr) /\ transcript_fields_match_signature tr. </pre>	$\text{transcript_valid} \subseteq \text{mode} \times \text{transcript}$ is a Boolean-valued predicate.	Names a well-formedness or support condition so it can be reused in lemmas
transcript_public_equation	94	operation <pre> op transcript_public_equation (md : mode) (tr : transcript) : bool = verify_public_equation md (transcript_pk tr) (transcript_signature tr). </pre>	$\text{transcript_public_equation} \subseteq \text{mode} \times \text{transcript}$ is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.
	97	operation		

Line	Construct	What was written	Symbolic correspondence
	<pre> same_fork_target op same_fork_target (tr1 tr2 : transcript) : bool = transcript_pk tr1 = transcript_pk tr2 /\ transcript_message tr1 = transcript_message tr2 /\ transcript_context tr1 = transcript_context tr2 /\ transcript_commitment_highbits tr1 = transcript_commitment_highbits tr2 /\ transcript_commitment_lowbits tr1 = transcript_commitment_lowbits tr2 /\ transcript_mes... </pre>	<pre> same_fork_target $\subseteq$ transcript $\times$ transcript is a Boolean-valued predicate. </pre>	Names a Boolean mathematical condition used in statements and invariants.
105	<pre> distinct_fork_challenges op distinct_fork_challenges (tr1 tr2 : transcript) : bool = transcript_challenge tr1 <> transcript_challenge tr2. </pre>	<pre> distinct_fork_challenges $\subseteq$ transcript $\times$ transcript is a Boolean-valued predicate. </pre>	Names a Boolean mathematical condition used in statements and invariants.
108	<pre> valid_forking_pair op valid_forking_pair (md : mode) (tr1 tr2 : transcript) : bool = same_fork_target tr1 tr2 /\ distinct_fork_challenges tr1 tr2 /\ transcript_valid md tr1 /\ transcript_valid md tr2. </pre>	<pre> valid_forking_pair $\subseteq$ mode $\times$ transcript $\times$ transcript is a Boolean-valued predicate. </pre>	Names a well-formedness or support condition so it can be reused in lemmas
114	<pre> transcript_response op transcript_response (tr : transcript) : polyvecl = sig_response (transcript_signature tr). </pre>	<pre> transcript__response : transcript $\rightarrow$ polyvecl </pre>	Defines transcript or extraction data used in the forking/special-soundness arguments
116	<pre> transcript_response_aux op transcript_response_aux (tr : transcript) : polyveck = sig_response_aux (transcript_signature tr). </pre>	<pre> transcript__response__aux : transcript $\rightarrow$ polyveck </pre>	Defines transcript or extraction data used in the forking/special-soundness arguments
119	<pre> operation </pre>		

	Line	Construct	What was written	Symbolic correspondence
transcript_public_key_challenge_term		<pre> op transcript_public_key_challenge_term (md : mode) (tr : transcript) : polyveck = public_key_challenge_term md (transcript_pk tr) (sig_challenge (transcript_signature tr)). </pre>	<pre> transcript_public_key_challenge_term : mode × transcript → polyveck </pre>	Defines transcript or extraction data used in the forking/special-soundness arguments.
active_public_reconstruction	125	operation		
		<pre> op active_public_reconstruction (md : mode) (tr : transcript) : bool = transcript_public_key_challenge_term md tr <> polyveck_zero md. </pre>	<pre> active_public_reconstruction ⊆ mode × transcript is a Boolean-valued predicate. </pre>	Names a Boolean mathematical condition used in statements and invariants.
inactive_public_reconstruction	128	operation		
		<pre> op inactive_public_reconstruction (md : mode) (tr : transcript) : bool = transcript_public_key_challenge_term md tr = polyveck_zero md. </pre>	<pre> inactive_public_reconstruction ⊆ mode × transcript is a Boolean-valued predicate. </pre>	Names a Boolean mathematical condition used in statements and invariants.
forking_difference_solution	131	operation		
		<pre> op forking_difference_solution (md : mode) (tr1 tr2 : transcript) : bimodal_selftarget_msis_solution = polyveck_sub md (transcript_response tr1) (transcript_response tr2). </pre>	<pre> forking_difference_solution : mode × transcript × transcript → bimodal_selftarget_msis_solution </pre>	Defines transcript or extraction data used in the forking/special-soundness arguments.
fork_response_aux_difference	135	operation		
		<pre> op fork_response_aux_difference (md : mode) (tr1 tr2 : transcript) : polyveck = polyveck_sub md (transcript_response_aux tr1) (transcript_response_aux tr2). </pre>	<pre> fork_response_aux_difference : mode × transcript × transcript → polyveck </pre>	Defines transcript or extraction data used in the forking/special-soundness arguments.
	139	operation		

	Line	Construct	What was written	Symbolic correspondence
fork_public_challenge_relation		op fork_public_challenge_relation (md : mode) (tr1 tr2 : transcript) : bool = polyveck_add md (transcript_response_aux tr1) (transcript_public_key_challenge_term md tr1) = polyveck_add md (transcript_response_aux tr2) (transcript_public_key_challenge_term md tr2).	fork_public_challenge_relation $\subseteq$ mode $\times$ transcript $\times$ transcript is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.
	147	operation		
fork_response_aux_relation		op fork_response_aux_relation (tr1 tr2 : transcript) : bool = transcript_response_aux tr1 = transcript_response_aux tr2.	fork_response_aux_relation $\subseteq$ transcript $\times$ transcript is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.
	150	operation		
fork_left_active_public_challenge_relation		op fork_left_active_public_challenge_relation (md : mode) (tr1 tr2 : transcript) : bool = polyveck_add md (transcript_response_aux tr1) (transcript_public_key_challenge_term md tr1) = transcript_response_aux tr2.	fork_left_active_public_challenge_relation $\subseteq$ mode $\times$ transcript $\times$ transcript is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.
	157	operation		
fork_right_active_public_challenge_relation		op fork_right_active_public_challenge_relation (md : mode) (tr1 tr2 : transcript) : bool = transcript_response_aux tr1 = polyveck_add md (transcript_response_aux tr2) (transcript_public_key_challenge_term md tr2).	fork_right_active_public_challenge_relation $\subseteq$ mode $\times$ transcript $\times$ transcript is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.
	164	operation		

	Line	Construct	What was written	Symbolic correspondence
		<pre> fork_public_reconstruction_cases op fork_public_reconstruction_cases (md : mode) (tr1 tr2 : transcript) : bool = fork_public_challenge_relation md tr1 tr2. </pre>	$\text{fork_public_reconstruction_cases} \subseteq$ $\text{mode} \times \text{transcript} \times \text{transcript}$ is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.
	168	<pre> fork_matrix_source op fork_matrix_source (tr1 tr2 : transcript) : byte list = encode_pkey (transcript_pk tr1) ++ transcript_context tr1 ++ transcript_message tr1 ++ encode_polyveck (transcript_commitment_highbits tr1) ++ encode_poly (transcript_commitment_lowbits tr1) ++ transcript_message_hash tr1 ++ encode_poly (transcript_challenge tr1) ++ encode_poly (transcript_challen... </pre>	$\text{fork_matrix_source} :$ $\text{transcript} \times \text{transcript} \rightarrow \text{List}(\text{byte})$	Defines transcript or extraction data used in the forking/special-soundness argument.
	178	<pre> fork_module_matrix op fork_module_matrix (md : mode) (tr1 tr2 : transcript) : matrix = mkseq (fun i => deterministic_unit_polyveck md (53 + i) (fork_matrix_source tr1 tr2)) (mode_k md). </pre>	$\text{fork_module_matrix} :$ $\text{mode} \times \text{transcript} \times \text{transcript} \rightarrow \text{matrix}$	Defines transcript or extraction data used in the forking/special-soundness argument.
	184	<pre> fork_module_target op fork_module_target (md : mode) (tr1 tr2 : transcript) : polyveck = matrix_vec_mul md (fork_module_matrix md tr1 tr2) (forking_difference_solution md tr1 tr2). </pre>	$\text{fork_module_target} :$ $\text{mode} \times \text{transcript} \times \text{transcript} \rightarrow \text{polyveck}$	Defines transcript or extraction data used in the forking/special-soundness argument.
	189	<pre> operation </pre>		

	Line	Construct	What was written	Symbolic correspondence
		bimodal_selftarget_msis_instance_of_fork	op bimodal_selftarget_msis_instance_of_fork : mode -> transcript -> transcript -> bimodal_selftarget_msis_instance = fun md tr1 tr2 => (fork_module_matrix md tr1 tr2, fork_module_target md tr1 tr2).	bimodal_selftarget_msis_instance_of_fork : Defines transcript or extraction data used in the forking/special-soundness argument
	194	operation		
		extract_bimodal_selftarget_msis_solution	op extract_bimodal_selftarget_msis_solution : mode -> transcript -> transcript -> bimodal_selftarget_msis_solution option = fun md tr1 tr2 => if valid_forking_pair md tr1 tr2 then Some (forking_difference_solution md tr1 tr2) else None.	extract_bimodal_selftarget_msis_solution : Defines transcript or extraction data used in the forking/special-soundness argument
	202	operation		
		module_sis_instance_of_fork	op module_sis_instance_of_fork : mode -> transcript -> transcript -> module_sis_instance = fun md tr1 tr2 => bimodal_to_module_sis_instance md (bimodal_selftarget_msis_instance_of_fork md tr1 tr2).	module_sis_instance_of_fork : 1 → Defines transcript or extraction data used in the forking/special-soundness argument
	208	operation		
		extract_module_sis_solution	op extract_module_sis_solution : mode -> transcript -> transcript -> module_sis_solution option = fun md tr1 tr2 => let x = bimodal_selftarget_msis_instance_of_fork md tr1 tr2 in let sol = extract_bimodal_selftarget_msis_solution md tr1 tr2 in if sol = None then None else Some (bimodal_to_module_sis_solution md x (oget sol)).	extract_module_sis_solution : 1 → Defines transcript or extraction data used in the forking/special-soundness argument
	216	operation		

	Line	Construct	What was written	Symbolic correspondence
		<pre> bimodal_extraction_success op bimodal_extraction_success (md : mode) (tr1 tr2 : transcript) : bool = extract_bimodal_selftarget_msis_solution md tr1 tr2 <> None /\ bimodal_selftarget_msis_valid md (bimodal_selftarget_msis_instance_of_fork md tr1 tr2) (oget (extract_bimodal_selftarget_msis_solution md tr1 tr2)). </pre>	$\text{bimodal_extraction_success} \subseteq$ $\text{mode} \times \text{transcript} \times \text{transcript}$ is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.
	222	<pre> lemma extract_bimodal_forking_some lemma extract_bimodal_forking_some md tr1 tr2 : valid_forking_pair md tr1 tr2 => extract_bimodal_selftarget_msis_solution md tr1 tr2 = Some (forking_difference_solution md tr1 tr2). </pre>	$\text{extract_bimodal_forking_some} : P \Rightarrow Q$ is an implication used as a proof obligation.	Records a reusable theorem so later proofs can rewrite, apply, or compose it
	231	<pre> lemma extract_bimodal_forking_nonempty lemma extract_bimodal_forking_nonempty md tr1 tr2 : valid_forking_pair md tr1 tr2 => extract_bimodal_selftarget_msis_solution md tr1 tr2 <> None. </pre>	$\text{extract_bimodal_forking_nonempty} :$ $P \Rightarrow Q$ is an implication used as a proof obligation.	Records a reusable theorem so later proofs can rewrite, apply, or compose it
	240	<pre> lemma transcript_fields_match_signature_challenge_query lemma transcript_fields_match_signature_challenge_query md tr : transcript_fields_match_signature tr => transcript_challenge_query md tr = transcript_signature_challenge_query md tr. </pre>	$\text{transcript_fields_match_signature_challenge_query} :$ $P \Rightarrow Q$ is an implication used as a proof obligation.	Provides a bound that contributes to a game-hop or final security inequality.
	254	<pre> lemma </pre>		

	Line	Construct	What was written	Symbolic correspondence
transcript_fields_match_signature_challenge_matches		<pre> lemma transcript_fields_match_signature_challenge_matches : tr y : transcript_fields_match_signature tr => transcript_challenge_matches tr y = transcript_signature_challenge_matches tr y. </pre>	<pre> transcript_fields_match_signature_challenge_matches : P => Q is an implication used as a proof obligation. </pre>	Proves a bound that contributes to a game-hop or final security inequality.
transcript_valid_signature_challenge_query	268	<pre> lemma transcript_valid_signature_challenge_query : md tr : transcript_valid md tr => transcript_challenge_query md tr = transcript_signature_challenge_query md tr. </pre>	<pre> transcript_valid_signature_challenge_query : P => Q is an implication used as a proof obligation. </pre>	Proves a bound that contributes to a game-hop or final security inequality.
transcript_valid_signature_challenge_matches	278	<pre> lemma transcript_valid_signature_challenge_matches : md tr y : transcript_valid md tr => transcript_challenge_matches tr y = transcript_signature_challenge_matches tr y. </pre>	<pre> transcript_valid_signature_challenge_matches : P => Q is an implication used as a proof obligation. </pre>	Proves a bound that contributes to a game-hop or final security inequality.
transcript_valid_signature_challengeE	288	<pre> lemma transcript_valid_signature_challengeE : md tr : transcript_valid md tr => transcript_challenge tr = sig_challenge (transcript_signature tr). </pre>	<pre> transcript_valid_signature_challengeE : P => Q is an implication used as a proof obligation. </pre>	Proves a bound that contributes to a game-hop or final security inequality.
transcript_valid_public_equation	299	<pre> lemma transcript_valid_public_equation : md tr : transcript_valid md tr => transcript_public_equation md tr. </pre>	<pre> transcript_valid_public_equation : P => Q is an implication used as a proof obligation. </pre>	Proves a structural correctness fact used to justify later reductions.
	313	lemma		

	Line	Construct	What was written	Symbolic correspondence
transcript_valid_commitment_reconstruction		<pre> lemma transcript_valid_commitment_reconstruction : md tr : transcript_valid md tr => transcript_commitment_highbits tr = verify_reconstructed_highbits md (transcript_pk tr) (transcript_signature tr). </pre>	<pre> transcript_valid_commitment_reconstruction : P => Q </pre> $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a structural correctness fact used to justify later reductions.
valid_forking_pair_public_equations	333	lemma <pre> lemma valid_forking_pair_public_equations : md tr1 tr2 : valid_forking_pair md tr1 tr2 => transcript_public_equation md tr1 /\ transcript_public_equation md tr2. </pre>	<pre> valid_forking_pair_public_equations : P => Q </pre> $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a structural correctness fact used to justify later reductions.
same_fork_target_challenge_query	345	lemma <pre> lemma same_fork_target_challenge_query : md tr1 tr2 : same_fork_target tr1 tr2 => transcript_challenge_query md tr1 = transcript_challenge_query md tr2. </pre>	<pre> same_fork_target_challenge_query : P => Q </pre> $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a bound that contributes to a game-hop or final security inequality.
valid_forking_pair_challenge_query	354	lemma <pre> lemma valid_forking_pair_challenge_query : md tr1 tr2 : valid_forking_pair md tr1 tr2 => transcript_challenge_query md tr1 = transcript_challenge_query md tr2. </pre>	<pre> valid_forking_pair_challenge_query : P => Q </pre> $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a bound that contributes to a game-hop or final security inequality.
	363	lemma		

	Line	Construct	What was written	Symbolic correspondence
		<pre> valid_forking_pair_signature_challenge_query lemma valid_forking_pair_signature_challenge_query md tr1 tr2 : valid_forking_pair md tr1 tr2 => transcript_signature_challenge_query md tr1 = transcript_signature_challenge_query md tr2. </pre>	<pre> valid_forking_pair_signature_challenge_query P => Q </pre>	Proves a bound that contributes to a game-hop or final security inequality.
	378	<pre> lemma valid_forking_pair_distinct_signature_challenges lemma valid_forking_pair_distinct_signature_challenges md tr1 tr2 : valid_forking_pair md tr1 tr2 => sig_challenge (transcript_signature tr1) <> sig_challenge (transcript_signature tr2). </pre>	<pre> valid_forking_pair_distinct_signature_challenges P => Q </pre>	Proves a bound that contributes to a game-hop or final security inequality.
	392	<pre> lemma same_fork_target_commitment_highbits lemma same_fork_target_commitment_highbits tr1 tr2 : same_fork_target tr1 tr2 => transcript_commitment_highbits tr1 = transcript_commitment_highbits tr2. </pre>	<pre> same_fork_target_commitment_highbits : P => Q </pre>	Records a reusable theorem so later proofs can rewrite, apply, or compose it.
	397	<pre> lemma valid_forking_pair_commitment_highbits lemma valid_forking_pair_commitment_highbits md tr1 tr2 : valid_forking_pair md tr1 tr2 => transcript_commitment_highbits tr1 = transcript_commitment_highbits tr2. </pre>	<pre> valid_forking_pair_commitment_highbits : P => Q </pre>	Proves a structural correctness fact used to justify later reductions.
	406	<pre> lemma </pre>		

	Line	Construct	What was written	Symbolic correspondence
valid_forking_pair_reconstructed_highbits_equal		<pre> lemma valid_forking_pair_reconstructed_highbits_equal md tr1 tr2 : valid_forking_pair md tr1 tr2 => verify_reconstructed_highbits md (transcript_pk tr1) (transcript_signature tr1) = verify_reconstructed_highbits md (transcript_pk tr2) (transcript_signature tr2).</pre>	<pre> valid_forking_pair_reconstructed_highbits_equal P => Q is an implication used as a proof obligation.</pre>	Proves a structural correctness fact used to justify later reductions.
transcript_reconstructionE	423	<pre> lemma transcript_reconstructionE md tr : verify_reconstructed_highbits md (transcript_pk tr) (transcript_signature tr) = polyveck_add md (transcript_response_aux tr) (transcript_public_key_challenge_term md tr).</pre>	<pre> transcript_reconstructionE : L = R is an equality/rewriting fact.</pre>	Records a reusable theorem so later proofs can rewrite, apply, or compose it.
transcript_active_reconstructionE	436	<pre> lemma transcript_active_reconstructionE md tr : active_public_reconstruction md tr => verify_reconstructed_highbits md (transcript_pk tr) (transcript_signature tr) = polyveck_add md (transcript_response_aux tr) (transcript_public_key_challenge_term md tr).</pre>	<pre> transcript_active_reconstructionE : P => Q is an implication used as a proof obligation.</pre>	Records a reusable theorem so later proofs can rewrite, apply, or compose it.
valid_forking_pair_public_challenge_relation	448	<pre> lemma valid_forking_pair_public_challenge_relation md tr1 tr2 : valid_forking_pair md tr1 tr2 => fork_public_challenge_relation md tr1 tr2.</pre>	<pre> valid_forking_pair_public_challenge_relation P => Q is an implication used as a proof obligation.</pre>	Proves a bound that contributes to a game-hop or final security inequality.
	461	<pre> lemma</pre>		

	Line	Construct	What was written	Symbolic correspondence
		<code>valid_forking_pair_public_reconstruction_cases</code> <code>lemma</code> <code>valid_forking_pair_public_reconstruction_cases</code> <code>md tr1 tr2 : valid_forking_pair</code> <code>md tr1 tr2 =></code> <code>fork_public_reconstruction_cases</code> <code>md tr1 tr2.</code>	<code>valid_forking_pair_public_reconstruction_cases</code> $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a structural correctness fact used to justify later reductions.
	470	operation <code>extraction_success</code> <code>op extraction_success (md :</code> <code>mode) (tr1 tr2 : transcript) :</code> <code>bool =</code> <code>extract_module_sis_solution md</code> <code>tr1 tr2 <> None /\</code> <code>module_sis_valid md</code> <code>(module_sis_instance_of_fork md</code> <code>tr1 tr2) (oget</code> <code>(extract_module_sis_solution md</code> <code>tr1 tr2)).</code>	<code>extraction_success</code> $\subseteq$ <code>mode</code> $\times$ <code>transcript</code> $\times$ <code>transcript</code> is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.
	476	operation <code>forking_extraction_obligation</code> <code>op forking_extraction_obligation</code> <code>(md : mode) : bool = forall</code> <code>(tr1 tr2 : transcript),</code> <code>valid_forking_pair md tr1 tr2 =></code> <code>bimodal_extraction_success md</code> <code>tr1 tr2.</code>	<code>forking_extraction_obligation</code> $\subseteq$ <code>mode</code> $\times$ <code>transcript</code> $\times$ <code>transcript</code> is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.
	481	operation <code>fork_public_reconstruction_obligation</code> <code>op</code> <code>fork_public_reconstruction_obligation</code> <code>(md : mode) : bool = forall</code> <code>(tr1 tr2 : transcript),</code> <code>valid_forking_pair md tr1 tr2 =></code> <code>fork_public_reconstruction_cases</code> <code>md tr1 tr2.</code>	<code>fork_public_reconstruction_obligation</code> $\subseteq$ <code>mode</code> $\times$ <code>transcript</code> $\times$ <code>transcript</code> is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.
	486	lemma <code>forking_difference_solution_wf</code> <code>lemma</code> <code>forking_difference_solution_wf</code> <code>md tr1 tr2 :</code> <code>module_sis_solution_wf md</code> <code>(forking_difference_solution md</code> <code>tr1 tr2).</code>	<code>forking_difference_solution_wf</code> : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact used to justify later reductions.
	493	lemma		

	Line	Construct	What was written	Symbolic correspondence
fork_module_matrix_rows_wf		lemma fork_module_matrix_rows_wf md tr1 tr2 : all (polyvec1_wf md) (fork_module_matrix md tr1 tr2).	fork_module_matrix_rows_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.
	501	lemma		
fork_module_matrix_size		lemma fork_module_matrix_size md tr1 tr2 : size (fork_module_matrix md tr1 tr2) = mode_k md.	fork_module_matrix_size : $L = R$ is an equality/rewriting fact.	Proves a bound that contributes to a game-hop or final security inequality.
	505	lemma		
bimodal_forking_solution_valid		lemma bimodal_forking_solution_valid md tr1 tr2 : bimodal_selftarget_msis_valid md (bimodal_selftarget_msis_instance_of_fork md tr1 tr2) (forking_difference_solution md tr1 tr2).	bimodal_forking_solution_valid : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact used justify later reductions.
	516	lemma		
forking_extraction_obligation_holds		lemma forking_extraction_obligation_holds md : forking_extraction_obligation md.	forking_extraction_obligation_holds : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later pro scripts can rewrite, apply, or compose it
	527	lemma		
fork_public_reconstruction_obligation_holds		lemma fork_public_reconstruction_obligation_holds md : fork_public_reconstruction_obligation md.	fork_public_reconstruction_obligation_holds : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later pro scripts can rewrite, apply, or compose it
	535	operation		

	Line	Construct	What was written	Symbolic correspondence
bimodal_to_module_sis_lift_obligation		<pre> op bimodal_to_module_sis_lift_obligation (md : mode) : bool = forall (x : bimodal_selftarget_msis_instance) (sol : bimodal_selftarget_msis_solution), bimodal_selftarget_msis_valid md x sol => module_sis_valid md (bimodal_to_module_sis_instance md x) (bimodal_to_module_sis_solution md x sol).</pre>	$\text{bimodal_to_module_sis_lift_obligation} \subseteq \text{mode} \times \text{bimodal_selftarget_msis_instance} \times \text{bimodal_selftarget_msis_solution}$ is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.
bimodal_to_module_sis_lift_obligation_holds	543	lemma <pre> lemma bimodal_to_module_sis_lift_obligation_holds md : bimodal_to_module_sis_lift_obligation md.</pre>	$\text{bimodal_to_module_sis_lift_obligation_holds}$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.
special_soundness_obligation	551	operation <pre> op special_soundness_obligation (md : mode) : bool = forall (tr1 tr2 : transcript), valid_forking_pair md tr1 tr2 => extraction_success md tr1 tr2.</pre>	$\text{special_soundness_obligation} \subseteq \text{mode} \times \text{transcript} \times \text{transcript}$ is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.
special_soundness_with_public_reconstruction_obligation	556	operation <pre> op special_soundness_with_public_reconstruction_obligation (md : mode) : bool = forall (tr1 tr2 : transcript), valid_forking_pair md tr1 tr2 => fork_public_reconstruction_cases md tr1 tr2 /\ extraction_success md tr1 tr2.</pre>	$\text{special_soundness_with_public_reconstruction_obligation}$ is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.
	562	lemma		

	Line	Construct	What was written	Symbolic correspondence
module_sis_extraction_from_bimodal		<pre> lemma module_sis_extraction_from_bimodal md tr1 tr2 : bimodal_extraction_success md tr1 tr2 => bimodal_to_module_sis_lift_obligation md => extraction_success md tr1 tr2. </pre>	<pre> module_sis_extraction_from_bimodal : $P \Rightarrow Q$ is an implication used as a proof obligation. </pre>	Proves a bound that contributes to a game-hop or final security inequality.
special_soundness_from_bimodal_lift	582	lemma		
		<pre> lemma special_soundness_from_bimodal_lift md : forking_extraction_obligation md => bimodal_to_module_sis_lift_obligation md => special_soundness_obligation md. </pre>	<pre> special_soundness_from_bimodal_lift : $P \Rightarrow Q$ is an implication used as a proof obligation. </pre>	Proves a structural correctness fact used to justify later reductions.
special_soundness_with_public_reconstruction_from_bimodal_lift	592	lemma		
		<pre> lemma special_soundness_with_public_reconstruction_from_bimodal_lift md : fork_public_reconstruction_obligation md => forking_extraction_obligation md => bimodal_to_module_sis_lift_obligation md => special_soundness_with_public_reconstruction_obligation md. </pre>	<pre> special_soundness_with_public_reconstruction_from_bimodal_lift : $P \Rightarrow Q$ is an implication used as a proof obligation. </pre>	Proves a structural correctness fact used to justify later reductions.
transcript_from_honest_signing	607	operation		
		<pre> op transcript_from_honest_signing (md : mode) (sk : skey) (m : message) (ctx : context) (coins : random_coins) : transcript = transcript_of_signature md (public_key_of_secret md sk) m ctx (sign_internal md sk m ctx coins). </pre>	<pre> transcript_from_honest_signing : mode $\times$ skey $\times$ message $\times$ context $\times$ random_coins $\rightarrow$ transcript </pre>	Defines the random-oracle/hash interface data used by the games.
	615	lemma		

Line	Construct	What was written	Symbolic correspondence
honest_transcript_verifies	<pre> lemma honest_transcript_verifies md sk m ctx coins : transcript_verifies md (transcript_from_honest_signing md sk m ctx coins). </pre>	$\text{honest_transcript_verifies} : \forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later pro scripts can rewrite, apply, or compose it
625	lemma		
transcript_of_signature_fields	<pre> lemma transcript_of_signature_fields md pk m ctx sig : transcript_commitment_highbits (transcript_of_signature md pk m ctx sig) = sig_commitment_highbits sig /\ transcript_commitment_lowbits (transcript_of_signature md pk m ctx sig) = sig_commitment_lowbits sig /\ transcript_message_hash (transcript_of_signature md pk m ctx sig) = sig_message_hash sig /\... </pre>	$\text{transcript_of_signature_fields} : L = R$ is an equality/rewriting fact.	Records a reusable theorem so later pro scripts can rewrite, apply, or compose it
640	lemma		
transcript_of_signature_matches_signature	<pre> lemma transcript_of_signature_matches_signature md pk m ctx sig : transcript_fields_match_signature (transcript_of_signature md pk m ctx sig). </pre>	$\text{transcript_of_signature_matches_signature} : \forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later pro scripts can rewrite, apply, or compose it
647	lemma		
honest_transcript_challenge	<pre> lemma honest_transcript_challenge md sk m ctx coins : transcript_challenge (transcript_from_honest_signing md sk m ctx coins) = challenge_hash md (commitment_highbits md sk m ctx coins) (commitment_lowbits md sk m ctx coins) (message_hash (public_key_of_secret md sk) ctx m). </pre>	$\text{honest_transcript_challenge} : L = R$ is an equality/rewriting fact.	Proves a bound that contributes to a game-hop or final security inequality.

11 HAETAE_Events.ec

named rejection-failure events

Line	Construct	What was written	Symbolic correspondence	Why it is
9	operation			
	keygen_rejection_failure	op keygen_rejection_failure (md : mode) (pk : pkey) (sk : skey) : bool = ! valid_keypair md pk sk \/\ ! secret_norm_ok md sk.	keygen_rejection_failure $\subseteq$ mode $\times$ pkey $\times$ skey is a Boolean-valued predicate.	Names a bad event whose probability is later shown to be zero or bounded.
12	operation			
	signing_rejection_failure	op signing_rejection_failure (md : mode) (sig : signature) : bool = ! valid_signature md sig \/\ ! response_norm_ok md sig.	signing_rejection_failure $\subseteq$ mode $\times$ signature is a Boolean-valued predicate.	Names a bad event whose probability is later shown to be zero or bounded.
15	operation			
	verification_rejection_failure	op verification_rejection_failure (md : mode) (sig : signature) : bool = ! valid_signature md sig \/\ ! verify_norm_ok md sig.	verification_rejection_failure $\subseteq$ mode $\times$ signature is a Boolean-valued predicate.	Names a bad event whose probability is later shown to be zero or bounded.
18	operation			
	any_rejection_failure	op any_rejection_failure (md : mode) (pk : pkey) (sk : skey) (sig : signature) : bool = keygen_rejection_failure md pk sk \/\ signing_rejection_failure md sig \/\ verification_rejection_failure md sig.	any_rejection_failure $\subseteq$ mode $\times$ pkey $\times$ skey $\times$ signature is a Boolean-valued predicate.	Names a bad event whose probability is later shown to be zero or bounded.
24	lemma			
	keygen_rejection_free_current	lemma keygen_rejection_free_current md sd : ! keygen_rejection_failure md (keygen_internal md sd).‘1 (keygen_internal md sd).‘2.	keygen_rejection_free_current : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.
32	lemma			

Line	Construct	What was written	Symbolic correspondence	Why it is
signing_rejection_free_current	lemma signing_rejection_free_current md sk m ctx coins : ! signing_rejection_failure md (sign_internal md sk m ctx coins).	signing_rejection_free_current : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
40	lemma			
verification_rejection_free_current	lemma verification_rejection_free_current md sk m ctx coins : ! verification_rejection_failure md (sign_internal md sk m ctx coins).	verification_rejection_free_current : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
48	lemma			
accepted_transcript_rejection_free_current	lemma accepted_transcript_rejection_free_current md sd m ctx coins : ! any_rejection_failure md (keygen_internal md sd).‘1 (keygen_internal md sd).‘2 (sign_internal md (keygen_internal md sd).‘2 m ctx coins).	accepted_transcript_rejection_free_current : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	

12 HAETAE_Assumptions.ec

abstract MLWE, Module-SIS, and bimodal self-target games

Line	Construct	What was written	Symbolic correspondence	Why it
9	type mlwe_instance type mlwe_instance.	mlwe_instance is an abstract carrier set.	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.	
10	type module_sis_instance type module_sis_instance = matrix * polyveck.	module_sis_instance $\equiv$ matrix $\times$ polyveck	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.	
11	type module_sis_solution type module_sis_solution = polyvecl.	module_sis_solution $\equiv$ polyvecl	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.	
12	type bimodal_selftarget_msis_instance type bimodal_selftarget_msis_instance = module_sis_instance.	bimodal_selftarget_msis_instance $\equiv$ module_sis_instance	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.	
13	type bimodal_selftarget_msis_solution type bimodal_selftarget_msis_solution = module_sis_solution.	bimodal_selftarget_msis_solution $\equiv$ module_sis_solution	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.	
15	operation dmlwe_real op [lossless] dmlwe_real : mode -> mlwe_instance distr.	dmlwe_real : 1 $\rightarrow$ Distr(mode $\rightarrow$ mlwe_instance), with total probability mass 1.	Defines a sampler/distribution used by game procedures and probability bounds.	
16	operation dmlwe_random op [lossless] dmlwe_random : mode -> mlwe_instance distr.	dmlwe_random : 1 $\rightarrow$ Distr(mode $\rightarrow$ mlwe_instance), with total probability mass 1.	Defines a sampler/distribution used by game procedures and probability bounds.	
17	operation dmodule_sis_instance op [lossless] dmodule_sis_instance : mode -> module_sis_instance distr.	dmodule_sis_instance : 1 $\rightarrow$ Distr(mode $\rightarrow$ module_sis_instance), with total probability mass 1.	Defines a sampler/distribution used by game procedures and probability bounds.	
18	operation dbimodal_selftarget_msis_instance op [lossless] dbimodal_selftarget_msis_instance : mode -> bimodal_selftarget_msis_instance distr.	dbimodal_selftarget_msis_instance : 1 $\rightarrow$ Distr(mode $\rightarrow$ bimodal_selftarget_msis_instance), with total probability mass 1.	Defines a sampler/distribution used by game procedures and probability bounds.	

Line	Construct	What was written	Symbolic correspondence	Why it
21	operation			
module_sis_eval	<pre> op module_sis_eval (md : mode) (x : module_sis_instance) (sol : module_sis_solution) : polyveck = polyveck_sub md (matrix_vec_mul md x.'1 sol) x.'2. </pre>	<pre> module_sis_eval : mode × module_sis_instance × module_sis_solution → polyveck </pre>	Defines a total mathematical function used by the EasyCrypt model.	
26	operation			
module_sis_zero_instance	<pre> op module_sis_zero_instance (md : mode) : module_sis_instance = ([], polyveck_zero md). </pre>	<pre> module_sis_zero_instance : mode → module_sis_instance </pre>	Defines a total mathematical function used by the EasyCrypt model.	
29	operation			
module_sis_solution_wf	<pre> op module_sis_solution_wf (md : mode) (sol : module_sis_solution) : bool = polyvecl_wf md sol. </pre>	<pre> module_sis_solution_wf ⊆ mode × module_sis_solution is a Boolean-valued predicate. </pre>	Names a well-formedness or support condition so it can be reused in lemmas.	
32	operation			
module_sis_valid	<pre> op module_sis_valid (md : mode) (x : module_sis_instance) (sol : module_sis_solution) : bool = module_sis_solution_wf md sol /\ matrix_vec_mul md x.'1 sol = x.'2. </pre>	<pre> module_sis_valid ⊆ mode × module_sis_instance × module_sis_solution is a Boolean-valued predicate. </pre>	Names a well-formedness or support condition so it can be reused in lemmas.	
38	operation			
bimodal_selftarget_msis_valid	<pre> op bimodal_selftarget_msis_valid (md : mode) (x : bimodal_selftarget_msis_instance) (sol : bimodal_selftarget_msis_solution) : bool = module_sis_valid md x sol. </pre>	<pre> bimodal_selftarget_msis_valid ⊆ mode × bimodal_selftarget_msis_instance × bimodal_selftarget_msis_solution is a Boolean-valued predicate. </pre>	Names a well-formedness or support condition so it can be reused in lemmas.	
43	lemma			
module_sis_eval_zero_instance	<pre> lemma module_sis_eval_zero_instance md sol : module_sis_eval md (module_sis_zero_instance md) sol = polyveck_zero md. </pre>	<pre> module_sis_eval_zero_instance : L = R is an equality/rewriting fact. </pre>	Proves a bound that contributes to a game-hop or final security inequality.	
50	lemma			
module_sis_zero_valid	<pre> lemma module_sis_zero_valid md : module_sis_valid md (module_sis_zero_instance md) (polyvecl_zero md). </pre>	<pre> module_sis_zero_valid : ∀x̄. P(x̄) is a universally quantified fact. </pre>	Proves a bound that contributes to a game-hop or final security inequality.	

Line	Construct	What was written	Symbolic correspondence	Why it
59	lemma			
module_sis_eval_wf	lemma module_sis_eval_wf md x sol : polyveck_wf md (module_sis_eval md x sol).	module__sis__eval__wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.	
63	lemma			
module_sis_valid_target_wf	lemma module_sis_valid_target_wf md x sol : module_sis_valid md x sol => polyveck_wf md x.'2.	module__sis__valid__target__wf : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a bound that contributes to a game-hop or final security inequality.	
72	operation			
bimodal_to_module_sis_instance	op bimodal_to_module_sis_instance : mode -> bimodal_selftarget_msis_instance -> module_sis_instance = fun (_ : mode) x => x.	bimodal__to__module__sis__instance : mode → mode- > bimodal_selftarget_msis_instance- > module_sis_instance	Defines a total mathematical function used by the EasyCrypt model.	
75	operation			
bimodal_to_module_sis_solution	op bimodal_to_module_sis_solution : mode -> bimodal_selftarget_msis_instance -> bimodal_selftarget_msis_solution -> module_sis_solution = fun (_ : mode) (_ : bimodal_selftarget_msis_instance) sol => sol.	bimodal__to__module__sis__solution : mode × bimodal_selftarget_msis_instance → mode- > bimodal_selftarget_msis_instance- > bimodal_selftarget_msis_solution- > module_sis	Defines a total mathematical function used by the EasyCrypt model.	
80	lemma			
bimodal_to_module_sis_valid	lemma bimodal_to_module_sis_valid md x sol : bimodal_selftarget_msis_valid md x sol => module_sis_valid md (bimodal_to_module_sis_instance md x) (bimodal_to_module_sis_solution md x sol).	bimodal__to__module__sis__valid : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a bound that contributes to a game-hop or final security inequality.	
150	operation			

Line	Construct	What was written	Symbolic correspondence	Why it
	<pre> lift_bimodal_solution_option op lift_bimodal_solution_option (md : mode) (x : bimodal_selftarget_msis_instance) (sol : bimodal_selftarget_msis_solution option) : module_sis_solution option = if sol = None then None else Some (bimodal_to_module_sis_solution md x (oget sol)). </pre>	<pre> lift_bimodal_solution_option : mode × bimodal_selftarget_msis_instance × bimodal_selftarget_msis_solutionoption → module_sis_solutionoption </pre>	Defines a total mathematical function used by the EasyCrypt model.	
157	operation <pre> dmodule_sis_from_bimodal op dmodule_sis_from_bimodal (md : mode) : module_sis_instance distr = dmap (dbimodal_selftarget_msis_instance md) (bimodal_to_module_sis_instance md). </pre>	<pre> dmodule_sis_from_bimodal : mode → Distr(module_sis_instance) </pre>	Defines a sampler/distribution used by game procedures and probability bounds.	
161	lemma <pre> dmodule_sis_from_bimodalE lemma dmodule_sis_from_bimodalE md : dmodule_sis_from_bimodal md = dbimodal_selftarget_msis_instance md. </pre>	<pre> dmodule_sis_from_bimodalE : L = R is an equality/rewriting fact. </pre>	Proves a bound that contributes to a game-hop or final security inequality.	
167	lemma <pre> dmodule_sis_from_bimodal_lossless lemma dmodule_sis_from_bimodal_lossless md : is_lossless (dmodule_sis_from_bimodal md). </pre>	<pre> dmodule_sis_from_bimodal_lossless : ∀x̄. P(x̄) is a universally quantified fact. </pre>	Proves a distribution fact needed for sampling or probability mass reasoning.	
191	lemma <pre> bimodal_lift_successE lemma bimodal_lift_successE md x sol : (sol <> None /\ bimodal_selftarget_msis_valid md x (oget sol)) = (lift_bimodal_solution_option md x sol <> None /\ module_sis_valid md (bimodal_to_module_sis_instance md x) (oget (lift_bimodal_solution_option md x sol))). </pre>	<pre> bimodal_lift_successE : L = R is an equality/rewriting fact. </pre>	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	

	Line	Construct	What was written	Symbolic correspondence	Why it
	209	lemma			
bimodal_solver_lift_exact		<pre>lemma bimodal_solver_lift_exact &m md : Pr[BimodalSelfTargetMSIS_Relation_Game(B).main(md) @ &m : res] = Pr[BimodalToModuleSIS_Lift_Game(B).main(md) @ &m : res].</pre>	<pre>bimodal_solver_lift_exact : Pr[c : E] = B</pre> is a probability bound or exact game probability.	Proves an exact game equivalence or simulator transition.	
	292	lemma			
bimodal_mode_solver_lifted_distribution_exact		<pre>lemma bimodal_mode_solver_lifted_distribution_exact &m md : Pr[BimodalSelfTargetMSIS_ModeRelation_Game(B).main(md) @ &m : res] = Pr[ModuleSIS_LiftedDistribution_Relation_Game(BimodalModeSolver_As_ModuleSIS(B)).main(md) @ &m : res].</pre>	<pre>bimodal_mode_solver_lifted_distribution_exact : Pr[c : E] = B</pre> is a probability bound or exact game probability.	Proves an exact game equivalence or simulator transition.	
	362	lemma			
bimodal_msis_as_module_sis_exact		<pre>lemma bimodal_msis_as_module_sis_exact &m : Pr[BimodalSelfTargetMSIS_Game(B).main() @ &m : res] = Pr[ModuleSIS_Game(BimodalMSIS_As_ModuleSIS(B)).main() @ &m : res].</pre>	<pre>bimodal_msis_as_module_sis_exact : Pr[c : E] = B</pre> is a probability bound or exact game probability.	Proves a bound that contributes to a game-hop or final security inequality.	

13 HAETAEDistributions.ec

sampling distributions and support/cardinality facts

	Line	Construct	What was written	Symbolic correspondence	V
	10	type			
signing_randomness_source		type signing_randomness_source = int.	signing_randomness_source $\equiv \mathbb{Z}$	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.	
	12	operation			
dbyte		op dbyte : byte distr = dinter 0 255.	dbyte : $1 \rightarrow \text{Distr}(\text{byte})$	Defines a sampler/distribution used by game procedures and probability bounds.	
	13	operation			
dseed		op dseed : seed distr = dlist dbyte seedbytes.	dseed : $1 \rightarrow \text{Distr}(\text{seed})$	Defines a sampler/distribution used by game procedures and probability bounds.	
	14	operation			
dcrh		op dcrh : crh distr = dlist dbyte crhbytes.	dcrh : $1 \rightarrow \text{Distr}(\text{crh})$	Defines a sampler/distribution used by game procedures and probability bounds.	
	15	operation			
signing_entropy_token_cardinality		op signing_entropy_token_cardinality : int = 288230376151711744.	signing_entropy_token_cardinality : $1 \rightarrow \mathbb{Z}$	Defines a total mathematical function used by the EasyCrypt model.	
	16	operation			
signing_entropy_token_in_range		op signing_entropy_token_in_range (x : int) : bool = 0 <= x /\ x < signing_entropy_token_cardinality.	signing_entropy_token_in_range $\subseteq \mathbb{Z}$ is a Boolean-valued predicate.	Names a well-formedness or support condition so it can be reused in lemmas.	
	18	operation			
signing_randomness_byte_ok		op signing_randomness_byte_ok (b : byte) : bool = 0 <= b /\ b < 256.	signing_randomness_byte_ok $\subseteq \text{byte}$ is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.	
	19	operation			
signing_entropy_token_to_coins		op signing_entropy_token_to_coins (x : int) : random_coins = [x %% 256; (x %/ 256) %% 256; (x %/ 65536) %% 256; (x %/ 16777216) %% 256; (x %/ 4294967296) %% 256; (x %/ 1099511627776) %% 256; (x %/ 281474976710656) %% 256; x %/ 72057594037927936] ++ nseq (seedbytes - 8) 0.	signing_entropy_token_to_coins : $\mathbb{Z} \rightarrow \text{random_coins}$	Defines a total mathematical function used by the EasyCrypt model.	

	Line	Construct	What was written	Symbolic correspondence	V
	29	operation			
signing_randomness_domain		<pre> op signing_randomness_domain (coins : random_coins) : bool = size coins = seedbytes /\ all signing_randomness_byte_ok coins /\ signing_entropy_token_in_range (signing_entropy_token_of_coins coins). </pre>	$\text{signing_randomness_domain} \subseteq$ random_coins is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.	
	33	operation			
dsigning_entropy_token		<pre> op dsigning_entropy_token : int distr = dinter 0 (signing_entropy_token_cardinality - 1). </pre>	$\text{dsigning_entropy_token} : 1 \rightarrow \text{Distr}(\mathbb{Z})$	Defines a sampler/distribution used by game procedures and probability bounds.	
	35	operation			
dsigning_randomness_source		<pre> op dsigning_randomness_source : signing_randomness_source distr = dsigning_entropy_token. </pre>	$\text{dsigning_randomness_source} : 1 \rightarrow$ $\text{Distr}(\text{signing_randomness_source})$	Defines a sampler/distribution used by game procedures and probability bounds.	
	37	operation			
signing_randomness_source_valid		<pre> op signing_randomness_source_valid (src : signing_randomness_source) : bool = signing_entropy_token_in_range src. </pre>	$\text{signing_randomness_source_valid} \subseteq$ $\text{signing_randomness_source}$ is a Boolean-valued predicate.	Names a well-formedness or support condition so it can be reused in lemmas.	
	39	operation			
signing_randomness_from_source		<pre> op signing_randomness_from_source (src : signing_randomness_source) : random_coins = signing_entropy_token_to_coins src. </pre>	$\text{signing_randomness_from_source} :$ $\text{signing_randomness_source} \rightarrow$ random_coins	Defines the random-oracle/hash interface data used by the games.	
	42	operation			
dsigning_randomness		<pre> op dsigning_randomness : random_coins distr = dmap dsigning_randomness_source signing_randomness_from_source. </pre>	$\text{dsigning_randomness} : 1 \rightarrow$ $\text{Distr}(\text{random_coins})$	Defines a sampler/distribution used by game procedures and probability bounds.	
	44	operation			
drandom_coins		<pre> op drandom_coins : random_coins distr = dsigning_randomness. </pre>	$\text{drandom_coins} : 1 \rightarrow$ $\text{Distr}(\text{random_coins})$	Defines a sampler/distribution used by game procedures and probability bounds.	

	Line	Construct	What was written	Symbolic correspondence	V
	45	operation			
		signing_randomness_point_bound	op signing_randomness_point_bound : $\mathbb{R}$ real = 1%r / (signing_entropy_token_cardinality)%r.	signing_randomness_point_bound : $1 \rightarrow \mathbb{R}$	Defines a concrete numeric loss term used in the final security inequality.
	47	operation			
		signing_randomness_token_spread	op signing_randomness_token_spread (d : random_coins distr) : bool = forall token, mu d (fun coins => signing_entropy_token_of_coins coins = token) <= signing_randomness_point_bound.	signing_randomness_token_spread $\subseteq$ Distr(random_coins) is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.
	51	operation			
		signing_randomness_distribution_ok	op signing_randomness_distribution_ok (d : random_coins distr) : bool = is_lossless d /\ (forall coins, coins \in d => signing_randomness_domain coins) \/ signing_randomness_token_spread d.	signing_randomness_distribution_ok $\subseteq$ Distr(random_coins) is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.
	55	operation			
		dcoeff	op dcoeff : coeff distr = dinter 0 (q - 1).	dcoeff : $1 \rightarrow \text{Distr}(\text{coeff})$	Defines a sampler/distribution used by game procedures and probability bounds.
	56	operation			
		dpoly	op dpoly : poly distr = dlist dcoeff n.	dpoly : $1 \rightarrow \text{Distr}(\text{poly})$	Defines a sampler/distribution used by game procedures and probability bounds.
	57	operation			
		dpolyveck	op dpolyveck (md : mode) : polyveck distr = dlist dpoly (mode_k md).	dpolyveck : mode $\rightarrow \text{Distr}(\text{polyveck})$	Defines a sampler/distribution used by game procedures and probability bounds.
	58	operation			
		dpolyvecl	op dpolyvecl (md : mode) : polyvecl distr = dlist dpoly (mode_l md).	dpolyvecl : mode $\rightarrow \text{Distr}(\text{polyvecl})$	Defines a sampler/distribution used by game procedures and probability bounds.
	59	operation			
		dmatrix	op dmatrix (md : mode) : matrix distr = dlist (dpolyvecl md) (mode_k md).	dmatrix : mode $\rightarrow \text{Distr}(\text{matrix})$	Defines a sampler/distribution used by game procedures and probability bounds.

Line	Construct	What was written	Symbolic correspondence	V
65	operation			
dchallenge	op dchallenge (md : mode) : challenge distr = dmap dcrh (challenge_from_seed md).	dchallenge : mode $\rightarrow$ Distr(challenge)	Defines a sampler/distribution used by game procedures and probability bounds.	
68	type			
signing_sample_pair	type signing_sample_pair = polyvecl * polyveck.	signing_sample_pair $\equiv$ polyvecl $\times$ polyveck	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.	
70	operation			
signing_sample_pair_y1	op signing_sample_pair_y1 (s : signing_sample_pair) : polyvecl = s.'1.	signing_sample_pair_y1 : signing_sample_pair $\rightarrow$ polyvecl	Defines a total mathematical function used by the EasyCrypt model.	
71	operation			
signing_sample_pair_y2	op signing_sample_pair_y2 (s : signing_sample_pair) : polyveck = s.'2.	signing_sample_pair_y2 : signing_sample_pair $\rightarrow$ polyveck	Defines a total mathematical function used by the EasyCrypt model.	
73	operation			
signing_sample_pair_wf	op signing_sample_pair_wf (md : mode) (s : signing_sample_pair) : bool = polyvecl_wf md (signing_sample_pair_y1 s) /\ polyveck_wf md (signing_sample_pair_y2 s).	signing_sample_pair_wf $\subseteq$ mode $\times$ signing_sample_pair is a Boolean-valued predicate.	Names a well-formedness or support condition so it can be reused in lemmas.	
77	operation			
signing_sample_pair_unit	op signing_sample_pair_unit (md : mode) (s : signing_sample_pair) : bool = polyvecl_unit md (signing_sample_pair_y1 s) /\ polyveck_unit md (signing_sample_pair_y2 s).	signing_sample_pair_unit $\subseteq$ mode $\times$ signing_sample_pair is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.	
81	operation			
signing_sample_bound	op signing_sample_bound (md : mode) : int = mode_ln md.	signing_sample_bound : mode $\rightarrow \mathbb{Z}$	Defines a concrete numeric loss term used in the final security inequality.	
83	operation			
coeff_bounded_by	op coeff_bounded_by (b : int) (x : coeff) : bool = -b <= x /\ x <= b.	coeff_bounded_by $\subseteq \mathbb{Z} \times$ coeff is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.	
86	operation			

Line	Construct	What was written	Symbolic correspondence	V
	poly_bounded_by <pre> op poly_bounded_by (b : int) (p : poly) : bool = poly_wf p /\ forall i, 0 <= i < n => coeff_bounded_by b (poly_coeff p i).</pre>	poly_bounded_by $\subseteq \mathbb{Z} \times \text{poly}$ is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.	
91	operation polyvecl_bounded_by <pre> op polyvecl_bounded_by (md : mode) (b : int) (xs : polyvecl) : bool = polyvecl_wf md xs /\ forall row, 0 <= row < mode_l md => poly_bounded_by b (nth poly_zero xs row).</pre>	polyvecl_bounded_by $\subseteq \text{mode} \times \mathbb{Z} \times \text{polyvecl}$ is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.	
96	operation polyveck_bounded_by <pre> op polyveck_bounded_by (md : mode) (b : int) (xs : polyveck) : bool = polyveck_wf md xs /\ forall row, 0 <= row < mode_k md => poly_bounded_by b (nth poly_zero xs row).</pre>	polyveck_bounded_by $\subseteq \text{mode} \times \mathbb{Z} \times \text{polyveck}$ is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.	
101	operation signing_sample_pair_bounded <pre> op signing_sample_pair_bounded (md : mode) (s : signing_sample_pair) : bool = polyvecl_bounded_by md (signing_sample_bound md) (signing_sample_pair_y1 s) /\ polyveck_bounded_by md (signing_sample_bound md) (signing_sample_pair_y2 s).</pre>	signing_sample_pair_bounded $\subseteq \text{mode} \times \text{signing_sample_pair}$ is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.	
107	operation signing_sample_pair_sample_ok <pre> op signing_sample_pair_sample_ok (md : mode) (s : signing_sample_pair) : bool = signing_sample_pair_wf md s /\ signing_sample_pair_bounded md s.</pre>	signing_sample_pair_sample_ok $\subseteq \text{mode} \times \text{signing_sample_pair}$ is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.	
111	operation			

Line	Construct	What was written	Symbolic correspondence	V
	signing_sample_pair_of_coins op signing_sample_pair_of_coins (md : mode) (sk : skey) (m : message) (ctx : context) (coins : random_coins) : signing_sample_pair = (signing_sample_y1 md sk m ctx coins, signing_sample_y2 md sk m ctx coins).	signing_sample_pair_of_coins : mode $\times$ skey $\times$ message $\times$ context $\times$ random_coins $\rightarrow$ signing_sample_pair	Defines a total mathematical function used by the EasyCrypt model.	
117	operation dstructural_signing_sample_pair op dstructural_signing_sample_pair (md : mode) (sk : skey) (m : message) (ctx : context) (coins : random_coins) : signing_sample_pair distr = dunit (signing_sample_pair_of_coins md sk m ctx coins).	dstructural_signing_sample_pair : mode $\times$ skey $\times$ message $\times$ context $\times$ random_coins $\rightarrow$ Distr(signing_sample_pair)	Defines a sampler/distribution used by game procedures and probability bounds.	
122	operation dsigning_unit_coeff op dsigning_unit_coeff : coeff distr = duniform [-1; 1].	dsigning_unit_coeff : 1 $\rightarrow$ Distr(coeff)	Defines a sampler/distribution used by game procedures and probability bounds.	
123	operation dsigning_unit_poly op dsigning_unit_poly : poly distr = dlist dsigning_unit_coeff n.	dsigning_unit_poly : 1 $\rightarrow$ Distr(poly)	Defines a sampler/distribution used by game procedures and probability bounds.	
124	operation dsigning_unit_polyvecl op dsigning_unit_polyvecl (md : mode) : polyvecl distr = dlist dsigning_unit_poly (mode_l md).	dsigning_unit_polyvecl : mode $\rightarrow$ Distr(polyvecl)	Defines a sampler/distribution used by game procedures and probability bounds.	
126	operation dsigning_unit_polyveck op dsigning_unit_polyveck (md : mode) : polyveck distr = dlist dsigning_unit_poly (mode_k md).	dsigning_unit_polyveck : mode $\rightarrow$ Distr(polyveck)	Defines a sampler/distribution used by game procedures and probability bounds.	
129	operation signing_bounded_coeff_cardinality op signing_bounded_coeff_cardinality (md : mode) : int = 2 * signing_sample_bound md + 1.	signing_bounded_coeff_cardinality : mode $\rightarrow \mathbb{Z}$	Defines a concrete numeric loss term used in the final security inequality.	
131	operation			

Line	Construct	What was written	Symbolic correspondence	V
	<code>signing_bounded_coeff_point_bound</code> <code>op</code> <code>signing_bounded_coeff_point_bound</code> <code>(md : mode) : real = 1%r /</code> <code>(signing_bounded_coeff_cardinality</code> <code>md)%r.</code>	<code>signing_bounded_coeff_point_bound :</code> <code>mode → ℝ</code>	Defines a concrete numeric loss term used in the final security inequality.	
133	operation			
	<code>dsigning_bounded_coeff</code> <code>op dsigning_bounded_coeff (md :</code> <code>mode) : coeff distr = dinter</code> <code>(-(signing_sample_bound md))</code> <code>(signing_sample_bound md).</code>	<code>dsigning_bounded_coeff : mode →</code> <code>Distr(coeff)</code>	Defines a sampler/distribution used by game procedures and probability bounds.	
135	operation			
	<code>dsigning_bounded_poly</code> <code>op dsigning_bounded_poly (md :</code> <code>mode) : poly distr = dlist</code> <code>(dsigning_bounded_coeff md) n.</code>	<code>dsigning_bounded_poly : mode →</code> <code>Distr(poly)</code>	Defines a sampler/distribution used by game procedures and probability bounds.	
137	operation			
	<code>dsigning_bounded_polyvecl</code> <code>op dsigning_bounded_polyvecl (md</code> <code>: mode) : polyvecl distr =</code> <code>dlist (dsigning_bounded_poly md)</code> <code>(mode_l md).</code>	<code>dsigning_bounded_polyvecl : mode →</code> <code>Distr(polyvecl)</code>	Defines a sampler/distribution used by game procedures and probability bounds.	
139	operation			
	<code>dsigning_bounded_polyveck</code> <code>op dsigning_bounded_polyveck (md</code> <code>: mode) : polyveck distr =</code> <code>dlist (dsigning_bounded_poly md)</code> <code>(mode_k md).</code>	<code>dsigning_bounded_polyveck : mode →</code> <code>Distr(polyveck)</code>	Defines a sampler/distribution used by game procedures and probability bounds.	
145	operation			
	<code>dbounded_unit_signing_sample_pair</code> <code>op</code> <code>dbounded_unit_signing_sample_pair</code> <code>(md : mode) :</code> <code>signing_sample_pair distr = dlet</code> <code>(dsigning_unit_polyvecl md) (fun</code> <code>y1 => dmap</code> <code>(dsigning_unit_polyveck md) (fun</code> <code>y2 => (y1, y2)))</code> .	<code>dbounded_unit_signing_sample_pair :</code> <code>mode → Distr(signing_sample_pair)</code>	Defines a sampler/distribution used by game procedures and probability bounds.	
156	operation			
	<code>dchecked_hyperball_signing_sample_pair</code> <code>op</code> <code>dchecked_hyperball_signing_sample_pair</code> <code>(md : mode) :</code> <code>signing_sample_pair distr = dlet</code> <code>(dsigning_bounded_polyvecl md)</code> <code>(fun y1 => dmap</code> <code>(dsigning_bounded_polyveck md)</code> <code>(fun y2 => (y1, y2)))</code> .	<code>dchecked_hyperball_signing_sample_pair :</code> <code>mode → Distr(signing_sample_pair)</code>	Defines a sampler/distribution used by game procedures and probability bounds.	

Line	Construct	What was written	Symbolic correspondence	V
163	operation			
hyperball_coeff_ok	op hyperball_coeff_ok (md : mode) (c : coeff) : bool = coeff_bounded_by (signing_sample_bound md) c.	hyperball_coeff_ok $\subseteq$ mode $\times$ coeff is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.	
165	operation			
hyperball_poly_ok	op hyperball_poly_ok (md : mode) (p : poly) : bool = poly_bounded_by (signing_sample_bound md) p.	hyperball_poly_ok $\subseteq$ mode $\times$ poly is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.	
167	operation			
hyperball_polyvecl_ok	op hyperball_polyvecl_ok (md : mode) (y1 : polyvecl) : bool = polyvecl_bounded_by md (signing_sample_bound md) y1.	hyperball_polyvecl_ok $\subseteq$ mode $\times$ polyvecl is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.	
169	operation			
hyperball_polyveck_ok	op hyperball_polyveck_ok (md : mode) (y2 : polyveck) : bool = polyveck_bounded_by md (signing_sample_bound md) y2.	hyperball_polyveck_ok $\subseteq$ mode $\times$ polyveck is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.	
171	operation			
hyperball_sample_ok	op hyperball_sample_ok (md : mode) (s : signing_sample_pair) : bool = hyperball_polyvecl_ok md (signing_sample_pair_y1 s) /\ hyperball_polyveck_ok md (signing_sample_pair_y2 s).	hyperball_sample_ok $\subseteq$ mode $\times$ signing_sample_pair is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.	
175	operation			
dhyperball_coeff	op dhyperball_coeff (md : mode) : coeff distr = dsigning_bounded_coeff md.	dhyperball_coeff : mode $\rightarrow$ Distr(coeff)	Defines a sampler/distribution used by game procedures and probability bounds.	
177	operation			
dhyperball_poly	op dhyperball_poly (md : mode) : poly distr = dsigning_bounded_poly md.	dhyperball_poly : mode $\rightarrow$ Distr(poly)	Defines a sampler/distribution used by game procedures and probability bounds.	
179	operation			
dhyperball_polyvecl	op dhyperball_polyvecl (md : mode) : polyvecl distr = dsigning_bounded_polyvecl md.	dhyperball_polyvecl : mode $\rightarrow$ Distr(polyvecl)	Defines a sampler/distribution used by game procedures and probability bounds.	
181	operation			

	Line	Construct	What was written	Symbolic correspondence	V
		dhyperball_polyveck	op dhyperball_polyveck (md : mode) : polyveck distr = dsigning_bounded_polyveck md.	dhyperball_polyveck : mode → Distr(polyveck)	Defines a sampler/distribution used by game procedures and probability bounds.
	184	operation			
		hyperball_coeff_point_bound	op hyperball_coeff_point_bound (md : mode) : real = signing_bounded_coeff_point_bound md.	hyperball_coeff_point_bound : mode → $\mathbb{R}$	Defines a concrete numeric loss term used in the final security inequality.
	186	operation			
		hyperball_poly_point_bound	op hyperball_poly_point_bound (md : mode) : real = hyperball_coeff_point_bound md ^ n.	hyperball_poly_point_bound : mode → $\mathbb{R}$	Defines a concrete numeric loss term used in the final security inequality.
	188	operation			
		hyperball_polyvecl_point_bound	op hyperball_polyvecl_point_bound (md : mode) : real = hyperball_poly_point_bound md ^ (mode_l md).	hyperball_polyvecl_point_bound : mode → $\mathbb{R}$	Defines a concrete numeric loss term used in the final security inequality.
	190	operation			
		hyperball_polyveck_point_bound	op hyperball_polyveck_point_bound (md : mode) : real = hyperball_poly_point_bound md ^ (mode_k md).	hyperball_polyveck_point_bound : mode → $\mathbb{R}$	Defines a concrete numeric loss term used in the final security inequality.
	192	operation			
		hyperball_sample_pair_point_bound	op hyperball_sample_pair_point_bound (md : mode) : real = hyperball_polyvecl_point_bound md * hyperball_polyveck_point_bound md.	hyperball_sample_pair_point_bound : mode → $\mathbb{R}$	Defines a concrete numeric loss term used in the final security inequality.
	201	operation			
		dexact_hyperball_signing_sample_pair	op dexact_hyperball_signing_sample_pair (md : mode) : signing_sample_pair distr = (dhyperball_polyvecl md) ‘*‘ (dhyperball_polyveck md).	dexact_hyperball_signing_sample_pair : mode → Distr(signing_sample_pair)	Defines a sampler/distribution used by game procedures and probability bounds.
	205	operation			

Line	Construct	What was written	Symbolic correspondence	V
	<pre> signing_sample_pair_distribution_ok op signing_sample_pair_distribution_ok (md : mode) (d : signing_sample_pair distr) : bool = is_lossless d /\ (forall s, s \in d => signing_sample_pair_sample_ok md s).</pre>	<pre> signing_sample_pair_distribution_ok ⊆ mode × Distr(signing_sample_pair) is a Boolean-valued predicate.</pre>	Names a Boolean mathematical condition used in statements and invariants.	
211	lemma			
byte_distribution_lossless	<pre> lemma byte_distribution_lossless : is_lossless dbyte.</pre>	<pre> byte_distribution_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.</pre>	Proves a distribution fact needed for sampling or probability mass reasoning.	
214	lemma			
seed_distribution_lossless	<pre> lemma seed_distribution_lossless : is_lossless dseed.</pre>	<pre> seed_distribution_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.</pre>	Proves a distribution fact needed for sampling or probability mass reasoning.	
220	lemma			
crh_distribution_lossless	<pre> lemma crh_distribution_lossless : is_lossless dcrh.</pre>	<pre> crh_distribution_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.</pre>	Proves a distribution fact needed for sampling or probability mass reasoning.	
226	lemma			
signing_entropy_token_cardinality_positive	<pre> lemma signing_entropy_token_cardinality_positive : 0 < signing_entropy_token_cardinality.</pre>	<pre> signing_entropy_token_cardinality_positive : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.</pre>	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
230	lemma			
signing_entropy_token_distribution_lossless	<pre> lemma signing_entropy_token_distribution_lossless : is_lossless dsigning_entropy_token.</pre>	<pre> signing_entropy_token_distribution_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.</pre>	Proves a distribution fact needed for sampling or probability mass reasoning.	
238	lemma			
signing_randomness_source_distribution_lossless	<pre> lemma signing_randomness_source_distribution_lossless : is_lossless dsigning_randomness_source.</pre>	<pre> signing_randomness_source_distribution_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.</pre>	Proves a distribution fact needed for sampling or probability mass reasoning.	
245	lemma			
signing_coin_distribution_lossless	<pre> lemma signing_coin_distribution_lossless : is_lossless drandom_coins.</pre>	<pre> signing_coin_distribution_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.</pre>	Proves a distribution fact needed for sampling or probability mass reasoning.	
251	lemma			

	Line	Construct	What was written	Symbolic correspondence	V
signing_entropy_token_to_coins_tokenK		lemma	signing_entropy_token_to_coins_tokenK :	Records a reusable theorem so later proof	
		signing_entropy_token_to_coins_tokenK	$L = R$ is an equality/rewriting fact.	scripts can rewrite, apply, or compose it.	
		x :			
		signing_entropy_token_of_coins			
		(signing_entropy_token_to_coins			
		x) = x.			
	260	lemma			
signing_randomness_from_source_tokenK		lemma	signing_randomness_from_source_tokenK :	Records a reusable theorem so later proof	
		signing_randomness_from_source_tokenK	$L = R$ is an equality/rewriting fact.	scripts can rewrite, apply, or compose it.	
		src :			
		signing_entropy_token_of_coins			
		(signing_randomness_from_source			
		src) = src.			
	267	lemma			
signing_entropy_token_to_coins_size		lemma	signing_entropy_token_to_coins_size :	Records a reusable theorem so later proof	
		signing_entropy_token_to_coins_size	$L = R$ is an equality/rewriting fact.	scripts can rewrite, apply, or compose it.	
		x : size			
		(signing_entropy_token_to_coins			
		x) = seedbytes.			
	279	lemma			
signing_entropy_token_to_coins_byte_domain		lemma	signing_entropy_token_to_coins_byte_domain :	Records a reusable theorem so later proof	
		signing_entropy_token_to_coins_byte_domain	$R \Rightarrow Q$ is an implication used as a proof	scripts can rewrite, apply, or compose it.	
		x :	obligation.		
		signing_entropy_token_in_range x			
		=> all			
		signing_randomness_byte_ok			
		(signing_entropy_token_to_coins			
		x).			
	292	lemma			
signing_entropy_token_to_coins_domain		lemma	signing_entropy_token_to_coins_domain :	Records a reusable theorem so later proof	
		signing_entropy_token_to_coins_domain	$R \Rightarrow Q$ is an implication used as a proof	scripts can rewrite, apply, or compose it.	
		x :	obligation.		
		signing_entropy_token_in_range x			
		=> signing_randomness_domain			
		(signing_entropy_token_to_coins			
		x).			
	305	lemma			

	Line	Construct	What was written	Symbolic correspondence	V
		<pre> signing_randomness_from_source_domain lemma signing_randomness_from_source_domain : src : signing_randomness_source_valid src => signing_randomness_domain (signing_randomness_from_source src). </pre>	<pre> signing_randomness_from_source_domain : P => Q is an implication used as a proof obligation. </pre>	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
	314	<pre> lemma signing_entropy_token_distribution_point_bound lemma signing_entropy_token_distribution_point_bound : x : mu dsigning_entropy_token (pred1 x) <= signing_randomness_point_bound. </pre>	<pre> signing_entropy_token_distribution_point_bound : X <= Y is an arithmetic or probability upper bound. </pre>	Bounds a bound that contributes to a game-hop or final security inequality.	
	326	<pre> lemma signing_randomness_source_distribution_point_bound lemma signing_randomness_source_distribution_point_bound : src : mu dsigning_randomness_source (pred1 src) <= signing_randomness_point_bound. </pre>	<pre> signing_randomness_source_distribution_point_bound : X <= Y is an arithmetic or probability upper bound. </pre>	Probes a bound that contributes to a game-hop or final security inequality.	
	334	<pre> lemma signing_coin_distribution_token_point_bound lemma signing_coin_distribution_token_point_bound : token : mu drandom_coins (fun coins => signing_entropy_token_of_coins coins = token) <= signing_randomness_point_bound. </pre>	<pre> signing_coin_distribution_token_point_bound : X <= Y is an arithmetic or probability upper bound. </pre>	Proves a bound that contributes to a game-hop or final security inequality.	
	348	<pre> lemma signing_coin_distribution_token_spread lemma signing_coin_distribution_token_spread : : signing_randomness_token_spread drandom_coins. </pre>	<pre> signing_coin_distribution_token_spread : forall x. P(x) is a universally quantified fact. </pre>	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
	355	<pre> lemma signing_coin_distribution_domain lemma signing_coin_distribution_domain : coins : coins \in drandom_coins => signing_randomness_domain coins. </pre>	<pre> signing_coin_distribution_domain : P => Q is an implication used as a proof obligation. </pre>	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
	371	<pre> lemma </pre>			

Line	Construct	What was written	Symbolic correspondence	V
signing_coin_distribution_domain_full	lemma signing_coin_distribution_domain_full : mu drandom_coins signing_randomness_domain = 1%r.	signing_coin_distribution_domain_full : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
379	lemma			
signing_coin_distribution_ok	lemma signing_coin_distribution_ok : signing_randomness_distribution_ok drandom_coins.	signing_coin_distribution_ok : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
390	lemma			
coeff_distribution_lossless	lemma coeff_distribution_lossless : is_lossless dcoeff.	coeff_distribution_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a distribution fact needed for sampling or probability mass reasoning.	
393	lemma			
poly_distribution_lossless	lemma poly_distribution_lossless : is_lossless dpoly.	poly_distribution_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a distribution fact needed for sampling or probability mass reasoning.	
399	lemma			
polyveck_distribution_lossless	lemma polyveck_distribution_lossless md : is_lossless (dpolyveck md).	polyveck_distribution_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a distribution fact needed for sampling or probability mass reasoning.	
405	lemma			
polyvecl_distribution_lossless	lemma polyvecl_distribution_lossless md : is_lossless (dpolyvecl md).	polyvecl_distribution_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a distribution fact needed for sampling or probability mass reasoning.	
411	lemma			
matrix_distribution_lossless	lemma matrix_distribution_lossless md : is_lossless (dmatrix md).	matrix_distribution_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a distribution fact needed for sampling or probability mass reasoning.	
417	lemma			
challenge_distribution_lossless	lemma challenge_distribution_lossless md : is_lossless (dchallenge md).	challenge_distribution_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a distribution fact needed for sampling or probability mass reasoning.	
423	lemma			
signing_sample_pair_of_coins_wf	lemma signing_sample_pair_of_coins_wf md sk m ctx coins : signing_sample_pair_wf md (signing_sample_pair_of_coins md sk m ctx coins).	signing_sample_pair_of_coins_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.	

Line	Construct	What was written	Symbolic correspondence	V
434	lemma signing_sample_pair_of_coins_unit	lemma signing_sample_pair_of_coins_unit md sk m ctx coins : signing_sample_pair_unit md (signing_sample_pair_of_coins md sk m ctx coins).	signing_sample_pair_of_coins_unit : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.
445	lemma signing_sample_bound_gt0	lemma signing_sample_bound_gt0 md : 0 < signing_sample_bound md.	signing_sample_bound_gt0 : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.
449	lemma unit_coeff_bounded_by_signing_sample_bound	lemma unit_coeff_bounded_by_signing_sample_bound md c : unit_coeff_ok c => coeff_bounded_by (signing_sample_bound md) c.	unit_coeff_bounded_by_signing_sample_bound : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a bound that contributes to a game-hop or final security inequality.
457	lemma poly_unit_bounded_by_signing_sample_bound	lemma poly_unit_bounded_by_signing_sample_bound md p : poly_unit p => poly_bounded_by (signing_sample_bound md) p.	poly_unit_bounded_by_signing_sample_bound : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a bound that contributes to a game-hop or final security inequality.
470	lemma polyvecl_unit_bounded_by_signing_sample_bound	lemma polyvecl_unit_bounded_by_signing_sample_bound md y1 : polyvecl_unit md y1 => polyvecl_bounded_by md (signing_sample_bound md) y1.	polyvecl_unit_bounded_by_signing_sample_bound : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a bound that contributes to a game-hop or final security inequality.
482	lemma polyveck_unit_bounded_by_signing_sample_bound	lemma polyveck_unit_bounded_by_signing_sample_bound md y2 : polyveck_unit md y2 => polyveck_bounded_by md (signing_sample_bound md) y2.	polyveck_unit_bounded_by_signing_sample_bound : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a bound that contributes to a game-hop or final security inequality.
494	lemma signing_sample_pair_unit_bounded	lemma signing_sample_pair_unit_bounded md s : signing_sample_pair_unit md s => signing_sample_pair_bounded md s.	signing_sample_pair_unit_bounded : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a bound that contributes to a game-hop or final security inequality.

	Line	Construct	What was written	Symbolic correspondence	V
	505	lemma			
signing_sample_pair_of_coins_sample_ok		lemma signing_sample_pair_of_coins_sample_ok md sk m ctx coins : signing_sample_pair_sample_ok md (signing_sample_pair_of_coins md sk m ctx coins).	signing_sample_pair_of_coins_sample_ok : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.	
	516	lemma			
structural_signing_sample_pair_lossless		lemma structural_signing_sample_pair_lossless md sk m ctx coins : is_lossless (dstructural_signing_sample_pair md sk m ctx coins).	structural_signing_sample_pair_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a distribution fact needed for sampling or probability mass reasoning.	
	523	lemma			
structural_signing_sample_pair_support		lemma structural_signing_sample_pair_support md sk m ctx coins s : s \in dstructural_signing_sample_pair md sk m ctx coins => signing_sample_pair_wf md s /\ signing_sample_pair_unit md s.	structural_signing_sample_pair_support : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a distribution fact needed for sampling or probability mass reasoning.	
	534	lemma			
structural_signing_sample_pair_sample_ok		lemma structural_signing_sample_pair_sample_ok md sk m ctx coins s : s \in dstructural_signing_sample_pair md sk m ctx coins => signing_sample_pair_sample_ok md s.	structural_signing_sample_pair_sample_ok : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a bound that contributes to a game-hop or final security inequality.	
	543	lemma			
structural_signing_sample_pair_distribution_ok		lemma structural_signing_sample_pair_distribution_ok md sk m ctx coins : signing_sample_pair_distribution_ok md (dstructural_signing_sample_pair md sk m ctx coins).	structural_signing_sample_pair_distribution_ok : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.	
	553	lemma			
signing_unit_coeff_lossless		lemma signing_unit_coeff_lossless : is_lossless dsigning_unit_coeff.	signing_unit_coeff_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a distribution fact needed for sampling or probability mass reasoning.	

	Line	Construct	What was written	Symbolic correspondence	V
	560	lemma			
signing_unit_coeff_support		lemma signing_unit_coeff_support c : c \in dsigning_unit_coeff => unit_coeff_ok c.	signing_unit_coeff_support : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a distribution fact needed for sampling or probability mass reasoning.	
	568	lemma			
signing_bounded_coeff_cardinality_positive		lemma signing_bounded_coeff_cardinality_positive md : 0 < signing_bounded_coeff_cardinality md.	signing_bounded_coeff_cardinality_positive : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.	
	575	lemma			
signing_bounded_coeff_lossless		lemma signing_bounded_coeff_lossless md : is_lossless (dsigning_bounded_coeff md).	signing_bounded_coeff_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a distribution fact needed for sampling or probability mass reasoning.	
	583	lemma			
signing_bounded_coeff_support		lemma signing_bounded_coeff_support md c : c \in dsigning_bounded_coeff md => coeff_bounded_by (signing_sample_bound md) c.	signing_bounded_coeff_support : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a distribution fact needed for sampling or probability mass reasoning.	
	590	lemma			
signing_bounded_coeff_supportP		lemma signing_bounded_coeff_supportP md c : c \in dsigning_bounded_coeff md <=> coeff_bounded_by (signing_sample_bound md) c.	signing_bounded_coeff_supportP : $X \leq Y$ is an arithmetic or probability upper bound.	Proves a distribution fact needed for sampling or probability mass reasoning.	
	597	lemma			
signing_bounded_coeff_point_bound		lemma signing_bounded_coeff_point_bound md c : mu (dsigning_bounded_coeff md) (pred1 c) <= signing_bounded_coeff_point_bound md.	signing_bounded_coeff_point_bound : $X \leq Y$ is an arithmetic or probability upper bound.	Proves a bound that contributes to a game-hop or final security inequality.	
	616	lemma			

Line	Construct	What was written	Symbolic correspondence	V
625	signing_bounded_coeff_point_bound_nonnegative lemma signing_bounded_coeff_point_bound_nonnegative md : 0%r <= signing_bounded_coeff_point_bound md. lemma	signing_bounded_coeff_point_bound_nonnegative : $X \leq Y$ is an arithmetic or probability upper bound.	Proves a bound that contributes to a game-hop or final security inequality.	
647	dlist_size_point_bound lemma dlist_size_point_bound ['a] (d : 'a distr) bd xs : 0%r <= bd => (forall x, mu d (pred1 x) <= bd) => mu (dlist d (size xs)) (pred1 xs) <= bd ^ (size xs). lemma	dlist_size_point_bound : $X \leq Y$ is an arithmetic or probability upper bound.	Proves a bound that contributes to a game-hop or final security inequality.	
661	dlist_point_bound lemma dlist_point_bound ['a] (d : 'a distr) len bd xs : 0 <= len => 0%r <= bd => (forall x, mu d (pred1 x) <= bd) => mu (dlist d len) (pred1 xs) <= bd ^ len. lemma	dlist_point_bound : $X \leq Y$ is an arithmetic or probability upper bound.	Proves a bound that contributes to a game-hop or final security inequality.	
668	hyperball_coeff_point_bound_nonnegative lemma hyperball_coeff_point_bound_nonnegative md : 0%r <= hyperball_coeff_point_bound md. lemma	hyperball_coeff_point_bound_nonnegative : $X \leq Y$ is an arithmetic or probability upper bound.	Proves a bound that contributes to a game-hop or final security inequality.	
676	hyperball_coeff_point_boundE lemma hyperball_coeff_point_boundE md c : mu (dhyperball_coeff md) (pred1 c) <= hyperball_coeff_point_bound md. lemma	hyperball_coeff_point_boundE : $X \leq Y$ is an arithmetic or probability upper bound.	Proves a bound that contributes to a game-hop or final security inequality.	
683	signing_unit_poly_lossless lemma signing_unit_poly_lossless : is_lossless dsigning_unit_poly. lemma	signing_unit_poly_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a distribution fact needed for sampling or probability mass reasoning.	
695	signing_unit_poly_support lemma signing_unit_poly_support p : p \in dsigning_unit_poly => poly_unit p. lemma	signing_unit_poly_support : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a distribution fact needed for sampling or probability mass reasoning.	

	Line	Construct	What was written	Symbolic correspondence	V
		signing_bounded_poly_lossless	lemma signing_bounded_poly_lossless md : is_lossless (dsigning_bounded_poly md).	signing_bounded_poly_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a distribution fact needed for sampling or probability mass reasoning.
	702	lemma			
		signing_bounded_poly_support	lemma signing_bounded_poly_support md p : p \in dsigning_bounded_poly md => poly_bounded_by (signing_sample_bound md) p.	signing_bounded_poly_support : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a distribution fact needed for sampling or probability mass reasoning.
	717	lemma			
		signing_bounded_poly_supportP	lemma signing_bounded_poly_supportP md p : p \in dsigning_bounded_poly md <=> poly_bounded_by (signing_sample_bound md) p.	signing_bounded_poly_supportP : $X \leq Y$ is an arithmetic or probability upper bound.	Proves a distribution fact needed for sampling or probability mass reasoning.
	738	lemma			
		hyperball_poly_point_bound_nonnegative	lemma hyperball_poly_point_bound_nonnegative md md : 0%r <= hyperball_poly_point_bound md.	hyperball_poly_point_bound_nonnegative : $X \leq Y$ is an arithmetic or probability upper bound.	Proves a bound that contributes to a game-hop or final security inequality.
	745	lemma			
		hyperball_poly_lossless	lemma hyperball_poly_lossless md : is_lossless (dhyperball_poly md).	hyperball_poly_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a distribution fact needed for sampling or probability mass reasoning.
	751	lemma			
		hyperball_poly_support	lemma hyperball_poly_support md p : p \in dhyperball_poly md => hyperball_poly_ok md p.	hyperball_poly_support : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a distribution fact needed for sampling or probability mass reasoning.
	759	lemma			
		hyperball_poly_supportP	lemma hyperball_poly_supportP md p : p \in dhyperball_poly md <=> hyperball_poly_ok md p.	hyperball_poly_supportP : $X \leq Y$ is an arithmetic or probability upper bound.	Proves a distribution fact needed for sampling or probability mass reasoning.
	766	lemma			
		hyperball_poly_point_boundE	lemma hyperball_poly_point_boundE md p : mu (dhyperball_poly md) (pred1 p) <= hyperball_poly_point_bound md.	hyperball_poly_point_boundE : $X \leq Y$ is an arithmetic or probability upper bound.	Proves a bound that contributes to a game-hop or final security inequality.
	778	lemma			

	Line	Construct	What was written	Symbolic correspondence	V
		signing_unit_polyvecl_lossless lemma signing_unit_polyvecl_lossless md : is_lossless (dsigning_unit_polyvecl md). 785 lemma	signing_unit_polyvecl_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a distribution fact needed for sampling or probability mass reasoning.	
		signing_unit_polyveck_lossless lemma signing_unit_polyveck_lossless md : is_lossless (dsigning_unit_polyveck md). 792 lemma	signing_unit_polyveck_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a distribution fact needed for sampling or probability mass reasoning.	
		signing_bounded_polyvecl_lossless lemma signing_bounded_polyvecl_lossless md : is_lossless (dsigning_bounded_polyvecl md). 799 lemma	signing_bounded_polyvecl_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a distribution fact needed for sampling or probability mass reasoning.	
		signing_bounded_polyveck_lossless lemma signing_bounded_polyveck_lossless md : is_lossless (dsigning_bounded_polyveck md). 806 lemma	signing_bounded_polyveck_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a distribution fact needed for sampling or probability mass reasoning.	
		signing_unit_polyvecl_support lemma signing_unit_polyvecl_support md y1 : y1 \in dsigning_unit_polyvecl md => polyvecl_unit md y1. 825 lemma	signing_unit_polyvecl_support : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a distribution fact needed for sampling or probability mass reasoning.	
		signing_unit_polyveck_support lemma signing_unit_polyveck_support md y2 : y2 \in dsigning_unit_polyveck md => polyveck_unit md y2. 844 lemma	signing_unit_polyveck_support : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a distribution fact needed for sampling or probability mass reasoning.	
		signing_bounded_polyvecl_support lemma signing_bounded_polyvecl_support md y1 : y1 \in dsigning_bounded_polyvecl md => polyvecl_bounded_by md (signing_sample_bound md) y1. 865 lemma	signing_bounded_polyvecl_support : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a distribution fact needed for sampling or probability mass reasoning.	

Line	Construct	What was written	Symbolic correspondence	V
	<code>signing_bounded_polyvecl_supportP</code> <code>lemma</code> <code>signing_bounded_polyvecl_supportP</code> <code>md y1 : y1 \in</code> <code>dsigning_bounded_polyvecl md <=></code> <code>polyvecl_bounded_by md</code> <code>(signing_sample_bound md) y1.</code>	<code>signing_bounded_polyvecl_supportP :</code> $X \leq Y$ is an arithmetic or probability upper bound.	Proves a distribution fact needed for sampling or probability mass reasoning.	
892	lemma			
	<code>signing_bounded_polyveck_support</code> <code>lemma</code> <code>signing_bounded_polyveck_support</code> <code>md y2 : y2 \in</code> <code>dsigning_bounded_polyveck md =></code> <code>polyveck_bounded_by md</code> <code>(signing_sample_bound md) y2.</code>	<code>signing_bounded_polyveck_support :</code> $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a distribution fact needed for sampling or probability mass reasoning.	
913	lemma			
	<code>signing_bounded_polyveck_supportP</code> <code>lemma</code> <code>signing_bounded_polyveck_supportP</code> <code>md y2 : y2 \in</code> <code>dsigning_bounded_polyveck md <=></code> <code>polyveck_bounded_by md</code> <code>(signing_sample_bound md) y2.</code>	<code>signing_bounded_polyveck_supportP :</code> $X \leq Y$ is an arithmetic or probability upper bound.	Proves a distribution fact needed for sampling or probability mass reasoning.	
940	lemma			
	<code>hyperball_polyvecl_point_bound_nonnegative</code> <code>lemma</code> <code>hyperball_polyvecl_point_bound_nonnegative</code> <code>md : 0%r <=</code> <code>hyperball_polyvecl_point_bound</code> <code>md.</code>	<code>hyperball_polyvecl_point_bound_nonnegativeP :</code> $X \leq Y$ is an arithmetic or probability upper bound.	Proves a bound that contributes to a game-hop or final security inequality.	
947	lemma			
	<code>hyperball_polyveck_point_bound_nonnegative</code> <code>lemma</code> <code>hyperball_polyveck_point_bound_nonnegative</code> <code>md : 0%r <=</code> <code>hyperball_polyveck_point_bound</code> <code>md.</code>	<code>hyperball_polyveck_point_bound_nonnegativeP :</code> $X \leq Y$ is an arithmetic or probability upper bound.	Proves a bound that contributes to a game-hop or final security inequality.	
954	lemma			
	<code>hyperball_sample_pair_point_bound_nonnegative</code> <code>lemma</code> <code>hyperball_sample_pair_point_bound_nonnegative</code> <code>md : 0%r <=</code> <code>hyperball_sample_pair_point_bound</code> <code>md.</code>	<code>hyperball_sample_pair_point_bound_nonnegativeP :</code> $X \leq Y$ is an arithmetic or probability upper bound.	Proves a bound that contributes to a game-hop or final security inequality.	
963	lemma			

	Line	Construct	What was written	Symbolic correspondence	V
		hyperball_polyvecl_lossless	lemma hyperball_polyvecl_lossless md : is_lossless (dhyperball_polyvecl md).	hyperball__polyvecl__lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a distribution fact needed for sampling or probability mass reasoning.
	969	lemma			
		hyperball_polyveck_lossless	lemma hyperball_polyveck_lossless md : is_lossless (dhyperball_polyveck md).	hyperball__polyveck__lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a distribution fact needed for sampling or probability mass reasoning.
	975	lemma			
		hyperball_polyvecl_support	lemma hyperball_polyvecl_support md y1 : y1 \in dhyperball_polyvecl md => hyperball_polyvecl_ok md y1.	hyperball__polyvecl__support : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a distribution fact needed for sampling or probability mass reasoning.
	983	lemma			
		hyperball_polyvecl_supportP	lemma hyperball_polyvecl_supportP md y1 : y1 \in dhyperball_polyvecl md <=> hyperball_polyvecl_ok md y1.	hyperball__polyvecl__supportP : $X \leq Y$ is an arithmetic or probability upper bound.	Proves a distribution fact needed for sampling or probability mass reasoning.
	990	lemma			
		hyperball_polyveck_support	lemma hyperball_polyveck_support md y2 : y2 \in dhyperball_polyveck md => hyperball_polyveck_ok md y2.	hyperball__polyveck__support : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a distribution fact needed for sampling or probability mass reasoning.
	998	lemma			
		hyperball_polyveck_supportP	lemma hyperball_polyveck_supportP md y2 : y2 \in dhyperball_polyveck md <=> hyperball_polyveck_ok md y2.	hyperball__polyveck__supportP : $X \leq Y$ is an arithmetic or probability upper bound.	Proves a distribution fact needed for sampling or probability mass reasoning.
	1005	lemma			
		hyperball_polyvecl_point_boundE	lemma hyperball_polyvecl_point_boundE md y1 : mu (dhyperball_polyvecl md) (pred1 y1) <= hyperball_polyvecl_point_bound md.	hyperball__polyvecl__point__boundE : $X \leq$ Y is an arithmetic or probability upper bound.	Proves a bound that contributes to a game-hop or final security inequality.
	1017	lemma			

	Line	Construct	What was written	Symbolic correspondence	V
		hyperball_polyveck_point_boundE	lemma hyperball_polyveck_point_boundE md y2 : mu (dhyperball_polyveck md) (pred1 y2) <= hyperball_polyveck_point_bound md.	hyperball_polyveck_point_boundE : $X \leq Y$ is an arithmetic or probability upper bound.	Proves a bound that contributes to a game-hop or final security inequality.
	1029	lemma	lemma bounded_unit_signing_sample_pair_lossless md : is_lossless (dbounded_unit_signing_sample_pair md).	bounded_unit_signing_sample_pair_lossless $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a distribution fact needed for sampling or probability mass reasoning.
	1039	lemma	lemma bounded_unit_signing_sample_pair_support md s : s \in dbounded_unit_signing_sample_pair md => signing_sample_pair_wf md s /\ signing_sample_pair_unit md s.	bounded_unit_signing_sample_pair_support $B \Rightarrow Q$ is an implication used as a proof obligation.	Proves a distribution fact needed for sampling or probability mass reasoning.
	1060	lemma	lemma bounded_unit_signing_sample_pair_sample_ok md s : s \in dbounded_unit_signing_sample_pair md => signing_sample_pair_sample_ok md s.	bounded_unit_signing_sample_pair_sample_ok $B \Rightarrow Q$ is an implication used as a proof obligation.	Proves a bound that contributes to a game-hop or final security inequality.
	1072	lemma	lemma bounded_unit_signing_sample_pair_distribution_ok md : signing_sample_pair_distribution_ok md (dbounded_unit_signing_sample_pair md).	bounded_unit_signing_sample_pair_distribution_ok $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.
	1082	lemma	lemma checked_hyperball_signing_sample_pair_lossless md : is_lossless (dchecked_hyperball_signing_sample_pair md).	checked_hyperball_signing_sample_pair_lossless $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a distribution fact needed for sampling or probability mass reasoning.

	Line	Construct	What was written	Symbolic correspondence	V
	1092	lemma			
checked_hyperball_signing_sample_pair_support		lemma	checked_hyperball_signing_sample_pair_support	Proves a distribution fact needed for	
		checked_hyperball_signing_sample_pair_support	$P \Rightarrow Q$ is an implication used as a proof	sampling or probability mass reasoning.	
		md s : s \in	obligation.		
		dchecked_hyperball_signing_sample_pair			
		md =>			
		signing_sample_pair_sample_ok md			
		s.			
	1114	lemma			
checked_hyperball_signing_sample_pair_distribution_ok		lemma	checked_hyperball_signing_sample_pair_distribution_ok	Proves a bound that contributes to a	
		checked_hyperball_signing_sample_pair_distribution_ok	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	game-hop or final security inequality.	
		md :			
		signing_sample_pair_distribution_ok			
		md			
		(dchecked_hyperball_signing_sample_pair			
		md).			
	1124	lemma			
hyperball_sample_ok_sample_ok		lemma	hyperball_sample_ok_sample_ok : $P \Rightarrow$	Proves a bound that contributes to a	
		hyperball_sample_ok_sample_ok md	Q is an implication used as a proof	game-hop or final security inequality.	
		s : hyperball_sample_ok md s =>	obligation.		
		signing_sample_pair_sample_ok md			
		s.			
	1139	lemma			
exact_hyperball_signing_sample_pair_lossless		lemma	exact_hyperball_signing_sample_pair_lossless	Proves a distribution fact needed for	
		exact_hyperball_signing_sample_pair_lossless	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	sampling or probability mass reasoning.	
		md : is_lossless			
		(dexact_hyperball_signing_sample_pair			
		md).			
	1149	lemma			
exact_hyperball_signing_sample_pair_support		lemma	exact_hyperball_signing_sample_pair_support	Proves a distribution fact needed for	
		exact_hyperball_signing_sample_pair_support	$P \Rightarrow Q$ is an implication used as a proof	sampling or probability mass reasoning.	
		md s : s \in	obligation.		
		dexact_hyperball_signing_sample_pair			
		md => hyperball_sample_ok md s.			
	1163	lemma			
exact_hyperball_signing_sample_pair_supportP		lemma	exact_hyperball_signing_sample_pair_supportP	Proves a distribution fact needed for	
		exact_hyperball_signing_sample_pair_supportP	$X \leq Y$ is an arithmetic or probability	sampling or probability mass reasoning.	
		md s : s \in	upper bound.		
		dexact_hyperball_signing_sample_pair			
		md <=> hyperball_sample_ok md s.			
	1173	lemma			

Line	Construct	What was written	Symbolic correspondence	V
	exact_hyperball_signing_sample_pair_sample_ok	lemma exact_hyperball_signing_sample_pair_sample_ok md s : s \in dexact_hyperball_signing_sample_pair md => signing_sample_pair_sample_ok md s.	exact_hyperball_signing_sample_pair_sample_ok $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a bound that contributes to a game-hop or final security inequality.
1182	lemma			
	exact_hyperball_signing_sample_pair_distribution_ok	lemma exact_hyperball_signing_sample_pair_distribution_ok md : signing_sample_pair_distribution_ok md (dexact_hyperball_signing_sample_pair md).	exact_hyperball_signing_sample_pair_distribution_ok $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.
1192	lemma			
	exact_hyperball_signing_sample_pair_point_bound	lemma exact_hyperball_signing_sample_pair_point_bound md s : mu (dexact_hyperball_signing_sample_pair md) (pred1 s) <= hyperball_sample_pair_point_bound md.	exact_hyperball_signing_sample_pair_point_bound $X \leq Y$ is an arithmetic or probability upper bound.	Proves a bound that contributes to a game-hop or final security inequality.
1206	lemma			
	exact_hyperball_signing_sample_pair_checkedE	lemma exact_hyperball_signing_sample_pair_checkedE md : dexact_hyperball_signing_sample_pair md = dchecked_hyperball_signing_sample_pair md.	exact_hyperball_signing_sample_pair_checkedE $L = R$ is an equality/rewriting fact.	Proves a bound that contributes to a game-hop or final security inequality.

14 HAETAE_Rejection.ec

rejection predicates and rejection-sampling probability bounds

	Line	Construct	What was written	Symbolic correspondence
	16	type		
signing_transcript_record		type signing_transcript_record = message * context * signature * transcript.	signing__transcript__record $\equiv$ message $\times$ context $\times$ signature $\times$ transcript	Fixes the mathematical ca later declarations without an implementation represe
	17	type		
signing_attempt_sample		type signing_attempt_sample = polyvecl * polyveck * polyveck.	signing__attempt__sample $\equiv$ polyvecl $\times$ polyveck $\times$ polyveck	Fixes the mathematical ca later declarations without an implementation represe
	18	type		
signing_attempt_state		type signing_attempt_state = random_coins * signing_attempt_sample * polyveck * poly * challenge * polyvecl * polyveck * hint_t * signature.	signing__attempt__state $\equiv$ random_coins $\times$ signing__attempt__sample $\times$ polyveck $\times$ poly $\times$ challenge $\times$ polyvecl $\times$ polyveck $\times$ hint_t $\times$ signature	Fixes the mathematical ca later declarations without an implementation represe
	22	operation		
signing_attempt_state_coins		op signing_attempt_state_coins (st : signing_attempt_state) : random_coins = st.'1.	signing__attempt__state_coins : signing__attempt__state $\rightarrow$ random_coins	Defines a total mathemati by the EasyCrypt model.
	25	operation		
signing_attempt_state_sample		op signing_attempt_state_sample (st : signing_attempt_state) : signing_attempt_sample = st.'2.	signing__attempt__state_sample : signing__attempt__state $\rightarrow$ signing__attempt__sample	Defines a total mathemati by the EasyCrypt model.
	28	operation		
signing_attempt_sample_y1		op signing_attempt_sample_y1 (s : signing_attempt_sample) : polyvecl = s.'1.	signing__attempt__sample_y1 : signing__attempt__sample $\rightarrow$ polyvecl	Defines a total mathemati by the EasyCrypt model.
	30	operation		
signing_attempt_sample_y2		op signing_attempt_sample_y2 (s : signing_attempt_sample) : polyveck = s.'2.	signing__attempt__sample_y2 : signing__attempt__sample $\rightarrow$ polyveck	Defines a total mathemati by the EasyCrypt model.
	32	operation		
signing_attempt_sample_commitment_raw		op signing_attempt_sample_commitment_raw (s : signing_attempt_sample) : polyveck = s.'3.	signing__attempt__sample_commitment_raw : signing__attempt__sample $\rightarrow$ polyveck	Defines a total mathemati by the EasyCrypt model.
	35	operation		

	Line	Construct	What was written	Symbolic correspondence
		<pre> signing_attempt_state_sampled_y1 op signing_attempt_state_sampled_y1 (st : signing_attempt_state) : polyvecl = signing_attempt_sample_y1 (signing_attempt_state_sample st). </pre>	<pre> signing__attempt_state_sampled_y1 : signing__attempt_state → polyvecl </pre>	Defines a total mathematical model by the EasyCrypt model.
	38	operation		
		<pre> signing_attempt_state_sampled_y2 op signing_attempt_state_sampled_y2 (st : signing_attempt_state) : polyveck = signing_attempt_sample_y2 (signing_attempt_state_sample st). </pre>	<pre> signing__attempt_state_sampled_y2 : signing__attempt_state → polyveck </pre>	Defines a total mathematical model by the EasyCrypt model.
	41	operation		
		<pre> signing_attempt_state_sampled_y op signing_attempt_state_sampled_y (st : signing_attempt_state) : polyveck = signing_attempt_sample_commitment_raw (signing_attempt_state_sample st). </pre>	<pre> signing__attempt_state_sampled_y : signing__attempt_state → polyveck </pre>	Defines a total mathematical model by the EasyCrypt model.
	44	operation		
		<pre> signing_attempt_state_highbits op signing_attempt_state_highbits (st : signing_attempt_state) : polyveck = st.'3. </pre>	<pre> signing__attempt_state_highbits : signing__attempt_state → polyveck </pre>	Defines a total mathematical model by the EasyCrypt model.
	47	operation		
		<pre> signing_attempt_state_lowbits op signing_attempt_state_lowbits (st : signing_attempt_state) : poly = st.'4. </pre>	<pre> signing__attempt_state_lowbits : signing__attempt_state → poly </pre>	Defines a total mathematical model by the EasyCrypt model.
	50	operation		
		<pre> signing_attempt_state_challenge op signing_attempt_state_challenge (st : signing_attempt_state) : challenge = st.'5. </pre>	<pre> signing__attempt_state_challenge : signing__attempt_state → challenge </pre>	Defines a total mathematical model by the EasyCrypt model.
	53	operation		
		<pre> signing_attempt_state_response op signing_attempt_state_response (st : signing_attempt_state) : polyvecl = st.'6. </pre>	<pre> signing__attempt_state_response : signing__attempt_state → polyvecl </pre>	Defines a total mathematical model by the EasyCrypt model.

	Line	Construct	What was written	Symbolic correspondence
	56	operation		
		signing_attempt_state_response_aux	op signing_attempt_state_response_aux : (st : signing_attempt_state) : polyveck = st.'7.	signing_attempt_state_response_aux : signing_attempt_state $\rightarrow$ polyveck
	59	operation		
		signing_attempt_state_hint	op signing_attempt_state_hint (st : signing_attempt_state) : hint_t = st.'8.	signing_attempt_state_hint : signing_attempt_state $\rightarrow$ hint_t
	62	operation		
		signing_attempt_state_signature	op signing_attempt_state_signature (st : signing_attempt_state) : signature = st.'9.	signing_attempt_state_signature : signing_attempt_state $\rightarrow$ signature
	65	operation		
		signature_rejection_branch_byte	op signature_rejection_branch_byte (sig : signature) : int = sig.'6.	signature_rejection_branch_byte : signature $\rightarrow \mathbb{Z}$
	67	operation		
		signing_attempt_state_reject2_branch_byte	op signing_attempt_state_reject2_branch_byte (st : signing_attempt_state) : int = signature_rejection_branch_byte (signing_attempt_state_signature st).	signing_attempt_state_reject2_branch_byte : signing_attempt_state $\rightarrow \mathbb{Z}$
	70	operation		
		signing_attempt_state_reject2_branch_bit	op signing_attempt_state_reject2_branch_bit (st : signing_attempt_state) : bool = (signing_attempt_state_reject2_branch_byte st %/ 2) %% 2 = 1.	signing_attempt_state_reject2_branch_bit : signing_attempt_state is a Boolean-valued predicate.
	73	operation		
		polyvecl_double	op polyvecl_double (md : mode) (xs : polyvecl) : polyvecl = mkseq (fun i => poly_double (nth poly_zero xs i)) (mode_l md).	polyvecl_double : mode $\times$ polyvecl $\rightarrow$ polyvecl
	77	operation		

	Line	Construct	What was written	Symbolic correspondence
		signing_attempt_state_reject2_sampled_y1	op signing_attempt_state_reject2_sampled_y1 (st : signing_attempt_state) : polyvecl = signing_attempt_state_sampled_y1 st.	signing_attempt_state_reject2_sampled_y1 Defines a total mathematical model by the EasyCrypt model.
	80	operation		
		signing_attempt_state_reject2_sampled_y2	op signing_attempt_state_reject2_sampled_y2 (st : signing_attempt_state) : polyveck = signing_attempt_state_sampled_y2 st.	signing_attempt_state_reject2_sampled_y2 Defines a total mathematical model by the EasyCrypt model.
	83	operation		
		signing_attempt_state_reject2_balance_response	op signing_attempt_state_reject2_balance_response (md : mode) (st : signing_attempt_state) : polyvecl = polyvecl_sub md (polyvecl_double md (signing_attempt_state_response st)) (signing_attempt_state_reject2_sampled_y1 st).	signing_attempt_state_reject2_balance_response Defines a total mathematical model by the EasyCrypt model.
	88	operation		
		signing_attempt_state_reject2_balance_aux	op signing_attempt_state_reject2_balance_aux (md : mode) (st : signing_attempt_state) : polyveck = polyveck_sub md (polyveck_double md (signing_attempt_state_response_aux st)) (signing_attempt_state_reject2_sampled_y2 st).	signing_attempt_state_reject2_balance_aux Defines a total mathematical model by the EasyCrypt model.
	93	operation		
		signing_attempt_state_lowbits_carrier	op signing_attempt_state_lowbits_carrier (st : signing_attempt_state) : poly = nth poly_zero (signing_attempt_state_sampled_y st) 0.	signing_attempt_state_lowbits_carrier : Defines a total mathematical model by the EasyCrypt model.

	Line	Construct	What was written	Symbolic correspondence
	96	operation		
		<pre> signing_attempt_state_token op signing_attempt_state_token (st : signing_attempt_state) : sig_token = (signing_attempt_state_response st, signing_attempt_state_response_aux st, signing_attempt_state_hint st). </pre>	<pre> signing__attempt__state__token : signing__attempt__state → sig__token </pre>	Defines a total mathematical model by the EasyCrypt model.
	102	operation		
		<pre> signing_attempt_state_programmed_lowbits op signing_attempt_state_programmed_lowbits (st : signing_attempt_state) : poly = mkseq (fun i => if i = 0 then signing_entropy_token_of_coins (signing_attempt_state_coins st) else poly_coeff (signing_attempt_state_lowbits_carrier st) i) n. </pre>	<pre> signing__attempt__state__programmed__lowbits signing__attempt__state → poly </pre>	Defines a total mathematical model by the EasyCrypt model.
	111	operation		
		<pre> signing_attempt_state_lowbits_consistent op signing_attempt_state_lowbits_consistent (st : signing_attempt_state) : bool = signing_attempt_state_lowbits st = signing_attempt_state_programmed_lowbits st. </pre>	<pre> signing__attempt__state__lowbits__consistent signing__attempt__state is a Boolean-valued predicate. </pre>	$\sqsubseteq$ Names a Boolean mathematical model used in statements and invariants.
	116	operation		
		<pre> signing_attempt_state_programmed_challenge op signing_attempt_state_programmed_challenge (md : mode) (pk : pkey) (m : message) (ctx : context) (st : signing_attempt_state) : challenge = challenge_hash md (signing_attempt_state_response_aux st) (signing_attempt_state_lowbits st) (message_hash pk ctx m). </pre>	<pre> signing__attempt__state__programmed__challenge mode × pkey × message × context × signing__attempt__state → challenge </pre>	Defines a total mathematical model by the EasyCrypt model.
	124	operation		

	Line	Construct	What was written	Symbolic correspondence
signing_attempt_state_challenge_consistent		<pre> op signing_attempt_state_challenge_consistent (md : mode) (pk : pkey) (m : message) (ctx : context) (st : signing_attempt_state) : bool = signing_attempt_state_challenge st = signing_attempt_state_programmed_challenge md pk m ctx st.</pre>	<pre> signing_attempt_state_challenge_consistent mode × pkey × message × context × signing_attempt_state is a Boolean-valued predicate.</pre>	Names a Boolean mathematical statement used in statements and invariants.
signing_attempt_state_reconstructed_highbits	130	<pre> operation op signing_attempt_state_reconstructed_highbits (md : mode) (pk : pkey) (st : signing_attempt_state) : polyveck = reconstructed_highbits md (signing_attempt_state_response_aux st) (public_key_challenge_term md pk (signing_attempt_state_challenge st)).</pre>	<pre> signing_attempt_state_reconstructed_highbits mode × pkey × signing_attempt_state → polyveck</pre>	Defines a total mathematical statement by the EasyCrypt model.
signing_attempt_state_public_equation_consistent	136	<pre> operation op signing_attempt_state_public_equation_consistent (md : mode) (pk : pkey) (st : signing_attempt_state) : bool = signing_attempt_state_highbits st = signing_attempt_state_reconstructed_highbits md pk st.</pre>	<pre> signing_attempt_state_public_equation_consistent mode × pkey × signing_attempt_state is a Boolean-valued predicate.</pre>	States a Boolean mathematical statement used in statements and invariants.
	141	operation		

	Line	Construct	What was written	Symbolic correspondence
signing_attempt_state_signature_of_fields		<pre> op signing_attempt_state_signature_of_fields (pk : pkey) (m : message) (ctx : context) (st : signing_attempt_state) : signature = (signing_attempt_state_highbits st, signing_attempt_state_lowbits st, message_hash pk ctx m, signing_attempt_state_challenge st, signing_attempt_state_token st, 0, 0). </pre>	<pre> signing_attempt_state_signature_of_fields pkey × message × context × signing_attempt_state → signature </pre>	Defines a total mathematical function by the EasyCrypt model.
signing_attempt_state_fields_match_signature	152	<pre> operation signing_attempt_state_fields_match_signature (pk : pkey) (m : message) (ctx : context) (st : signing_attempt_state) : bool = signing_attempt_state_signature st = signing_attempt_state_signature_of_fields pk m ctx st. </pre>	<pre> signing_attempt_state_fields_match_signature pkey × message × context × signing_attempt_state is a Boolean-valued predicate. </pre>	Names a Boolean mathematical function used in statements and invariants.
signing_attempt_state_field_accepts	158	<pre> operation signing_attempt_state_field_accepts (md : mode) (pk : pkey) (m : message) (ctx : context) (st : signing_attempt_state) : bool = signing_attempt_state_lowbits_consistent st /\ signing_attempt_state_challenge_consistent md pk m ctx st /\ signing_attempt_state_public_equation_consistent md pk st /\ signing_attempt_state_fields_match_signature pk m ctx st. </pre>	<pre> signing_attempt_state_field_accepts mode × pkey × message × context × signing_attempt_state is a Boolean-valued predicate. </pre>	Names a Boolean mathematical function used in statements and invariants.
	166	<pre> operation </pre>		

Line	Construct	What was written	Symbolic correspondence
	<pre> signing_attempt_state_of_coins op signing_attempt_state_of_coins (md : mode) (sk : skey) (m : message) (ctx : context) (coins : random_coins) : signing_attempt_state = (coins, (signing_sample_y1 md sk m ctx coins, signing_sample_y2 md sk m ctx coins, commitment_raw md sk m ctx coins), commitment_highbits md sk m ctx coins, commitment_lowbits md sk m ctx coins, commitment_challenge md...</pre>	<pre> signing_attempt_state_of_coins : mode × skey × message × context × random_coins → signing_attempt_state</pre>	Defines a total mathematical model by the EasyCrypt model.
181	operation		
	<pre> signing_attempt_sample_of_pair op signing_attempt_sample_of_pair (md : mode) (sk : skey) (m : message) (ctx : context) (coins : random_coins) (sample : signing_sample_pair) : signing_attempt_sample = (signing_sample_pair_y1 sample, signing_sample_pair_y2 sample, commitment_raw md sk m ctx coins).</pre>	<pre> signing_attempt_sample_of_pair : mode × skey × message × context × random_coins × signing_sample_pair → signing_attempt_sample</pre>	Defines a total mathematical model by the EasyCrypt model.
189	operation		
	<pre> signing_attempt_state_of_sample_pair op signing_attempt_state_of_sample_pair (md : mode) (sk : skey) (m : message) (ctx : context) (coins : random_coins) (sample : signing_sample_pair) : signing_attempt_state = (coins, signing_attempt_sample_of_pair md sk m ctx coins sample, commitment_highbits md sk m ctx coins, commitment_lowbits md sk m ctx coins, commitment_challenge md sk m ctx coins,...</pre>	<pre> signing_attempt_state_of_sample_pair : mode × skey × message × context × random_coins × signing_sample_pair → signing_attempt_state</pre>	Defines a total mathematical model by the EasyCrypt model.

	Line	Construct	What was written	Symbolic correspondence
	203	operation		
signing_attempt_state_valid_signature_accepts		<pre> op signing_attempt_state_valid_signature_accepts (md : mode) (st : signing_attempt_state) : bool = valid_signature md (signing_attempt_state.signature st).</pre>	$\text{signing_attempt_state_valid_signature_accepts} \subseteq \text{mode} \times \text{signing_attempt_state}$ is a Boolean-valued predicate.	Names a well-formedness condition so it can be reused
	207	operation		
signing_attempt_state_response_norm_accepts		<pre> op signing_attempt_state_response_norm_accepts (md : mode) (st : signing_attempt_state) : bool = response_norm_ok md (signing_attempt_state.signature st).</pre>	$\text{signing_attempt_state_response_norm_accepts} \subseteq \text{mode} \times \text{signing_attempt_state}$ is a Boolean-valued predicate.	Names a Boolean mathematical expression used in statements and invariants
	211	operation		
signing_attempt_state_hint_norm_accepts		<pre> op signing_attempt_state_hint_norm_accepts (md : mode) (st : signing_attempt_state) : bool = verify_norm_ok md (signing_attempt_state.signature st).</pre>	$\text{signing_attempt_state_hint_norm_accepts} \subseteq \text{mode} \times \text{signing_attempt_state}$ is a Boolean-valued predicate.	Names a Boolean mathematical expression used in statements and invariants
	215	operation		
signing_attempt_state_accepts		<pre> op signing_attempt_state_accepts (md : mode) (st : signing_attempt_state) : bool = signing_attempt_state_valid_signature_accepts md st /\ signing_attempt_state_response_norm_accepts md st /\ signing_attempt_state_hint_norm_accepts md st.</pre>	$\text{signing_attempt_state_accepts} \subseteq \text{mode} \times \text{signing_attempt_state}$ is a Boolean-valued predicate.	Names a Boolean mathematical expression used in statements and invariants
	221	operation		
signing_attempt_reject1_bound		<pre> op signing_attempt_reject1_bound (md : mode) : int = mode_b1sq md * mode_ln md * mode_ln md.</pre>	$\text{signing_attempt_reject1_bound} : \text{mode} \rightarrow \mathbb{Z}$	Defines a concrete numeric value in the final security inequality
	224	operation		
signing_attempt_reject2_bound		<pre> op signing_attempt_reject2_bound (md : mode) : int = mode_b0sq md * mode_ln md * mode_ln md.</pre>	$\text{signing_attempt_reject2_bound} : \text{mode} \rightarrow \mathbb{Z}$	Defines a concrete numeric value in the final security inequality

	Line	Construct	What was written	Symbolic correspondence
	227	operation		
signing_attempt_reject2_balance_norm_sq		<pre> op signing_attempt_reject2_balance_norm_sq (md : mode) (z : polyvecl) (zaux : polyveck) (y1 : polyvecl) (y2 : polyveck) : int = polyvecl_norm_sq md (polyvecl_sub md (polyvecl_double md z) y1) + polyveck_norm_sq md (polyveck_sub md (polyveck_double md zaux) y2).</pre>	$\text{signing_attempt_reject2_balance_norm_sq} : \text{mode} \times \text{polyvecl} \times \text{polyveck} \times \text{polyvecl} \times \text{polyveck} \rightarrow \mathbb{Z}$	Defines a total mathematical object by the EasyCrypt model.
	233	operation		
signing_attempt_reject2_concrete_aborts		<pre> op signing_attempt_reject2_concrete_aborts (md : mode) (branch : bool) (z : polyvecl) (zaux : polyveck) (y1 : polyvecl) (y2 : polyveck) : bool = branch /\ signing_attempt_reject2_balance_norm_sq md z zaux y1 y2 < signing_attempt_reject2_bound md.</pre>	$\text{signing_attempt_reject2_concrete_aborts} : \text{mode} \times \text{Bool} \times \text{polyvecl} \times \text{polyveck} \times \text{polyvecl} \times \text{polyveck} \rightarrow \text{bool}$	Names a Boolean mathematical object used in statements and invariants. $\text{polyvecl} \times \text{polyveck}$ is a Boolean-valued predicate.
	240	operation		
signing_attempt_state_reject1_norm_sq		<pre> op signing_attempt_state_reject1_norm_sq (md : mode) (st : signing_attempt_state) : int = response_norm_sq md (signing_attempt_state_signature st).</pre>	$\text{signing_attempt_state_reject1_norm_sq} : \text{mode} \times \text{signing_attempt_state} \rightarrow \mathbb{Z}$	Defines a total mathematical object by the EasyCrypt model.
	244	operation		

	Line	Construct	What was written	Symbolic correspondence
		<pre> signing_attempt_state_reject1_accepts op signing_attempt_state_reject1_accepts (md : mode) (st : signing_attempt_state) : bool = 0 <= signing_attempt_state_reject1_norm_sq md st /\ signing_attempt_state_reject1_norm_sq md st <= signing_attempt_reject1_bound md. </pre>	<pre> signing_attempt_state_reject1_accepts $\subseteq$ mode $\times$ signing_attempt_state is a Boolean-valued predicate. </pre>	Names a Boolean mathematical object used in statements and invariants.
	250	<pre> signing_attempt_state_balance_norm_sq op signing_attempt_state_balance_norm_sq (md : mode) (st : signing_attempt_state) : int = signing_attempt_reject2_balance_norm_sq md (signing_attempt_state_response st) (signing_attempt_state_response_aux st) (signing_attempt_state_reject2_sampled_y1 st) (signing_attempt_state_reject2_sampled_y2 st). </pre>	<pre> signing_attempt_state_balance_norm_sq : mode $\times$ signing_attempt_state $\rightarrow \mathbb{Z}$ </pre>	Defines a total mathematical object by the EasyCrypt model.
	258	<pre> signing_attempt_state_bimodal_reject2_branch op signing_attempt_state_bimodal_reject2_branch (st : signing_attempt_state) : bool = signing_attempt_state_reject2_branch_bit st. </pre>	<pre> signing_attempt_state_bimodal_reject2_branch $\subseteq$ signing_attempt_state is a Boolean-valued predicate. </pre>	Names a Boolean mathematical object used in statements and invariants.
	262	<pre> signing_attempt_state_reject2_under_bound op signing_attempt_state_reject2_under_bound (md : mode) (st : signing_attempt_state) : bool = signing_attempt_state_balance_norm_sq md st < signing_attempt_reject2_bound md. </pre>	<pre> signing_attempt_state_reject2_under_bound : mode $\times$ signing_attempt_state is a Boolean-valued predicate. </pre>	Names a Boolean mathematical object used in statements and invariants.

	Line	Construct	What was written	Symbolic correspondence
	267	operation		
signing_attempt_state_reject2_aborts		<pre> op signing_attempt_state_reject2_aborts (md : mode) (st : signing_attempt_state) : bool = signing_attempt_reject2_concrete_aborts md (signing_attempt_state_bimodal_reject2_branch st) (signing_attempt_state_response st) (signing_attempt_state_response_aux st) (signing_attempt_state_reject2_sampled_y1 st) (signing_attempt_state_reject2_sampled_y2 st). </pre>	$\text{signing_attempt_state_reject2_aborts} \subseteq \text{mode} \times \text{signing_attempt_state}$ is a Boolean-valued predicate.	Names a Boolean mathematical object used in statements and invariants.
	276	operation		
signing_attempt_state_reject2_accepts		<pre> op signing_attempt_state_reject2_accepts (md : mode) (st : signing_attempt_state) : bool = ! signing_attempt_state_reject2_aborts md st. </pre>	$\text{signing_attempt_state_reject2_accepts} \subseteq \text{mode} \times \text{signing_attempt_state}$ is a Boolean-valued predicate.	Names a Boolean mathematical object used in statements and invariants.
	280	operation		
signing_attempt_state_pack_accepts		<pre> op signing_attempt_state_pack_accepts (md : mode) (st : signing_attempt_state) : bool = signing_attempt_state_valid_signature_accepts md st /\ 0 < mode_crypto_bytes md. </pre>	$\text{signing_attempt_state_pack_accepts} \subseteq \text{mode} \times \text{signing_attempt_state}$ is a Boolean-valued predicate.	Names a Boolean mathematical object used in statements and invariants.
	285	operation		

Line	Construct	What was written	Symbolic correspondence
	<pre> signing_attempt_state_reference_rejection_accepts op signing_attempt_state_reference_rejection_accepts (md : mode) (st : signing_attempt_state) : bool = signing_attempt_state_reject1_accepts md st /\ signing_attempt_state_reject2_accepts md st /\ signing_attempt_state_pack_accepts md st /\ signing_attempt_state_hint_norm_accepts md st. </pre>	<pre> signing_attempt_state_reference_rejection_accepts mode $\times$ signing_attempt_state is a Boolean-valued predicate. </pre>	<pre> signing_attempt_state_reference_rejection_accepts Names a bad event whose later shown to be zero or 1 </pre>
292	operation		
	<pre> signing_attempt_state_reference_rejection_aborts op signing_attempt_state_reference_rejection_aborts (md : mode) (st : signing_attempt_state) : bool = ! signing_attempt_state_reference_rejection_accepts md st. </pre>	<pre> signing_attempt_state_reference_rejection_aborts mode $\times$ signing_attempt_state is a Boolean-valued predicate. </pre>	<pre> signing_attempt_state_reference_rejection_aborts Names a bad event whose later shown to be zero or 1 </pre>
296	operation		
	<pre> signing_attempt_state_reject1_aborts op signing_attempt_state_reject1_aborts (md : mode) (st : signing_attempt_state) : bool = ! signing_attempt_state_reject1_accepts md st. </pre>	<pre> signing_attempt_state_reject1_aborts mode $\times$ signing_attempt_state is a Boolean-valued predicate. </pre>	<pre> signing_attempt_state_reject1_aborts Names a Boolean mathem used in statements and inv </pre>
300	operation		
	<pre> signing_attempt_state_pack_aborts op signing_attempt_state_pack_aborts (md : mode) (st : signing_attempt_state) : bool = ! signing_attempt_state_pack_accepts md st. </pre>	<pre> signing_attempt_state_pack_aborts mode $\times$ signing_attempt_state is a Boolean-valued predicate. </pre>	<pre> signing_attempt_state_pack_aborts Names a Boolean mathem used in statements and inv </pre>
304	operation		

	Line	Construct	What was written	Symbolic correspondence
signing_attempt_state_verification_accepts		<pre> op signing_attempt_state_verification_accepts (md : mode) (pk : pkey) (m : message) (ctx : context) (st : signing_attempt_state) : bool = signing_attempt_state_field_accepts md pk m ctx st /\ signing_attempt_state_accepts md st.</pre>	<pre> signing_attempt_state_verification_accepts mode × pkey × message × context × signing_attempt_state is a Boolean-valued predicate.</pre>	Names a Boolean mathematical object used in statements and invariants
	310	operation		
signing_attempt_state_rejection_accepts		<pre> op signing_attempt_state_rejection_accepts (md : mode) (pk : pkey) (m : message) (ctx : context) (st : signing_attempt_state) : bool = signing_attempt_state_verification_accepts md pk m ctx st /\ signing_attempt_state_reference_rejection_accepts md st.</pre>	<pre> signing_attempt_state_rejection_accepts ⊆ mode × pkey × message × context × signing_attempt_state is a Boolean-valued predicate.</pre>	Names a bad event whose value is later shown to be zero or one
	316	operation		
signing_attempt_state_rejection_aborts		<pre> op signing_attempt_state_rejection_aborts (md : mode) (pk : pkey) (m : message) (ctx : context) (st : signing_attempt_state) : bool = ! signing_attempt_state_rejection_accepts md pk m ctx st.</pre>	<pre> signing_attempt_state_rejection_aborts ⊆ mode × pkey × message × context × signing_attempt_state is a Boolean-valued predicate.</pre>	Names a bad event whose value is later shown to be zero or one
	321	operation		
signing_attempt_state_response_norm_aborts		<pre> op signing_attempt_state_response_norm_aborts (md : mode) (st : signing_attempt_state) : bool = ! signing_attempt_state_response_norm_accepts md st.</pre>	<pre> signing_attempt_state_response_norm_aborts ⊆ mode × signing_attempt_state is a Boolean-valued predicate.</pre>	Names a Boolean mathematical object used in statements and invariants
	325	operation		

	Line	Construct	What was written	Symbolic correspondence
signing_attempt_state_hint_norm_aborts		<pre> op signing_attempt_state_hint_norm_aborts (md : mode) (st : signing_attempt_state) : bool = ! signing_attempt_state_hint_norm_accepts md st.</pre>	<pre> signing_attempt_state_hint_norm_aborts ⊆ mode × signing_attempt_state is a Boolean-valued predicate.</pre>	Names a Boolean mathematical object used in statements and invariants
signing_attempt_state_rejection_sampling_aborts	329	<pre> op signing_attempt_state_rejection_sampling_aborts (md : mode) (st : signing_attempt_state) : bool = ! signing_attempt_state_valid_signature_accepts md st.</pre> operation	<pre> signing_attempt_state_rejection_sampling_aborts ⊆ mode × signing_attempt_state is a Boolean-valued predicate.</pre>	Names a bad event whose probability is later shown to be zero or less
signing_attempt_state_aborts	333	<pre> op signing_attempt_state_aborts (md : mode) (st : signing_attempt_state) : bool = signing_attempt_state_rejection_sampling_aborts md st \/ signing_attempt_state_response_norm_aborts md st \/ signing_attempt_state_hint_norm_aborts md st.</pre> operation	<pre> signing_attempt_state_aborts ⊆ mode × signing_attempt_state is a Boolean-valued predicate.</pre>	Names a Boolean mathematical object used in statements and invariants
signing_accepts	339	<pre> op signing_accepts (md : mode) (sk : skey) (m : message) (ctx : context) (coins : random_coins) : bool = signing_attempt_state_accepts md (signing_attempt_state_of_coins md sk m ctx coins).</pre> operation	<pre> signing_accepts ⊆ mode × skey × message × context × random_coins is a Boolean-valued predicate.</pre>	Names a Boolean mathematical object used in statements and invariants
dsigning_attempt_state	345	<pre> op dsigning_attempt_state (md : mode) (sk : skey) (m : message) (ctx : context) : signing_attempt_state distr = dmap drandom_coins (signing_attempt_state_of_coins md sk m ctx).</pre> operation	<pre> dsigning_attempt_state : mode × skey × message × context → Distr(signing_attempt_state)</pre>	Defines a sampler/distribution over game procedures and probabilities

	Line	Construct	What was written	Symbolic correspondence
	350	operation		
		<pre> signing_sample_pair_sampler_ok op signing_sample_pair_sampler_ok (md : mode) (dsample : random_coins -> signing_sample_pair distr) : bool = forall coins, signing_randomness_domain coins => signing_sample_pair_distribution_ok md (dsample coins).</pre>	<pre> signing_sample_pair_sampler_ok ⊆ mode × Distr(random_coins → signing_sample_pair) is a Boolean-valued predicate.</pre>	Names a Boolean mathematical object used in statements and invariants.
	357	operation		
		<pre> dsigning_attempt_state_from_sample_pair_sampler op dsigning_attempt_state_from_sample_pair_sampler (md : mode) (sk : skey) (m : message) (ctx : context) (dsample : random_coins -> signing_sample_pair distr) : signing_attempt_state distr = dlet drandom_coins (fun coins => dmap (dsample coins) (signing_attempt_state_of_sample_pair md sk m ctx coins)).</pre>	<pre> dsigning_attempt_state_from_sample_pair_sampler mode × skey × message × context × Distr(random_coins → signing_sample_pair) → Distr(signing_attempt_state)</pre>	Defines a sampler/distribution over game procedures and probabilities.
	366	type		
		<pre> signing_attempt_coin_sample_pair type signing_attempt_coin_sample_pair = random_coins * signing_sample_pair.</pre>	<pre> signing_attempt_coin_sample_pair ≡ random_coins × signing_sample_pair</pre>	Fixes the mathematical carrier of the later declarations without changing the implementation representation.
	368	operation		
		<pre> signing_attempt_state_of_coin_sample_pair op signing_attempt_state_of_coin_sample_pair (md : mode) (sk : skey) (m : message) (ctx : context) (cs : signing_attempt_coin_sample_pair) : signing_attempt_state = signing_attempt_state_of_sample_pair md sk m ctx cs.'1 cs.'2.</pre>	<pre> signing_attempt_state_of_coin_sample_pair mode × skey × message × context × signing_attempt_coin_sample_pair → signing_attempt_state</pre>	Defines a total mathematical object used by the EasyCrypt model.
	373	operation		

Line	Construct	What was written	Symbolic correspondence
	dstructural_signing_attempt_coin_sample_pair	<pre> op dstructural_signing_attempt_coin_sample_pair (md : mode) (sk : skey) (m : message) (ctx : context) : signing_attempt_coin_sample_pair distr = dlet drandom_coins (fun coins => dmap (dstructural_signing_sample_pair md sk m ctx coins) (fun sample => (coins, sample))). </pre>	Defines a sampler/distribution over game procedures and probabilities
381	operation		
	dexact_hyperball_signing_attempt_coin_sample_pair	<pre> op dexact_hyperball_signing_attempt_coin_sample_pair (md : mode) (sk : skey) (m : message) (ctx : context) : signing_attempt_coin_sample_pair distr = dlet drandom_coins (fun coins => dmap (dexact_hyperball_signing_sample_pair md) (fun sample => (coins, sample))). </pre>	Defines a sampler/distribution over game procedures and probabilities
389	operation		
	dstructural_signing_attempt_state	<pre> op dstructural_signing_attempt_state (md : mode) (sk : skey) (m : message) (ctx : context) : signing_attempt_state distr = dmap drandom_coins (fun coins => signing_attempt_state_of_sample_pair md sk m ctx coins (signing_sample_pair_of_coins md sk m ctx coins)). </pre>	Defines a sampler/distribution over game procedures and probabilities
397	operation		
	dstructural_signing_attempt_state_from_sampler	<pre> op dstructural_signing_attempt_state_from_sampler (md : mode) (sk : skey) (m : message) (ctx : context) : signing_attempt_state distr = dsigning_attempt_state_from_sample_pair_sampler md sk m ctx (dstructural_signing_sample_pair md sk m ctx). </pre>	Defines a sampler/distribution over game procedures and probabilities

	Line	Construct	What was written	Symbolic correspondence
	403	operation		
		dbounded_unit_signing_attempt_state	op dbounded_unit_signing_attempt_state : $\text{mode} \times \text{skey} \times \text{message} \times \text{context} \rightarrow \text{Distr}(\text{signing_attempt_state})$ (md : mode) (sk : skey) (m : message) (ctx : context) : signing_attempt_state distr = dsigning_attempt_state_from_sample_pair_sampler md sk m ctx (fun _ => dbounded_unit_signing_sample_pair md).	Defines a sampler/distribution over game procedures and probabilities
	409	operation		
		dchecked_hyperball_signing_attempt_state	op dchecked_hyperball_signing_attempt_state : $\text{mode} \times \text{skey} \times \text{message} \times \text{context} \rightarrow \text{Distr}(\text{signing_attempt_state})$ (md : mode) (sk : skey) (m : message) (ctx : context) : signing_attempt_state distr = dsigning_attempt_state_from_sample_pair_sampler md sk m ctx (fun _ => dchecked_hyperball_signing_sample_pair md).	Defines a sampler/distribution over game procedures and probabilities
	415	operation		
		dexact_hyperball_signing_attempt_state	op dexact_hyperball_signing_attempt_state : $\text{mode} \times \text{skey} \times \text{message} \times \text{context} \rightarrow \text{Distr}(\text{signing_attempt_state})$ (md : mode) (sk : skey) (m : message) (ctx : context) : signing_attempt_state distr = dsigning_attempt_state_from_sample_pair_sampler md sk m ctx (fun _ => dexact_hyperball_signing_sample_pair md).	Defines a sampler/distribution over game procedures and probabilities
	421	operation		
		signing_attempt_state_sample_pair_wf	op signing_attempt_state_sample_pair_wf : $\text{mode} \times \text{signing_attempt_state}$ is a Boolean-valued predicate. (md : mode) (st : signing_attempt_state) : bool = polyvecl_wf md (signing_attempt_state_sampled_y1 st) /\ polyveck_wf md (signing_attempt_state_sampled_y2 st).	Names a well-formedness condition so it can be reused
	426	operation		

	Line	Construct	What was written	Symbolic correspondence
		<pre> signing_attempt_state_sample_pair_unit op signing_attempt_state_sample_pair_unit (md : mode) (st : signing_attempt_state) : bool = polyvecl_unit md (signing_attempt_state_sampled_y1 st) /\ polyveck_unit md (signing_attempt_state_sampled_y2 st). </pre>	<pre> signing_attempt_state_sample_pair_unit ⊆ mode × signing_attempt_state is a Boolean-valued predicate. </pre>	Names a Boolean mathematical object used in statements and invariants.
	431	<pre> signing_attempt_state_sample_pair_bounded op signing_attempt_state_sample_pair_bounded (md : mode) (st : signing_attempt_state) : bool = polyvecl_bounded_by md (signing_sample_bound md) (signing_attempt_state_sampled_y1 st) /\ polyveck_bounded_by md (signing_sample_bound md) (signing_attempt_state_sampled_y2 st). </pre>	<pre> signing_attempt_state_sample_pair_bounded ⊆ mode × signing_attempt_state is a Boolean-valued predicate. </pre>	Names a Boolean mathematical object used in statements and invariants.
	438	<pre> signing_attempt_relation op signing_attempt_relation (md : mode) (pk : pkey) (sk : skey) (m : message) (ctx : context) (coins : random_coins) (sig : signature) : bool = valid_keypair md pk sk /\ signing_accepts md sk m ctx coins /\ sig = sign_internal md sk m ctx coins. </pre>	<pre> signing_attempt_relation ⊆ mode × pkey × skey × message × context × random_coins × signature is a Boolean-valued predicate. </pre>	Names a Boolean mathematical object used in statements and invariants.
	445	<pre> signing_simulation_relation op signing_simulation_relation (md : mode) (pk : pkey) (m : message) (ctx : context) (sig : signature) : bool = verify_internal md pk m ctx sig /\ exists (sk : skey) (coins : random_coins), signing_attempt_relation md pk sk m ctx coins sig. </pre>	<pre> signing_simulation_relation ⊆ mode × pkey × message × context × signature × skey × random_coins is a Boolean-valued predicate. </pre>	Names a Boolean mathematical object used in statements and invariants.

	Line	Construct	What was written	Symbolic correspondence
	452	operation		
signing_transcript_relation		op signing_transcript_relation (md : mode) (pk : pkey) (m : message) (ctx : context) (sig : signature) (tr : transcript) : bool = signing_simulation_relation md pk m ctx sig /\ tr = transcript_of_signature md pk m ctx sig /\ transcript_valid md tr.	signing__transcript_relation $\subseteq$ $\text{mode} \times \text{pkey} \times \text{message} \times \text{context} \times$ $\text{signature} \times \text{transcript}$ is a Boolean-valued predicate.	Names a Boolean mathematical object used in statements and invariants.
	459	operation		
signing_record_message		op signing_record_message (r : signing_transcript_record) : message = r.'1.	signing__record_message : signing__transcript_record $\rightarrow$ message	Defines a total mathematical object by the EasyCrypt model.
	460	operation		
signing_record_context		op signing_record_context (r : signing_transcript_record) : context = r.'2.	signing__record_context : signing__transcript_record $\rightarrow$ context	Defines a total mathematical object by the EasyCrypt model.
	461	operation		
signing_record_signature		op signing_record_signature (r : signing_transcript_record) : signature = r.'3.	signing__record_signature : signing__transcript_record $\rightarrow$ signature	Defines a total mathematical object by the EasyCrypt model.
	462	operation		
signing_record_transcript		op signing_record_transcript (r : signing_transcript_record) : transcript = r.'4.	signing__record_transcript : signing__transcript_record $\rightarrow$ transcript	Defines transcript or extra data used in the forking/special-source games.
	465	operation		
signing_record_query		op signing_record_query (r : signing_transcript_record) : message * context = (signing_record_message r, signing_record_context r).	signing__record_query : signing__transcript_record $\rightarrow$ message $\times$ context	Defines the random-oracle data used by the games.
	468	operation		

	Line	Construct	What was written	Symbolic correspondence
		signing_record_relation op signing_record_relation (md : mode) (pk : pkey) (r : signing_transcript_record) : bool = signing_transcript_relation md pk (signing_record_message r) (signing_record_context r) (signing_record_signature r) (signing_record_transcript r).	signing_record_relation $\subseteq$ mode $\times$ pkey $\times$ signing_transcript_record is a Boolean-valued predicate.	Names a Boolean mathematical object used in statements and invariants.
	476	operation		
		signing_record_queries op signing_record_queries (rs : signing_transcript_record list) : (message * context) list = map signing_record_query rs.	signing_record_queries : List(signing_transcript_record) $\rightarrow$ List((message $\times$ context))	Defines a total mathematical object by the EasyCrypt model.
	480	operation		
		signing_record_transcripts op signing_record_transcripts (rs : signing_transcript_record list) : transcript list = map signing_record_transcript rs.	signing_record_transcripts : List(signing_transcript_record) $\rightarrow$ List(transcript)	Defines transcript or extra objects in the forking/special-soundness model.
	484	operation		
		signing_record_log_sound op signing_record_log_sound (md : mode) (pk : pkey) (rs : signing_transcript_record list) : bool = all (signing_record_relation md pk) rs.	signing_record_log_sound $\subseteq$ mode $\times$ pkey $\times$ List(signing_transcript_record) is a Boolean-valued predicate.	Names a Boolean mathematical object used in statements and invariants.
	488	operation		
		transcript_log_valid op transcript_log_valid (md : mode) (trs : transcript list) : bool = all (transcript_valid md) trs.	transcript_log_valid $\subseteq$ mode $\times$ List(transcript) is a Boolean-valued predicate.	Names a well-formedness condition so it can be reused.
	491	operation		

	Line	Construct	What was written	Symbolic correspondence
structural_to_exact_hyperball_sample_pair_point_loss_obligation		<pre> op structural_to_exact_hyperball_sample_pair_point_loss_obligation : bool = forall (md : mode) (sk : skey) (m : message) (ctx : context) (coins : random_coins), 1%r <= mu (dexact_hyperball_signing_sample_pair md) (pred1 (signing_sample_pair_of_coins md sk m ctx coins)) + rejection_sampling_loss_term. </pre>	<pre> structural_to_exact_hyperball_sample_pair_point_loss_obligation mode × skey × message × context × random_coins is a Boolean-valued predicate. </pre>	<pre> points_loss_obligation used in statements and inv </pre>
	499	operation		
structural_to_exact_hyperball_sample_pair_loss_obligation		<pre> op structural_to_exact_hyperball_sample_pair_loss_obligation : bool = forall (md : mode) (sk : skey) (m : message) (ctx : context) (coins : random_coins) (p : signing_sample_pair -> bool), mu (dstructural_signing_sample_pair md sk m ctx coins) p <= mu (dexact_hyperball_signing_sample_pair md) p + rejection_sampling_loss_term. </pre>	<pre> structural_to_exact_hyperball_sample_pair_loss_obligation mode × skey × message × context × random_coins × signing_sample_pair -> bool is a Boolean-valued predicate. </pre>	<pre> points_loss_obligation used in statements and inv </pre>
	506	operation		
structural_to_exact_hyperball_coin_sample_pair_loss_obligation		<pre> op structural_to_exact_hyperball_coin_sample_pair_loss_obligation : bool = forall (md : mode) (sk : skey) (m : message) (ctx : context) (p : signing_attempt_coin_sample_pair -> bool), mu (dstructural_signing_attempt_coin_sample_pair md sk m ctx) p <= mu (dexact_hyperball_signing_attempt_coin_sample_pair md sk m ctx) p + rejection_sampling_loss_term. </pre>	<pre> structural_to_exact_hyperball_coin_sample_pair_loss_obligation mode × skey × message × context × signing_attempt_coin_sample_pair -> bool is a Boolean-valued predicate. </pre>	<pre> points_loss_obligation used in statements and inv </pre>
	513	operation		

Line	Construct	What was written	Symbolic correspondence
	structural_to_exact_hyperball_attempt_loss_obligation	<pre> op structural_to_exact_hyperball_attempt_loss : mode × skey × message × context × signing_attempt_state → bool : bool = forall (md : mode) (sk : skey) (m : message) (ctx : context) (p : signing_attempt_state → bool), mu (dsigning_attempt_state md sk m ctx) p ≤ mu (dexact_hyperball_signing_attempt_state md sk m ctx) p + rejection_sampling_loss_term. </pre>	obligation Boolean mathematical statement used in statements and in
520	rejection_sampling_bound_obligation	<pre> op rejection_sampling_bound_obligation : bool = 0%r ≤ rejection_sampling_loss_term /\ 0%r ≤ fs_with_aborts_reprogramming_term /\ 0%r ≤ fs_with_aborts_min_entropy_term. </pre>	rejection_sampling_bound_obligation $\subseteq 1$ is a Boolean-valued predicate. Names a bad event whose later shown to be zero or 1
525	signing_attempt_verifies	<pre> lemma signing_attempt_verifies md pk sk m ctx coins sig : signing_attempt_relation md pk sk m ctx coins sig => verify_internal md pk m ctx sig. </pre>	signing_attempt_verifies : $P \Rightarrow Q$ is an implication used as a proof obligation. Records a reusable theorem scripts can rewrite, apply,
535	signing_simulation_verifies	<pre> lemma signing_simulation_verifies md pk m ctx sig : signing_simulation_relation md pk m ctx sig => verify_internal md pk m ctx sig. </pre>	signing_simulation_verifies : $P \Rightarrow Q$ is an implication used as a proof obligation. Proves a distribution fact sampling or probability m
542	signing_attempt_simulates	<pre> lemma signing_attempt_simulates md pk sk m ctx coins sig : signing_attempt_relation md pk sk m ctx coins sig => signing_simulation_relation md pk m ctx sig. </pre>	signing_attempt_simulates : $P \Rightarrow Q$ is an implication used as a proof obligation. Proves a distribution fact sampling or probability m
554		<pre> lemma </pre>	

	Line	Construct	What was written	Symbolic correspondence
		sign_internal_attempt_relation lemma sign_internal_attempt_relation md sk m ctx coins : signing_attempt_relation md (public_key_of_secret md sk) sk m ctx coins (sign_internal md sk m ctx coins).	sign_internal_attempt_relation : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem scripts can rewrite, apply,
	571	lemma signing_attempt_sample_of_pair_currentE lemma signing_attempt_sample_of_pair_currentE md sk m ctx coins : signing_attempt_sample_of_pair md sk m ctx coins (signing_sample_pair_of_coins md sk m ctx coins) = (signing_sample_y1 md sk m ctx coins, signing_sample_y2 md sk m ctx coins, commitment_raw md sk m ctx coins).	signing_attempt_sample_of_pair_currentE : $L \equiv R$ is an equality/rewriting fact.	Proves a bound that controls game-hop or final security
	583	lemma signing_attempt_state_of_sample_pair_currentE lemma signing_attempt_state_of_sample_pair_currentE md sk m ctx coins : signing_attempt_state_of_sample_pair md sk m ctx coins (signing_sample_pair_of_coins md sk m ctx coins) = signing_attempt_state_of_coins md sk m ctx coins.	signing_attempt_state_of_sample_pair_currentE : $L \equiv R$ is an equality/rewriting fact.	Proves a bound that controls game-hop or final security
	593	lemma structural_signing_sample_pair_sampler_ok lemma structural_signing_sample_pair_sampler_ok md sk m ctx : signing_sample_pair_sampler_ok md (dstructural_signing_sample_pair md sk m ctx).	structural_signing_sample_pair_sampler_ok : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that controls game-hop or final security
	601	lemma		

	Line	Construct	What was written	Symbolic correspondence
		<pre> bounded_unit_signing_sample_pair_sampler_ok lemma bounded_unit_signing_sample_pair_sampler_ok md : signing_sample_pair_sampler_ok md (fun _ => dbounded_unit_signing_sample_pair md). </pre>	<pre> bounded_unit_signing_sample_pair_sampler_ok P => Q </pre> $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a bound that controls game-hop or final security
	609	<pre> checked_hyperball_signing_sample_pair_sampler_ok lemma checked_hyperball_signing_sample_pair_sampler_ok md : signing_sample_pair_sampler_ok md (fun _ => dchecked_hyperball_signing_sample_pair md). </pre>	<pre> checked_hyperball_signing_sample_pair_sampler_ok P => Q </pre> $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a bound that controls game-hop or final security
	617	<pre> exact_hyperball_signing_sample_pair_sampler_ok lemma exact_hyperball_signing_sample_pair_sampler_ok md : signing_sample_pair_sampler_ok md (fun _ => dexact_hyperball_signing_sample_pair md). </pre>	<pre> exact_hyperball_signing_sample_pair_sampler_ok P => Q </pre> $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a bound that controls game-hop or final security
	625	<pre> dsigning_attempt_state_from_sample_pair_sampler_lossless lemma dsigning_attempt_state_from_sample_pair_sampler_lossless md sk m ctx dsample : signing_sample_pair_sampler_ok md dsample => is_lossless (dsigning_attempt_state_from_sample_pair_sampler md sk m ctx dsample). </pre>	<pre> dsigning_attempt_state_from_sample_pair_sampler_lossless P => Q </pre> $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a lossless sampling or probability measure
	643	<pre> lemma </pre>		

Line	Construct	What was written	Symbolic correspondence
	dsigning_attempt_state_from_sample_pair_sampler_sample_ok lemma dsigning_attempt_state_from_sample_pair_sampler_sample_ok md sk m ctx dsample st : signing_sample_pair_sampler_ok md dsample => st \in dsigning_attempt_state_from_sample_pair_sampler md sk m ctx dsample => signing_attempt_state_sample_pair_wf md st /\n signing_attempt_state_sample_pair_bounded md st.	dsigning_attempt_state_from_sample_pair_sampler_sample_ok $P \Rightarrow Q$ is an implication used as a proof obligation.	dsigning_attempt_state_from_sample_pair_sampler_sample_ok : controls game-hop or final security
678	lemma dstructural_signing_attempt_state_lossless lemma dstructural_signing_attempt_state_lossless md sk m ctx : is_lossless (dstructural_signing_attempt_state md sk m ctx).	dstructural_signing_attempt_state_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	dstructural_signing_attempt_state_lossless : Proves a distribution fact sampling or probability measure
685	lemma dstructural_signing_attempt_state_from_sampler_lossless lemma dstructural_signing_attempt_state_from_sampler_lossless md sk m ctx : is_lossless (dstructural_signing_attempt_state_from_sampler md sk m ctx).	dstructural_signing_attempt_state_from_sampler_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	dstructural_signing_attempt_state_from_sampler_lossless : Proves a distribution fact sampling or probability measure
695	lemma dbounded_unit_signing_attempt_state_lossless lemma dbounded_unit_signing_attempt_state_lossless md sk m ctx : is_lossless (dbounded_unit_signing_attempt_state md sk m ctx).	dbounded_unit_signing_attempt_state_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	dbounded_unit_signing_attempt_state_lossless : Proves a distribution fact sampling or probability measure
703	lemma dbounded_unit_signing_attempt_state_sample_ok lemma dbounded_unit_signing_attempt_state_sample_ok md sk m ctx st : st \in dbounded_unit_signing_attempt_state md sk m ctx => signing_attempt_state_sample_pair_wf md st /\n signing_attempt_state_sample_pair_bounded md st.	dbounded_unit_signing_attempt_state_sample_ok $P \Rightarrow Q$ is an implication used as a proof obligation.	dbounded_unit_signing_attempt_state_sample_ok : Proves a bound that controls game-hop or final security
713	lemma		

Line	Construct	What was written	Symbolic correspondence
	dchecked_hyperball_signing_attempt_state_lossless	lemma dchecked_hyperball_signing_attempt_state_lossless md sk m ctx : is_lossless (dchecked_hyperball_signing_attempt_state md sk m ctx).	lossless is a distribution fact sampling or probability measure
721	lemma		
	dchecked_hyperball_signing_attempt_state_sample_ok	lemma dchecked_hyperball_signing_attempt_state_sample_ok md sk m ctx st : st \in dchecked_hyperball_signing_attempt_state md sk m ctx => signing_attempt_state_sample_pair_wf md st /\ signing_attempt_state_sample_pair_bounded md st.	sample_ok bound that controls game-hop or final security obligation.
731	lemma		
	dexact_hyperball_signing_attempt_state_lossless	lemma dexact_hyperball_signing_attempt_state_lossless md sk m ctx : is_lossless (dexact_hyperball_signing_attempt_state md sk m ctx).	lossless is a distribution fact sampling or probability measure
739	lemma		
	dexact_hyperball_signing_attempt_state_sample_ok	lemma dexact_hyperball_signing_attempt_state_sample_ok md sk m ctx st : st \in dexact_hyperball_signing_attempt_state md sk m ctx => signing_attempt_state_sample_pair_wf md st /\ signing_attempt_state_sample_pair_bounded md st.	sample_ok bound that controls game-hop or final security obligation.
749	lemma		

	Line	Construct	What was written	Symbolic correspondence
dexact_hyperball_signing_attempt_state_supportP		lemma	dexact_hyperball_signing_attempt_state_supportP	Prove a distribution fact
		dexact_hyperball_signing_attempt_state_supportP	$X \leq Y$ is an arithmetic or probability	sampling or probability m
		md sk m ctx st : st \in	upper bound.	
		dexact_hyperball_signing_attempt_state		
		md sk m ctx <=> exists (coins :		
		random_coins) (sample :		
		signing_sample_pair), coins \in		
		drandom_coins /\		
		hyperball_sample_ok md sample /\		
		st =		
		signing_attempt_state_of_sample_pair		
		md sk m ctx coins sample.		
	778	lemma	dexact_hyperball_signing_attempt_state_reference_rejection_abort_bound1	Prove a rejection that aborts
dexact_hyperball_signing_attempt_state_reference_rejection_abort_bound1		lemma	dexact_hyperball_signing_attempt_state_reference_rejection_abort_bound1	Prove a rejection that aborts
		dexact_hyperball_signing_attempt_state_reference_rejection_abort_bound1	$X \leq Y$ is an arithmetic or probability	game-hop or final security
		md sk m ctx : mu	upper bound.	
		(dexact_hyperball_signing_attempt_state		
		md sk m ctx)		
		(signing_attempt_state_reference_rejection_aborts		
		md) <= 1/r.		
	786	lemma	dexact_hyperball_signing_attempt_state_mu_pullback	Prove a distribution fact
dexact_hyperball_signing_attempt_state_mu_pullback		lemma	dexact_hyperball_signing_attempt_state_mu_pullback	Prove a distribution fact
		dexact_hyperball_signing_attempt_state_mu_pullback	$P \Rightarrow Q$ is an implication used as a proof	sampling or probability m
		md sk m ctx (P :	obligation.	
		signing_attempt_state -> bool) :		
		mu		
		(dexact_hyperball_signing_attempt_state		
		md sk m ctx) P = mu (dlet		
		drandom_coins (fun coins => dmap		
		(dexact_hyperball_signing_sample_pair		
		md) (fun sample =>		
		signing_attempt_state_of_sample_pair		
		md sk m ctx coins sample))) P.		
	801	lemma		

	Line	Construct	What was written	Symbolic correspondence
signing_attempt_state_reference_rejection_aborts_componentE		<pre> lemma signing_attempt_state_reference_rejection_aborts_componentE md st : signing_attempt_state_reference_rejection_aborts md st = (signing_attempt_state_reject1_aborts md st \ / signing_attempt_state_reject2_aborts md st \ / signing_attempt_state_pack_aborts md st \ / signing_attempt_state_hint_norm_aborts md st).</pre>	signing_attempt_state_reference_rejection_aborts_componentE Let P be an equality/rewriting fact.	Aborts componentE preserves oracle, adversary call, or game procedure.
dexact_hyperball_signing_attempt_state_reference_rejection_abort_split	817	<pre> lemma dexact_hyperball_signing_attempt_state_reference_rejection_abort_split md sk m ctx : mu (dexact_hyperball_signing_attempt_state md sk m ctx) (signing_attempt_state_reference_rejection_aborts md) = mu (dexact_hyperball_signing_attempt_state md sk m ctx) (predU (signing_attempt_state_reject1_aborts md) (predU (signing_attempt_state_reject2_aborts md))...</pre>	dexact_hyperball_signing_attempt_state_reference_rejection_abort_split Let P be an equality/rewriting fact.	Reference rejection aborts simulator transition.
dexact_hyperball_signing_attempt_state_reference_rejection_abort_union_bound	834	<pre> lemma dexact_hyperball_signing_attempt_state_reference_rejection_abort_union_bound md sk m ctx : mu (dexact_hyperball_signing_attempt_state md sk m ctx) (signing_attempt_state_reference_rejection_aborts md) <= mu (dexact_hyperball_signing_attempt_state md sk m ctx) (signing_attempt_state_reject1_aborts md) + mu (dexact_hyperball_signing_attempt_state md s...</pre>	dexact_hyperball_signing_attempt_state_reference_rejection_abort_union_bound $X \leq Y$ is an arithmetic comparison upper bound.	Reference rejection abort union game-hop or final security

	Line	Construct	What was written	Symbolic correspondence
signing_attempt_state_of_sample_pair_signatureE	852	lemma lemma signing_attempt_state_of_sample_pair_signatureE md sk m ctx coins sample : signing_attempt_state_signature (signing_attempt_state_of_sample_pair md sk m ctx coins sample) = sign_internal md sk m ctx coins.	signing_attempt_state_of_sample_pair_signatureE $L \equiv R$ is an equality/rewriting fact.	LemmaE a bound that controls game-hop or final security
signing_attempt_state_of_sample_pair_pack_accepts	862	lemma lemma signing_attempt_state_of_sample_pair_pack_accepts md sk m ctx coins sample : signing_attempt_state_pack_accepts md (signing_attempt_state_of_sample_pair md sk m ctx coins sample).	signing_attempt_state_of_sample_pair_pack_accepts $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	PropAccepts a bound that controls game-hop or final security
signing_attempt_state_of_sample_pair_pack_no_abort	875	lemma lemma signing_attempt_state_of_sample_pair_pack_no_abort md sk m ctx coins sample : ! signing_attempt_state_pack_aborts md (signing_attempt_state_of_sample_pair md sk m ctx coins sample).	signing_attempt_state_of_sample_pair_pack_no_abort $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	PropNoAbort a bound that controls game-hop or final security
dexact_hyperball_signing_attempt_state_pack_abort_zero	884	lemma lemma dexact_hyperball_signing_attempt_state_pack_abort_zero md sk m ctx : mu (dexact_hyperball_signing_attempt_state md sk m ctx) (signing_attempt_state_pack_aborts md) = 0%r.	dexact_hyperball_signing_attempt_state_pack_abort_zero $L \equiv R$ is an equality/rewriting fact.	PropAbortZero not game equivalence simulator transition.
dstructural_signing_attempt_state_from_samplerE	904	lemma lemma dstructural_signing_attempt_state_from_samplerE md sk m ctx : dstructural_signing_attempt_state_from_sampler md sk m ctx = dstructural_signing_attempt_state md sk m ctx.	dstructural_signing_attempt_state_from_samplerE $L \equiv R$ is an equality/rewriting fact.	PropE a bound that controls game-hop or final security
	925	lemma		

	Line	Construct	What was written	Symbolic correspondence
		dstructural_signing_attempt_stateE	lemma dstructural_signing_attempt_stateE md sk m ctx : dstructural_signing_attempt_state md sk m ctx = dsigning_attempt_state md sk m ctx.	dstructural_signing_attempt_stateE : $L = R$ is an equality/rewriting fact. Proves an invariant preserved by oracle, adversary call, or game hop.
	934	lemma	lemma dstructural_signing_attempt_state_from_sampler_coin_sample_pairE md sk m ctx : dmap (dstructural_signing_attempt_coin_sample_pair md sk m ctx) (signing_attempt_state_of_coin_sample_pair md sk m ctx) = dstructural_signing_attempt_state_from_sampler md sk m ctx.	dstructural_signing_attempt_state_from_sampler_coin_sample_pairE : $L = R$ is an equality/rewriting fact. Proves coin sample pair invariant in game-hop or final security game.
	950	lemma	lemma dexact_hyperball_signing_attempt_state_coin_sample_pairE md sk m ctx : dmap (dexact_hyperball_signing_attempt_coin_sample_pair md sk m ctx) (signing_attempt_state_of_coin_sample_pair md sk m ctx) = dexact_hyperball_signing_attempt_state md sk m ctx.	dexact_hyperball_signing_attempt_state_coin_sample_pairE : $L = R$ is an equality/rewriting fact. Proves coin sample pair invariant in game-hop or final security game.
	966	lemma	lemma mu_dlet_le_add_all ['a 'b 'c] (d : 'a distr) (F1 : 'a -> 'b distr) (F2 : 'a -> 'c distr) (P1 : 'b -> bool) (P2 : 'c -> bool) (eps : real) : 0%r <= eps => (forall x, mu (F1 x) P1 <= mu (F2 x) P2 + eps) => mu (dlet d F1) P1 <= mu (dlet d F2) P2 + eps.	mu_dlet_le_add_all : $X \leq Y$ is an arithmetic or probability upper bound. Proves a distribution fact about sampling or probability measure.
	1004	lemma		

	Line	Construct	What was written	Symbolic correspondence
structural_to_exact_hyperball_sample_pair_loss_from_point_loss		lemma	structural_to_exact_hyperball_sample_pair_loss_from_point_loss	Proves an invariant preservation
		structural_to_exact_hyperball_sample_pair_loss_from_point_loss	$P \Rightarrow Q$ is an implication used as a proof obligation.	game-hop or final security
		:		
		structural_to_exact_hyperball_sample_pair_point_loss_obligation		
		=>		
		structural_to_exact_hyperball_sample_pair_loss_obligation.		
	1023	lemma		
structural_to_exact_hyperball_coin_sample_pair_loss_from_sample_pair_loss		lemma	structural_to_exact_hyperball_coin_sample_pair_loss_from_sample_pair_loss	Proves a bound that controls
		structural_to_exact_hyperball_coin_sample_pair_loss_from_sample_pair_loss	$P \Rightarrow Q$ is an implication used as a proof obligation.	game-hop or final security
		:		
		structural_to_exact_hyperball_sample_pair_loss_obligation		
		=>		
		structural_to_exact_hyperball_coin_sample_pair_loss_obligation.		
	1039	lemma		
structural_to_exact_hyperball_attempt_loss_from_coin_sample_pair_loss		lemma	structural_to_exact_hyperball_attempt_loss_from_coin_sample_pair_loss	Proves a bound that controls
		structural_to_exact_hyperball_attempt_loss_from_coin_sample_pair_loss	$P \Rightarrow Q$ is an implication used as a proof obligation.	game-hop or final security
		:		
		structural_to_exact_hyperball_coin_sample_pair_loss_obligation		
		=>		
		structural_to_exact_hyperball_attempt_loss_obligation.		
	1055	lemma		
signing_attempt_state_of_coins_signatureE		lemma	signing_attempt_state_of_coins_signatureE	Proves an invariant preservation
		signing_attempt_state_of_coins_signatureE	$L \Rightarrow R$ is an equality/rewriting fact.	oracle, adversary call, or game
		md sk m ctx coins :		procedure.
		signing_attempt_state_signature		
		(signing_attempt_state_of_coins		
		md sk m ctx coins) =		
		sign_internal md sk m ctx coins.		
	1062	lemma		
signing_attempt_state_of_coins_sampled_y1E		lemma	signing_attempt_state_of_coins_sampled_y1E	Proves a bound that controls
		signing_attempt_state_of_coins_sampled_y1E	$L \Rightarrow R$ is an equality/rewriting fact.	game-hop or final security
		md sk m ctx coins :		
		signing_attempt_state_sampled_y1		
		(signing_attempt_state_of_coins		
		md sk m ctx coins) =		
		signing_sample_y1 md sk m ctx		
		coins.		
	1073	lemma		

Line	Construct	What was written	Symbolic correspondence
signing_attempt_state_of_coins_sampled_y2E	lemma	signing_attempt_state_of_coins_sampled_y2E	Proves a bound that controls game-hop or final security
	signing_attempt_state_of_coins_sampled_y2E	$L_{d=y2E}$ is an equality/rewriting fact.	
	md sk m ctx coins :		
	signing_attempt_state_sampled_y2		
	(signing_attempt_state_of_coins		
	md sk m ctx coins) =		
	signing_sample_y2 md sk m ctx		
	coins.		
1084	lemma		
signing_attempt_state_of_coins_sampled_yE	lemma	signing_attempt_state_of_coins_sampled_yE	Proves a bound that controls game-hop or final security
	signing_attempt_state_of_coins_sampled_yE	$L_{d=yE}$ is an equality/rewriting fact.	
	md sk m ctx coins :		
	signing_attempt_state_sampled_y		
	(signing_attempt_state_of_coins		
	md sk m ctx coins) =		
	commitment_raw md sk m ctx		
	coins.		
1095	lemma		
signing_attempt_state_of_coins_highbitsE	lemma	signing_attempt_state_of_coins_highbitsE	Proves an invariant preserved by oracle, adversary call, or game procedure.
	signing_attempt_state_of_coins_highbitsE	L_{tsE} is an equality/rewriting fact.	
	md sk m ctx coins :		
	signing_attempt_state_highbits		
	(signing_attempt_state_of_coins		
	md sk m ctx coins) =		
	commitment_highbits md sk m ctx		
	coins.		
1102	lemma		
signing_attempt_state_of_coins_lowbitsE	lemma	signing_attempt_state_of_coins_lowbitsE	Proves an invariant preserved by oracle, adversary call, or game procedure.
	signing_attempt_state_of_coins_lowbitsE	L_{tsE} is an equality/rewriting fact.	
	md sk m ctx coins :		
	signing_attempt_state_lowbits		
	(signing_attempt_state_of_coins		
	md sk m ctx coins) =		
	commitment_lowbits md sk m ctx		
	coins.		
1109	lemma		

	Line	Construct	What was written	Symbolic correspondence
signing_attempt_state_of_coins_challengeE		lemma signing_attempt_state_of_coins_challengeE md sk m ctx coins : signing_attempt_state_challenge (signing_attempt_state_of_coins md sk m ctx coins) = commitment_challenge md sk m ctx coins.	signing_attempt_state_of_coins_challengeE $L = R$ is an equality/rewriting fact.	Proves a bound that controls game-hop or final security
signing_attempt_state_of_coins_responseE	1116	lemma signing_attempt_state_of_coins_responseE md sk m ctx coins : signing_attempt_state_response (signing_attempt_state_of_coins md sk m ctx coins) = response_vector md sk m ctx coins.	signing_attempt_state_of_coins_responseE $L = R$ is an equality/rewriting fact.	Proves an invariant preserved oracle, adversary call, or game procedure.
signing_attempt_state_of_coins_response_auxE	1123	lemma signing_attempt_state_of_coins_response_auxE md sk m ctx coins : signing_attempt_state_response_aux (signing_attempt_state_of_coins md sk m ctx coins) = response_aux_vector md sk m ctx coins.	signing_attempt_state_of_coins_response_auxE $L = R$ is an equality/rewriting fact.	Proves an invariant preserved oracle, adversary call, or game procedure.
signing_attempt_state_of_coins_hintE	1130	lemma signing_attempt_state_of_coins_hintE md sk m ctx coins : signing_attempt_state_hint (signing_attempt_state_of_coins md sk m ctx coins) = hint_vector md sk m ctx coins.	signing_attempt_state_of_coins_hintE : $L = R$ is an equality/rewriting fact.	Proves an invariant preserved oracle, adversary call, or game procedure.
	1137	lemma		

	Line	Construct	What was written	Symbolic correspondence
signing_attempt_state_of_coins_lowbits_carrierE		lemma	signing_attempt_state_of_coins_lowbits_carrierE	Proves an invariant preserved by oracle, adversary call, or game procedure.
		signing_attempt_state_of_coins_lowbits_carrierE	$L = R$ is an equality/rewriting fact.	
		md sk m ctx coins :		
		signing_attempt_state_lowbits_carrier		
		(signing_attempt_state_of_coins		
		md sk m ctx coins) = nth		
		poly_zero (commitment_raw md sk		
		m ctx coins) 0.		
	1146	lemma	signing_attempt_state_of_coins_tokenE	Proves an invariant preserved by oracle, adversary call, or game procedure.
signing_attempt_state_of_coins_tokenE		lemma	signing_attempt_state_of_coins_tokenE	Proves an invariant preserved by oracle, adversary call, or game procedure.
		signing_attempt_state_of_coins_tokenE	$L = R$ is an equality/rewriting fact.	
		md sk m ctx coins :		
		signing_attempt_state_token		
		(signing_attempt_state_of_coins		
		md sk m ctx coins) =		
		signature_token md sk m ctx		
		coins.		
	1158	lemma	signing_attempt_state_of_coins_programmed_lowbitsE	Proves an invariant preserved by oracle, adversary call, or game procedure.
signing_attempt_state_of_coins_programmed_lowbitsE		lemma	signing_attempt_state_of_coins_programmed_lowbitsE	Proves an invariant preserved by oracle, adversary call, or game procedure.
		signing_attempt_state_of_coins_programmed_lowbitsE	$L = R$ is an equality/rewriting fact.	
		md sk m ctx coins :		
		signing_attempt_state_programmed_lowbits		
		(signing_attempt_state_of_coins		
		md sk m ctx coins) =		
		commitment_lowbits md sk m ctx		
		coins.		
	1169	lemma	signing_attempt_state_of_coins_lowbits_consistentE	Proves an invariant preserved by oracle, adversary call, or game procedure.
signing_attempt_state_of_coins_lowbits_consistentE		lemma	signing_attempt_state_of_coins_lowbits_consistentE	Proves an invariant preserved by oracle, adversary call, or game procedure.
		signing_attempt_state_of_coins_lowbits_consistentE	$\forall \vec{x}. B(\vec{x})$ is a universally quantified fact.	
		md sk m ctx coins :		
		signing_attempt_state_lowbits_consistent		
		(signing_attempt_state_of_coins		
		md sk m ctx coins).		
	1178	lemma		

Line	Construct	What was written	Symbolic correspondence
	<pre> signing_attempt_state_of_coins_programmed_challengeE lemma signing_attempt_state_of_coins_programmed_challengeE md sk m ctx coins : signing_attempt_state_programmed_challenge md (public_key_of_secret md sk) m ctx (signing_attempt_state_of_coins md sk m ctx coins) = commitment_challenge md sk m ctx coins. </pre>	<pre> signing_attempt_state_of_coins_programmed_challengeE $\text{Lemma } P$ is an equality/rewriting fact. </pre>	<pre> End that contr game-hop or final security </pre>
1189	<pre> lemma signing_attempt_state_of_coins_challenge_consistent md sk m ctx coins : signing_attempt_state_challenge_consistent md (public_key_of_secret md sk) m ctx (signing_attempt_state_of_coins md sk m ctx coins). </pre>	<pre> signing_attempt_state_of_coins_challenge_consistent $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact. </pre>	<pre> Consistent bound that contr game-hop or final security </pre>
1198	<pre> lemma signing_attempt_state_of_coins_reconstructed_highbitsE md sk m ctx coins : signing_attempt_state_reconstructed_highbits md (public_key_of_secret md sk) (signing_attempt_state_of_coins md sk m ctx coins) = commitment_highbits md sk m ctx coins. </pre>	<pre> signing_attempt_state_of_coins_reconstructed_highbitsE $\text{Lemma } P$ is an equality/rewriting fact. </pre>	<pre> Ed highbitsE variant preserv oracle, adversary call, or g procedure. </pre>
1210	<pre> lemma signing_attempt_state_of_coins_public_equation_consistent md sk m ctx coins : signing_attempt_state_public_equation_consistent md (public_key_of_secret md sk) (signing_attempt_state_of_coins md sk m ctx coins). </pre>	<pre> signing_attempt_state_of_coins_public_equation_consistent $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact. </pre>	<pre> $\text{Equation consistent}$ variant preserv oracle, adversary call, or g procedure. </pre>
1221	<pre> lemma </pre>	<pre> </pre>	<pre> </pre>

	Line	Construct	What was written	Symbolic correspondence
signing_attempt_state_of_coins_signature_of_fieldsE		lemma	signing_attempt_state_of_coins_signature_of_fieldsE	Proves an invariant preservation
		signing_attempt_state_of_coins_signature_of_fieldsE is an equality/rewriting fact.		oracle, adversary call, or game procedure.
		md sk m ctx coins :		
		signing_attempt_state_signature_of_fields		
		(public_key_of_secret md sk) m		
		ctx		
		(signing_attempt_state_of_coins		
		md sk m ctx coins) =		
		sign_internal md sk m ctx coins.		
	1234	lemma	signing_attempt_state_of_coins_fields_match_signature	Proves an invariant preservation
signing_attempt_state_of_coins_fields_match_signature		lemma	signing_attempt_state_of_coins_fields_match_signature	Proves an invariant preservation
		signing_attempt_state_of_coins_fields_match_signature is a universally quantified fact.		oracle, adversary call, or game procedure.
		md sk m ctx coins :		
		signing_attempt_state_fields_match_signature		
		(public_key_of_secret md sk) m		
		ctx		
		(signing_attempt_state_of_coins		
		md sk m ctx coins).		
	1244	lemma	signing_attempt_state_of_coins_field_accepts	Proves an invariant preservation
signing_attempt_state_of_coins_field_accepts		lemma	signing_attempt_state_of_coins_field_accepts	Proves an invariant preservation
		signing_attempt_state_of_coins_field_accepts is a universally quantified fact.		oracle, adversary call, or game procedure.
		md sk m ctx coins :		
		signing_attempt_state_field_accepts		
		md (public_key_of_secret md sk)		
		m ctx		
		(signing_attempt_state_of_coins		
		md sk m ctx coins).		
	1258	lemma	signing_attempt_state_of_coins_accepts_current	Proves an invariant preservation
signing_attempt_state_of_coins_accepts_current		lemma	signing_attempt_state_of_coins_accepts_current	Proves an invariant preservation
		signing_attempt_state_of_coins_accepts_current is a universally quantified fact.		oracle, adversary call, or game procedure.
		md sk m ctx coins :		
		signing_attempt_state_accepts md		
		(signing_attempt_state_of_coins		
		md sk m ctx coins).		
	1272	lemma	signing_attempt_reject1_bound_ge_response_bound	Proves a bound that controls
signing_attempt_reject1_bound_ge_response_bound		lemma	signing_attempt_reject1_bound_ge_response_bound	Proves a bound that controls
		signing_attempt_reject1_bound_ge_response_bound is an arithmetic or probability upper bound.		game-hop or final security
		md : mode_blsq md <=		
		signing_attempt_reject1_bound		
		md.		
	1276	lemma		

	Line	Construct	What was written	Symbolic correspondence
signing_attempt_state_reject1_accepts_from_response_norm_ok		lemma	signing_attempt_state_reject1_accepts_from_response_norm_ok	Proves an invariant preserved
		signing_attempt_state_reject1_accepts_from_response_norm_ok	$P \Rightarrow Q$ is an implication used as a proof	oracle, adversary call, or game hop
		md st :	obligation.	procedure.
		signing_attempt_state_response_norm_accepts		
		md st =>		
		signing_attempt_state_reject1_accepts		
		md st.		
	1289	lemma	signing_attempt_state_balance_norm_sq_concreteE	ConcreteE invariant preserved
signing_attempt_state_balance_norm_sq_concreteE		lemma	signing_attempt_state_balance_norm_sq_concreteE	ConcreteE invariant preserved
		signing_attempt_state_balance_norm_sq_concreteE	$L \Leftarrow R$ is an equality/rewriting fact.	oracle, adversary call, or game hop
		md st :		procedure.
		signing_attempt_state_balance_norm_sq		
		md st =		
		signing_attempt_reject2_balance_norm_sq		
		md		
		(signing_attempt_state_response		
		st)		
		(signing_attempt_state_response_aux		
		st)		
		(signing_attempt_state_reject2_sampled_y1		
		st)		
		(signing_attempt_state_reject2_sampled_y2		
		st).		
	1298	lemma	signing_attempt_state_reject2_under_bound_concreteE	Proves that bound that controls
signing_attempt_state_reject2_under_bound_concreteE		lemma	signing_attempt_state_reject2_under_bound_concreteE	Proves that bound that controls
		signing_attempt_state_reject2_under_bound_concreteE	$L \Leftarrow R$ is an equality/rewriting fact.	game-hop or final security
		md st :		
		signing_attempt_state_reject2_under_bound		
		md st =		
		(signing_attempt_reject2_balance_norm_sq		
		md		
		(signing_attempt_state_response		
		st)		
		(signing_attempt_state_response_aux		
		st)		
		(signing_attempt_state_reject2_sampled_y1		
		st)		
		(signing_attempt_state_reject2_sampled_y2		
		st) <		
		signing_attempt_reject2_bou...		
	1311	lemma		

	Line	Construct	What was written	Symbolic correspondence
		<pre> signing_attempt_state_reject2_aborts_concreteE lemma signing_attempt_state_reject2_aborts_concreteE md st : signing_attempt_state_reject2_aborts md st = signing_attempt_reject2_concrete_aborts md (signing_attempt_state_bimodal_reject2_branch st) (signing_attempt_state_response st) (signing_attempt_state_response_aux st) (signing_attempt_state_reject2_sampled_y1 st) (signing_attempt_state_reject2_sampled_y2 st) </pre>	<pre> signing_attempt_state_reject2_aborts_concreteE </pre>	Proves an invariant preserved by the oracle, adversary call, or game hop procedure.
	1321	<pre> lemma signing_attempt_state_reject2_accepts_from_branch_clear signing_attempt_state_reject2_accepts_from_branch_clear md st : ! signing_attempt_state_bimodal_reject2_branch st => signing_attempt_state_reject2_accepts md st. </pre>	<pre> signing_attempt_state_reject2_accepts_from_branch_clear </pre>	Proves a branch bound: that controlling the game-hop or final security obligation.
	1332	<pre> lemma signing_attempt_state_of_sample_pair_reject2_branch_clear signing_attempt_state_of_sample_pair_reject2_branch_clear md sk m ctx coins sample : ! signing_attempt_state_bimodal_reject2_branch (signing_attempt_state_of_sample_pair md sk m ctx coins sample). </pre>	<pre> signing_attempt_state_of_sample_pair_reject2_branch_clear </pre>	Proves a branch bound: that controlling the game-hop or final security obligation.
	1345	<pre> lemma signing_attempt_state_of_sample_pair_reject2_accepts signing_attempt_state_of_sample_pair_reject2_accepts md sk m ctx coins sample : signing_attempt_state_reject2_accepts md (signing_attempt_state_of_sample_pair md sk m ctx coins sample). </pre>	<pre> signing_attempt_state_of_sample_pair_reject2_accepts </pre>	Proves a branch bound: that controlling the game-hop or final security obligation.
	1354	<pre> lemma </pre>		

	Line	Construct	What was written	Symbolic correspondence
signing_attempt_state_of_sample_pair_reject2_no_abort		lemma	signing_attempt_state_of_sample_pair_reject2_no_abort	Pt2 over a bound that controls
		signing_attempt_state_of_sample_pair_reject2_no_abort	$\forall \vec{z}. P(\vec{z})$ is a universally quantified fact.	game-hop or final security
		md sk m ctx coins sample :	!	
		signing_attempt_state_reject2_aborts		
		md		
		(signing_attempt_state_of_sample_pair		
		md sk m ctx coins sample).		
	1364	lemma		
dexact_hyperball_signing_attempt_state_reject2_abort_zero		lemma	dexact_hyperball_signing_attempt_state_reject2_abort_zero	Pt2 over a bound that controls
		dexact_hyperball_signing_attempt_state_reject2_abort_zero	$L = R$ is an equality/rewriting fact.	game-equivalence
		md sk m ctx :	mu	
		(dexact_hyperball_signing_attempt_state		
		md sk m ctx)		
		(signing_attempt_state_reject2_aborts		
		md) = 0%r.		
	1384	lemma		
dexact_hyperball_signing_attempt_state_reference_rejection_abort_reduced_bound		lemma	dexact_hyperball_signing_attempt_state_reference_rejection_abort_reduced_bound	Pt2 over a bound that controls
		dexact_hyperball_signing_attempt_state_reference_rejection_abort_reduced_bound	$X \leq K$ is an arithmetic or probability	game-hop or final security
		md sk m ctx :	mu	
		(dexact_hyperball_signing_attempt_state		
		md sk m ctx)		
		(signing_attempt_state_reference_rejection_aborts		
		md) <= mu		
		(dexact_hyperball_signing_attempt_state		
		md sk m ctx)		
		(signing_attempt_state_reject1_aborts		
		md) + mu		
		(dexact_hyperball_signing_attempt_state		
		md...		
	1401	lemma		
signing_attempt_state_of_sample_pair_response_norm_accepts		lemma	signing_attempt_state_of_sample_pair_response_norm_accepts	Pt2 over a bound that controls
		signing_attempt_state_of_sample_pair_response_norm_accepts	$\forall \vec{z}. P(\vec{z})$ is a universally quantified fact.	game-hop or final security
		md sk m ctx coins sample :		
		signing_attempt_state_response_norm_accepts		
		md		
		(signing_attempt_state_of_sample_pair		
		md sk m ctx coins sample).		
	1411	lemma		

	Line	Construct	What was written	Symbolic correspondence
signing_attempt_state_of_sample_pair_reject1_accepts		lemma	signing_attempt_state_of_sample_pair_reject1_accepts	Prove accepts and that contr
		signing_attempt_state_of_sample_pair_reject1_accepts	$\forall \vec{x}; P(\vec{x})$ is a universally quantified fact.	game-hop or final security
		md sk m ctx coins sample :		
		signing_attempt_state_reject1_accepts		
		md		
		(signing_attempt_state_of_sample_pair		
		md sk m ctx coins sample).		
	1420	lemma	signing_attempt_state_of_sample_pair_reject1_no_abort	Prove no abort and that contr
signing_attempt_state_of_sample_pair_reject1_no_abort		lemma	signing_attempt_state_of_sample_pair_reject1_no_abort	Prove no abort and that contr
		signing_attempt_state_of_sample_pair_reject1_no_abort	$\forall \vec{x}; P(\vec{x})$ is a universally quantified fact.	game-hop or final security
		md sk m ctx coins sample : !		
		signing_attempt_state_reject1_aborts		
		md		
		(signing_attempt_state_of_sample_pair		
		md sk m ctx coins sample).		
	1429	lemma	dexact_hyperball_signing_attempt_state_reject1_abort_zero	Prove abort zero game equi
dexact_hyperball_signing_attempt_state_reject1_abort_zero		lemma	dexact_hyperball_signing_attempt_state_reject1_abort_zero	Prove abort zero game equi
		dexact_hyperball_signing_attempt_state_reject1_abort_zero	$L = R$ is an equality/rewriting fact.	simulator transition.
		md sk m ctx : mu		
		(dexact_hyperball_signing_attempt_state		
		md sk m ctx)		
		(signing_attempt_state_reject1_aborts		
		md) = 0%r.		
	1449	lemma	dexact_hyperball_signing_attempt_state_reference_rejection_abort_hint_bound	Prove a rejection abort hint
dexact_hyperball_signing_attempt_state_reference_rejection_abort_hint_bound		lemma	dexact_hyperball_signing_attempt_state_reference_rejection_abort_hint_bound	Prove a rejection abort hint
		dexact_hyperball_signing_attempt_state_reference_rejection_abort_hint_bound	$X \leq Y$ is an arithmetic comparison	game-hop or final security
		md sk m ctx : mu	upper bound.	
		(dexact_hyperball_signing_attempt_state		
		md sk m ctx)		
		(signing_attempt_state_reference_rejection_aborts		
		md) <= mu		
		(dexact_hyperball_signing_attempt_state		
		md sk m ctx)		
		(signing_attempt_state_hint_norm_aborts		
		md).		
	1463	lemma		

	Line	Construct	What was written	Symbolic correspondence
signing_attempt_state_of_sample_pair_hint_norm_accepts		lemma	signing_attempt_state_of_sample_pair_hint_norm_accepts	Proves an accept that contr
		signing_attempt_state_of_sample_pair_hint_norm_accepts	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	game-hop or final security
		md sk m ctx coins sample :		
		signing_attempt_state_hint_norm_accepts		
		md		
		(signing_attempt_state_of_sample_pair		
		md sk m ctx coins sample).		
	1473	lemma	signing_attempt_state_of_sample_pair_hint_norm_no_abort	Proves a no abort that contr
signing_attempt_state_of_sample_pair_hint_norm_no_abort		lemma	signing_attempt_state_of_sample_pair_hint_norm_no_abort	Proves a no abort that contr
		signing_attempt_state_of_sample_pair_hint_norm_no_abort	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	game-hop or final security
		md sk m ctx coins sample : !		
		signing_attempt_state_hint_norm_aborts		
		md		
		(signing_attempt_state_of_sample_pair		
		md sk m ctx coins sample).		
	1482	lemma	dexact_hyperball_signing_attempt_state_hint_norm_abort_zero	Proves an abort game equi
dexact_hyperball_signing_attempt_state_hint_norm_abort_zero		lemma	dexact_hyperball_signing_attempt_state_hint_norm_abort_zero	Proves an abort game equi
		dexact_hyperball_signing_attempt_state_hint_norm_abort_zero	$L \equiv R$ is an equality/rewriting fact.	simulator transition.
		md sk m ctx : mu		
		(dexact_hyperball_signing_attempt_state		
		md sk m ctx)		
		(signing_attempt_state_hint_norm_aborts		
		md) = 0%r.		
	1502	lemma	dexact_hyperball_signing_attempt_state_reference_rejection_abort_zero	Proves a rejection abort equi
dexact_hyperball_signing_attempt_state_reference_rejection_abort_zero		lemma	dexact_hyperball_signing_attempt_state_reference_rejection_abort_zero	Proves a rejection abort equi
		dexact_hyperball_signing_attempt_state_reference_rejection_abort_zero	$L \equiv R$ is an equality/rewriting fact.	simulator transition.
		md sk m ctx : mu		
		(dexact_hyperball_signing_attempt_state		
		md sk m ctx)		
		(signing_attempt_state_reference_rejection_aborts		
		md) = 0%r.		
	1515	lemma	signing_attempt_state_of_coins_reject2_branch_clear_current	Proves a clear current that contr
signing_attempt_state_of_coins_reject2_branch_clear_current		lemma	signing_attempt_state_of_coins_reject2_branch_clear_current	Proves a clear current that contr
		signing_attempt_state_of_coins_reject2_branch_clear_current	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	game-hop or final security
		md sk m ctx coins : !		
		signing_attempt_state_bimodal_reject2_branch		
		(signing_attempt_state_of_coins		
		md sk m ctx coins).		
	1528	lemma		

Line	Construct	What was written	Symbolic correspondence
	<pre> signing_attempt_state_of_coins_reject2_sampled_y1E lemma signing_attempt_state_of_coins_reject2_sampled_y1E md sk m ctx coins : signing_attempt_state_reject2_sampled_y1 (signing_attempt_state_of_coins md sk m ctx coins) = signing_sample_y1 md sk m ctx coins. </pre>	<pre> signing_attempt_state_of_coins_reject2_sampled_y1E L2-B is an equality/rewriting fact. </pre>	<pre> signing_attempt_state_of_coins_reject2_sampled_y1E Implies y1E found that contr game-hop or final security </pre>
1537	lemma		
	<pre> signing_attempt_state_of_coins_reject2_sampled_y2E lemma signing_attempt_state_of_coins_reject2_sampled_y2E md sk m ctx coins : signing_attempt_state_reject2_sampled_y2 (signing_attempt_state_of_coins md sk m ctx coins) = signing_sample_y2 md sk m ctx coins. </pre>	<pre> signing_attempt_state_of_coins_reject2_sampled_y2E L2-B is an equality/rewriting fact. </pre>	<pre> signing_attempt_state_of_coins_reject2_sampled_y2E Implies y2E found that contr game-hop or final security </pre>
1546	lemma		
	<pre> signing_attempt_state_of_coins_reject2_balance_norm_sqE lemma signing_attempt_state_of_coins_reject2_balance_norm_sqE md sk m ctx coins : signing_attempt_state_balance_norm_sq md (signing_attempt_state_of_coins md sk m ctx coins) = signing_attempt_reject2_balance_norm_sq md (response_vector md sk m ctx coins) (response_aux_vector md sk m ctx coins) (signing_sample_y1 md sk m ctx coins) (signing_sample_y2 md sk... </pre>	<pre> signing_attempt_state_of_coins_reject2_balance_norm_sqE L2-B is an equality/rewriting fact. </pre>	<pre> signing_attempt_state_of_coins_reject2_balance_norm_sqE Implies norm_sqE ant preserv oracle, adversary call, or g procedure. </pre>
1563	lemma		

Line	Construct	What was written	Symbolic correspondence
	<pre> signing_attempt_state_of_coins_balance_norm_sq_current lemma signing_attempt_state_of_coins_balance_norm_sq_current md sk m ctx coins : signing_attempt_state_balance_norm_sq md (signing_attempt_state_of_coins md sk m ctx coins) = polyvecl_norm_sq md (signing_attempt_state_reject2_balance_response md (signing_attempt_state_of_coins md sk m ctx coins)) + polyveck_norm_sq md (signing_attempt_state_reject2_balanc...</pre>	<pre> signing_attempt_state_of_coins_balance_norm_sq_current $L \Rightarrow R$ is an equality/rewriting fact.</pre>	Proves a current invariant preserved by oracle, adversary call, or g procedure.
1580	<pre> lemma signing_attempt_state_of_coins_reject2_accepts_current md sk m ctx coins : signing_attempt_state_reject2_accepts md (signing_attempt_state_of_coins md sk m ctx coins).</pre>	<pre> signing_attempt_state_of_coins_reject2_accepts_current $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.</pre>	Proves a current invariant preserved by oracle, adversary call, or g procedure.
1589	<pre> lemma signing_attempt_state_pack_accepts_from_valid md st : signing_attempt_state_valid_signature_accepts md st => signing_attempt_state_pack_accepts md st.</pre>	<pre> signing_attempt_state_pack_accepts_from_valid $P \Rightarrow Q$ is an implication used as a proof obligation.</pre>	Validates an invariant preserved by oracle, adversary call, or g procedure.
1599	<pre> lemma signing_attempt_state_reference_rejection_accepts_from_accepts md st : signing_attempt_state_accepts md st => signing_attempt_state_reject2_accepts md st => signing_attempt_state_reference_rejection_accepts md st.</pre>	<pre> signing_attempt_state_reference_rejection_accepts_from_accepts $P \Rightarrow Q$ is an implication used as a proof obligation.</pre>	Proves a from invariant preserved by oracle, adversary call, or g procedure.
1616	<pre> lemma</pre>		

	Line	Construct	What was written	Symbolic correspondence
signing_attempt_state_of_coins_reference_rejection_accepts_current		lemma	signing_attempt_state_of_coins_reference_rejection_accepts_current	Rejection and acceptance current
		signing_attempt_state_of_coins_reference_rejection_accepts_current	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	oracle, adversary call, or g
		md sk m ctx coins :		procedure.
		signing_attempt_state_reference_rejection_accepts		
		md		
		(signing_attempt_state_of_coins		
		md sk m ctx coins).		
	1626	lemma	signing_attempt_state_of_coins_verification_accepts_current	Accepts invariant preser
signing_attempt_state_of_coins_verification_accepts_current		lemma	signing_attempt_state_of_coins_verification_accepts_current	Accepts invariant preser
		signing_attempt_state_of_coins_verification_accepts_current	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	oracle, adversary call, or g
		md sk m ctx coins :		procedure.
		signing_attempt_state_verification_accepts		
		md (public_key_of_secret md sk)		
		m ctx		
		(signing_attempt_state_of_coins		
		md sk m ctx coins).		
	1637	lemma	signing_attempt_state_of_coins_rejection_accepts_current	Accepts invariant preser
signing_attempt_state_of_coins_rejection_accepts_current		lemma	signing_attempt_state_of_coins_rejection_accepts_current	Accepts invariant preser
		signing_attempt_state_of_coins_rejection_accepts_current	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	oracle, adversary call, or g
		md sk m ctx coins :		procedure.
		signing_attempt_state_rejection_accepts		
		md (public_key_of_secret md sk)		
		m ctx		
		(signing_attempt_state_of_coins		
		md sk m ctx coins).		
	1648	lemma	signing_attempt_state_of_coins_rejection_no_abort_current	No abort invariant preser
signing_attempt_state_of_coins_rejection_no_abort_current		lemma	signing_attempt_state_of_coins_rejection_no_abort_current	No abort invariant preser
		signing_attempt_state_of_coins_rejection_no_abort_current	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	oracle, adversary call, or g
		md sk m ctx coins : !		procedure.
		signing_attempt_state_rejection_aborts		
		md (public_key_of_secret md sk)		
		m ctx		
		(signing_attempt_state_of_coins		
		md sk m ctx coins).		
	1657	lemma	signing_attempt_state_of_coins_no_abort_current	Current invariant preser
signing_attempt_state_of_coins_no_abort_current		lemma	signing_attempt_state_of_coins_no_abort_current	Current invariant preser
		signing_attempt_state_of_coins_no_abort_current	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	oracle, adversary call, or g
		md sk m ctx coins : !		procedure.
		signing_attempt_state_aborts md		
		(signing_attempt_state_of_coins		
		md sk m ctx coins).		

	Line	Construct	What was written	Symbolic correspondence
	1674	lemma		
signing_attempt_state_distribution_lossless		lemma signing_attempt_state_distribution_lossless md sk m ctx : is_lossless (dsigning_attempt_state md sk m ctx).	signing_attempt_state_distribution_lossless : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a distribution fact sampling or probability measure.
	1681	lemma		
signing_attempt_state_distribution_token_spread		lemma signing_attempt_state_distribution_token_spread md sk m ctx token : mu (dsigning_attempt_state md sk m ctx) (fun st => signing_entropy_token_of_coins (signing_attempt_state_coins st) = token) <= signing_randomness_point_bound.	signing_attempt_state_distribution_token_spread : $X \leq Y$ is an arithmetic or probability upper bound.	Proves an invariant preserved by oracle, adversary call, or g procedure.
	1698	lemma		
signing_attempt_state_distribution_reference_rejection_abort_zero_current		lemma signing_attempt_state_distribution_reference_rejection_abort_zero_current md sk m ctx : mu (dsigning_attempt_state md sk m ctx) (signing_attempt_state_reference_rejection_aborts md) = 0%r.	signing_attempt_state_distribution_reference_rejection_abort_zero_current : $L = R$ is an equality/rewriting fact.	Projection of abort zero current oracle, adversary call, or g procedure.
	1712	lemma		
signing_attempt_state_distribution_rejection_abort_zero_current		lemma signing_attempt_state_distribution_rejection_abort_zero_current md sk m ctx : mu (dsigning_attempt_state md sk m ctx) (signing_attempt_state_rejection_aborts md (public_key_of_secret md sk) m ctx) = 0%r.	signing_attempt_state_distribution_rejection_abort_zero_current : $L = R$ is an equality/rewriting fact.	Projection of abort zero current oracle, adversary call, or g procedure.
	1726	lemma		
sign_internal_simulation_relation		lemma sign_internal_simulation_relation md sk m ctx coins : signing_simulation_relation md (public_key_of_secret md sk) m ctx (sign_internal md sk m ctx coins).	sign_internal_simulation_relation : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a distribution fact sampling or probability measure.

	Line	Construct	What was written	Symbolic correspondence
	1736	lemma <code>signing_attempt_transcript_valid</code> <code>lemma</code> <code>signing_attempt_transcript_valid</code> <code>md pk sk m ctx coins sig :</code> <code>signing_attempt_relation md pk</code> <code>sk m ctx coins sig =></code> <code>transcript_valid md</code> <code>(transcript_of_signature md pk m</code> <code>ctx sig).</code>	<code>signing_attempt_transcript_valid : $P \Rightarrow Q$</code> Q is an implication used as a proof obligation.	Proves a structural correctness property that can be used to justify later reductions.
	1753	lemma <code>signing_attempt_transcript_relation</code> <code>lemma</code> <code>signing_attempt_transcript_relation</code> <code>md pk sk m ctx coins sig :</code> <code>signing_attempt_relation md pk</code> <code>sk m ctx coins sig =></code> <code>signing_transcript_relation md</code> <code>pk m ctx sig</code> <code>(transcript_of_signature md pk m</code> <code>ctx sig).</code>	<code>signing_attempt_transcript_relation :</code> $P \Rightarrow Q$ is an implication used as a proof obligation.	Records a reusable theorem that can be used to rewrite, apply, or simplify expressions.
	1766	lemma <code>sign_internal_transcript_relation</code> <code>lemma</code> <code>sign_internal_transcript_relation</code> <code>md sk m ctx coins :</code> <code>signing_transcript_relation md</code> <code>(public_key_of_secret md sk) m</code> <code>ctx (sign_internal md sk m ctx</code> <code>coins) (transcript_of_signature</code> <code>md (public_key_of_secret md sk)</code> <code>m ctx (sign_internal md sk m ctx</code> <code>coins)).</code>	<code>sign_internal_transcript_relation :</code> $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem that can be used to rewrite, apply, or simplify expressions.
	1779	lemma <code>sign_internal_record_relation</code> <code>lemma</code> <code>sign_internal_record_relation md</code> <code>sk m ctx coins :</code> <code>signing_record_relation md</code> <code>(public_key_of_secret md sk) (m,</code> <code>ctx, sign_internal md sk m ctx</code> <code>coins, transcript_of_signature</code> <code>md (public_key_of_secret md sk)</code> <code>m ctx (sign_internal md sk m ctx</code> <code>coins)).</code>	<code>sign_internal_record_relation : $\forall \vec{x}. P(\vec{x})$</code> is a universally quantified fact.	Records a reusable theorem that can be used to rewrite, apply, or simplify expressions.

	Line	Construct	What was written	Symbolic corresponden
	1792	lemma		
signing_record_relation_sound		<pre> lemma signing_record_relation_sound md pk r : signing_record_relation md pk r => signing_simulation_relation md pk (signing_record_message r) (signing_record_context r) (signing_record_signature r) /\ transcript_valid md (signing_record_transcript r).</pre>	$\text{signing_record_relation_sound} : P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a structural correctness property that justifies later reductions.
	1809	lemma		
signing_record_log_sound_cons		<pre> lemma signing_record_log_sound_cons md pk r rs : signing_record_relation md pk r => signing_record_log_sound md pk rs => signing_record_log_sound md pk (r :: rs).</pre>	$\text{signing_record_log_sound_cons} : P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a structural correctness property that justifies later reductions.
	1817	lemma		
signing_record_log_sound_transcripts_valid		<pre> lemma signing_record_log_sound_transcripts_valid md md pk rs : signing_record_log_sound md pk rs => transcript_log_valid md (signing_record_transcripts rs).</pre>	$\text{signing_record_log_sound_transcripts_valid} : P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a structural correctness property that justifies later reductions.
	1830	lemma		
signing_record_queries_cons		<pre> lemma signing_record_queries_cons r rs : signing_record_queries (r :: rs) = signing_record_query r :: signing_record_queries rs.</pre>	$\text{signing_record_queries_cons} : L = R$ is an equality/rewriting fact.	Records a reusable theorem that transcripts can rewrite, apply,
	1835	lemma		
signing_record_transcripts_cons		<pre> lemma signing_record_transcripts_cons r rs : signing_record_transcripts (r :: rs) = signing_record_transcript r :: signing_record_transcripts rs.</pre>	$\text{signing_record_transcripts_cons} : L = R$ is an equality/rewriting fact.	Records a reusable theorem that transcripts can rewrite, apply,

	Line	Construct	What was written	Symbolic correspondence
	1840	lemma rejection_bound_parts_nonnegative	lemma rejection_bound_parts_nonnegative : rejection_sampling_bound_obligation => 0%r <= rejection_sampling_loss_term.	rejection_bound_parts_nonnegative : $X \leq Y$ is an arithmetic or probability upper bound.
	1845	lemma rejection_sampling_bound_obligation_holds	lemma rejection_sampling_bound_obligation_holds : rejection_sampling_bound_obligation.	rejection_sampling_bound_obligation_holds $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.
	1856	lemma signing_attempt_state_of_sample_pair_sampled_yE	lemma signing_attempt_state_of_sample_pair_sampled_yE : md sk m ctx coins sample : signing_attempt_state_sampled_y (signing_attempt_state_of_sample_pair md sk m ctx coins sample) = commitment_raw md sk m ctx coins.	signing_attempt_state_of_sample_pair_sampled_yE $L_{\text{sampled_y}}$ is an equality/rewriting fact.
	1868	lemma signing_attempt_state_of_sample_pair_highbitsE	lemma signing_attempt_state_of_sample_pair_highbitsE : md sk m ctx coins sample : signing_attempt_state_highbits (signing_attempt_state_of_sample_pair md sk m ctx coins sample) = commitment_highbits md sk m ctx coins.	signing_attempt_state_of_sample_pair_highbitsE L_{highbits} is an equality/rewriting fact.
	1877	lemma signing_attempt_state_of_sample_pair_lowbitsE	lemma signing_attempt_state_of_sample_pair_lowbitsE : md sk m ctx coins sample : signing_attempt_state_lowbits (signing_attempt_state_of_sample_pair md sk m ctx coins sample) = commitment_lowbits md sk m ctx coins.	signing_attempt_state_of_sample_pair_lowbitsE L_{lowbits} is an equality/rewriting fact.
	1886	lemma		

Line	Construct	What was written	Symbolic correspondence
1895	signing_attempt_state_of_sample_pair_challengeE lemma signing_attempt_state_of_sample_pair_challengeE md sk m ctx coins sample : signing_attempt_state_challenge (signing_attempt_state_of_sample_pair md sk m ctx coins sample) = commitment_challenge md sk m ctx coins.	signing_attempt_state_of_sample_pair_challengeE $L_{challenge}$ is an equality/rewriting fact.	PengeE a bound that controls game-hop or final security
1904	signing_attempt_state_of_sample_pair_responseE lemma signing_attempt_state_of_sample_pair_responseE md sk m ctx coins sample : signing_attempt_state_response (signing_attempt_state_of_sample_pair md sk m ctx coins sample) = response_vector md sk m ctx coins.	signing_attempt_state_of_sample_pair_responseE $L_{response}$ is an equality/rewriting fact.	PenseE a bound that controls game-hop or final security
1914	signing_attempt_state_of_sample_pair_response_auxE lemma signing_attempt_state_of_sample_pair_response_auxE md sk m ctx coins sample : signing_attempt_state_response_aux (signing_attempt_state_of_sample_pair md sk m ctx coins sample) = response_aux_vector md sk m ctx coins.	signing_attempt_state_of_sample_pair_response_auxE $L_{response_aux}$ is an equality/rewriting fact.	PenseAuxE a bound that controls game-hop or final security
1923	signing_attempt_state_of_sample_pair_hintE lemma signing_attempt_state_of_sample_pair_hintE md sk m ctx coins sample : signing_attempt_state_hint (signing_attempt_state_of_sample_pair md sk m ctx coins sample) = hint_vector md sk m ctx coins.	signing_attempt_state_of_sample_pair_hintE L_{hint} is an equality/rewriting fact.	PenseHintE a bound that controls game-hop or final security

Line	Construct	What was written	Symbolic correspondence
1933	lemma signing_attempt_state_of_sample_pair_lowbits_carrierE signing_attempt_state_of_sample_pair_lowbits_carrierE md sk m ctx coins sample : signing_attempt_state_lowbits_carrier (signing_attempt_state_of_sample_pair md sk m ctx coins sample) = nth poly_zero (commitment_raw md sk m ctx coins) 0.	signing_attempt_state_of_sample_pair_lowbits_carrierE $L_{lowbits_carrier}$ is an equality/rewriting fact.	It is easy to find that contr game-hop or final security
1945	lemma signing_attempt_state_of_sample_pair_tokenE signing_attempt_state_of_sample_pair_tokenE md sk m ctx coins sample : signing_attempt_state_token (signing_attempt_state_of_sample_pair md sk m ctx coins sample) = signature_token md sk m ctx coins.	signing_attempt_state_of_sample_pair_tokenE L_{token} is an equality/rewriting fact.	It gives a bound that contr game-hop or final security
1957	lemma signing_attempt_state_of_sample_pair_programmed_lowbitsE signing_attempt_state_of_sample_pair_programmed_lowbitsE md sk m ctx coins sample : signing_attempt_state_programmed_lowbits (signing_attempt_state_of_sample_pair md sk m ctx coins sample) = commitment_lowbits md sk m ctx coins.	signing_attempt_state_of_sample_pair_programmed_lowbitsE $L_{programmed_lowbits}$ is an equality/rewriting fact.	It gives a bound that contr game-hop or final security
1967	lemma signing_attempt_state_of_sample_pair_lowbits_consistent signing_attempt_state_of_sample_pair_lowbits_consistent md sk m ctx coins sample : signing_attempt_state_lowbits_consistent (signing_attempt_state_of_sample_pair md sk m ctx coins sample).	signing_attempt_state_of_sample_pair_lowbits_consistent $\forall \vec{x}. B(\vec{x})$ is a universally quantified fact.	It is consistent: that contr game-hop or final security

	Line	Construct	What was written	Symbolic correspondence
		signing_attempt_state_of_sample_pair_programmed_challengeE lemma signing_attempt_state_of_sample_pair_programmed_challengeE md sk m ctx coins sample : signing_attempt_state_programmed_challenge md (public_key_of_secret md sk) m ctx (signing_attempt_state_of_sample_pair md sk m ctx coins sample) = commitment_challenge md sk m ctx coins.	signing_attempt_state_of_sample_pair_programmed_challengeE $L_{\text{programmed_challenge}}$ is an equality/rewriting fact.	Framed challenge game-hop or final security
	1979	lemma signing_attempt_state_of_sample_pair_challenge_consistent lemma signing_attempt_state_of_sample_pair_challenge_consistent md sk m ctx coins sample : signing_attempt_state_challenge_consistent md (public_key_of_secret md sk) m ctx (signing_attempt_state_of_sample_pair md sk m ctx coins sample).	signing_attempt_state_of_sample_pair_challenge_consistent $\forall \vec{z}. P(\vec{z})$ is a universally quantified fact.	Challenge consistent game-hop or final security
	1989	lemma signing_attempt_state_of_sample_pair_reconstructed_highbitsE lemma signing_attempt_state_of_sample_pair_reconstructed_highbitsE md sk m ctx coins sample : signing_attempt_state_reconstructed_highbits md (public_key_of_secret md sk) (signing_attempt_state_of_sample_pair md sk m ctx coins sample) = commitment_highbits md sk m ctx coins.	signing_attempt_state_of_sample_pair_reconstructed_highbitsE $L_{\text{reconstructed_highbits}}$ is an equality/rewriting fact.	Trusted highbits game-hop or final security
	2001	lemma signing_attempt_state_of_sample_pair_public_equation_consistent lemma signing_attempt_state_of_sample_pair_public_equation_consistent md sk m ctx coins sample : signing_attempt_state_public_equation_consistent md (public_key_of_secret md sk) (signing_attempt_state_of_sample_pair md sk m ctx coins sample).	signing_attempt_state_of_sample_pair_public_equation_consistent $\forall \vec{z}. P(\vec{z})$ is a universally quantified fact.	Public equation consistent game-hop or final security
	2012	lemma	lemma	lemma

	Line	Construct	What was written	Symbolic correspondence
signing_attempt_state_of_sample_pair_signature_of_fieldsE		lemma	signing_attempt_state_of_sample_pair_signature_of_fieldsE	Properties of fields that control game-hop or final security
		signing_attempt_state_of_sample_pair_signature_of_fieldsE	L is an equality/rewriting fact.	
		md sk m ctx coins sample :		
		signing_attempt_state_signature_of_fields		
		(public_key_of_secret md sk) m		
		ctx		
		(signing_attempt_state_of_sample_pair		
		md sk m ctx coins sample) =		
		sign_internal md sk m ctx coins.		
	2026	lemma	signing_attempt_state_of_sample_pair_fields_match_signature	Properties of fields that control game-hop or final security
signing_attempt_state_of_sample_pair_fields_match_signature		lemma	signing_attempt_state_of_sample_pair_fields_match_signature	Properties of fields that control game-hop or final security
		signing_attempt_state_of_sample_pair_fields_match_signature	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	
		md sk m ctx coins sample :		
		signing_attempt_state_fields_match_signature		
		(public_key_of_secret md sk) m		
		ctx		
		(signing_attempt_state_of_sample_pair		
		md sk m ctx coins sample).		
	2037	lemma	signing_attempt_state_of_sample_pair_field_accepts	Properties of fields that control game-hop or final security
signing_attempt_state_of_sample_pair_field_accepts		lemma	signing_attempt_state_of_sample_pair_field_accepts	Properties of fields that control game-hop or final security
		signing_attempt_state_of_sample_pair_field_accepts	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	
		md sk m ctx coins sample :		
		signing_attempt_state_field_accepts		
		md (public_key_of_secret md sk)		
		m ctx		
		(signing_attempt_state_of_sample_pair		
		md sk m ctx coins sample).		
	2052	lemma	signing_attempt_state_of_sample_pair_valid_signature_accepts	Properties of fields that control game-hop or final security
signing_attempt_state_of_sample_pair_valid_signature_accepts		lemma	signing_attempt_state_of_sample_pair_valid_signature_accepts	Properties of fields that control game-hop or final security
		signing_attempt_state_of_sample_pair_valid_signature_accepts	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	
		md sk m ctx coins sample :		
		signing_attempt_state_valid_signature_accepts		
		md		
		(signing_attempt_state_of_sample_pair		
		md sk m ctx coins sample).		
	2062	lemma	signing_attempt_state_of_sample_pair_accepts_current	Properties of fields that control game-hop or final security
signing_attempt_state_of_sample_pair_accepts_current		lemma	signing_attempt_state_of_sample_pair_accepts_current	Properties of fields that control game-hop or final security
		signing_attempt_state_of_sample_pair_accepts_current	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	
		md sk m ctx coins sample :		
		signing_attempt_state_accepts md		
		(signing_attempt_state_of_sample_pair		
		md sk m ctx coins sample).		

	Line	Construct	What was written	Symbolic correspondence
	2075	lemma		
signing_attempt_state_of_sample_pair_verification_accepts_current		lemma signing_attempt_state_of_sample_pair_verification_accepts_current md sk m ctx coins sample : signing_attempt_state_verification_accepts md (public_key_of_secret md sk) m ctx (signing_attempt_state_of_sample_pair md sk m ctx coins sample).	signing_attempt_state_of_sample_pair_verification_accepts_current $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	It is a <u>current</u> state that accepts the current game-hop or final security
	2086	lemma		
signing_attempt_state_of_sample_pair_reference_rejection_accepts		lemma signing_attempt_state_of_sample_pair_reference_rejection_accepts (md : mode) (sk : skey) (m : message) (ctx : context) (coins : random_coins) (sample : signing_sample_pair) : signing_attempt_state_reference_rejection_accepts md (signing_attempt_state_of_sample_pair md sk m ctx coins sample).	signing_attempt_state_of_sample_pair_reference_rejection_accepts $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	It is a <u>reference</u> state that rejects the current game-hop or final security
	2102	lemma		
signing_attempt_state_of_sample_pair_reference_rejection_no_abort		lemma signing_attempt_state_of_sample_pair_reference_rejection_no_abort (md : mode) (sk : skey) (m : message) (ctx : context) (coins : random_coins) (sample : signing_sample_pair) : ! signing_attempt_state_reference_rejection_aborts md (signing_attempt_state_of_sample_pair md sk m ctx coins sample).	signing_attempt_state_of_sample_pair_reference_rejection_no_abort $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	It is a <u>reference</u> state that <u>aborts</u> the current game-hop or final security
	2112	lemma		
signing_attempt_state_of_sample_pair_rejection_accepts_current		lemma signing_attempt_state_of_sample_pair_rejection_accepts_current md sk m ctx coins sample : signing_attempt_state_rejection_accepts md (public_key_of_secret md sk) m ctx (signing_attempt_state_of_sample_pair md sk m ctx coins sample).	signing_attempt_state_of_sample_pair_rejection_accepts_current $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	It is a <u>current</u> state that rejects the current game-hop or final security
	2123	lemma		

	Line	Construct	What was written	Symbolic correspondence
signing_attempt_state_of_sample_pair_rejection_no_abort_current		lemma	signing_attempt_state_of_sample_pair_rejection_no_abort_current	Proves no abort in current
		signing_attempt_state_of_sample_pair_rejection_no_abort_current	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	game-hop or final security
		md sk m ctx coins sample : !		
		signing_attempt_state_rejection_aborts		
		md (public_key_of_secret md sk)		
		m ctx		
		(signing_attempt_state_of_sample_pair		
		md sk m ctx coins sample).		
	2132	lemma		
dexact_hyperball_signing_attempt_state_reference_rejection_accepts		lemma	dexact_hyperball_signing_attempt_state_reference_rejection_accepts	Proves rejection accepts
		dexact_hyperball_signing_attempt_state_reference_rejection_accepts	$P \Rightarrow Q$ is an implication used as a proof	simulator transition.
		(md : mode) (sk : skey) (m : message) (ctx : context) (st : signing_attempt_state) : st \in dexact_hyperball_signing_attempt_state	obligation.	
		md sk m ctx =>		
		signing_attempt_state_reference_rejection_accepts		
		md st.		
	2144	lemma		
dexact_hyperball_signing_attempt_state_reference_rejection_no_abort		lemma	dexact_hyperball_signing_attempt_state_reference_rejection_no_abort	Proves rejection no abort
		dexact_hyperball_signing_attempt_state_reference_rejection_no_abort	$P \Rightarrow Q$ is an implication used as a proof	simulator transition.
		(md : mode) (sk : skey) (m : message) (ctx : context) (st : signing_attempt_state) : st \in dexact_hyperball_signing_attempt_state	obligation.	
		md sk m ctx => !		
		signing_attempt_state_reference_rejection_aborts		
		md st.		
	2156	lemma		
dexact_hyperball_signing_attempt_state_rejection_accepts		lemma	dexact_hyperball_signing_attempt_state_rejection_accepts	Proves rejection accepts
		dexact_hyperball_signing_attempt_state_rejection_accepts	$P \Rightarrow Q$ is an implication used as a proof	simulator transition.
		(md : mode) (sk : skey) (m : message) (ctx : context) (st : signing_attempt_state) : st \in dexact_hyperball_signing_attempt_state	obligation.	
		md sk m ctx =>		
		signing_attempt_state_rejection_accepts		
		md (public_key_of_secret md sk)		
		m ctx st.		
	2169	lemma		

	Line	Construct	What was written	Symbolic correspondence
dexact_hyperball_signing_attempt_state_rejection_no_abort		lemma	dexact_hyperball_signing_attempt_state_rejection_no_abort	Rejection no abort game equivalence
		dexact_hyperball_signing_attempt_state_rejection_no_abort	$P \Rightarrow Q$ is an implication used as a proof obligation.	simulator transition.
		(md : mode) (sk : skey) (m : message) (ctx : context) (st : signing_attempt_state) : st \in dexact_hyperball_signing_attempt_state		
		md sk m ctx => ! signing_attempt_state_rejection_aborts		
		md (public_key_of_secret md sk)		
		m ctx st.		
	2182	lemma	dexact_hyperball_signing_attempt_state_rejection_abort	Rejection abort game equivalence
dexact_hyperball_signing_attempt_state_rejection_abort_zero		lemma	dexact_hyperball_signing_attempt_state_rejection_abort	Rejection abort game equivalence
		dexact_hyperball_signing_attempt_state_rejection_abort	$L = R$ is an equality/rewriting fact.	simulator transition.
		md sk m ctx : mu		
		(dexact_hyperball_signing_attempt_state		
		md sk m ctx)		
		(signing_attempt_state_rejection_aborts		
		md (public_key_of_secret md sk)		
		m ctx) = 0%r.		

15 HAETAE_Algebra.ec

deterministic polynomial/vector/matrix algebra used by the model

	Line	Construct	What was written	Symbolic correspondence
	9	type		
	byte	type byte = int.	$\text{byte} \equiv \mathbb{Z}$	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.
	10	type		
	seed	type seed = byte list.	$\text{seed} \equiv \text{List}(\text{byte})$	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.
	11	type		
	crh	type crh = byte list.	$\text{crh} \equiv \text{List}(\text{byte})$	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.
	12	type		
	random_coins	type random_coins = byte list.	$\text{random_coins} \equiv \text{List}(\text{byte})$	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.
	13	type		
	xof256_state	type xof256_state = byte list.	$\text{xof256_state} \equiv \text{List}(\text{byte})$	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.
	14	type		
	message	type message = byte list.	$\text{message} \equiv \text{List}(\text{byte})$	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.
	15	type		
	context	type context = byte list.	$\text{context} \equiv \text{List}(\text{byte})$	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.
	16	type		
	coeff	type coeff = int.	$\text{coeff} \equiv \mathbb{Z}$	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.
	17	type		
	poly	type poly = coeff list.	$\text{poly} \equiv \text{List}(\text{coeff})$	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.
	18	type		

	Line	Construct	What was written	Symbolic correspondence
	challenge	type challenge = poly.	challenge $\equiv$ poly	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.
	19	type		
	polyveck	type polyveck = poly list.	polyveck $\equiv$ List(poly)	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.
	20	type		
	polyvecl	type polyvecl = poly list.	polyvecl $\equiv$ List(poly)	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.
	21	type		
	polyvecm	type polyvecm = poly list.	polyvecm $\equiv$ List(poly)	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.
	22	type		
	matrix	type matrix = poly list list.	matrix $\equiv$ List(List(poly))	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.
	23	type		
	hint_t	type hint_t = polyveck.	hint_t $\equiv$ polyveck	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.
	24	type		
	sig_token	type sig_token = polyvecl * polyveck * hint_t.	sig_token $\equiv$ polyvecl $\times$ polyveck $\times$ hint_t	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.
	26	type		
	pkey	type pkey = seed * polyveck.	pkey $\equiv$ seed $\times$ polyveck	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.
	27	type		
	encoded_pkey	type encoded_pkey = byte list.	encoded_pkey $\equiv$ List(byte)	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.
	28	type		
	skey	type skey = seed * int.	skey $\equiv$ seed $\times \mathbb{Z}$	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.
	29	type		

	Line	Construct	What was written	Symbolic correspondence	
		signature	type signature = polyveck * poly * crh * challenge * sig_token * int * int.	signature $\equiv$ polyveck $\times$ poly $\times$ crh $\times$ challenge $\times$ sig_token $\times \mathbb{Z} \times \mathbb{Z}$	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.
	31	operation			
	secret_key_of_seed	op secret_key_of_seed (_ : mode) (sd : seed) : skey = (sd, 0).	secret_key_of_seed : mode $\times$ seed $\rightarrow$ skey		Defines a total mathematical function by the EasyCrypt model.
	32	operation			
	poly_zero	op poly_zero : poly = nseq n 0.	poly_zero : 1 $\rightarrow$ poly		Defines a total mathematical function by the EasyCrypt model.
	33	operation			
	polyveck_zero	op polyveck_zero (md : mode) : polyveck = nseq (mode_k md) poly_zero.	polyveck_zero : mode $\rightarrow$ polyveck		Defines a total mathematical function by the EasyCrypt model.
	34	operation			
	polyvecl_zero	op polyvecl_zero (md : mode) : polyvecl = nseq (mode_l md) poly_zero.	polyvecl_zero : mode $\rightarrow$ polyvecl		Defines a total mathematical function by the EasyCrypt model.
	35	operation			
	polyvecm_zero	op polyvecm_zero (md : mode) : polyvecm = nseq (mode_m md) poly_zero.	polyvecm_zero : mode $\rightarrow$ polyvecm		Defines a total mathematical function by the EasyCrypt model.
	36	operation			
	matrix_zero	op matrix_zero (md : mode) : matrix = nseq (mode_k md) (polyvecl_zero md).	matrix_zero : mode $\rightarrow$ matrix		Defines a total mathematical function by the EasyCrypt model.
	39	lemma			
	poly_zero_size	lemma poly_zero_size : size poly_zero = n.	poly_zero_size : $L = R$ is an equality/rewriting fact.		Records a reusable theorem so later p scripts can rewrite, apply, or compose
	43	lemma			
	polyveck_zero_size	lemma polyveck_zero_size md : size (polyveck_zero md) = mode_k md.	polyveck_zero_size : $L = R$ is an equality/rewriting fact.		Records a reusable theorem so later p scripts can rewrite, apply, or compose
	47	lemma			
	polyvecl_zero_size	lemma polyvecl_zero_size md : size (polyvecl_zero md) = mode_l md.	polyvecl_zero_size : $L = R$ is an equality/rewriting fact.		Records a reusable theorem so later p scripts can rewrite, apply, or compose
	51	lemma			
	polyvecm_zero_size	lemma polyvecm_zero_size md : size (polyvecm_zero md) = mode_m md.	polyvecm_zero_size : $L = R$ is an equality/rewriting fact.		Records a reusable theorem so later p scripts can rewrite, apply, or compose

Line	Construct	What was written	Symbolic correspondence
55	operation		
poly_wf	op poly_wf (p : poly) : bool = size p = n.	poly_wf $\subseteq$ poly is a Boolean-valued predicate.	Names a well-formedness or support condition so it can be reused in lemm
56	operation		
polyveck_wf	op polyveck_wf (md : mode) (xs : polyveck) : bool = size xs = mode_k md /\ all poly_wf xs.	polyveck_wf $\subseteq$ mode $\times$ polyveck is a Boolean-valued predicate.	Names a well-formedness or support condition so it can be reused in lemm
58	operation		
polyvecl_wf	op polyvecl_wf (md : mode) (xs : polyvecl) : bool = size xs = mode_l md /\ all poly_wf xs.	polyvecl_wf $\subseteq$ mode $\times$ polyvecl is a Boolean-valued predicate.	Names a well-formedness or support condition so it can be reused in lemm
61	lemma		
polyveck_wf_nth	lemma polyveck_wf_nth md b row : polyveck_wf md b => 0 <= row < mode_k md => poly_wf (nth poly_zero b row).	polyveck_wf_nth : $X \leq Y$ is an arithmetic or probability upper bound.	Proves a structural correctness fact u justify later reductions.
72	operation		
crh_wf	op crh_wf (mu : crh) : bool = size mu = crhbytes.	crh_wf $\subseteq$ crh is a Boolean-valued predicate.	Names a well-formedness or support condition so it can be reused in lemm
73	operation		
challenge_coeff_ok	op challenge_coeff_ok (x : coeff) : bool = x = -1 \/ x = 0 \/ x = 1.	challenge_coeff_ok $\subseteq$ coeff is a Boolean-valued predicate.	Names a Boolean mathematical cond used in statements and invariants.
74	operation		
challenge_wf	op challenge_wf (ch : challenge) : bool = poly_wf ch /\ all challenge_coeff_ok ch.	challenge_wf $\subseteq$ challenge is a Boolean-valued predicate.	Names a well-formedness or support condition so it can be reused in lemm
76	operation		
unit_coeff_ok	op unit_coeff_ok (x : coeff) : bool = x = -1 \/ x = 1.	unit_coeff_ok $\subseteq$ coeff is a Boolean-valued predicate.	Names a Boolean mathematical cond used in statements and invariants.
77	operation		
challenge_sparse_at	op challenge_sparse_at (md : mode) (i : int) (x : coeff) : bool = if i < mode_tau md then unit_coeff_ok x else x = 0.	challenge_sparse_at $\subseteq$ mode $\times \mathbb{Z} \times$ coeff is a Boolean-valued predicate.	Names a Boolean mathematical cond used in statements and invariants.
79	operation		
challenge_sparse	op challenge_sparse (md : mode) (ch : challenge) : bool = size ch = n /\ forall (i : int), 0 <= i /\ i < n => challenge_sparse_at md i (nth 0 ch i).	challenge_sparse $\subseteq$ mode $\times$ challenge $\times \mathbb{Z}$ is a Boolean-valued predicate.	Names a Boolean mathematical cond used in statements and invariants.

Line	Construct	What was written	Symbolic correspondence
83	operation		
poly_unit	op poly_unit (p : poly) : bool = poly_wf p /\ all unit_coeff_ok p.	poly_unit $\subseteq$ poly is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.
84	operation		
polyveck_unit	op polyveck_unit (md : mode) (xs : polyveck) : bool = polyveck_wf md xs /\ all poly_unit xs.	polyveck_unit $\subseteq$ mode $\times$ polyveck is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.
86	operation		
polyvecl_unit	op polyvecl_unit (md : mode) (xs : polyvecl) : bool = polyvecl_wf md xs /\ all poly_unit xs.	polyvecl_unit $\subseteq$ mode $\times$ polyvecl is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.
89	lemma		
poly_zero_wf	lemma poly_zero_wf : poly_wf poly_zero.	poly_zero_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact to justify later reductions.
92	lemma		
polyveck_zero_wf	lemma polyveck_zero_wf md : polyveck_wf md (polyveck_zero md).	polyveck_zero_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact to justify later reductions.
100	lemma		
polyvecl_zero_wf	lemma polyvecl_zero_wf md : polyvecl_wf md (polyvecl_zero md).	polyvecl_zero_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact to justify later reductions.
108	operation		
coeff_mod	op coeff_mod (x : coeff) : coeff = x %% q.	coeff_mod : coeff $\rightarrow$ coeff	Defines a total mathematical function by the EasyCrypt model.
109	operation		
coeff_add	op coeff_add (x y : coeff) : coeff = coeff_mod (x + y).	coeff_add : coeff $\times$ coeff $\rightarrow$ coeff	Defines a total mathematical function by the EasyCrypt model.
110	operation		
coeff_neg	op coeff_neg (x : coeff) : coeff = coeff_mod (-x).	coeff_neg : coeff $\rightarrow$ coeff	Defines a total mathematical function by the EasyCrypt model.
111	operation		
coeff_sub	op coeff_sub (x y : coeff) : coeff = coeff_add x (coeff_neg y).	coeff_sub : coeff $\times$ coeff $\rightarrow$ coeff	Defines a total mathematical function by the EasyCrypt model.
112	operation		
coeff_mul	op coeff_mul (x y : coeff) : coeff = coeff_mod (x * y).	coeff_mul : coeff $\times$ coeff $\rightarrow$ coeff	Defines a total mathematical function by the EasyCrypt model.

Line	Construct	What was written	Symbolic correspondence
113	operation		
coeff_double	op coeff_double (x : coeff) : coeff = coeff_add x x.	coeff__double : coeff → coeff	Defines a total mathematical function by the EasyCrypt model.
115	operation		
poly_coeff	op poly_coeff (p : poly) (i : int) : coeff = nth 0 p i.	poly__coeff : poly × ℤ → coeff	Defines a total mathematical function by the EasyCrypt model.
116	operation		
poly_normalize	op poly_normalize (p : poly) : poly = mkseq (fun i => coeff_mod (poly_coeff p i)) n.	poly__normalize : poly → poly	Defines a total mathematical function by the EasyCrypt model.
118	operation		
poly_add	op poly_add (p r : poly) : poly = if p = poly_zero /\ r = poly_zero then poly_zero else mkseq (fun i => coeff_add (poly_coeff p i) (poly_coeff r i)) n.	poly__add : poly × poly → poly	Defines a total mathematical function by the EasyCrypt model.
122	operation		
poly_neg	op poly_neg (p : poly) : poly = if p = poly_zero then poly_zero else mkseq (fun i => coeff_neg (poly_coeff p i)) n.	poly__neg : poly → poly	Defines a total mathematical function by the EasyCrypt model.
125	operation		
poly_sub	op poly_sub (p r : poly) : poly = poly_add p (poly_neg r).	poly__sub : poly × poly → poly	Defines a total mathematical function by the EasyCrypt model.
126	operation		
poly_double	op poly_double (p : poly) : poly = if p = poly_zero then poly_zero else mkseq (fun i => coeff_double (poly_coeff p i)) n.	poly__double : poly → poly	Defines a total mathematical function by the EasyCrypt model.
130	operation		
poly_mul_term	op poly_mul_term (p r : poly) (i j : int) : coeff = if j <= i then poly_coeff p j * poly_coeff r (i - j) else - (poly_coeff p j * poly_coeff r (n + i - j)).	poly__mul__term : poly × poly × ℤ × ℤ → coeff	Defines a total mathematical function by the EasyCrypt model.
133	operation		

	Line	Construct	What was written	Symbolic correspondence
		<pre> poly_mul op poly_mul (p r : poly) : poly = if p = poly_zero \ / r = poly_zero then poly_zero else mkseq (fun i => coeff_mod (sumz (map (fun j => poly_mul_term p r i j) (range 0 n)))) n. </pre>	$\text{poly_mul} : \text{poly} \times \text{poly} \rightarrow \text{poly}$	Defines a total mathematical function by the EasyCrypt model.
	140	operation		
		<pre> poly_dot op poly_dot (row : poly list) (v : poly list) : poly = if row = [] \ / v = [] then poly_zero else foldl (fun acc (xy : poly * poly) => poly_add acc (poly_mul xy.'1 xy.'2)) poly_zero (zip row v). </pre>	$\text{poly_dot} : \text{List}(\text{poly}) \times \text{List}(\text{poly}) \times \text{poly} \times \text{poly} \rightarrow \text{poly}$	Defines a total mathematical function by the EasyCrypt model.
	147	operation		
		<pre> matrix_vec_mul op matrix_vec_mul (md : mode) (a : matrix) (v : polyveck) : polyveck = if a = [] \ / a = matrix_zero md then polyveck_zero md else mkseq (fun i => poly_dot (nth [] a i) v) (mode_k md). </pre>	$\text{matrix_vec_mul} : \text{mode} \times \text{matrix} \times \text{polyveck} \rightarrow \text{polyveck}$	Defines a total mathematical function by the EasyCrypt model.
	151	operation		
		<pre> polyveck_add op polyveck_add (md : mode) (xs ys : polyveck) : polyveck = if xs = polyveck_zero md /\ ys = polyveck_zero md then polyveck_zero md else mkseq (fun i => poly_add (nth poly_zero xs i) (nth poly_zero ys i)) (mode_k md). </pre>	$\text{polyveck_add} : \text{mode} \times \text{polyveck} \times \text{polyveck} \rightarrow \text{polyveck}$	Defines a total mathematical function by the EasyCrypt model.
	156	operation		
		<pre> polyveck_neg op polyveck_neg (md : mode) (xs : polyveck) : polyveck = if xs = polyveck_zero md then polyveck_zero md else mkseq (fun i => poly_neg (nth poly_zero xs i)) (mode_k md). </pre>	$\text{polyveck_neg} : \text{mode} \times \text{polyveck} \rightarrow \text{polyveck}$	Defines a total mathematical function by the EasyCrypt model.
	159	operation		

Line	Construct	What was written	Symbolic correspondence
	polyveck_sub op polyveck_sub (md : mode) (xs ys : polyveck) : polyveck = polyveck_add md xs (polyveck_neg md ys).	polyveck_sub : mode $\times$ polyveck $\times$ polyveck $\rightarrow$ polyveck	Defines a total mathematical function by the EasyCrypt model.
161	operation		
	polyveck_double op polyveck_double (md : mode) (xs : polyveck) : polyveck = if xs = polyveck_zero md then polyveck_zero md else mkseq (fun i => poly_double (nth poly_zero xs i)) (mode_k md).	polyveck_double : mode $\times$ polyveck $\rightarrow$ polyveck	Defines a total mathematical function by the EasyCrypt model.
165	operation		
	polyvecl_sub op polyvecl_sub (md : mode) (xs ys : polyvecl) : polyvecl = mkseq (fun i => poly_sub (nth poly_zero xs i) (nth poly_zero ys i)) (mode_l md).	polyvecl_sub : mode $\times$ polyvecl $\times$ polyvecl $\rightarrow$ polyvecl	Defines a total mathematical function by the EasyCrypt model.
170	lemma		
	polyvecl_sub_size lemma polyvecl_sub_size md xs ys : size (polyvecl_sub md xs ys) = mode_l md.	polyvecl_sub_size : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
174	lemma		
	poly_add_wf lemma poly_add_wf p r : poly_wf (poly_add p r).	poly_add_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
182	lemma		
	poly_normalize_wf lemma poly_normalize_wf p : poly_wf (poly_normalize p).	poly_normalize_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
185	lemma		
	poly_neg_wf lemma poly_neg_wf p : poly_wf (poly_neg p).	poly_neg_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
193	lemma		
	poly_sub_wf lemma poly_sub_wf p r : poly_wf (poly_sub p r).	poly_sub_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
196	lemma		
	poly_double_wf lemma poly_double_wf p : poly_wf (poly_double p).	poly_double_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.
204	lemma		
	poly_mul_wf lemma poly_mul_wf p r : poly_wf (poly_mul p r).	poly_mul_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a distribution fact needed for sampling or probability mass reasoning
212	lemma		
	poly_dot_wf lemma poly_dot_wf row v : poly_wf (poly_dot row v).	poly_dot_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.

	Line	Construct	What was written	Symbolic correspondence
	231	lemma		
matrix_vec_mul_wf		lemma matrix_vec_mul_wf md a v : polyveck_wf md (matrix_vec_mul md a v).	matrix_vec_mul_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a distribution fact needed for sampling or probability mass reasoning.
	243	lemma		
polyveck_add_wf		lemma polyveck_add_wf md xs ys : polyveck_wf md (polyveck_add md xs ys).	polyveck_add_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	255	lemma		
polyveck_neg_wf		lemma polyveck_neg_wf md xs : polyveck_wf md (polyveck_neg md xs).	polyveck_neg_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	267	lemma		
polyveck_sub_wf		lemma polyveck_sub_wf md xs ys : polyveck_wf md (polyveck_sub md xs ys).	polyveck_sub_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	270	lemma		
polyveck_double_wf		lemma polyveck_double_wf md xs : polyveck_wf md (polyveck_double md xs).	polyveck_double_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.
	282	lemma		
polyvecl_sub_wf		lemma polyvecl_sub_wf md xs ys : polyvecl_wf md (polyvecl_sub md xs ys).	polyvecl_sub_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	291	operation		
coeff_q_bound		op coeff_q_bound (x : coeff) : bool = 0 <= x < q.	coeff_q_bound $\subseteq$ coeff is a Boolean-valued predicate.	Names a Boolean mathematical cond used in statements and invariants.
	292	operation		
poly_coeffs_q_bound		op poly_coeffs_q_bound (p : poly) : bool = forall i, 0 <= i < n => coeff_q_bound (poly_coeff p i).	poly_coeffs_q_bound $\subseteq$ poly is a Boolean-valued predicate.	Names a Boolean mathematical cond used in statements and invariants.
	294	operation		
polyveck_coeffs_q_bound		op polyveck_coeffs_q_bound (md : mode) (xs : polyveck) : bool = forall row, 0 <= row < mode_k md => poly_coeffs_q_bound (nth poly_zero xs row).	polyveck_coeffs_q_bound $\subseteq$ mode $\times$ polyveck is a Boolean-valued predicate.	Names a Boolean mathematical cond used in statements and invariants.
	298	lemma		
coeff_mod_q_bound		lemma coeff_mod_q_bound x : coeff_q_bound (coeff_mod x).	coeff_mod_q_bound : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.

	Line	Construct	What was written	Symbolic correspondence
	301	lemma		
	poly_zero_coeffs_q_bound	lemma poly_zero_coeffs_q_bound : poly_coeffs_q_bound poly_zero.	poly_zero_coeffs_q_bound : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.
	307	lemma		
	poly_add_coeffs_q_bound	lemma poly_add_coeffs_q_bound p r : poly_coeffs_q_bound (poly_add p r).	poly_add_coeffs_q_bound : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.
	317	lemma		
	poly_neg_coeffs_q_bound	lemma poly_neg_coeffs_q_bound p : poly_coeffs_q_bound (poly_neg p).	poly_neg_coeffs_q_bound : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.
	327	lemma		
	poly_sub_coeffs_q_bound	lemma poly_sub_coeffs_q_bound p r : poly_coeffs_q_bound (poly_sub p r).	poly_sub_coeffs_q_bound : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.
	331	lemma		
	poly_double_coeffs_q_bound	lemma poly_double_coeffs_q_bound p : poly_coeffs_q_bound (poly_double p).	poly_double_coeffs_q_bound : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.
	341	lemma		
	poly_mul_coeffs_q_bound	lemma poly_mul_coeffs_q_bound p r : poly_coeffs_q_bound (poly_mul p r).	poly_mul_coeffs_q_bound : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a distribution fact needed for sampling or probability mass reasoning.
	351	lemma		
	poly_dot_coeffs_q_bound	lemma poly_dot_coeffs_q_bound row v : poly_coeffs_q_bound (poly_dot row v).	poly_dot_coeffs_q_bound : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.
	371	lemma		
	matrix_vec_mul_coeffs_q_bound	lemma matrix_vec_mul_coeffs_q_bound md a v : polyveck_coeffs_q_bound md (matrix_vec_mul md a v).	matrix_vec_mul_coeffs_q_bound : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a distribution fact needed for sampling or probability mass reasoning.
	384	lemma		
	polyveck_zero_coeffs_q_bound	lemma polyveck_zero_coeffs_q_bound md : polyveck_coeffs_q_bound md (polyveck_zero md).	polyveck_zero_coeffs_q_bound : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.
	393	lemma		
	polyveck_add_coeffs_q_bound	lemma polyveck_add_coeffs_q_bound md xs ys : polyveck_coeffs_q_bound md (polyveck_add md xs ys).	polyveck_add_coeffs_q_bound : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.

	Line	Construct	What was written	Symbolic correspondence
	403	lemma		
		polyveck_neg_coeffs_q_bound	lemma polyveck_neg_coeffs_q_bound md xs : polyveck_coeffs_q_bound md (polyveck_neg md xs).	polyveck_neg_coeffs_q_bound : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.
	413	lemma		
		polyveck_sub_coeffs_q_bound	lemma polyveck_sub_coeffs_q_bound md xs ys : polyveck_coeffs_q_bound md (polyveck_sub md xs ys).	polyveck_sub_coeffs_q_bound : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.
	417	lemma		
		polyveck_double_coeffs_q_bound	lemma polyveck_double_coeffs_q_bound md xs : polyveck_coeffs_q_bound md (polyveck_double md xs).	polyveck_double_coeffs_q_bound : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.
	427	operation		
		z1_alpha	op z1_alpha : int = 256.	z1_alpha : $1 \rightarrow \mathbb{Z}$
	429	operation		
		coeff_decompose_z1_low	op coeff_decompose_z1_low (x : coeff) : coeff = let lb = x %% z1_alpha in if z1_alpha %% 2 <= lb then lb - z1_alpha else lb.	coeff_decompose_z1_low : coeff $\rightarrow$ coeff
	432	operation		
		coeff_decompose_z1_high	op coeff_decompose_z1_high (x : coeff) : coeff = (x + (z1_alpha %% 2)) %% z1_alpha.	coeff_decompose_z1_high : coeff $\rightarrow$ coeff
	434	operation		
		coeff_lsb	op coeff_lsb (x : coeff) : coeff = x %% 2.	coeff_lsb : coeff $\rightarrow$ coeff
	435	operation		
		coeff_decompose_vk_low	op coeff_decompose_vk_low (x : coeff) : coeff = if x %% 2 = 0 then 0 else if (x %% 2) %% 2 = 0 then 1 else -1.	coeff_decompose_vk_low : coeff $\rightarrow$ coeff
	439	operation		
		coeff_decompose_vk_high	op coeff_decompose_vk_high (x : coeff) : coeff = (x - coeff_decompose_vk_low x) %% 2.	coeff_decompose_vk_high : coeff $\rightarrow$ coeff
	441	operation		

Line	Construct	What was written	Symbolic correspondence
	<pre> coeff_decompose_hint_high op coeff_decompose_hint_high (md : mode) (x : coeff) : coeff = let alpha = mode_alpha_hint md in let hb = (x + (alpha %/ 2)) %/ alpha in let top = (dq - 2) %/ alpha in if top <= hb then hb - top else hb. </pre>	<pre> coeff__decompose_hint_high : mode × coeff → coeff </pre>	Defines a total mathematical function by the EasyCrypt model.
446	operation		
	<pre> coeff_compose_z1 op coeff_compose_z1 (hi lo : coeff) : coeff = hi * z1_alpha + lo. </pre>	<pre> coeff__compose_z1 : coeff × coeff → coeff </pre>	Defines a total mathematical function by the EasyCrypt model.
448	operation		
	<pre> poly_highbits op poly_highbits (p : poly) : poly = mkseq (fun i => coeff_decompose_z1_high (poly_coeff p i)) n. </pre>	<pre> poly__highbits : poly → poly </pre>	Defines a total mathematical function by the EasyCrypt model.
450	operation		
	<pre> poly_lowbits op poly_lowbits (p : poly) : poly = mkseq (fun i => coeff_decompose_z1_low (poly_coeff p i)) n. </pre>	<pre> poly__lowbits : poly → poly </pre>	Defines a total mathematical function by the EasyCrypt model.
452	operation		
	<pre> poly_lsb op poly_lsb (p : poly) : poly = mkseq (fun i => coeff_lsb (poly_coeff p i)) n. </pre>	<pre> poly__lsb : poly → poly </pre>	Defines a total mathematical function by the EasyCrypt model.
454	operation		
	<pre> poly_vk_lowbits op poly_vk_lowbits (p : poly) : poly = mkseq (fun i => coeff_decompose_vk_low (poly_coeff p i)) n. </pre>	<pre> poly__vk_lowbits : poly → poly </pre>	Defines a total mathematical function by the EasyCrypt model.
456	operation		
	<pre> poly_vk_highbits op poly_vk_highbits (p : poly) : poly = mkseq (fun i => coeff_decompose_vk_high (poly_coeff p i)) n. </pre>	<pre> poly__vk_highbits : poly → poly </pre>	Defines a total mathematical function by the EasyCrypt model.
458	operation		
	<pre> poly_compose_z1 op poly_compose_z1 (hi lo : poly) : poly = mkseq (fun i => coeff_compose_z1 (poly_coeff hi i) (poly_coeff lo i)) n. </pre>	<pre> poly__compose_z1 : poly × poly → poly </pre>	Defines a total mathematical function by the EasyCrypt model.
462	operation		

Line	Construct	What was written	Symbolic correspondence
	poly_hint_highbits <pre> op poly_hint_highbits (md : mode) (p : poly) : poly = mkseq (fun i => coeff_decompose_hint_high md (poly_coeff p i)) n.</pre>	poly_hint_highbits : mode $\times$ poly $\rightarrow$ poly	Defines a total mathematical function by the EasyCrypt model.
465	operation		
	polyvecl_highbits <pre> op polyvecl_highbits (md : mode) (xs : polyvecl) : polyvecl = mkseq (fun i => poly_highbits (nth poly_zero xs i)) (mode_l md).</pre>	polyvecl_highbits : mode $\times$ polyvecl $\rightarrow$ polyvecl	Defines a total mathematical function by the EasyCrypt model.
467	operation		
	polyvecl_lowbits <pre> op polyvecl_lowbits (md : mode) (xs : polyvecl) : polyvecl = mkseq (fun i => poly_lowbits (nth poly_zero xs i)) (mode_l md).</pre>	polyvecl_lowbits : mode $\times$ polyvecl $\rightarrow$ polyvecl	Defines a total mathematical function by the EasyCrypt model.
469	operation		
	polyveck_highbits_hint <pre> op polyveck_highbits_hint (md : mode) (xs : polyveck) : polyveck = mkseq (fun i => poly_hint_highbits md (nth poly_zero xs i)) (mode_k md).</pre>	polyveck_highbits_hint : mode $\times$ polyveck $\rightarrow$ polyveck	Defines a total mathematical function by the EasyCrypt model.
471	operation		
	polyveck_vk_lowbits <pre> op polyveck_vk_lowbits (md : mode) (xs : polyveck) : polyveck = mkseq (fun i => poly_vk_lowbits (nth poly_zero xs i)) (mode_k md).</pre>	polyveck_vk_lowbits : mode $\times$ polyveck $\rightarrow$ polyveck	Defines a total mathematical function by the EasyCrypt model.
473	operation		
	polyveck_vk_highbits <pre> op polyveck_vk_highbits (md : mode) (xs : polyveck) : polyveck = mkseq (fun i => poly_vk_highbits (nth poly_zero xs i)) (mode_k md).</pre>	polyveck_vk_highbits : mode $\times$ polyveck $\rightarrow$ polyveck	Defines a total mathematical function by the EasyCrypt model.
475	operation		
	polyveck_mul_alpha <pre> op polyveck_mul_alpha (md : mode) (xs : polyveck) : polyveck = mkseq (fun i => mkseq (fun j => mode_alpha_hint md * poly_coeff (nth poly_zero xs i) j) n) (mode_k md).</pre>	polyveck_mul_alpha : mode $\times$ polyveck $\rightarrow$ polyveck	Defines a total mathematical function by the EasyCrypt model.

Line	Construct	What was written	Symbolic correspondence
483	lemma		
poly_highbits_wf	lemma poly_highbits_wf p : poly_wf (poly_highbits p).	poly_highbits_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
486	lemma		
poly_lowbits_wf	lemma poly_lowbits_wf p : poly_wf (poly_lowbits p).	poly_lowbits_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
489	lemma		
poly_lsb_wf	lemma poly_lsb_wf p : poly_wf (poly_lsb p).	poly_lsb_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
492	lemma		
poly_vk_lowbits_wf	lemma poly_vk_lowbits_wf p : poly_wf (poly_vk_lowbits p).	poly_vk_lowbits_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
495	lemma		
poly_vk_highbits_wf	lemma poly_vk_highbits_wf p : poly_wf (poly_vk_highbits p).	poly_vk_highbits_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
498	lemma		
poly_compose_z1_wf	lemma poly_compose_z1_wf hi lo : poly_wf (poly_compose_z1 hi lo).	poly_compose_z1_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
501	lemma		
poly_hint_highbits_wf	lemma poly_hint_highbits_wf md p : poly_wf (poly_hint_highbits md p).	poly_hint_highbits_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
504	lemma		
polyvecl_highbits_wf	lemma polyvecl_highbits_wf md xs : polyvecl_wf md (polyvecl_highbits md xs).	polyvecl_highbits_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
514	lemma		
polyvecl_lowbits_wf	lemma polyvecl_lowbits_wf md xs : polyvecl_wf md (polyvecl_lowbits md xs).	polyvecl_lowbits_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
524	lemma		
polyveck_highbits_hint_wf	lemma polyveck_highbits_hint_wf md xs : polyveck_wf md (polyveck_highbits_hint md xs).	polyveck_highbits_hint_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
534	lemma		
polyveck_vk_lowbits_wf	lemma polyveck_vk_lowbits_wf md xs : polyveck_wf md (polyveck_vk_lowbits md xs).	polyveck_vk_lowbits_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
544	lemma		
polyveck_vk_highbits_wf	lemma polyveck_vk_highbits_wf md xs : polyveck_wf md (polyveck_vk_highbits md xs).	polyveck_vk_highbits_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.

	Line	Construct	What was written	Symbolic correspondence
	554	lemma		
polyveck_mul_alpha_wf		lemma polyveck_mul_alpha_wf md xs : polyveck_wf md (polyveck_mul_alpha md xs).	polyveck_mul_alpha_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a distribution fact needed for sampling or probability mass reasoning
	564	operation		
public_key_seed_poly		op public_key_seed_poly (tag : int) (src : byte list) : poly = mkseq (fun i => coeff_mod (tag + i + nth 0 src i)) n.	public_key_seed_poly : $\mathbb{Z} \times \text{List}(\text{byte}) \rightarrow$ poly	Defines a total mathematical function by the EasyCrypt model.
	566	operation		
public_key_seed_polyvecl		op public_key_seed_polyvecl (md : mode) (tag : int) (src : byte list) : polyvecl = mkseq (fun i => public_key_seed_poly (tag + i) (i :: src)) (mode_l md).	public_key_seed_polyvecl : $\text{mode} \times \mathbb{Z} \times \text{List}(\text{byte}) \times : \text{src} \rightarrow \text{polyvecl}$	Defines a total mathematical function by the EasyCrypt model.
	571	operation		
public_key_seed_polyveck		op public_key_seed_polyveck (md : mode) (tag : int) (src : byte list) : polyveck = mkseq (fun i => public_key_seed_poly (tag + i) (i :: src)) (mode_k md).	public_key_seed_polyveck : $\text{mode} \times \mathbb{Z} \times \text{List}(\text{byte}) \times : \text{src} \rightarrow \text{polyveck}$	Defines a total mathematical function by the EasyCrypt model.
	576	operation		
haetae_keygen_seedbuf_bytes		op haetae_keygen_seedbuf_bytes : int = 2 * seedbytes + crhbytes.	haetae_keygen_seedbuf_bytes : $1 \rightarrow \mathbb{Z}$	Defines a total mathematical function by the EasyCrypt model.
	577	operation		
haetae_reference_keygen_seedbuf_bytes		op haetae_reference_keygen_seedbuf_bytes : int = 2 * seedbytes + crhbytes.	haetae_reference_keygen_seedbuf_bytes : $1 \rightarrow \mathbb{Z}$	Defines a total mathematical function by the EasyCrypt model.
	579	operation		
haetae_reference_keygen_xof_absorb_bytes		op haetae_reference_keygen_xof_absorb_bytes : int = seedbytes.	haetae_reference_keygen_xof_absorb_bytes : $1 \rightarrow \mathbb{Z}$	Defines a total mathematical function by the EasyCrypt model.
	580	operation		
haetae_reference_keygen_xof_squeeze_bytes		op haetae_reference_keygen_xof_squeeze_bytes : int = haetae_reference_keygen_seedbuf_bytes.	haetae_reference_keygen_xof_squeeze_bytes : $1 \rightarrow \mathbb{Z}$	Defines a total mathematical function by the EasyCrypt model.
	582	operation		

	Line	Construct	What was written	Symbolic correspondence
		haetae_keygen_rho_prime_offset : int = 0.	haetae_keygen_rho_prime_offset : $\mathbf{1} \rightarrow \mathbb{Z}$	Defines a total mathematical function by the EasyCrypt model.
	583	operation		
		haetae_keygen_sigma_offset : int = seedbytes.	haetae_keygen_sigma_offset : $\mathbf{1} \rightarrow \mathbb{Z}$	Defines a total mathematical function by the EasyCrypt model.
	584	operation		
		haetae_keygen_key_offset : int = seedbytes + crhbytes.	haetae_keygen_key_offset : $\mathbf{1} \rightarrow \mathbb{Z}$	Defines a total mathematical function by the EasyCrypt model.
	585	operation		
		haetae_shake256_rate_bytes : int = shake256_rate_bytes.	haetae_shake256_rate_bytes : $\mathbf{1} \rightarrow \mathbb{Z}$	Defines a total mathematical function by the EasyCrypt model.
	586	operation		
		haetae_shake256_domain_separator : byte = shake256_domain_separator.	haetae_shake256_domain_separator : $\mathbf{1} \rightarrow \text{byte}$	Defines a total mathematical function by the EasyCrypt model.
	587	operation		
		haetae_shake256_bytes (input : byte list) (outlen : int) : byte list = shake256 input (size input) outlen.	haetae_shake256_bytes : $\text{List}(\text{byte}) \times \mathbb{Z} \rightarrow \text{List}(\text{byte})$	Defines a total mathematical function by the EasyCrypt model.
	589	operation		
		haetae_seed_slice (src : byte list) (offset len : int) : byte list = mkseq (fun i => nth 0 src (offset + i)) len.	haetae_seed_slice : $\text{List}(\text{byte}) \times \mathbb{Z} \times \mathbb{Z} \rightarrow \text{List}(\text{byte})$	Defines a total mathematical function by the EasyCrypt model.
	591	operation		
		haetae_xof256_absorb_input (input : byte list) (inlen : int) : byte list = haetae_seed_slice input 0 inlen.	haetae_xof256_absorb_input : $\text{List}(\text{byte}) \times \mathbb{Z} \rightarrow \text{List}(\text{byte})$	Defines a total mathematical function by the EasyCrypt model.
	594	operation		
		haetae_xof256_absorb_once (input : byte list) (inlen : int) : xof256_state = shake256_absorb_once (haetae_xof256_absorb_input input inlen) inlen.	haetae_xof256_absorb_once : $\text{List}(\text{byte}) \times \mathbb{Z} \rightarrow \text{xof256_state}$	Defines a total mathematical function by the EasyCrypt model.
	597	operation		

	Line	Construct	What was written	Symbolic correspondence
		<pre> haetae_xof256_squeeze op haetae_xof256_squeeze (st : xof256_state) (outlen : int) : byte list = shake256_squeeze st outlen. </pre>	<pre> haetae_xof256_squeeze : xof256_state $\times \mathbb{Z} \rightarrow \text{List}(\text{byte})$ </pre>	Defines a total mathematical function by the EasyCrypt model.
	600	operation		
		<pre> haetae_xof256_absorb_once_squeeze op haetae_xof256_absorb_once_squeeze (input : byte list) (inlen outlen : int) : byte list = haetae_xof256_squeeze (haetae_xof256_absorb_once input inlen) outlen. </pre>	<pre> haetae_xof256_absorb_once_squeeze : List(byte) $\times \mathbb{Z} \times \mathbb{Z} \rightarrow \text{List}(\text{byte})$ </pre>	Defines a total mathematical function by the EasyCrypt model.
	605	operation		
		<pre> haetae_reference_keygen_seedbuf_after_memcpy op haetae_reference_keygen_seedbuf_after_memcpy (sd : seed) : byte list = mkseq (fun i => if i < seedbytes then nth 0 sd i else 0) haetae_reference_keygen_seedbuf_bytes. </pre>	<pre> haetae_reference_keygen_seedbuf_after_memcpy : seed $\rightarrow \text{List}(\text{byte})$ </pre>	Defines a total mathematical function by the EasyCrypt model.
	609	operation		
		<pre> haetae_reference_keygen_xof_input op haetae_reference_keygen_xof_input (sd : seed) : byte list = haetae_seed_slice (haetae_reference_keygen_seedbuf_after_memcpy sd) 0 haetae_reference_keygen_xof_absorb_bytes. </pre>	<pre> haetae_reference_keygen_xof_input : seed $\rightarrow \text{List}(\text{byte})$ </pre>	Defines a total mathematical function by the EasyCrypt model.
	613	operation		
		<pre> haetae_keygen_xof_seedbuf op haetae_keygen_xof_seedbuf (sd : seed) : byte list = haetae_xof256_absorb_once_squeeze (haetae_reference_keygen_seedbuf_after_memcpy sd) haetae_reference_keygen_xof_absorb_bytes haetae_reference_keygen_xof_squeeze_bytes. </pre>	<pre> haetae_keygen_xof_seedbuf : seed $\rightarrow \text{List}(\text{byte})$ </pre>	Defines a total mathematical function by the EasyCrypt model.
	618	operation		
		<pre> haetae_reference_keygen_xof256_seedbuf op haetae_reference_keygen_xof256_seedbuf (sd : seed) : byte list = haetae_keygen_xof_seedbuf sd. </pre>	<pre> haetae_reference_keygen_xof256_seedbuf : seed $\rightarrow \text{List}(\text{byte})$ </pre>	Defines a total mathematical function by the EasyCrypt model.
	620	operation		

	Line	Construct	What was written	Symbolic correspondence
haetae_keygen_seedbuf_rhoprime		op haetae_keygen_seedbuf_rhoprime (seedbuf : byte list) : seed = haetae_seed_slice seedbuf haetae_keygen_rhoprime_offset seedbytes.	haetae__keygen__seedbuf__rhoprime : List(byte) $\rightarrow$ seed	Defines a total mathematical function by the EasyCrypt model.
	622	operation		
haetae_keygen_seedbuf_sigma		op haetae_keygen_seedbuf_sigma (seedbuf : byte list) : crh = haetae_seed_slice seedbuf haetae_keygen_sigma_offset crhbytes.	haetae__keygen__seedbuf__sigma : List(byte) $\rightarrow$ crh	Defines a total mathematical function by the EasyCrypt model.
	624	operation		
haetae_keygen_seedbuf_key		op haetae_keygen_seedbuf_key (seedbuf : byte list) : seed = haetae_seed_slice seedbuf haetae_keygen_key_offset seedbytes.	haetae__keygen__seedbuf__key : List(byte) $\rightarrow$ seed	Defines a total mathematical function by the EasyCrypt model.
	626	operation		
haetae_keygen_seedbuf_layout_wf		op haetae_keygen_seedbuf_layout_wf (seedbuf : byte list) : bool = size seedbuf = haetae_reference_keygen_seedbuf_bytes /\ haetae_keygen_rhoprime_offset = 0 /\ haetae_keygen_sigma_offset = seedbytes /\ haetae_keygen_key_offset = seedbytes + crhbytes /\ size (haetae_keygen_seedbuf_rhoprime seedbuf) = seedbytes /\ size (haetae_keygen_seedbuf_sigma seedbuf) =...	haetae__keygen__seedbuf__layout__wf $\subseteq$ List(byte) is a Boolean-valued predicate.	Names a well-formedness or support condition so it can be reused in lemm
	634	operation		

	Line	Construct	What was written	Symbolic correspondence
		<pre> haetae_reference_keygen_xof_call_wf op haetae_reference_keygen_xof_call_wf seed × List(byte) is a Boolean-valued (sd : seed) (seedbuf : byte predicate. list) : bool = size sd = haetae_reference_keygen_xof_absorb_bytes /\ size (haetae_reference_keygen_seedbuf_after_memcpy sd) = haetae_reference_keygen_seedbuf_bytes /\ haetae_xof256_absorb_input (haetae_reference_keygen_seedbuf_after_memcpy sd) haetae_reference_keygen_xof_absorb_bytes = hae...</pre>	$\text{haetae_reference_keygen_xof_call_wf} \subseteq \text{seed} \times \text{List}(\text{byte})$ is a Boolean-valued predicate.	Names a well-formedness or support condition so it can be reused in lemmas.
	663	operation		
		<pre> haetae_keygen_rhoprime op haetae_keygen_rhoprime (sd : seed) : seed = haetae_keygen_seedbuf_rhoprime (haetae_reference_keygen_xof256_seedbuf sd).</pre>	$\text{haetae_keygen_rhoprime} : \text{seed} \rightarrow \text{seed}$	Defines a total mathematical function by the EasyCrypt model.
	666	operation		
		<pre> haetae_keygen_sigma op haetae_keygen_sigma (sd : seed) : crh = haetae_keygen_seedbuf_sigma (haetae_reference_keygen_xof256_seedbuf sd).</pre>	$\text{haetae_keygen_sigma} : \text{seed} \rightarrow \text{crh}$	Defines a total mathematical function by the EasyCrypt model.
	669	operation		
		<pre> haetae_keygen_key op haetae_keygen_key (sd : seed) : seed = haetae_keygen_seedbuf_key (haetae_reference_keygen_xof256_seedbuf sd).</pre>	$\text{haetae_keygen_key} : \text{seed} \rightarrow \text{seed}$	Defines a total mathematical function by the EasyCrypt model.
	672	operation		
		<pre> haetae_keygen_counter_next op haetae_keygen_counter_next (md : mode) (counter : int) : int = counter + mode_m md + mode_k md.</pre>	$\text{haetae_keygen_counter_next} : \text{mode} \times \mathbb{Z} \rightarrow \mathbb{Z}$	Defines a total mathematical function by the EasyCrypt model.
	674	operation		
		<pre> haetae_keygen_first_accept_counter op haetae_keygen_first_accept_counter (_ : mode) (_ : seed) : int = 0.</pre>	$\text{haetae_keygen_first_accept_counter} : \text{mode} \times \text{seed} \rightarrow \mathbb{Z}$	Defines a total mathematical function by the EasyCrypt model.

	Line	Construct	What was written	Symbolic correspondence
	675	operation		
haetae_keygen_sampler_seed		op haetae_keygen_sampler_seed (sigma : crh) (counter : int) : seed = mkseq (fun i => nth 0 sigma (i %% crhbytes) + counter + i) seedbytes.	haetae__keygen__sampler__seed : $\text{crh} \times \mathbb{Z} \rightarrow$ seed	Defines a total mathematical function by the EasyCrypt model.
	680	operation		
public_matrix_seed		op public_matrix_seed (md : mode) (sd : seed) : matrix = mkseq (fun i => public_key_seed_polyvecl md (101 + i) (i :: sd)) (mode_k md).	public__matrix__seed : $\text{mode} \times \text{seed} \times \text{sd} \rightarrow \text{matrix}$	Defines a total mathematical function by the EasyCrypt model.
	684	operation		
public_secret_vector_seed		op public_secret_vector_seed (md : mode) (sd : seed) : polyvecl = public_key_seed_polyvecl md 211 sd.	public__secret__vector__seed : $\text{mode} \times \text{seed} \rightarrow \text{polyvecl}$	Defines a total mathematical function by the EasyCrypt model.
	686	operation		
public_error_vector_seed		op public_error_vector_seed (md : mode) (sd : seed) : polyveck = public_key_seed_polyveck md 307 sd.	public__error__vector__seed : $\text{mode} \times \text{seed} \rightarrow \text{polyveck}$	Defines a total mathematical function by the EasyCrypt model.
	688	operation		
public_rounding_vector_seed		op public_rounding_vector_seed (md : mode) (sd : seed) : polyveck = public_key_seed_polyveck md 401 sd.	public__rounding__vector__seed : $\text{mode} \times \text{seed} \rightarrow \text{polyveck}$	Defines a total mathematical function by the EasyCrypt model.
	691	operation		
haetae_keygen_a0_matrix		op haetae_keygen_a0_matrix (md : mode) (sd : seed) : matrix = public_matrix_seed md sd.	haetae__keygen__a0__matrix : $\text{mode} \times \text{seed} \rightarrow \text{matrix}$	Defines a total mathematical function by the EasyCrypt model.
	693	operation		
haetae_keygen_secret_vector		op haetae_keygen_secret_vector (md : mode) (sd : seed) : polyvecl = public_secret_vector_seed md sd.	haetae__keygen__secret__vector : $\text{mode} \times \text{seed} \rightarrow \text{polyvecl}$	Defines a total mathematical function by the EasyCrypt model.
	695	operation		

	Line	Construct	What was written	Symbolic correspondence
haetae_keygen_error_vector		op haetae_keygen_error_vector (md : mode) (sd : seed) : polyveck = public_error_vector_seed md sd.	haetae_keygen_error_vector : $\text{mode} \times \text{seed} \rightarrow \text{polyveck}$	Defines a total mathematical function by the EasyCrypt model.
	697	operation		
haetae_keygen_raw_public_vector		op haetae_keygen_raw_public_vector (md : mode) (sd : seed) : polyveck = polyveck_add md (matrix_vec_mul md (haetae_keygen_a0_matrix md sd) (haetae_keygen_secret_vector md sd)) (haetae_keygen_error_vector md sd).	haetae_keygen_raw_public_vector : $\text{mode} \times \text{seed} \rightarrow \text{polyveck}$	Defines a total mathematical function by the EasyCrypt model.
	703	operation		
haetae_keygen_mode23_predecompose_vector		op haetae_keygen_mode23_predecompose_vector (md : mode) (sd : seed) : polyveck = polyveck_add md (public_rounding_vector_seed md sd) (haetae_keygen_raw_public_vector md sd).	haetae_keygen_mode23_predecompose_vector : $\text{mode} \times \text{seed} \rightarrow \text{polyveck}$	Defines a total mathematical function by the EasyCrypt model.
	708	operation		
haetae_keygen_mode23_public_vector		op haetae_keygen_mode23_public_vector (md : mode) (sd : seed) : polyveck = polyveck_vk_highbits md (haetae_keygen_mode23_predecompose_vector md sd).	haetae_keygen_mode23_public_vector : $\text{mode} \times \text{seed} \rightarrow \text{polyveck}$	Defines a total mathematical function by the EasyCrypt model.
	711	operation		
haetae_keygen_mode23_rounding_lowbits		op haetae_keygen_mode23_rounding_lowbits (md : mode) (sd : seed) : polyveck = polyveck_vk_lowbits md (haetae_keygen_mode23_predecompose_vector md sd).	haetae_keygen_mode23_rounding_lowbits : $\text{mode} \times \text{seed} \rightarrow \text{polyveck}$	Defines a total mathematical function by the EasyCrypt model.
	714	operation		

	Line	Construct	What was written	Symbolic correspondence
		<pre> haetae_keygen_mode23_adjusted_error_vector op haetae_keygen_mode23_adjusted_error_vector (md : mode) (sd : seed) : polyveck = polyveck_sub md (haetae_keygen_error_vector md sd) (haetae_keygen_mode23_rounding_lowbits md sd). </pre>	<pre> haetae_keygen_mode23_adjusted_error_vector : mode × seed → polyveck </pre>	Defines a total mathematical function by the EasyCrypt model.
	719	operation		
		<pre> haetae_keygen_mode5_public_vector op haetae_keygen_mode5_public_vector (md : mode) (sd : seed) : polyveck = polyveck_neg md (polyveck_double md (haetae_keygen_raw_public_vector md sd)). </pre>	<pre> haetae_keygen_mode5_public_vector : mode × seed → polyveck </pre>	Defines a total mathematical function by the EasyCrypt model.
	722	operation		
		<pre> haetae_public_key_vector_from_relation op haetae_public_key_vector_from_relation (md : mode) (sd : seed) (s : polyveck1) (e : polyveck) : polyveck = with md = Mode2 => let raw = polyveck_add md (matrix_vec_mul md (public_matrix_seed md sd) s) e in polyveck_vk_highbits md (polyveck_add md (public_rounding_vector_seed md sd) raw) with md = Mode3 => let raw = polyveck_add md (matrix_vec_mul md (pu... </pre>	<pre> haetae_public_key_vector_from_relation : mode × seed × polyveck1 × polyveck → polyveck </pre>	Defines the random-oracle/hash interaction data used by the games.
	744	operation		
		<pre> haetae_keygen_public_vector op haetae_keygen_public_vector (md : mode) (sd : seed) : polyveck = haetae_public_key_vector_from_relation md sd (haetae_keygen_secret_vector md sd) (haetae_keygen_error_vector md sd). </pre>	<pre> haetae_keygen_public_vector : mode × seed → polyveck </pre>	Defines a total mathematical function by the EasyCrypt model.
	748	operation		

	Line	Construct	What was written	Symbolic correspondence	
		public_key_vector_seed	op public_key_vector_seed (md : mode) (sd : seed) : polyveck = haetae_keygen_public_vector md sd.	public_key_vector_seed : mode $\times$ seed $\rightarrow$ polyveck	Defines a total mathematical function by the EasyCrypt model.
	750	operation			
		public_key_relation	op public_key_relation (md : mode) (pk : pkey) (s : polyvecl) (e : polyveck) : bool = pk.'2 = haetae_public_key_vector_from_relation md pk.'1 s e.	public_key_relation $\subseteq$ mode $\times$ pkey $\times$ polyvecl $\times$ polyveck is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.
	753	operation			
		public_key_equation_holds	op public_key_equation_holds (md : mode) (pk : pkey) : bool = public_key_relation md pk (public_secret_vector_seed md pk.'1) (public_error_vector_seed md pk.'1).	public_key_equation_holds $\subseteq$ mode $\times$ pkey is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.
	757	operation			
		haetae_reference_polymatkm_expand_matA	op haetae_reference_polymatkm_expand_matA (md : mode) (rho_prime : seed) : matrix = public_matrix_seed md rho_prime.	haetae_reference_polymatkm_expand_matA : mode $\times$ seed $\rightarrow$ matrix	Defines a total mathematical function by the EasyCrypt model.
	760	operation			
		haetae_reference_polyveck_expand_vecA	op haetae_reference_polyveck_expand_vecA (md : mode) (rho_prime : seed) : polyveck = public_rounding_vector_seed md rho_prime.	haetae_reference_polyveck_expand_vecA : mode $\times$ seed $\rightarrow$ polyveck	Defines a total mathematical function by the EasyCrypt model.
	763	operation			
		haetae_reference_polyvecmk_expand_S_s1	op haetae_reference_polyvecmk_expand_S_s1 (md : mode) (sigma : crh) (counter : int) : polyvecl = public_secret_vector_seed md (haetae_keygen_sampler_seed sigma counter).	haetae_reference_polyvecmk_expand_S_s1 : mode $\times$ crh $\times$ $\mathbb{Z} \rightarrow$ polyvecl	Defines a total mathematical function by the EasyCrypt model.
	766	operation			

	Line	Construct	What was written	Symbolic correspondence
haetae_reference_polyvecmk_expand_S_s2		op haetae_reference_polyvecmk_expand_S_s2 (md : mode) (sigma : crh) (counter : int) : polyveck = public_error_vector_seed md (haetae_keygen_sampler_seed sigma counter).	haetae_reference_polyvecmk_expand_S_s2 : $\text{mode} \times \text{crh} \times \mathbb{Z} \rightarrow \text{polyveck}$	Defines a total mathematical function by the EasyCrypt model.
	769	operation		
haetae_reference_keygen_secret_vector		op haetae_reference_keygen_secret_vector (md : mode) (sd : seed) : polyvecl = haetae_reference_polyvecmk_expand_S_s1 md (haetae_keygen_sigma sd) (haetae_keygen_first_accept_counter md sd).	haetae_reference_keygen_secret_vector : $\text{mode} \times \text{seed} \rightarrow \text{polyvecl}$	Defines a total mathematical function by the EasyCrypt model.
	774	operation		
haetae_reference_keygen_error_vector		op haetae_reference_keygen_error_vector (md : mode) (sd : seed) : polyveck = haetae_reference_polyvecmk_expand_S_s2 md (haetae_keygen_sigma sd) (haetae_keygen_first_accept_counter md sd).	haetae_reference_keygen_error_vector : $\text{mode} \times \text{seed} \rightarrow \text{polyveck}$	Defines a total mathematical function by the EasyCrypt model.
	779	operation		
haetae_reference_keygen_a0_matrix		op haetae_reference_keygen_a0_matrix (md : mode) (sd : seed) : matrix = haetae_reference_polymatkm_expand_mata md (haetae_keygen_rhopprime sd).	haetae_reference_keygen_a0_matrix : $\text{mode} \times \text{seed} \rightarrow \text{matrix}$	Defines a total mathematical function by the EasyCrypt model.
	782	operation		

	Line	Construct	What was written	Symbolic correspondence
		haetae_reference_keygen_raw_public_vector	op haetae_reference_keygen_raw_public_vector (md : mode) (sd : seed) : polyveck = polyveck_add md (matrix_vec_mul md (haetae_reference_keygen_a0_matrix md sd) (haetae_reference_keygen_secret_vector md sd)) (haetae_reference_keygen_error_vector md sd).	haetae_reference_keygen_raw_public_vector $\text{mode} \times \text{seed} \rightarrow \text{polyveck}$ Defines a total mathematical function by the EasyCrypt model.
	789	operation		
		haetae_reference_keygen_mode23_predecompose_vector	op haetae_reference_keygen_mode23_predecompose_vector (md : mode) (sd : seed) : polyveck = polyveck_add md (haetae_reference_polyveck_expand_vecA md (haetae_keygen_rhoprime sd)) (haetae_reference_keygen_raw_public_vector md sd).	haetae_reference_keygen_mode23_predecompose_vector $\text{mode} \times \text{seed} \rightarrow \text{polyveck}$ Defines a total mathematical function by the EasyCrypt model.
	794	operation		
		haetae_reference_keygen_mode23_public_vector	op haetae_reference_keygen_mode23_public_vector (md : mode) (sd : seed) : polyveck = polyveck_vk_highbits md (haetae_reference_keygen_mode23_predecompose_vector md sd).	haetae_reference_keygen_mode23_public_vector $\text{mode} \times \text{seed} \rightarrow \text{polyveck}$ Defines a total mathematical function by the EasyCrypt model.
	798	operation		
		haetae_reference_keygen_mode23_rounding_lowbits	op haetae_reference_keygen_mode23_rounding_lowbits (md : mode) (sd : seed) : polyveck = polyveck_vk_lowbits md (haetae_reference_keygen_mode23_predecompose_vector md sd).	haetae_reference_keygen_mode23_rounding_lowbits $\text{mode} \times \text{seed} \rightarrow \text{polyveck}$ Defines a total mathematical function by the EasyCrypt model.
	802	operation		

	Line	Construct	What was written	Symbolic correspondence
		<pre> haetae_reference_keygen_mode23_adjusted_error_vector op haetae_reference_keygen_mode23_adjusted_error_vector : (md : mode) (sd : seed) : polyveck = polyveck_sub md (haetae_reference_keygen_error_vector md sd) (haetae_reference_keygen_mode23_rounding_lowbits md sd). </pre>	<pre> haetae_reference_keygen_mode23_adjusted_error_vector : mode × seed → polyveck </pre>	Defines a total mathematical function by the EasyCrypt model.
	807	operation		
		<pre> haetae_reference_keygen_mode5_public_vector op haetae_reference_keygen_mode5_public_vector : (md : mode) (sd : seed) : polyveck = polyveck_neg md (polyveck_double md (haetae_reference_keygen_raw_public_vector md sd)). </pre>	<pre> haetae_reference_keygen_mode5_public_vector : mode × seed → polyveck </pre>	Defines a total mathematical function by the EasyCrypt model.
	811	operation		
		<pre> haetae_reference_keygen_public_vector op haetae_reference_keygen_public_vector : (md : mode) (sd : seed) : polyveck = haetae_public_key_vector_from_relation md (haetae_keygen_rhoprime sd) (haetae_reference_keygen_secret_vector md sd) (haetae_reference_keygen_error_vector md sd). </pre>	<pre> haetae_reference_keygen_public_vector : mode × seed → polyveck </pre>	Defines a total mathematical function by the EasyCrypt model.
	817	operation		
		<pre> packed_haetae_reference_public_key op packed_haetae_reference_public_key : (md : mode) (sd : seed) : pkey = (haetae_keygen_rhoprime sd, haetae_reference_keygen_public_vector md sd). </pre>	<pre> packed_haetae_reference_public_key : mode × seed → pkey </pre>	Defines a total mathematical function by the EasyCrypt model.
	820	operation		

	Line	Construct	What was written	Symbolic correspondence
		<pre> haetae_reference_keygen_seed_flow_wf op haetae_reference_keygen_seed_flow_wf (md : mode) (sd : seed) : bool = haetae_reference_keygen_a0_matrix md sd = haetae_reference_polymatkm_expand_matA md (haetae_keygen_rho_prime sd) /\ haetae_reference_keygen_secret_vector md sd = haetae_reference_polyvecmk_expand_S_s1 md (haetae_keygen_sigma sd) (haetae_keygen_first_accept_counter md sd) /\ haetae_ref...</pre>	$\text{haetae_reference_keygen_seed_flow_wf} \subseteq \text{mode} \times \text{seed}$ is a Boolean-valued predicate.	Names a well-formedness or support condition so it can be reused in lemmas.
	841	operation		
		<pre> haetae_mode23_qj_vector op haetae_mode23_qj_vector (md : mode) (sd : seed) : polyveck = public_rounding_vector_seed md sd.</pre>	$\text{haetae_mode23_qj_vector} : \text{mode} \times \text{seed} \rightarrow \text{polyveck}$	Defines a total mathematical function by the EasyCrypt model.
	843	operation		
		<pre> haetae_mode23_verification_column op haetae_mode23_verification_column (md : mode) (b qj : polyveck) : polyveck = polyveck_double md (polyveck_sub md qj (polyveck_double md b)).</pre>	$\text{haetae_mode23_verification_column} : \text{mode} \times \text{polyveck} \times \text{polyveck} \rightarrow \text{polyveck}$	Defines a total mathematical function by the EasyCrypt model.
	846	operation		
		<pre> haetae_mode5_verification_column op haetae_mode5_verification_column (md : mode) (b : polyveck) : polyveck = mkseq (fun i => poly_normalize (nth poly_zero b i)) (mode_k md).</pre>	$\text{haetae_mode5_verification_column} : \text{mode} \times \text{polyveck} \rightarrow \text{polyveck}$	Defines a total mathematical function by the EasyCrypt model.
	849	operation		
		<pre> public_key_mode23_column op public_key_mode23_column (md : mode) (pk : pkey) : polyveck = haetae_mode23_verification_column md pk.'2 (haetae_mode23_qj_vector md pk.'1).</pre>	$\text{public_key_mode23_column} : \text{mode} \times \text{pkey} \rightarrow \text{polyveck}$	Defines a total mathematical function by the EasyCrypt model.

	Line	Construct	What was written	Symbolic correspondence
	852	operation public_key_mode5_column op public_key_mode5_column (md : mode) (pk : pkey) : polyveck = haetae_mode5_verification_column md pk.'2.	public_key_mode5_column : $\text{mode} \times \text{pkey} \rightarrow \text{polyveck}$	Defines a total mathematical function by the EasyCrypt model.
	854	operation public_key_verification_column op public_key_verification_column (md : mode) (pk : pkey) : polyveck = with md = Mode2 => public_key_mode23_column md pk with md = Mode3 => public_key_mode23_column md pk with md = Mode5 => public_key_mode5_column md pk.	public_key_verification_column : $\text{mode} \times \text{pkey} \rightarrow \text{polyveck}$	Defines a total mathematical function by the EasyCrypt model.
	858	operation haetae_verification_column_from_unpacked op haetae_verification_column_from_unpacked (md : mode) (sd : seed) (b : polyveck) : polyveck = with md = Mode2 => haetae_mode23_verification_column md b (haetae_mode23_qj_vector md sd) with md = Mode3 => haetae_mode23_verification_column md b (haetae_mode23_qj_vector md sd) with md = Mode5 => haetae_mode5_verification_column md b.	haetae_verification_column_from_unpacked : $\text{mode} \times \text{seed} \times \text{polyveck} \rightarrow \text{polyveck}$	Defines the random-oracle/hash inter data used by the games.
	868	operation haetae_verification_matrix_from_unpacked op haetae_verification_matrix_from_unpacked (md : mode) (sd : seed) (b : polyveck) : matrix = mkseq (fun i => mkseq (fun j => if j = 0 then nth poly_zero (haetae_verification_column_from_unpacked md sd b) i else poly_double (nth poly_zero (nth [] (haetae_keygen_a0_matrix md sd) i) (j - 1))) (mode_l md)) (mode_k md).	haetae_verification_matrix_from_unpacked : $\text{mode} \times \text{seed} \times \text{polyveck} \rightarrow \text{matrix}$	Defines the random-oracle/hash inter data used by the games.

	Line	Construct	What was written	Symbolic correspondence
	883	operation packed_haetae_public_key op packed_haetae_public_key (md : mode) (sd : seed) : pkey = (sd, haetae_keygen_public_vector md sd).	packed__haetae_public__key : $\text{mode} \times \text{seed} \rightarrow \text{pkey}$	Defines a total mathematical function by the EasyCrypt model.
	885	operation packed_haetae_verification_matrix op packed_haetae_verification_matrix (md : mode) (pk : pkey) : matrix = haetae_verification_matrix_from_unpacked md pk.'1 pk.'2.	packed__haetae_verification_matrix : $\text{mode} \times \text{pkey} \rightarrow \text{matrix}$	Defines a total mathematical function by the EasyCrypt model.
	887	operation packed_haetae_keygen_verification_matrix op packed_haetae_keygen_verification_matrix (md : mode) (sd : seed) : matrix = packed_haetae_verification_matrix md (packed_haetae_public_key md sd).	packed__haetae_keygen_verification_matrix : $\text{mode} \times \text{seed} \rightarrow \text{matrix}$	Defines a total mathematical function by the EasyCrypt model.
	890	operation public_verification_matrix op public_verification_matrix (md : mode) (pk : pkey) : matrix = packed_haetae_verification_matrix md pk.	public_verification_matrix : $\text{mode} \times \text{pkey} \rightarrow \text{matrix}$	Defines a total mathematical function by the EasyCrypt model.
	892	operation challenge_embedding op challenge_embedding (md : mode) (ch : challenge) : polyvec1 = mkseq (fun i => if i = 0 then ch else poly_zero) (mode_1 md).	challenge_embedding : $\text{mode} \times \text{challenge} \rightarrow \text{polyvec1}$	Defines a total mathematical function by the EasyCrypt model.
	894	operation public_key_challenge_term op public_key_challenge_term (md : mode) (pk : pkey) (ch : challenge) : polyveck = matrix_vec_mul md (public_verification_matrix md pk) (challenge_embedding md ch).	public_key_challenge_term : $\text{mode} \times \text{pkey} \times \text{challenge} \rightarrow \text{polyveck}$	Defines a total mathematical function by the EasyCrypt model.
	898	operation		

	Line	Construct	What was written	Symbolic correspondence	
		reconstructed_highbits	op reconstructed_highbits (md : mode) (aux pk_ch : polyveck) : polyveck = polyveck_add md aux pk_ch.	reconstructed_highbits : $\text{mode} \times \text{polyveck} \times \text{polyveck} \rightarrow \text{polyveck}$	Defines a total mathematical function by the EasyCrypt model.
	902	operation			
		public_key_of_secret	op public_key_of_secret (md : mode) (sk : skey) : pkey = (sk.'1, public_key_vector_seed md sk.'1).	public_key_of_secret : $\text{mode} \times \text{skey} \rightarrow \text{pkey}$	Defines a total mathematical function by the EasyCrypt model.
	904	operation			
		haetae_public_key_body_bytes	op haetae_public_key_body_bytes (md : mode) : int = mode_k md * mode_polyq_packedbytes md.	haetae_public_key_body_bytes : $\text{mode} \rightarrow \mathbb{Z}$	Defines a total mathematical function by the EasyCrypt model.
	906	operation			
		haetae_public_key_bytes	op haetae_public_key_bytes (md : mode) : int = mode_publickeybytes md.	haetae_public_key_bytes : $\text{mode} \rightarrow \mathbb{Z}$	Defines a total mathematical function by the EasyCrypt model.
	908	operation			
		haetae_public_key_encoding_wf	op haetae_public_key_encoding_wf (md : mode) (enc : encoded_pkey) : bool = size enc = haetae_public_key_bytes md.	haetae_public_key_encoding_wf $\subseteq \text{mode} \times \text{encoded_pkey}$ is a Boolean-valued predicate.	Names a well-formedness or support condition so it can be reused in lemm
	910	operation			
		haetae_public_key_unpacked_wf	op haetae_public_key_unpacked_wf (md : mode) (pk : pkey) : bool = size pk.'1 = seedbytes /\ polyveck_wf md pk.'2.	haetae_public_key_unpacked_wf $\subseteq \text{mode} \times \text{pkey}$ is a Boolean-valued predicate.	Names a well-formedness or support condition so it can be reused in lemm
	912	operation			
		haetae_pack_seed	op haetae_pack_seed (sd : seed) : byte list = mkseq (fun i => nth 0 sd i) seedbytes.	haetae_pack_seed : $\text{seed} \rightarrow \text{List}(\text{byte})$	Defines a total mathematical function by the EasyCrypt model.
	914	operation			
		haetae_unpack_seed	op haetae_unpack_seed (enc : encoded_pkey) : seed = mkseq (fun i => nth 0 enc i) seedbytes.	haetae_unpack_seed : $\text{encoded_pkey} \rightarrow \text{seed}$	Defines a total mathematical function by the EasyCrypt model.
	916	operation			
		haetae_pack_poly_q	op haetae_pack_poly_q (md : mode) (p : poly) : byte list = mkseq (fun i => if i < n then poly_coeff p i else 0) (mode_polyq_packedbytes md).	haetae_pack_poly_q : $\text{mode} \times \text{poly} \rightarrow \text{List}(\text{byte})$	Defines a total mathematical function by the EasyCrypt model.

	Line	Construct	What was written	Symbolic correspondence	
	920	operation			
		haetae_unpack_poly_q_at	op haetae_unpack_poly_q_at (md : mode) (enc : encoded_pkey) (offset : int) : poly = mkseq (fun i => nth 0 enc (offset + i)) n.	haetae_unpack_poly_q_at : $\text{mode} \times \text{encoded_pkey} \times \mathbb{Z} \rightarrow \text{poly}$	Defines a total mathematical function by the EasyCrypt model.
	924	operation			
		haetae_byte_modulus	op haetae_byte_modulus : int = 256.	haetae_byte_modulus : $1 \rightarrow \mathbb{Z}$	Defines a total mathematical function by the EasyCrypt model.
	925	operation			
		haetae_polyq_mode23_bound	op haetae_polyq_mode23_bound : int = 32768.	haetae_polyq_mode23_bound : $1 \rightarrow \mathbb{Z}$	Defines a concrete numeric loss term in the final security inequality.
	926	operation			
		haetae_polyq_mode5_bound	op haetae_polyq_mode5_bound : int = 65536.	haetae_polyq_mode5_bound : $1 \rightarrow \mathbb{Z}$	Defines a concrete numeric loss term in the final security inequality.
	928	operation			
		haetae_byte_norm	op haetae_byte_norm (x : int) : byte = x %% haetae_byte_modulus.	haetae_byte_norm : $\mathbb{Z} \rightarrow \text{byte}$	Defines a total mathematical function by the EasyCrypt model.
	930	operation			
		haetae_polyq_coeff_bound	op haetae_polyq_coeff_bound (md : mode) : int = with md = Mode2 => haetae_polyq_mode23_bound with md = Mode3 => haetae_polyq_mode23_bound with md = Mode5 => haetae_polyq_mode5_bound.	haetae_polyq_coeff_bound : $\text{mode} \rightarrow \mathbb{Z}$	Defines a concrete numeric loss term in the final security inequality.
	935	operation			
		haetae_polyq_coeffs_wf	op haetae_polyq_coeffs_wf (md : mode) (p : poly) : bool = poly_wf p /\ forall i, 0 <= i < n => 0 <= poly_coeff p i < haetae_polyq_coeff_bound md.	haetae_polyq_coeffs_wf $\subseteq \text{mode} \times \text{poly}$ is a Boolean-valued predicate.	Names a well-formedness or support condition so it can be reused in lemmas.
	940	operation			
		haetae_polyveck_polyq_coeffs_wf	op haetae_polyveck_polyq_coeffs_wf (md : mode) (b : polyveck) : bool = polyveck_wf md b /\ forall row, 0 <= row < mode_k md => haetae_polyq_coeffs_wf md (nth poly_zero b row).	haetae_polyveck_polyq_coeffs_wf $\subseteq \text{mode} \times \text{polyveck}$ is a Boolean-valued predicate.	Names a well-formedness or support condition so it can be reused in lemmas.

	Line	Construct	What was written	Symbolic correspondence
	946	lemma coeff_decompose_hint_high_polyq_bound lemma coeff_decompose_hint_high_polyq_bound md x : coeff_q_bound x => 0 <= coeff_decompose_hint_high md x < haetae_polyq_coeff_bound md.	coeff_decompose_hint_high_polyq_bound : Proves a bound that contributes to a $X \leq Y$ is an arithmetic or probability upper bound.	game-hop or final security inequality.
	956	lemma poly_hint_highbits_polyq_wf lemma poly_hint_highbits_polyq_wf md p : poly_coeffs_q_bound p => haetae_polyq_coeffs_wf md (poly_hint_highbits md p).	poly_hint_highbits_polyq_wf : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a structural correctness fact u justify later reductions.
	970	lemma polyveck_highbits_hint_polyq_wf lemma polyveck_highbits_hint_polyq_wf md xs : polyveck_coeffs_q_bound md xs => haetae_polyveck_polyq_coeffs_wf md (polyveck_highbits_hint md xs).	polyveck_highbits_hint_polyq_wf : $P \Rightarrow$ Q is an implication used as a proof obligation.	Proves a structural correctness fact u justify later reductions.
	984	lemma coeff_decompose_vk_high_mode23_polyq_bound lemma coeff_decompose_vk_high_mode23_polyq_bound md x : (md = Mode2 \/ Mode3) => coeff_q_bound x => 0 <= coeff_decompose_vk_high x < haetae_polyq_coeff_bound md.	coeff_decompose_vk_high_mode23_polyq_bound : Proves a bound that contributes to a $X \leq Y$ is an arithmetic or probability upper bound.	game-hop or final security inequality.
	995	lemma poly_vk_highbits_mode23_polyq_wf lemma poly_vk_highbits_mode23_polyq_wf md p : (md = Mode2 \/ Mode3) => poly_coeffs_q_bound p => haetae_polyq_coeffs_wf md (poly_vk_highbits p).	poly_vk_highbits_mode23_polyq_wf : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a structural correctness fact u justify later reductions.
	1010	lemma polyveck_vk_highbits_mode23_polyq_wf lemma polyveck_vk_highbits_mode23_polyq_wf md xs : (md = Mode2 \/ Mode3) => polyveck_coeffs_q_bound md xs => haetae_polyveck_polyq_coeffs_wf md (polyveck_vk_highbits md xs).	polyveck_vk_highbits_mode23_polyq_wf : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a structural correctness fact u justify later reductions.

	Line	Construct	What was written	Symbolic correspondence
	1025	lemma		
		coeff_q_bound_mode5_polyq_bound	lemma coeff_q_bound_mode5_polyq_bound x : coeff_q_bound x => 0 <= x < haetae_polyq_coeff_bound Mode5.	coeff_q_bound_mode5_polyq_bound : $X \leq Y$ is an arithmetic or probability upper bound.
	1033	lemma		
		poly_q_bound_mode5_polyq_wf	lemma poly_q_bound_mode5_polyq_wf p : poly_wf p => poly_coeffs_q_bound p => haetae_polyq_coeffs_wf Mode5 p.	poly_q_bound_mode5_polyq_wf : $P \Rightarrow$ Q is an implication used as a proof obligation.
	1045	lemma		
		polyveck_q_bound_mode5_polyq_wf	lemma polyveck_q_bound_mode5_polyq_wf xs : polyveck_wf Mode5 xs => polyveck_coeffs_q_bound Mode5 xs => haetae_polyveck_polyq_coeffs_wf Mode5 xs.	polyveck_q_bound_mode5_polyq_wf : $P \Rightarrow Q$ is an implication used as a proof obligation.
	1059	operation		
		haetae_pack_poly_q_mode23_byte	op haetae_pack_poly_q_mode23_byte (p : poly) (blk ofs : int) : byte = let c0 = poly_coeff p (8 * blk + 0) in let c1 = poly_coeff p (8 * blk + 1) in let c2 = poly_coeff p (8 * blk + 2) in let c3 = poly_coeff p (8 * blk + 3) in let c4 = poly_coeff p (8 * blk + 4) in let c5 = poly_coeff p (8 * blk + 5) in let c6 = poly_coeff p (8 * blk + 6) in let c7 = poly...	haetae_pack_poly_q_mode23_byte : $\text{poly} \times \mathbb{Z} \times \mathbb{Z} \rightarrow \text{byte}$
	1092	operation		
		haetae_pack_poly_q_mode5_byte	op haetae_pack_poly_q_mode5_byte (p : poly) (j : int) : byte = let c = poly_coeff p (j %/ 2) in if j %% 2 = 0 then haetae_byte_norm c else haetae_byte_norm (c %/ 256).	haetae_pack_poly_q_mode5_byte : $\text{poly} \times \mathbb{Z} \rightarrow \text{byte}$
	1097	operation		

Line	Construct	What was written	Symbolic correspondence
haetae_pack_poly_q_ref	<pre> op haetae_pack_poly_q_ref (md : mode) (p : poly) : byte list = with md = Mode2 => mkseq (fun j => haetae_pack_poly_q_mode23_byte p (j %/ 15) (j %% 15)) (mode_polyq_packedbytes Mode2) with md = Mode3 => mkseq (fun j => haetae_pack_poly_q_mode23_byte p (j %/ 15) (j %% 15)) (mode_polyq_packedbytes Mode3) with md = Mode5 => mkseq (fun j => haetae_pack_poly_q...</pre>	<pre> haetae__pack__poly__q__ref : mode $\times$ poly $\rightarrow$ List(byte)</pre>	Defines a total mathematical function by the EasyCrypt model.
1111	operation		
haetae_unpack_poly_q_mode23_coeff_at	<pre> op haetae_unpack_poly_q_mode23_coeff_at (enc : encoded_pkey) (base blk ofs : int) : coeff = let a0 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 0)) in let a1 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 1)) in let a2 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 2)) in let a3 = haetae_byte_norm (nth 0 enc (base + 15 * blk + 3)) in let a4 = haet...</pre>	<pre> haetae__unpack__poly__q__mode23__coeff__at : encoded_pkey $\times \mathbb{Z} \times \mathbb{Z} \rightarrow$ coeff</pre>	Defines a total mathematical function by the EasyCrypt model.
1143	operation		
haetae_unpack_poly_q_mode5_coeff_at	<pre> op haetae_unpack_poly_q_mode5_coeff_at (enc : encoded_pkey) (base i : int) : coeff = haetae_byte_norm (nth 0 enc (base + 2 * i)) + 256 * haetae_byte_norm (nth 0 enc (base + 2 * i + 1)).</pre>	<pre> haetae__unpack__poly__q__mode5__coeff__at : encoded_pkey $\times \mathbb{Z} \times \mathbb{Z} \rightarrow$ coeff</pre>	Defines a total mathematical function by the EasyCrypt model.
1148	operation		

	Line	Construct	What was written	Symbolic correspondence
		<pre> haetae_unpack_poly_q_ref_at op haetae_unpack_poly_q_ref_at (md : mode) (enc : encoded_pkey) (offset : int) : poly = with md = Mode2 => mkseq (fun i => haetae_unpack_poly_q_mode23_coeff_at enc offset (i % 8) (i % 8)) n with md = Mode3 => mkseq (fun i => haetae_unpack_poly_q_mode23_coeff_at enc offset (i % 8) (i % 8)) n with md = Mode5 => mkseq (fun i => haetae_unpack_poly_q_mode5...</pre>	<pre> haetae__unpack_poly_q_ref_at : mode $\times$ encoded_pkey $\times \mathbb{Z} \rightarrow$ poly</pre>	Defines a total mathematical function by the EasyCrypt model.
	1167	operation		
		<pre> haetae_pack_polyveck_body_ref op haetae_pack_polyveck_body_ref (md : mode) (b : polyveck) : byte list = mkseq (fun i => nth 0 (haetae_pack_poly_q_ref md (nth poly_zero b (i % mode_polyq_packedbytes md))) (i %% mode_polyq_packedbytes md)) (haetae_public_key_body_bytes md).</pre>	<pre> haetae__pack_polyveck_body_ref : mode $\times$ polyveck $\rightarrow$ List(byte)</pre>	Defines a total mathematical function by the EasyCrypt model.
	1176	operation		
		<pre> haetae_unpack_polyveck_body_ref op haetae_unpack_polyveck_body_ref (md : mode) (enc : encoded_pkey) (offset : int) : polyveck = mkseq (fun row => haetae_unpack_poly_q_ref_at md enc (offset + row * mode_polyq_packedbytes md)) (mode_k md).</pre>	<pre> haetae__unpack_polyveck_body_ref : mode $\times$ encoded_pkey $\times \mathbb{Z} \rightarrow$ polyveck</pre>	Defines a total mathematical function by the EasyCrypt model.
	1184	operation		
		<pre> haetae_pack_vk_ref op haetae_pack_vk_ref (md : mode) (b : polyveck) (sd : seed) : encoded_pkey = haetae_pack_seed sd ++ haetae_pack_polyveck_body_ref md b.</pre>	<pre> haetae__pack_vk_ref : mode $\times$ polyveck $\times$ seed $\rightarrow$ encoded_pkey</pre>	Defines a total mathematical function by the EasyCrypt model.
	1187	operation		

	Line	Construct	What was written	Symbolic correspondence
		<pre> haetae_unpack_vk_public_key_ref op haetae_unpack_vk_public_key_ref (md : mode) (enc : encoded_pkey) : pkey = (haetae_unpack_seed enc, haetae_unpack_polyveck_body_ref md enc seedbytes). </pre>	<pre> haetae__unpack_vk_public_key_ref : mode × encoded_pkey → pkey </pre>	Defines a total mathematical function by the EasyCrypt model.
	1191	<pre> haetae_pack_polyveck_body op haetae_pack_polyveck_body (md : mode) (b : polyveck) : byte list = mkseq (fun i => if i %% mode_polyq_packedbytes md < n then poly_coeff (nth poly_zero b (i %/ mode_polyq_packedbytes md)) (i %% mode_polyq_packedbytes md) else 0) (haetae_public_key_body_bytes md). </pre>	<pre> haetae__pack_polyveck_body : mode × polyveck → List(byte) </pre>	Defines a total mathematical function by the EasyCrypt model.
	1200	<pre> haetae_unpack_polyveck_body op haetae_unpack_polyveck_body (md : mode) (enc : encoded_pkey) (offset : int) : polyveck = mkseq (fun row => haetae_unpack_poly_q_at md enc (offset + row * mode_polyq_packedbytes md)) (mode_k md). </pre>	<pre> haetae__unpack_polyveck_body : mode × encoded_pkey × ℤ → polyveck </pre>	Defines a total mathematical function by the EasyCrypt model.
	1207	<pre> haetae_pack_vk op haetae_pack_vk (md : mode) (b : polyveck) (sd : seed) : encoded_pkey = haetae_pack_seed sd ++ haetae_pack_polyveck_body md b. </pre>	<pre> haetae__pack_vk : mode × polyveck × seed → encoded_pkey </pre>	Defines a total mathematical function by the EasyCrypt model.
	1209	<pre> haetae_unpack_vk_public_key op haetae_unpack_vk_public_key (md : mode) (enc : encoded_pkey) : pkey = (haetae_unpack_seed enc, haetae_unpack_polyveck_body md enc seedbytes). </pre>	<pre> haetae__unpack_vk_public_key : mode × encoded_pkey → pkey </pre>	Defines a total mathematical function by the EasyCrypt model.
	1213	<pre> operation </pre>		

	Line	Construct	What was written	Symbolic correspondence
haetae_unpack_vk_matrix		<pre> op haetae_unpack_vk_matrix (md : mode) (enc : encoded_pkey) : matrix = packed_haetae_verification_matrix md (haetae_unpack_vk_public_key md enc). </pre>	<pre> haetae__unpack_vk_matrix : mode × encoded_pkey → matrix </pre>	Defines a total mathematical function by the EasyCrypt model.
haetae_pack_public_key_bytes	1216	<pre> op haetae_pack_public_key_bytes (md : mode) (pk : pkey) : encoded_pkey = haetae_pack_vk md pk.'2 pk.'1. </pre>	<pre> haetae__pack_public_key_bytes : mode × pkey → encoded_pkey </pre>	Defines a total mathematical function by the EasyCrypt model.
haetae_unpack_public_key_bytes	1218	<pre> op haetae_unpack_public_key_bytes (md : mode) (enc : encoded_pkey) : pkey = haetae_unpack_vk_public_key md enc. </pre>	<pre> haetae__unpack_public_key_bytes : mode × encoded_pkey → pkey </pre>	Defines a total mathematical function by the EasyCrypt model.
concrete_haetae_keygen_packed_public_key	1221	<pre> op concrete_haetae_keygen_packed_public_key (md : mode) (sd : seed) : encoded_pkey = haetae_pack_public_key_bytes md (packed_haetae_public_key md sd). </pre>	<pre> concrete__haetae_keygen_packed_public_key : mode × seed → encoded_pkey </pre>	Defines a total mathematical function by the EasyCrypt model.
concrete_haetae_keygen_unpacked_public_key	1224	<pre> op concrete_haetae_keygen_unpacked_public_key (md : mode) (sd : seed) : pkey = haetae_unpack_public_key_bytes md (concrete_haetae_keygen_packed_public_key md sd). </pre>	<pre> concrete__haetae_keygen_unpacked_public_key : mode × seed → pkey </pre>	Defines a total mathematical function by the EasyCrypt model.
	1228	operation		

	Line	Construct	What was written	Symbolic correspondence
concrete_haetae_keygen_verification_matrix		<pre> op concrete_haetae_keygen_verification_matrix = (md : mode) (sd : seed) : matrix = packed_haetae_verification_matrix md (concrete_haetae_keygen_unpacked_public_key md sd). </pre>	concrete_haetae_keygen_verification_matrix : $\text{mode} \times \text{seed} \rightarrow \text{matrix}$	Defines a total mathematical function by the EasyCrypt model.
haetae_pack_public_key_bytes_ref	1232	<pre> op haetae_pack_public_key_bytes_ref = (md : mode) (pk : pkey) : encoded_pkey = haetae_pack_vk_ref md pk.'2 pk.'1. </pre>	haetae_pack_public_key_bytes_ref : $\text{mode} \times \text{pkey} \rightarrow \text{encoded_pkey}$	Defines a total mathematical function by the EasyCrypt model.
haetae_unpack_public_key_bytes_ref	1235	<pre> op haetae_unpack_public_key_bytes_ref = (md : mode) (enc : encoded_pkey) : pkey = haetae_unpack_vk_public_key_ref md enc. </pre>	haetae_unpack_public_key_bytes_ref : $\text{mode} \times \text{encoded_pkey} \rightarrow \text{pkey}$	Defines a total mathematical function by the EasyCrypt model.
concrete_haetae_keygen_packed_public_key_ref	1238	<pre> op concrete_haetae_keygen_packed_public_key_ref = (md : mode) (sd : seed) : encoded_pkey = haetae_pack_public_key_bytes_ref md (packed_haetae_public_key md sd). </pre>	concrete_haetae_keygen_packed_public_key_ref : $\text{mode} \times \text{seed} \rightarrow \text{encoded_pkey}$	Defines a total mathematical function by the EasyCrypt model.
concrete_haetae_keygen_unpacked_public_key_ref	1241	<pre> op concrete_haetae_keygen_unpacked_public_key_ref = (md : mode) (sd : seed) : pkey = haetae_unpack_public_key_bytes_ref md (concrete_haetae_keygen_packed_public_key_ref md sd). </pre>	concrete_haetae_keygen_unpacked_public_key_ref : $\text{mode} \times \text{seed} \rightarrow \text{pkey}$	Defines a total mathematical function by the EasyCrypt model.
	1245	operation		

Line	Construct	What was written	Symbolic correspondence
	<pre> concrete_haetae_keygen_verification_matrix_ref op concrete_haetae_keygen_verification_matrix_ref (md : mode) (sd : seed) : matrix = packed_haetae_verification_matrix md (concrete_haetae_keygen_unpacked_public_key_ref md sd). </pre>	<pre> concrete_haetae_keygen_verification_matrix_ref mode × seed → matrix </pre>	Defines a total mathematical function by the EasyCrypt model.
1249	operation		
	<pre> concrete_haetae_reference_keygen_packed_public_key_ref op concrete_haetae_reference_keygen_packed_public_key_ref (md : mode) (sd : seed) : encoded_pkey = haetae_pack_public_key_bytes_ref md (packed_haetae_reference_public_key md sd). </pre>	<pre> concrete_haetae_reference_keygen_packed_public_key_ref mode × seed → encoded_pkey </pre>	Defines a total mathematical function by the EasyCrypt model.
1253	operation		
	<pre> concrete_haetae_reference_keygen_unpacked_public_key_ref op concrete_haetae_reference_keygen_unpacked_public_key_ref (md : mode) (sd : seed) : pkey = haetae_unpack_public_key_bytes_ref md (concrete_haetae_reference_keygen_packed_public_key_ref md sd). </pre>	<pre> concrete_haetae_reference_keygen_unpacked_public_key_ref mode × seed → pkey </pre>	Defines a total mathematical function by the EasyCrypt model.
1257	operation		
	<pre> concrete_haetae_reference_keygen_verification_matrix_ref op concrete_haetae_reference_keygen_verification_matrix_ref (md : mode) (sd : seed) : matrix = packed_haetae_verification_matrix md (concrete_haetae_reference_keygen_unpacked_public_key_ref md sd). </pre>	<pre> concrete_haetae_reference_keygen_verification_matrix_ref mode × seed → matrix </pre>	Defines a total mathematical function by the EasyCrypt model.
1261	operation		

	Line	Construct	What was written	Symbolic correspondence
		<pre> haetae_public_key_roundtrip_ok op haetae_public_key_roundtrip_ok (md : mode) (pk : pkey) : bool = haetae_unpack_public_key_bytes md (haetae_pack_public_key_bytes md pk) = pk. </pre>	<pre> haetae_public_key_roundtrip_ok ⊆ mode × pkey is a Boolean-valued predicate. </pre>	Names a Boolean mathematical condition used in statements and invariants.
	1264	operation		
		<pre> haetae_keygen_public_key_roundtrip_ok op haetae_keygen_public_key_roundtrip_ok (md : mode) (sd : seed) : bool = haetae_public_key_roundtrip_ok md (packed_haetae_public_key md sd). </pre>	<pre> haetae_keygen_public_key_roundtrip_ok ⊆ mode × seed is a Boolean-valued predicate. </pre>	Names a Boolean mathematical condition used in statements and invariants.
	1267	lemma		
		<pre> public_key_seed_poly_wf lemma public_key_seed_poly_wf tag src : poly_wf (public_key_seed_poly tag src). </pre>	<pre> public_key_seed_poly_wf : ∀x̄. P(x̄) is a universally quantified fact. </pre>	Proves a structural correctness fact used to justify later reductions.
	1271	lemma		
		<pre> public_key_seed_polyvecl_wf lemma public_key_seed_polyvecl_wf md tag src : polyvecl_wf md (public_key_seed_polyvecl md tag src). </pre>	<pre> public_key_seed_polyvecl_wf : ∀x̄. P(x̄) is a universally quantified fact. </pre>	Proves a structural correctness fact used to justify later reductions.
	1281	lemma		
		<pre> public_key_seed_polyveck_wf lemma public_key_seed_polyveck_wf md tag src : polyveck_wf md (public_key_seed_polyveck md tag src). </pre>	<pre> public_key_seed_polyveck_wf : ∀x̄. P(x̄) is a universally quantified fact. </pre>	Proves a structural correctness fact used to justify later reductions.
	1291	lemma		
		<pre> haetae_keygen_seedbuf_bytes_gt0 lemma haetae_keygen_seedbuf_bytes_gt0 : 0 < haetae_keygen_seedbuf_bytes. </pre>	<pre> haetae_keygen_seedbuf_bytes_gt0 : ∀x̄. P(x̄) is a universally quantified fact. </pre>	Records a reusable theorem so later proofs and scripts can rewrite, apply, or compose with it.
	1295	lemma		
		<pre> haetae_reference_keygen_seedbuf_bytesE lemma haetae_reference_keygen_seedbuf_bytesE : haetae_reference_keygen_seedbuf_bytes = haetae_keygen_seedbuf_bytes. </pre>	<pre> haetae_reference_keygen_seedbuf_bytesE : L = R is an equality/rewriting fact. </pre>	Records a reusable theorem so later proofs and scripts can rewrite, apply, or compose with it.

	Line	Construct	What was written	Symbolic correspondence
	1302	lemma haetae_reference_keygen_xof_absorb_bytesE lemma haetae_reference_keygen_xof_absorb_bytesE : haetae_reference_keygen_xof_absorb_bytes = seedbytes.	haetae_reference_keygen_xof_absorb_bytesE $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	1306	lemma haetae_reference_keygen_xof_squeeze_bytesE lemma haetae_reference_keygen_xof_squeeze_bytesE : haetae_reference_keygen_xof_squeeze_bytes = haetae_reference_keygen_seedbuf_bytes.	haetae_reference_keygen_xof_squeeze_bytesE $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	1311	lemma haetae_keygen_rhoprime_offsetE lemma haetae_keygen_rhoprime_offsetE : haetae_keygen_rhoprime_offset = 0.	haetae_keygen_rhoprime_offsetE : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	1315	lemma haetae_keygen_sigma_offsetE lemma haetae_keygen_sigma_offsetE : haetae_keygen_sigma_offset = seedbytes.	haetae_keygen_sigma_offsetE : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	1319	lemma haetae_keygen_key_offsetE lemma haetae_keygen_key_offsetE : haetae_keygen_key_offset = seedbytes + crhbytes.	haetae_keygen_key_offsetE : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	1323	lemma haetae_shake256_rate_bytesE lemma haetae_shake256_rate_bytesE : haetae_shake256_rate_bytes = 136.	haetae_shake256_rate_bytesE : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	1329	lemma haetae_shake256_domain_separatorE lemma haetae_shake256_domain_separatorE : haetae_shake256_domain_separator = 31.	haetae_shake256_domain_separatorE : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	1335	lemma		

	Line	Construct	What was written	Symbolic correspondence
		haetae_shake256_bytes_size lemma haetae_shake256_bytes_size input outlen : 0 <= outlen => size (haetae_shake256_bytes input outlen) = outlen. 1343 lemma	haetae_shake256_bytes_size : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
		haetae_xof256_squeeze_size lemma haetae_xof256_squeeze_size st outlen : 0 <= outlen => size (haetae_xof256_squeeze st outlen) = outlen. 1352 lemma	haetae_xof256_squeeze_size : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
		haetae_xof256_absorb_once_squeeze_size lemma haetae_xof256_absorb_once_squeeze_size input inlen outlen : 0 <= outlen => size (haetae_xof256_absorb_once_squeeze input inlen outlen) = outlen. 1361 lemma	haetae_xof256_absorb_once_squeeze_size : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
		haetae_reference_keygen_seedbuf_after_memcpy_size lemma haetae_reference_keygen_seedbuf_after_memcpy_size sd : size (haetae_reference_keygen_seedbuf_after_memcpy sd) = haetae_reference_keygen_seedbuf_bytes. 1368 lemma	haetae_reference_keygen_seedbuf_after_memcpy_size : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
		haetae_keygen_xof_seedbuf_size lemma haetae_keygen_xof_seedbuf_size sd : size (haetae_keygen_xof_seedbuf sd) = haetae_keygen_seedbuf_bytes. 1379 lemma	haetae_keygen_xof_seedbuf_size : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
		haetae_reference_keygen_xof256_seedbuf_size lemma haetae_reference_keygen_xof256_seedbuf_size sd : size (haetae_reference_keygen_xof256_seedbuf sd) = haetae_reference_keygen_seedbuf_bytes. 1388 lemma	haetae_reference_keygen_xof256_seedbuf_size : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
		haetae_seed_slice_size lemma haetae_seed_slice_size src offset len : 0 <= len => size (haetae_seed_slice src offset len) = len.	haetae_seed_slice_size : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later p scripts can rewrite, apply, or compose

	Line	Construct	What was written	Symbolic correspondence
	1397	lemma		
haetae_reference_keygen_xof_input_size		lemma haetae_reference_keygen_xof_input_size sd : size (haetae_reference_keygen_xof_input sd) = haetae_reference_keygen_xof_absorb_bytes.	haetae_reference_keygen_xof_input_size : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	1406	lemma		
haetae_xof256_absorb_input_size		lemma haetae_xof256_absorb_input_size input inlen : 0 <= inlen => size (haetae_xof256_absorb_input input inlen) = inlen.	haetae_xof256_absorb_input_size : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	1415	lemma		
haetae_reference_keygen_xof_absorb_inputE		lemma haetae_reference_keygen_xof_absorb_inputE sd : haetae_xof256_absorb_input (haetae_reference_keygen_seedbuf_after_memcpy sd) haetae_reference_keygen_xof_absorb_bytes = haetae_reference_keygen_xof_input sd.	haetae_reference_keygen_xof_absorb_inputE : R is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	1425	lemma		
haetae_reference_keygen_shake256_domain		lemma haetae_reference_keygen_shake256_domain sd : shake256_absorb_once_short_domain (haetae_reference_keygen_xof_input sd) haetae_reference_keygen_xof_absorb_bytes.	haetae_reference_keygen_shake256_domain : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	1435	lemma		
haetae_reference_keygen_xof_squeeze_bytes_lt_rate		lemma haetae_reference_keygen_xof_squeeze_bytes_lt_rate : haetae_reference_keygen_xof_squeeze_bytes < haetae_shake256_rate_bytes.	haetae_reference_keygen_xof_squeeze_bytes_lt_rate : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	1444	lemma		

	Line	Construct	What was written	Symbolic correspondence
haetae_reference_keygen_xof_absorb_once_stateE		lemma	haetae_reference_keygen_xof_absorb_once_stateE	StateE is an invariant preservation fact
		haetae_reference_keygen_xof_absorb_once_stateE	$L = R$ is an equality/rewriting fact.	oracle, adversary call, or game main procedure.
		sd : haetae_xof256_absorb_once		
		(haetae_reference_keygen_seedbuf_after_memcpy		
		sd)		
		haetae_reference_keygen_xof_absorb_bytes		
		= shake256_absorb_once		
		(haetae_reference_keygen_xof_input		
		sd)		
		haetae_reference_keygen_xof_absorb_bytes.		
	1456	lemma		
haetae_keygen_xof_seedbuf_incrementalE		lemma	haetae_keygen_xof_seedbuf_incrementalE	E : Proves a bound that contributes to a
		haetae_keygen_xof_seedbuf_incrementalE	$L = R$ is an equality/rewriting fact.	game-hop or final security inequality.
		sd : haetae_keygen_xof_seedbuf		
		sd = haetae_xof256_squeeze		
		(haetae_xof256_absorb_once		
		(haetae_reference_keygen_seedbuf_after_memcpy		
		sd)		
		haetae_reference_keygen_xof_absorb_bytes)		
		haetae_reference_keygen_xof_squeeze_bytes.		
	1466	lemma		
haetae_keygen_xof_seedbuf_shake256E		lemma	haetae_keygen_xof_seedbuf_shake256E	E : Records a reusable theorem so later p
		haetae_keygen_xof_seedbuf_shake256E	$L = R$ is an equality/rewriting fact.	scripts can rewrite, apply, or compose
		sd : haetae_keygen_xof_seedbuf		
		sd = haetae_shake256_bytes		
		(haetae_reference_keygen_xof_input		
		sd)		
		haetae_reference_keygen_xof_squeeze_bytes.		
	1478	lemma		
haetae_keygen_seedbuf_rhoprime_size		lemma	haetae_keygen_seedbuf_rhoprime_size	: Records a reusable theorem so later p
		haetae_keygen_seedbuf_rhoprime_size	$L = R$ is an equality/rewriting fact.	scripts can rewrite, apply, or compose
		seedbuf : size		
		(haetae_keygen_seedbuf_rhoprime		
		seedbuf) = seedbytes.		
	1486	lemma		
haetae_keygen_seedbuf_sigma_size		lemma	haetae_keygen_seedbuf_sigma_size	: $L =$ Records a reusable theorem so later p
		haetae_keygen_seedbuf_sigma_size	R is an equality/rewriting fact.	scripts can rewrite, apply, or compose
		seedbuf : size		
		(haetae_keygen_seedbuf_sigma		
		seedbuf) = crhbytes.		
	1494	lemma		

	Line	Construct	What was written	Symbolic correspondence
haetae_keygen_seedbuf_key_size		lemma haetae_keygen_seedbuf_key_size seedbuf : size (haetae_keygen_seedbuf_key seedbuf) = seedbytes.	haetae_keygen_seedbuf_key_size : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
1502		lemma		
haetae_reference_keygen_xof_seedbuf_layout_wf		lemma haetae_reference_keygen_xof_seedbuf_layout_wf sd : haetae_keygen_seedbuf_layout_wf (haetae_reference_keygen_xof256_seedbuf sd).	haetae_reference_keygen_xof_seedbuf_layout_wf : $\forall \vec{x}. R(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
1516		lemma		
haetae_reference_keygen_xof_call_wf_ok		lemma haetae_reference_keygen_xof_call_wf_ok sd : size sd = seedbytes => haetae_reference_keygen_xof_call_wf sd (haetae_reference_keygen_xof256_seedbuf sd).	haetae_reference_keygen_xof_call_wf_ok : $B \Rightarrow Q$ is an implication used as a proof obligation.	Proves a structural correctness fact u justify later reductions.
1537		lemma		
haetae_keygen_rhoprime_ref_sliceE		lemma haetae_keygen_rhoprime_ref_sliceE sd : haetae_keygen_rhoprime sd = haetae_keygen_seedbuf_rhoprime (haetae_reference_keygen_xof256_seedbuf sd).	haetae_keygen_rhoprime_ref_sliceE : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
1543		lemma		
haetae_keygen_sigma_ref_sliceE		lemma haetae_keygen_sigma_ref_sliceE sd : haetae_keygen_sigma sd = haetae_keygen_seedbuf_sigma (haetae_reference_keygen_xof256_seedbuf sd).	haetae_keygen_sigma_ref_sliceE : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
1549		lemma		
haetae_keygen_key_ref_sliceE		lemma haetae_keygen_key_ref_sliceE sd : haetae_keygen_key sd = haetae_keygen_seedbuf_key (haetae_reference_keygen_xof256_seedbuf sd).	haetae_keygen_key_ref_sliceE : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose

	Line	Construct	What was written	Symbolic correspondence
	1555	lemma		
haetae_keygen_rho_prime_size		lemma haetae_keygen_rho_prime_size sd : size (haetae_keygen_rho_prime sd) = seedbytes.	haetae_keygen_rho_prime_size : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	1561	lemma		
haetae_keygen_sigma_size		lemma haetae_keygen_sigma_size sd : size (haetae_keygen_sigma sd) = crhbytes.	haetae_keygen_sigma_size : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	1567	lemma		
haetae_keygen_key_size		lemma haetae_keygen_key_size sd : size (haetae_keygen_key sd) = seedbytes.	haetae_keygen_key_size : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	1573	lemma		
haetae_keygen_sampler_seed_size		lemma haetae_keygen_sampler_seed_size sigma counter : size (haetae_keygen_sampler_seed sigma counter) = seedbytes.	haetae_keygen_sampler_seed_size : $L =$ R is an equality/rewriting fact.	Proves a bound that contributes to a game-hop or final security inequality.
	1577	lemma		
haetae_keygen_counter_next_gt		lemma haetae_keygen_counter_next_gt counter md : counter < haetae_keygen_counter_next md counter.	haetae_keygen_counter_next_gt : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	1581	lemma		
public_matrix_seed_size		lemma public_matrix_seed_size md sd : size (public_matrix_seed md sd) = mode_k md.	public_matrix_seed_size : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	1585	lemma		
public_matrix_seed_rows_wf		lemma public_matrix_seed_rows_wf md sd : all (polyvecl_wf md) (public_matrix_seed md sd).	public_matrix_seed_rows_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	1593	lemma		
public_secret_vector_seed_wf		lemma public_secret_vector_seed_wf md sd : polyvecl_wf md (public_secret_vector_seed md sd).	public_secret_vector_seed_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	1597	lemma		

	Line	Construct	What was written	Symbolic correspondence
		<pre> public_error_vector_seed_wf lemma public_error_vector_seed_wf md sd : polyveck_wf md (public_error_vector_seed md sd). </pre>	<pre> public_error_vector_seed_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact. </pre>	Proves a structural correctness fact u justify later reductions.
	1601	<pre> public_rounding_vector_seed_wf lemma public_rounding_vector_seed_wf md sd : polyveck_wf md (public_rounding_vector_seed md sd). </pre>	<pre> public_rounding_vector_seed_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact. </pre>	Proves a structural correctness fact u justify later reductions.
	1605	<pre> haetae_reference_polymatkm_expand_matA_size lemma haetae_reference_polymatkm_expand_matA_size md rhoprime : size (haetae_reference_polymatkm_expand_matA md rhoprime) = mode_k md. </pre>	<pre> haetae_reference_polymatkm_expand_matA_size $L=A, B$ is an equality/rewriting fact. </pre>	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	1611	<pre> haetae_reference_polymatkm_expand_matA_rows_wf lemma haetae_reference_polymatkm_expand_matA_rows_wf md rhoprime : all (polyvecl_wf md) (haetae_reference_polymatkm_expand_matA md rhoprime). </pre>	<pre> haetae_reference_polymatkm_expand_matA_rows_wf $\forall \vec{x}. B(\vec{x})$ is a universally quantified fact. </pre>	Proves a structural correctness fact u justify later reductions.
	1619	<pre> haetae_reference_polyveck_expand_vecA_wf lemma haetae_reference_polyveck_expand_vecA_wf md rhoprime : polyveck_wf md (haetae_reference_polyveck_expand_vecA md rhoprime). </pre>	<pre> haetae_reference_polyveck_expand_vecA_wf $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact. </pre>	Proves a structural correctness fact u justify later reductions.
	1626	<pre> haetae_reference_polyvecmk_expand_S_s1_wf lemma haetae_reference_polyvecmk_expand_S_s1_wf md sigma counter : polyvecl_wf md (haetae_reference_polyvecmk_expand_S_s1 md sigma counter). </pre>	<pre> haetae_reference_polyvecmk_expand_S_s1_wf $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact. </pre>	Proves a structural correctness fact u justify later reductions.
	1634	<pre> lemma </pre>		

	Line	Construct	What was written	Symbolic correspondence
haetae_reference_polyvecmk_expand_S_s2_wf		lemma	haetae_reference_polyvecmk_expand_S_s2_wf	Proves a structural correctness fact u
			$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	justify later reductions.
		md sigma counter : polyveck_wf		
		md		
		(haetae_reference_polyvecmk_expand_S_s2		
		md sigma counter).		
	1642	lemma		
haetae_reference_keygen_secret_vector_wf		lemma	haetae_reference_keygen_secret_vector_wf	Proves a structural correctness fact u
			$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	justify later reductions.
		md sd : polyvecl_wf md		
		(haetae_reference_keygen_secret_vector		
		md sd).		
	1649	lemma		
haetae_reference_keygen_error_vector_wf		lemma	haetae_reference_keygen_error_vector_wf	Proves a structural correctness fact u
			$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	justify later reductions.
		md sd : polyveck_wf md		
		(haetae_reference_keygen_error_vector		
		md sd).		
	1656	lemma		
haetae_reference_keygen_raw_public_vector_wf		lemma	haetae_reference_keygen_raw_public_vector_wf	Proves a structural correctness fact u
			$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	justify later reductions.
		md sd : polyveck_wf md		
		(haetae_reference_keygen_raw_public_vector		
		md sd).		
	1663	lemma		
haetae_reference_keygen_raw_public_vector_coeffs_q_bound		lemma	haetae_reference_keygen_raw_public_vector_coeffs_q_bound	Proves a bound that contributes to a
			$\forall \vec{c}. P(\vec{c})$ is a universally quantified fact.	game-hop or final security inequality.
		md sd : polyveck_coeffs_q_bound		
		md		
		(haetae_reference_keygen_raw_public_vector		
		md sd).		
	1671	lemma		
haetae_reference_keygen_mode23_predecompose_vector_wf		lemma	haetae_reference_keygen_mode23_predecompose_vector_wf	Proves a structural correctness fact u
			$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	justify later reductions.
		md sd : polyveck_wf md		
		(haetae_reference_keygen_mode23_predecompose_vector		
		md sd).		
	1679	lemma		

	Line	Construct	What was written	Symbolic correspondence
haetae_reference_keygen_mode23_predecompose_vector_coeffs_q_bound		lemma	haetae_reference_keygen_mode23_predecompose_vector_coeffs_q_bound	Proves vector coeffs q_bound contributes to a game-hop or final security inequality.
		haetae_reference_keygen_mode23_predecompose_vector_coeffs_q_bound	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	
		md sd : polyveck_coeffs_q_bound		
		md		
		(haetae_reference_keygen_mode23_predecompose_vector		
		md sd).		
	1688	lemma	haetae_reference_keygen_mode23_public_vector_polyq_wf	Proves polyq wf structural correctness fact u
haetae_reference_keygen_mode23_public_vector_polyq_wf		lemma	haetae_reference_keygen_mode23_public_vector_polyq_wf	$P \Rightarrow Q$ is an implication used as a proof justify later reductions.
		md sd : (md = Mode2 $\vee$ md = Mode3) =>	obligation.	
		haetae_polyveck_polyq_coeffs_wf		
		md		
		(haetae_reference_keygen_mode23_public_vector		
		md sd).		
	1699	lemma	haetae_reference_keygen_mode23_adjusted_error_vector_wf	Proves vector wf structural correctness fact u
haetae_reference_keygen_mode23_adjusted_error_vector_wf		lemma	haetae_reference_keygen_mode23_adjusted_error_vector_wf	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.
		md sd : polyveck_wf md		justify later reductions.
		(haetae_reference_keygen_mode23_adjusted_error_vector		
		md sd).		
	1707	lemma	haetae_reference_keygen_mode5_public_vector_wf	Proves a structural correctness fact u
haetae_reference_keygen_mode5_public_vector_wf		lemma	haetae_reference_keygen_mode5_public_vector_wf	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.
		sd : polyveck_wf Mode5		justify later reductions.
		(haetae_reference_keygen_mode5_public_vector		
		Mode5 sd).		
	1715	lemma	haetae_reference_keygen_mode5_public_vector_coeffs_q_bound	Proves coeffs q_bound contributes to a game-hop or final security inequality.
haetae_reference_keygen_mode5_public_vector_coeffs_q_bound		lemma	haetae_reference_keygen_mode5_public_vector_coeffs_q_bound	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.
		sd : polyveck_coeffs_q_bound		
		Mode5		
		(haetae_reference_keygen_mode5_public_vector		
		Mode5 sd).		
	1723	lemma	haetae_reference_keygen_mode5_public_vector_polyq_wf	Proves polyq wf structural correctness fact u
haetae_reference_keygen_mode5_public_vector_polyq_wf		lemma	haetae_reference_keygen_mode5_public_vector_polyq_wf	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.
		sd :		justify later reductions.
		haetae_polyveck_polyq_coeffs_wf		
		Mode5		
		(haetae_reference_keygen_mode5_public_vector		
		Mode5 sd).		

	Line	Construct	What was written	Symbolic correspondence
	1732	lemma		
haetae_keygen_raw_public_vector_wf		lemma haetae_keygen_raw_public_vector_wf md sd : polyveck_wf md (haetae_keygen_raw_public_vector md sd).	haetae_keygen_raw_public_vector_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	1736	lemma		
haetae_keygen_raw_public_vector_coeffs_q_bound		lemma haetae_keygen_raw_public_vector_coeffs_q_bound md sd : polyveck_coeffs_q_bound md (haetae_keygen_raw_public_vector md sd).	haetae_keygen_raw_public_vector_coeffs_q_bound : $\forall \vec{x}. B(\vec{x})$ is a universally quantified fact.	qP-bound: a bound that contributes to a game-hop or final security inequality.
	1743	lemma		
haetae_keygen_mode23_predecompose_vector_wf		lemma haetae_keygen_mode23_predecompose_vector_wf md sd : polyveck_wf md (haetae_keygen_mode23_predecompose_vector md sd).	haetae_keygen_mode23_predecompose_vector_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Shows: a structural correctness fact u justify later reductions.
	1750	lemma		
haetae_keygen_mode23_predecompose_vector_coeffs_q_bound		lemma haetae_keygen_mode23_predecompose_vector_coeffs_q_bound md sd : polyveck_coeffs_q_bound md (haetae_keygen_mode23_predecompose_vector md sd).	haetae_keygen_mode23_predecompose_vector_coeffs_q_bound : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	On coeffs q-bound: contributes to a game-hop or final security inequality.
	1758	lemma		
haetae_keygen_mode23_public_vector_polyq_wf		lemma haetae_keygen_mode23_public_vector_polyq_wf md sd : (md = Mode2 $\vee$ md = Mode3) => haetae_polyveck_polyq_coeffs_wf md (haetae_keygen_mode23_public_vector md sd).	haetae_keygen_mode23_public_vector_polyq_wf : $B \Rightarrow Q$ is an implication used as a proof obligation.	Proves a structural correctness fact u justify later reductions.
	1769	lemma		
haetae_keygen_mode23_rounding_lowbits_wf		lemma haetae_keygen_mode23_rounding_lowbits_wf md sd : polyveck_wf md (haetae_keygen_mode23_rounding_lowbits md sd).	haetae_keygen_mode23_rounding_lowbits_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.

	Line	Construct	What was written	Symbolic correspondence
	1776	lemma		
haetae_keygen_mode23_adjusted_error_vector_wf		lemma haetae_keygen_mode23_adjusted_error_vector_wf md sd : polyveck_wf md (haetae_keygen_mode23_adjusted_error_vector md sd).	haetae_keygen_mode23_adjusted_error_vector_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	1783	lemma		
haetae_keygen_mode5_public_vector_wf		lemma haetae_keygen_mode5_public_vector_wf sd : polyveck_wf Mode5 (haetae_keygen_mode5_public_vector Mode5 sd).	haetae_keygen_mode5_public_vector_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	1790	lemma		
haetae_keygen_mode5_public_vector_coeffs_q_bound		lemma haetae_keygen_mode5_public_vector_coeffs_q_bound sd : polyveck_coeffs_q_bound Mode5 (haetae_keygen_mode5_public_vector Mode5 sd).	haetae_keygen_mode5_public_vector_coeffs_q_bound : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Prove bound that contributes to a game-hop or final security inequality.
	1798	lemma		
haetae_keygen_mode5_public_vector_polyq_wf		lemma haetae_keygen_mode5_public_vector_polyq_wf sd : haetae_polyveck_polyq_coeffs_wf Mode5 (haetae_keygen_mode5_public_vector Mode5 sd).	haetae_keygen_mode5_public_vector_polyq_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	1807	lemma		
haetae_public_key_vector_from_relation_polyq_wf		lemma haetae_public_key_vector_from_relation_polyq_wf md sd s e : haetae_polyveck_polyq_coeffs_wf md (haetae_public_key_vector_from_relation md sd s e).	haetae_public_key_vector_from_relation_polyq_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	1824	lemma		

	Line	Construct	What was written	Symbolic correspondence
haetae_reference_keygen_public_vector_polyq_wf		lemma haetae_reference_keygen_public_vector_polyq_wf : md sd : haetae_polyveck_polyq_coeffs_wf md (haetae_reference_keygen_public_vector md sd).	haetae_reference_keygen_public_vector_polyq_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	1832	lemma		
haetae_keygen_public_vector_polyq_wf		lemma haetae_keygen_public_vector_polyq_wf : md sd : haetae_polyveck_polyq_coeffs_wf md (haetae_keygen_public_vector md sd).	haetae_keygen_public_vector_polyq_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	1839	lemma		
public_key_vector_seed_wf		lemma public_key_vector_seed_wf md sd : polyveck_wf md (public_key_vector_seed md sd).	public_key_vector_seed_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	1846	lemma		
public_key_mode23_column_wf		lemma public_key_mode23_column_wf md pk : polyveck_wf md (public_key_mode23_column md pk).	public_key_mode23_column_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	1851	lemma		
haetae_verification_column_from_unpacked_wf		lemma haetae_verification_column_from_unpacked_wf : md sd b : polyveck_wf md (haetae_verification_column_from_unpacked md sd b).	haetae_verification_column_from_unpacked_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	1865	lemma		
public_key_verification_column_wf		lemma public_key_verification_column_wf md pk : polyveck_wf md (public_key_verification_column md pk).	public_key_verification_column_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	1879	lemma		
public_verification_matrix_size		lemma public_verification_matrix_size md pk : size (public_verification_matrix md pk) = mode_k md.	public_verification_matrix_size : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose

	Line	Construct	What was written	Symbolic correspondence
	1886	lemma public_verification_matrix_rows_wf lemma public_verification_matrix_rows_wf md md pk : all (polyvecl_wf md) (public_verification_matrix md pk).	public_verification_matrix_rows_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	1903	lemma challenge_embedding_wf lemma challenge_embedding_wf md ch : challenge_wf ch => polyvecl_wf md (challenge_embedding md ch).	challenge_embedding_wf : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a bound that contributes to a game-hop or final security inequality.
	1916	lemma public_key_of_secret_wf lemma public_key_of_secret_wf md sk : polyveck_wf md (public_key_of_secret md sk).‘2.	public_key_of_secret_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	1920	lemma haetae_pack_seed_size lemma haetae_pack_seed_size sd : size (haetae_pack_seed sd) = seedbytes.	haetae_pack_seed_size : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	1924	lemma haetae_unpack_seed_size lemma haetae_unpack_seed_size enc : size (haetae_unpack_seed enc) = seedbytes.	haetae_unpack_seed_size : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	1928	lemma haetae_unpack_seed_pack lemma haetae_unpack_seed_pack sd : size sd = seedbytes => haetae_unpack_seed (haetae_pack_seed sd) = sd.	haetae_unpack_seed_pack : $P \Rightarrow Q$ is an implication used as a proof obligation.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	1942	lemma haetae_unpack_seed_pack_cat lemma haetae_unpack_seed_pack_cat sd tail : size sd = seedbytes => haetae_unpack_seed (haetae_pack_seed sd ++ tail) = sd.	haetae_unpack_seed_pack_cat : $P \Rightarrow Q$ is an implication used as a proof obligation.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	1955	lemma haetae_polyq_coeff_bound_gt0 lemma haetae_polyq_coeff_bound_gt0 md : 0 < haetae_polyq_coeff_bound md.	haetae_polyq_coeff_bound_gt0 : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.
	1962	lemma		

	Line	Construct	What was written	Symbolic correspondence
		<code>lemma haetae_byte_norm_bounds x</code> <code>: 0 <= haetae_byte_norm x <</code> <code>haetae_byte_modulus.</code>	<code>haetae_byte_norm_bounds</code> : $X \leq Y$ is an arithmetic or probability upper bound.	Proves a bound that contributes to a game-hop or final security inequality.
	1966	lemma		
		<code>lemma haetae_byte_norm_small x :</code> <code>0 <= x < haetae_byte_modulus =></code> <code>haetae_byte_norm x = x.</code>	<code>haetae_byte_norm_small</code> : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	1970	lemma		
		<code>lemma</code> <code>haetae_byte_norm_idempotent x :</code> <code>haetae_byte_norm</code> <code>(haetae_byte_norm x) =</code> <code>haetae_byte_norm x.</code>	<code>haetae_byte_norm_idempotent</code> : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	1974	lemma		
		<code>lemma haetae_unpack15_coeff0 c0</code> <code>c1 : 0 <= c0 < 32768 => 0 <= c1</code> <code>< 32768 => haetae_byte_norm c0 +</code> <code>256 * (haetae_byte_norm ((c0 %/</code> <code>256) %% 128 + (c1 %% 2) * 128)</code> <code>%% 128) = c0.</code>	<code>haetae_unpack15_coeff0</code> : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	1982	lemma		
		<code>lemma haetae_unpack15_coeff1 c0</code> <code>c1 c2 : 0 <= c0 < 32768 => 0 <=</code> <code>c1 < 32768 => 0 <= c2 < 32768 =></code> <code>((haetae_byte_norm ((c0 %/ 256)</code> <code>%% 128 + (c1 %% 2) * 128) %/</code> <code>128) %% 2) + 2 *</code> <code>haetae_byte_norm (c1 %/ 2) + 512</code> <code>* (haetae_byte_norm ((c1 %/ 512)</code> <code>%% 64 + (c2 %% 4) * 64) %% 64) =</code> <code>c1.</code>	<code>haetae_unpack15_coeff1</code> : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	1992	lemma		
		<code>lemma haetae_unpack15_coeff2 c1</code> <code>c2 c3 : 0 <= c1 < 32768 => 0 <=</code> <code>c2 < 32768 => 0 <= c3 < 32768 =></code> <code>((haetae_byte_norm ((c1 %/ 512)</code> <code>%% 64 + (c2 %% 4) * 64) %/ 64)</code> <code>%% 4) + 4 * haetae_byte_norm (c2</code> <code>%/ 4) + 1024 * (haetae_byte_norm</code> <code>((c2 %/ 1024) %% 32 + (c3 %% 8)</code> <code>* 32) %% 32) = c2.</code>	<code>haetae_unpack15_coeff2</code> : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later p scripts can rewrite, apply, or compose

	Line	Construct	What was written	Symbolic correspondence
	2002	lemma		
haetae_unpack15_coeff3		<pre> lemma haetae_unpack15_coeff3 c2 c3 c4 : 0 <= c2 < 32768 => 0 <= c3 < 32768 => 0 <= c4 < 32768 => ((haetae_byte_norm ((c2 %/ 1024) %% 32 + (c3 %% 8) * 32) %/ 32) %% 8) + 8 * haetae_byte_norm (c3 %/ 8) + 2048 * (haetae_byte_norm ((c3 %/ 2048) %% 16 + (c4 %% 16) * 16) %% 16) = c3. </pre>	haetae_unpack15_coeff3 : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	2012	lemma		
haetae_unpack15_coeff4		<pre> lemma haetae_unpack15_coeff4 c3 c4 c5 : 0 <= c3 < 32768 => 0 <= c4 < 32768 => 0 <= c5 < 32768 => ((haetae_byte_norm ((c3 %/ 2048) %% 16 + (c4 %% 16) * 16) %/ 16) %% 16) + 16 * haetae_byte_norm (c4 %/ 16) + 4096 * (haetae_byte_norm ((c4 %/ 4096) %% 8 + (c5 %% 32) * 8) %% 8) = c4. </pre>	haetae_unpack15_coeff4 : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	2022	lemma		
haetae_unpack15_coeff5		<pre> lemma haetae_unpack15_coeff5 c4 c5 c6 : 0 <= c4 < 32768 => 0 <= c5 < 32768 => 0 <= c6 < 32768 => ((haetae_byte_norm ((c4 %/ 4096) %% 8 + (c5 %% 32) * 8) %/ 8) %% 32) + 32 * haetae_byte_norm (c5 %/ 32) + 8192 * (haetae_byte_norm ((c5 %/ 8192) %% 4 + (c6 %% 64) * 4) %% 4) = c5. </pre>	haetae_unpack15_coeff5 : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	2032	lemma		

	Line	Construct	What was written	Symbolic correspondence
		<pre> lemma haetae_unpack15_coeff6 c5 c6 c7 : 0 <= c5 < 32768 => 0 <= c6 < 32768 => 0 <= c7 < 32768 => ((haetae_byte_norm ((c5 %/ 8192) %% 4 + (c6 %% 64) * 4) %/ 4) %% 64) + 64 * haetae_byte_norm (c6 %/ 64) + 16384 * (haetae_byte_norm ((c6 %/ 16384) %% 2 + (c7 %% 128) * 2) %% 2) = c6. </pre>	<code>haetae_unpack15_coeff6</code> : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	2042	lemma		
		<pre> lemma haetae_unpack15_coeff7 c6 c7 : 0 <= c6 < 32768 => 0 <= c7 < 32768 => ((haetae_byte_norm ((c6 %/ 16384) %% 2 + (c7 %% 128) * 2) %/ 2) %% 128) + 128 * haetae_byte_norm (c7 %/ 128) = c7. </pre>	<code>haetae_unpack15_coeff7</code> : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	2050	lemma		
		<pre> lemma haetae_pack_poly_q_ref_size md p : size (haetae_pack_poly_q_ref md p) = mode_polyq_packedbytes md. </pre>	<code>haetae_pack_poly_q_ref_size</code> : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	2057	lemma		
		<pre> lemma haetae_unpack_poly_q_ref_at_wf md enc offset : poly_wf (haetae_unpack_poly_q_ref_at md enc offset). </pre>	<code>haetae_unpack_poly_q_ref_at_wf</code> : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	2064	lemma		
		<pre> lemma haetae_pack_polyveck_body_ref_size md b : size (haetae_pack_polyveck_body_ref md b) = haetae_public_key_body_bytes md. </pre>	<code>haetae_pack_polyveck_body_ref_size</code> : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	2072	lemma		

	Line	Construct	What was written	Symbolic correspondence
haetae_unpack_polyveck_body_ref_wf		lemma haetae_unpack_polyveck_body_ref_wf md enc offset : polyveck_wf md (haetae_unpack_polyveck_body_ref md enc offset).	haetae_unpack_polyveck_body_ref_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	2082	lemma		
haetae_pack_vk_ref_size		lemma haetae_pack_vk_ref_size md b sd : size (haetae_pack_vk_ref md b sd) = haetae_public_key_bytes md.	haetae_pack_vk_ref_size : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	2091	lemma		
haetae_unpack_vk_public_key_ref_wf		lemma haetae_unpack_vk_public_key_ref_wf md enc : haetae_public_key_unpacked_wf md (haetae_unpack_vk_public_key_ref md enc).	haetae_unpack_vk_public_key_ref_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	2100	lemma		
nth_cat_size_plus		lemma nth_cat_size_plus ['a] (d : 'a) (xs ys : 'a list) j : 0 <= j => nth d (xs ++ ys) (size xs + j) = nth d ys j.	nth_cat_size_plus : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	2105	lemma		
haetae_pack_polyveck_body_ref_nth		lemma haetae_pack_polyveck_body_ref_nth md b row k : 0 <= row < mode_k md => 0 <= k < mode_polyq_packedbytes md => nth 0 (haetae_pack_polyveck_body_ref md b) (row * mode_polyq_packedbytes md + k) = nth 0 (haetae_pack_poly_q_ref md (nth poly_zero b row)) k.	haetae_pack_polyveck_body_ref_nth : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	2121	lemma		

	Line	Construct	What was written	Symbolic correspondence
haetae_pack_polyveck_body_ref_mode5_nth		lemma	haetae_pack_polyveck_body_ref_mode5_nth	Records a reusable theorem so later p
		haetae_pack_polyveck_body_ref_mode5_nth	$X \leq Y$ is an arithmetic or probability	scripts can rewrite, apply, or compose
		b row k : 0 <= row < mode_k	upper bound.	
		Mode5 => 0 <= k <		
		mode_polyq_packedbytes Mode5 =>		
		nth 0		
		(haetae_pack_polyveck_body_ref		
		Mode5 b) (row *		
		mode_polyq_packedbytes Mode5 +		
		k) = nth 0		
		(haetae_pack_poly_q_ref Mode5		
		(nth poly_zero b row)) k.		
	2131	lemma		
haetae_unpack_poly_q_mode23_coeff_at_cat		lemma	haetae_unpack_poly_q_mode23_coeff_at_cat	Records a reusable theorem so later p
		haetae_unpack_poly_q_mode23_coeff_at_cat	$X \leq Y$ is an arithmetic or probability	scripts can rewrite, apply, or compose
		prefix bytes blk ofs : 0 <= blk	upper bound.	
		=>		
		haetae_unpack_poly_q_mode23_coeff_at		
		(prefix ++ bytes) (size prefix)		
		blk ofs =		
		haetae_unpack_poly_q_mode23_coeff_at		
		bytes 0 blk ofs.		
	2142	lemma		
haetae_polyq_mode23_coeff_rng		lemma	haetae_polyq_mode23_coeff_rng : $X \leq$	Records a reusable theorem so later p
		haetae_polyq_mode23_coeff_rng md	Y is an arithmetic or probability upper	scripts can rewrite, apply, or compose
		p j : (md = Mode2 \/\ md =	bound.	
		Mode3) => haetae_polyq_coeffs_wf		
		md p => 0 <= j < n => 0 <=		
		poly_coeff p j < 32768.		
	2155	lemma		
haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs0		lemma	haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs0	Records a reusable theorem so later p
		haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs0	$X \leq Y$ is an arithmetic or probability	scripts can rewrite, apply, or compose
		md p blk : (md = Mode2 \/\ md =	upper bound.	
		Mode3) => haetae_polyq_coeffs_wf		
		md p => 0 <= blk < 32 =>		
		haetae_unpack_poly_q_mode23_coeff_at		
		(haetae_pack_poly_q_ref md p) 0		
		blk 0 = poly_coeff p (8 * blk +		
		0).		
	2182	lemma		

	Line	Construct	What was written	Symbolic correspondence
		<pre> haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs1 lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs1 md p blk : (md = Mode2 $\vee$ md = Mode3) => haetae_polyq_coeffs_wf md p => 0 <= blk < 32 => haetae_unpack_poly_q_mode23_coeff_at (haetae_pack_poly_q_ref md p) 0 blk 1 = poly_coeff p (8 * blk + 1). </pre>	<pre> haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs1 $X \leq Y$ is an arithmetic or probability upper bound. </pre>	Rcoeffofs1 reusable theorem so later p scripts can rewrite, apply, or compose
	2211	lemma		
		<pre> haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs2 lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs2 md p blk : (md = Mode2 $\vee$ md = Mode3) => haetae_polyq_coeffs_wf md p => 0 <= blk < 32 => haetae_unpack_poly_q_mode23_coeff_at (haetae_pack_poly_q_ref md p) 0 blk 2 = poly_coeff p (8 * blk + 2). </pre>	<pre> haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs2 $X \leq Y$ is an arithmetic or probability upper bound. </pre>	Rcoeffofs2 reusable theorem so later p scripts can rewrite, apply, or compose
	2240	lemma		
		<pre> haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs3 lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs3 md p blk : (md = Mode2 $\vee$ md = Mode3) => haetae_polyq_coeffs_wf md p => 0 <= blk < 32 => haetae_unpack_poly_q_mode23_coeff_at (haetae_pack_poly_q_ref md p) 0 blk 3 = poly_coeff p (8 * blk + 3). </pre>	<pre> haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs3 $X \leq Y$ is an arithmetic or probability upper bound. </pre>	Rcoeffofs3 reusable theorem so later p scripts can rewrite, apply, or compose
	2269	lemma		
		<pre> haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs4 lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs4 md p blk : (md = Mode2 $\vee$ md = Mode3) => haetae_polyq_coeffs_wf md p => 0 <= blk < 32 => haetae_unpack_poly_q_mode23_coeff_at (haetae_pack_poly_q_ref md p) 0 blk 4 = poly_coeff p (8 * blk + 4). </pre>	<pre> haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs4 $X \leq Y$ is an arithmetic or probability upper bound. </pre>	Rcoeffofs4 reusable theorem so later p scripts can rewrite, apply, or compose
	2298	lemma		

	Line	Construct	What was written	Symbolic correspondence
		<pre> lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs5 md p blk : (md = Mode2 $\vee$ md = Mode3) => haetae_polyq_coeffs_wf md p => 0 <= blk < 32 => haetae_unpack_poly_q_mode23_coeff_at (haetae_pack_poly_q_ref md p) 0 blk 5 = poly_coeff p (8 * blk + 5). </pre>	<pre> haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs5 X ≤ Y is an arithmetic or probability upper bound. </pre>	Rcoeffofs5 reusable theorem so later p scripts can rewrite, apply, or compose
	2327	lemma		
		<pre> lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs6 md p blk : (md = Mode2 $\vee$ md = Mode3) => haetae_polyq_coeffs_wf md p => 0 <= blk < 32 => haetae_unpack_poly_q_mode23_coeff_at (haetae_pack_poly_q_ref md p) 0 blk 6 = poly_coeff p (8 * blk + 6). </pre>	<pre> haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs6 X ≤ Y is an arithmetic or probability upper bound. </pre>	Rcoeffofs6 reusable theorem so later p scripts can rewrite, apply, or compose
	2356	lemma		
		<pre> lemma haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs7 md p blk : (md = Mode2 $\vee$ md = Mode3) => haetae_polyq_coeffs_wf md p => 0 <= blk < 32 => haetae_unpack_poly_q_mode23_coeff_at (haetae_pack_poly_q_ref md p) 0 blk 7 = poly_coeff p (8 * blk + 7). </pre>	<pre> haetae_unpack_poly_q_ref_pack_mode23_coeff_ofs7 X ≤ Y is an arithmetic or probability upper bound. </pre>	Rcoeffofs7 reusable theorem so later p scripts can rewrite, apply, or compose
	2383	lemma		
		<pre> lemma haetae_unpack_poly_q_ref_pack_mode23_coeff md p i : (md = Mode2 $\vee$ md = Mode3) => haetae_polyq_coeffs_wf md p => 0 <= i < n => haetae_unpack_poly_q_mode23_coeff_at (haetae_pack_poly_q_ref md p) 0 (i % 8) (i % 8) = poly_coeff p i. </pre>	<pre> haetae_unpack_poly_q_ref_pack_mode23_coeff X ≤ Y is an arithmetic or probability upper bound. </pre>	Rcoeff ds a reusable theorem so later p scripts can rewrite, apply, or compose
	2446	lemma		

	Line	Construct	What was written	Symbolic correspondence
haetae_unpack_poly_q_ref_pack_mode23_at		lemma	haetae_unpack_poly_q_ref_pack_mode23	Records a reusable theorem so later p
		haetae_unpack_poly_q_ref_pack_mode23	$P \Rightarrow Q$ is an implication used as a proof	scripts can rewrite, apply, or compose
		md prefix p : (md = Mode2 $\vee$ md = Mode3) =>	obligation.	
		haetae_polyq_coeffs_wf md p =>		
		haetae_unpack_poly_q_ref_at md		
		(prefix ++		
		haetae_pack_poly_q_ref md p)		
		(size prefix) = p.		
	2479	lemma		
haetae_unpack_poly_q_mode23_coeff_at_body		lemma	haetae_unpack_poly_q_mode23_coeff_at	Records a reusable theorem so later p
		haetae_unpack_poly_q_mode23_coeff_at_body	$X \leq Y$ is an arithmetic or probability	scripts can rewrite, apply, or compose
		md prefix b row blk ofs : (md = Mode2 $\vee$ md = Mode3) => 0 <= row	upper bound.	
		< mode_k md => 0 <= blk < 32 =>		
		haetae_unpack_poly_q_mode23_coeff_at		
		(prefix ++		
		haetae_pack_polyveck_body_ref md		
		b) (size prefix + row *		
		mode_polyq_packedbytes md) blk		
		ofs =		
		haetae_unpack_poly_q_mode23_coeff_at		
		(haetae_pack_poly_q_ref md (nth		
		P...		
	2495	lemma		
haetae_unpack_polyveck_body_ref_pack_mode23_coeff		lemma	haetae_unpack_polyveck_body_ref_pack_mode23	Records a reusable theorem so later p
		haetae_unpack_polyveck_body_ref_pack_mode23	$X \leq Y$ is an arithmetic or probability	scripts can rewrite, apply, or compose
		md prefix b row i : (md = Mode2 $\vee$ md = Mode3) =>	upper bound.	
		haetae_polyveck_polyq_coeffs_wf		
		md b => 0 <= row < mode_k md =>		
		0 <= i < n =>		
		haetae_unpack_poly_q_mode23_coeff_at		
		(prefix ++		
		haetae_pack_polyveck_body_ref md		
		b) (size prefix + row *		
		mode_polyq_packedbytes md) (i %/		
		8) (i %% 8) = poly_coeff (nth		
		poly...		
	2518	lemma		

	Line	Construct	What was written	Symbolic correspondence
		haetae_unpack_polyveck_body_ref_pack_mode23	lemma	haetae_unpack_polyveck_body_ref_pack_mode23
		haetae_unpack_polyveck_body_ref_pack_mode23	$P \Rightarrow Q$ is an implication used as a proof obligation.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
		md prefix b : (md = Mode2 $\vee$ md = Mode3) =>		
		haetae_polyveck_polyq_coeffs_wf		
		md b =>		
		haetae_unpack_polyveck_body_ref		
		md (prefix ++		
		haetae_pack_polyveck_body_ref md		
		b) (size prefix) = b.		
	2567	lemma		
		haetae_pack_vk_ref_roundtrip_mode23	haetae_pack_vk_ref_roundtrip_mode23 :	Records a reusable theorem so later p scripts can rewrite, apply, or compose
		haetae_pack_vk_ref_roundtrip_mode23	$P \Rightarrow Q$ is an implication used as a proof obligation.	
		md sd b : (md = Mode2 $\vee$ md = Mode3) => size sd = seedbytes =>		
		haetae_polyveck_polyq_coeffs_wf		
		md b =>		
		haetae_unpack_vk_public_key_ref		
		md (haetae_pack_vk_ref md b sd)		
		= (sd, b).		
	2585	lemma		
		haetae_unpack_poly_q_ref_pack_mode5_coeff	haetae_unpack_poly_q_ref_pack_mode5_coeff	Records a reusable theorem so later p scripts can rewrite, apply, or compose
		haetae_unpack_poly_q_ref_pack_mode5_coeff	$X \Leftarrow Y$ is an arithmetic or probability upper bound.	
		p i : haetae_polyq_coeffs_wf		
		Mode5 p => 0 <= i < n =>		
		haetae_unpack_poly_q_mode5_coeff_at		
		(haetae_pack_poly_q_ref Mode5 p)		
		0 i = poly_coeff p i.		
	2603	lemma		
		haetae_unpack_poly_q_ref_pack_mode5_at	haetae_unpack_poly_q_ref_pack_mode5_at	Records a reusable theorem so later p scripts can rewrite, apply, or compose
		haetae_unpack_poly_q_ref_pack_mode5_at	$P \Rightarrow Q$ is an implication used as a proof obligation.	
		prefix p :		
		haetae_polyq_coeffs_wf Mode5 p		
		=> haetae_unpack_poly_q_ref_at		
		Mode5 (prefix ++		
		haetae_pack_poly_q_ref Mode5 p)		
		(size prefix) = p.		
	2628	lemma		

	Line	Construct	What was written	Symbolic correspondence
		<pre> lemma haetae_unpack_polyveck_body_ref_pack_mode5_coeff haetae_unpack_polyveck_body_ref_pack_mode5_coeff prefix b row i : haetae_polyveck_polyq_coeffs_wf Mode5 b => 0 <= row < mode_k Mode5 => 0 <= i < n => haetae_unpack_poly_q_mode5_coeff_at (prefix ++ haetae_pack_polyveck_body_ref Mode5 b) (size prefix + row * mode_polyq_packedbytes Mode5) i = poly_coeff (nth poly_zero b row) i. </pre>	<pre> haetae_unpack_polyveck_body_ref_pack_mode5_coeff $X \leq Y$ is an arithmetic or probability upper bound. </pre>	<pre> Mode5_coeff Records a reusable theorem so later p scripts can rewrite, apply, or compose </pre>
	2684	<pre> lemma haetae_unpack_polyveck_body_ref_pack_mode5 haetae_unpack_polyveck_body_ref_pack_mode5 prefix b : haetae_polyveck_polyq_coeffs_wf Mode5 b => haetae_unpack_polyveck_body_ref Mode5 (prefix ++ haetae_pack_polyveck_body_ref Mode5 b) (size prefix) = b. </pre>	<pre> haetae_unpack_polyveck_body_ref_pack_mode5 $P \Rightarrow Q$ is an implication used as a proof obligation. </pre>	<pre> Mode5 Records a reusable theorem so later p scripts can rewrite, apply, or compose </pre>
	2715	<pre> lemma haetae_pack_vk_ref_roundtrip_mode5 haetae_pack_vk_ref_roundtrip_mode5 sd b : size sd = seedbytes => haetae_polyveck_polyq_coeffs_wf Mode5 b => haetae_unpack_vk_public_key_ref Mode5 (haetae_pack_vk_ref Mode5 b sd) = (sd, b). </pre>	<pre> haetae_pack_vk_ref_roundtrip_mode5 : $P \Rightarrow Q$ is an implication used as a proof obligation. </pre>	<pre> haetae_pack_vk_ref_roundtrip_mode5 Records a reusable theorem so later p scripts can rewrite, apply, or compose </pre>
	2732	<pre> lemma </pre>		

	Line	Construct	What was written	Symbolic correspondence
haetae_public_key_ref_roundtrip		<pre> lemma haetae_public_key_ref_roundtrip md (pk : pkey) : size pk.'1 = seedbytes => haetae_polyveck_polyq_coeffs_wf md pk.'2 => haetae_unpack_public_key_bytes_ref md (haetae_pack_public_key_bytes_ref md pk) = pk. </pre>	$\text{haetae_public_key_ref_roundtrip} : P \Rightarrow Q$ is an implication used as a proof obligation.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
2755		lemma		
haetae_keygen_public_key_ref_roundtrip		<pre> lemma haetae_keygen_public_key_ref_roundtrip md sd : size sd = seedbytes => haetae_unpack_vk_public_key_ref md (haetae_pack_vk_ref md (haetae_keygen_public_vector md sd) sd) = packed_haetae_public_key md sd. </pre>	$\text{haetae_keygen_public_key_ref_roundtrip} : P \Rightarrow Q$ is an implication used as a proof obligation.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
2771		lemma		
haetae_pack_polyveck_body_size		<pre> lemma haetae_pack_polyveck_body_size md b : size (haetae_pack_polyveck_body md b) = haetae_public_key_body_bytes md. </pre>	$\text{haetae_pack_polyveck_body_size} : L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
2778		lemma		
haetae_pack_poly_q_size		<pre> lemma haetae_pack_poly_q_size md p : size (haetae_pack_poly_q md p) = mode_polyq_packedbytes md. </pre>	$\text{haetae_pack_poly_q_size} : L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
2782		lemma		
haetae_unpack_poly_q_at_wf		<pre> lemma haetae_unpack_poly_q_at_wf md enc offset : poly_wf (haetae_unpack_poly_q_at md enc offset). </pre>	$\text{haetae_unpack_poly_q_at_wf} : \forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
2786		lemma		
haetae_unpack_poly_q_pack_at		<pre> lemma haetae_unpack_poly_q_pack_at md prefix p : poly_wf p => haetae_unpack_poly_q_at md (prefix ++ haetae_pack_poly_q md p) (size prefix) = p. </pre>	$\text{haetae_unpack_poly_q_pack_at} : P \Rightarrow Q$ is an implication used as a proof obligation.	Records a reusable theorem so later p scripts can rewrite, apply, or compose

	Line	Construct	What was written	Symbolic correspondence
	2803	lemma		
haetae_unpack_polyveck_body_wf		lemma haetae_unpack_polyveck_body_wf md enc offset : polyveck_wf md (haetae_unpack_polyveck_body md enc offset).	haetae_unpack_polyveck_body_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	2813	lemma		
haetae_body_index_div		lemma haetae_body_index_div md row col : 0 <= row => 0 <= col < n => (row * mode_polyq_packedbytes md + col) %/ mode_polyq_packedbytes md = row.	haetae_body_index_div : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	2820	lemma		
haetae_body_index_mod		lemma haetae_body_index_mod md row col : 0 <= row => 0 <= col < n => (row * mode_polyq_packedbytes md + col) %% mode_polyq_packedbytes md = col.	haetae_body_index_mod : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	2827	lemma		
haetae_body_index_lt		lemma haetae_body_index_lt md row col : 0 <= row < mode_k md => 0 <= col < n => row * mode_polyq_packedbytes md + col < haetae_public_key_body_bytes md.	haetae_body_index_lt : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	2837	lemma		
haetae_unpack_polyveck_body_pack_at		lemma haetae_unpack_polyveck_body_pack_at md prefix b : polyveck_wf md b => haetae_unpack_polyveck_body md (prefix ++ haetae_pack_polyveck_body md b) (size prefix) = b.	haetae_unpack_polyveck_body_pack_at : $P \Rightarrow Q$ is an implication used as a proof obligation.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	2860	lemma		

	Line	Construct	What was written	Symbolic correspondence
haetae_unpack_polyveck_body_pack		lemma haetae_unpack_polyveck_body_pack md sd b : polyveck_wf md b => haetae_unpack_polyveck_body md (haetae_pack_seed sd ++ haetae_pack_polyveck_body md b) seedbytes = b.	haetae_unpack_polyveck_body_pack : $P \Rightarrow Q$ is an implication used as a proof obligation.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	2872	lemma		
haetae_public_key_roundtrip		lemma haetae_public_key_roundtrip md pk : haetae_public_key_unpacked_wf md pk => haetae_public_key_roundtrip_ok md pk.	haetae_public_key_roundtrip : $P \Rightarrow Q$ is an implication used as a proof obligation.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	2886	lemma		
haetae_pack_public_key_bytes_size		lemma haetae_pack_public_key_bytes_size md pk : size (haetae_pack_public_key_bytes md pk) = haetae_public_key_bytes md.	haetae_pack_public_key_bytes_size : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	2894	lemma		
haetae_pack_public_key_bytes_wf		lemma haetae_pack_public_key_bytes_wf md pk : haetae_public_key_encoding_wf md (haetae_pack_public_key_bytes md pk).	haetae_pack_public_key_bytes_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	2900	lemma		
haetae_pack_public_key_bytes_ref_size		lemma haetae_pack_public_key_bytes_ref_size md pk : size (haetae_pack_public_key_bytes_ref md pk) = haetae_public_key_bytes md.	haetae_pack_public_key_bytes_ref_size : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	2907	lemma		

	Line	Construct	What was written	Symbolic correspondence
haetae_pack_public_key_bytes_ref_wf		lemma	haetae_pack_public_key_bytes_ref_wf :	Proves a structural correctness fact u
		haetae_pack_public_key_bytes_ref_wf	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	justify later reductions.
		md pk :		
		haetae_public_key_encoding_wf md		
		(haetae_pack_public_key_bytes_ref		
		md pk).		
	2915	lemma		
haetae_unpack_public_key_bytes_wf		lemma	haetae_unpack_public_key_bytes_wf :	Proves a structural correctness fact u
		haetae_unpack_public_key_bytes_wf	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	justify later reductions.
		md enc :		
		haetae_public_key_unpacked_wf md		
		(haetae_unpack_public_key_bytes		
		md enc).		
	2926	lemma		
haetae_unpack_public_key_bytes_ref_wf		lemma	haetae_unpack_public_key_bytes_ref_wf :	Proves a structural correctness fact u
		haetae_unpack_public_key_bytes_ref_wf	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	justify later reductions.
		md enc :		
		haetae_public_key_unpacked_wf md		
		(haetae_unpack_public_key_bytes_ref		
		md enc).		
	2934	lemma		
concrete_haetae_keygen_packed_public_key_wf		lemma	concrete_haetae_keygen_packed_public_key_wf :	Proves a structural correctness fact u
		concrete_haetae_keygen_packed_public_key_wf	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	justify later reductions.
		md sd :		
		haetae_public_key_encoding_wf md		
		(concrete_haetae_keygen_packed_public_key		
		md sd).		
	2942	lemma		
concrete_haetae_keygen_unpacked_public_key_wf		lemma	concrete_haetae_keygen_unpacked_public_key_wf :	Proves a structural correctness fact u
		concrete_haetae_keygen_unpacked_public_key_wf	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	justify later reductions.
		md sd :		
		haetae_public_key_unpacked_wf md		
		(concrete_haetae_keygen_unpacked_public_key		
		md sd).		
	2950	lemma		
concrete_haetae_keygen_packed_public_key_ref_wf		lemma	concrete_haetae_keygen_packed_public_key_ref_wf :	Proves a structural correctness fact u
		concrete_haetae_keygen_packed_public_key_ref_wf	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	justify later reductions.
		md sd :		
		haetae_public_key_encoding_wf md		
		(concrete_haetae_keygen_packed_public_key_ref		
		md sd).		

	Line	Construct	What was written	Symbolic correspondence
	2958	lemma		
concrete_haetae_keygen_unpacked_public_key_ref_wf		lemma	concrete_haetae_keygen_unpacked_public_key_ref_wf	Proves a structural correctness fact u
		concrete_haetae_keygen_unpacked_public_key_ref_wf	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	justify later reductions.
		md sd :		
		haetae_public_key_unpacked_wf md		
		(concrete_haetae_keygen_unpacked_public_key_ref		
		md sd).		
	2966	lemma		
packed_haetae_reference_public_key_wf		lemma	packed_haetae_reference_public_key_wf :	Proves a structural correctness fact u
		packed_haetae_reference_public_key_wf	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	justify later reductions.
		md sd : polyveck_wf md		
		(packed_haetae_reference_public_key		
		md sd). '2.		
	2974	lemma		
packed_haetae_reference_public_key_polyq_wf		lemma	packed_haetae_reference_public_key_polyq_wf	Proves a structural correctness fact u
		packed_haetae_reference_public_key_polyq_wf	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	justify later reductions.
		md sd :		
		haetae_polyveck_polyq_coeffs_wf		
		md		
		(packed_haetae_reference_public_key		
		md sd). '2.		
	2982	lemma		
packed_haetae_reference_public_key_unpacked_wf		lemma	packed_haetae_reference_public_key_unpacked_wf	Proves a structural correctness fact u
		packed_haetae_reference_public_key_unpacked_wf	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	justify later reductions.
		md sd :		
		haetae_public_key_unpacked_wf md		
		(packed_haetae_reference_public_key		
		md sd).		
	2994	lemma		
concrete_haetae_reference_keygen_packed_public_key_ref_wf		lemma	concrete_haetae_reference_keygen_packed_public_key_ref_wf	Proves a structural correctness fact u
		concrete_haetae_reference_keygen_packed_public_key_ref_wf	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	justify later reductions.
		md sd :		
		haetae_public_key_encoding_wf md		
		(concrete_haetae_reference_keygen_packed_public_key_ref		
		md sd).		
	3002	lemma		
concrete_haetae_reference_keygen_unpacked_public_key_ref_wf		lemma	concrete_haetae_reference_keygen_unpacked_public_key_ref_wf	Proves a structural correctness fact u
		concrete_haetae_reference_keygen_unpacked_public_key_ref_wf	$\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	justify later reductions.
		md sd :		
		haetae_public_key_unpacked_wf md		
		(concrete_haetae_reference_keygen_unpacked_public_key_ref		
		md sd).		

	Line	Construct	What was written	Symbolic correspondence
haetae_reference_keygen_public_key_ref_roundtrip	3010	lemma lemma haetae_reference_keygen_public_key_ref_roundtrip md sd : haetae_unpack_public_key_bytes_ref md (haetae_pack_public_key_bytes_ref md (packed_haetae_reference_public_key md sd)) = packed_haetae_reference_public_key md sd.	haetae_reference_keygen_public_key_ref_roundtrip $\text{Def-}P$ is an equality/rewriting fact.	Def-}P : a reusable theorem so later p scripts can rewrite, apply, or compose
public_key_of_secret_relation	3022	lemma lemma public_key_of_secret_relation md sk : public_key_relation md (public_key_of_secret md sk) (public_secret_vector_seed md sk.'1) (public_error_vector_seed md sk.'1).	public_key_of_secret_relation : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
public_key_of_secret_equation_holds	3033	lemma lemma public_key_of_secret_equation_holds md sk : public_key_equation_holds md (public_key_of_secret md sk).	public_key_of_secret_equation_holds : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
valid_mode	3039	operation op valid_mode : mode -> bool = fun _ => true.	valid_mode : $1 \rightarrow \text{mode} \rightarrow \text{bool}$	Defines a total mathematical function by the EasyCrypt model.
valid_keypair	3040	operation op valid_keypair (md : mode) (pk : pkey) (sk : skey) : bool = pk = public_key_of_secret md sk.	valid_keypair $\subseteq \text{mode} \times \text{pkey} \times \text{skey}$ is a Boolean-valued predicate.	Names a well-formedness or support condition so it can be reused in lemm
	3042	operation		

	Line	Construct	What was written	Symbolic correspondence
		valid_signature op valid_signature (md : mode) (sig : signature) : bool = polyveck_wf md sig.'1 /\ poly_wf sig.'2 /\ crh_wf sig.'3 /\ challenge_wf sig.'4 /\ polyvecl_wf md (sig.'5).'1 /\ polyveck_wf md (sig.'5).'2 /\ polyveck_wf md (sig.'5).'3.	valid_signature $\subseteq$ mode $\times$ signature is a Boolean-valued predicate.	Names a well-formedness or support condition so it can be reused in lemmas.
	3051	operation		
sig_commitment_highbits		op sig_commitment_highbits (sig : signature) : polyveck = sig.'1.	sig_commitment_highbits : signature $\rightarrow$ polyveck	Defines a total mathematical function by the EasyCrypt model.
	3052	operation		
sig_commitment_lowbits		op sig_commitment_lowbits (sig : signature) : poly = sig.'2.	sig_commitment_lowbits : signature $\rightarrow$ poly	Defines a total mathematical function by the EasyCrypt model.
	3053	operation		
sig_message_hash		op sig_message_hash (sig : signature) : crh = sig.'3.	sig_message_hash : signature $\rightarrow$ crh	Defines the random-oracle/hash interface data used by the games.
	3054	operation		
sig_challenge		op sig_challenge (sig : signature) : challenge = sig.'4.	sig_challenge : signature $\rightarrow$ challenge	Defines a total mathematical function by the EasyCrypt model.
	3055	operation		
sig_token_value		op sig_token_value (sig : signature) : sig_token = sig.'5.	sig_token_value : signature $\rightarrow$ sig_token	Defines a total mathematical function by the EasyCrypt model.
	3056	operation		
sig_response		op sig_response (sig : signature) : polyvecl = (sig_token_value sig).'1.	sig_response : signature $\rightarrow$ polyvecl	Defines a total mathematical function by the EasyCrypt model.
	3057	operation		
sig_response_aux		op sig_response_aux (sig : signature) : polyveck = (sig_token_value sig).'2.	sig_response_aux : signature $\rightarrow$ polyveck	Defines a total mathematical function by the EasyCrypt model.
	3058	operation		
sig_hint		op sig_hint (sig : signature) : hint_t = (sig_token_value sig).'3.	sig_hint : signature $\rightarrow$ hint_t	Defines a total mathematical function by the EasyCrypt model.
	3060	operation		
encode_poly		op encode_poly (p : poly) : byte list = p.	encode_poly : poly $\rightarrow$ List(byte)	Defines a total mathematical function by the EasyCrypt model.

Line	Construct	What was written	Symbolic correspondence
3061	operation		
encode_polyveck	op encode_polyveck (xs : polyveck) : byte list = flatten xs.	encode__polyveck : polyveck $\rightarrow$ List(byte)	Defines a total mathematical function by the EasyCrypt model.
3062	operation		
encode_pkey	op encode_pkey (pk : pkey) : byte list = pk.'1 ++ encode_polyveck pk.'2.	encode__pkey : pkey $\rightarrow$ List(byte)	Defines a total mathematical function by the EasyCrypt model.
3064	operation		
deterministic_poly	op deterministic_poly (tag : int) (src : byte list) : poly = mkseq (fun i => coeff_mod (tag + i + nth 0 src i)) n.	deterministic__poly : $\mathbb{Z} \times \text{List}(\text{byte}) \rightarrow \text{poly}$	Defines a sampler/distribution used by game procedures and probability bounds.
3066	operation		
deterministic_polyveck	op deterministic_polyveck (md : mode) (src : byte list) : polyveck = mkseq (fun i => deterministic_poly i (i :: src)) (mode_k md).	deterministic__polyveck : mode $\times$ List(byte) $\times$: src $\rightarrow$ polyveck	Defines a sampler/distribution used by game procedures and probability bounds.
3070	operation		
deterministic_unit_coeff	op deterministic_unit_coeff (tag : int) (src : byte list) (i : int) : coeff = if (tag + i + nth 0 src i) %% 2 = 0 then 1 else -1.	deterministic__unit__coeff : $\mathbb{Z} \times \text{List}(\text{byte}) \times \mathbb{Z} \rightarrow \text{coeff}$	Defines a sampler/distribution used by game procedures and probability bounds.
3073	operation		
deterministic_unit_poly	op deterministic_unit_poly (tag : int) (src : byte list) : poly = mkseq (fun i => deterministic_unit_coeff tag src i) n.	deterministic__unit__poly : $\mathbb{Z} \times \text{List}(\text{byte}) \rightarrow$ poly	Defines a sampler/distribution used by game procedures and probability bounds.
3075	operation		
deterministic_unit_polyveck	op deterministic_unit_polyveck (md : mode) (tag : int) (src : byte list) : polyveck = mkseq (fun i => deterministic_unit_poly (tag + i) (i :: src)) (mode_k md).	deterministic__unit__polyveck : mode $\times$ $\mathbb{Z} \times \text{List}(\text{byte}) \times$: src $\rightarrow$ polyveck	Defines a sampler/distribution used by game procedures and probability bounds.
3080	operation		

Line	Construct	What was written	Symbolic correspondence
deterministic_unit_polyvecl	<pre> op deterministic_unit_polyvecl (md : mode) (tag : int) (src : byte list) : polyvecl = mkseq (fun i => deterministic_unit_poly (tag + i) (i :: src)) (mode_l md). </pre>	<pre> deterministic_unit_polyvecl : mode $\times$ $\mathbb{Z}$ $\times$ List(byte) $\times$: src $\rightarrow$ polyvecl </pre>	Defines a sampler/distribution used by game procedures and probability bounds
3085	operation		
commitment_source	<pre> op commitment_source (sk : skey) (m : message) (ctx : context) (coins : random_coins) : byte list = coins ++ sk.'1 ++ ctx ++ m. </pre>	<pre> commitment_source : skey $\times$ message $\times$ context $\times$ random_coins $\rightarrow$ List(byte) </pre>	Defines a total mathematical function by the EasyCrypt model.
3089	operation		
commitment_raw	<pre> op commitment_raw : mode -> skey -> message -> context -> random_coins -> polyveck = fun md sk m ctx coins => deterministic_polyveck md (commitment_source sk m ctx coins). </pre>	<pre> commitment_raw : 1 $\rightarrow$ mode $\rightarrow$ skey $\rightarrow$ message $\rightarrow$ context $\rightarrow$ random_coins $\rightarrow$ polyveck </pre>	Defines a total mathematical function by the EasyCrypt model
3093	operation		
response_source	<pre> op response_source (sk : skey) (m : message) (ctx : context) (coins : random_coins) : byte list = coins ++ sk.'1 ++ ctx ++ m. </pre>	<pre> response_source : skey $\times$ message $\times$ context $\times$ random_coins $\rightarrow$ List(byte) </pre>	Defines a total mathematical function by the EasyCrypt model.
3097	operation		
response_aux_vector	<pre> op response_aux_vector : mode -> skey -> message -> context -> random_coins -> polyveck = fun md sk m ctx coins => deterministic_unit_polyveck md 29 (response_source sk m ctx coins). </pre>	<pre> response_aux_vector : 1 $\rightarrow$ mode $\rightarrow$ skey $\rightarrow$ message $\rightarrow$ context $\rightarrow$ random_coins $\rightarrow$ polyveck </pre>	Defines a total mathematical function by the EasyCrypt model
3101	operation		
signing_sample_y1	<pre> op signing_sample_y1 : mode -> skey -> message -> context -> random_coins -> polyvecl = fun md sk m ctx coins => deterministic_unit_polyvecl md 73 (response_source sk m ctx coins). </pre>	<pre> signing_sample_y1 : 1 $\rightarrow$ mode $\rightarrow$ skey $\rightarrow$ message $\rightarrow$ context $\rightarrow$ random_coins $\rightarrow$ polyvecl </pre>	Defines a total mathematical function by the EasyCrypt model

	Line	Construct	What was written	Symbolic correspondence
	3105	operation		
	signing_sample_y2	<pre> op signing_sample_y2 : mode -> skey -> message -> context -> random_coins -> polyveck = fun md sk m ctx coins => deterministic_unit_polyveck md 79 (response_source sk m ctx coins).</pre>	<pre> signing_sample_y2 : 1 -> mode- > skey- > message- > context- > random_coins -> polyveck</pre>	Defines a total mathematical function by the EasyCrypt model.
	3109	operation		
	signing_entropy_token_of_coins	<pre> op signing_entropy_token_of_coins (coins : random_coins) : int = nth 0 coins 0 + 256 * nth 0 coins 1 + 65536 * nth 0 coins 2 + 16777216 * nth 0 coins 3 + 4294967296 * nth 0 coins 4 + 1099511627776 * nth 0 coins 5 + 281474976710656 * nth 0 coins 6 + 72057594037927936 * nth 0 coins 7.</pre>	<pre> signing_entropy_token_of_coins : random_coins -> ℤ</pre>	Defines a total mathematical function by the EasyCrypt model.
	3118	operation		
	commitment_lowbits	<pre> op commitment_lowbits : mode -> skey -> message -> context -> random_coins -> poly = fun md sk m ctx coins => let raw = nth poly_zero (commitment_raw md sk m ctx coins) 0 in mkseq (fun i => if i = 0 then signing_entropy_token_of_coins coins else poly_coeff raw i) n.</pre>	<pre> commitment_lowbits : 1 -> mode- > skey- > message- > context- > random_coins -> poly</pre>	Defines a total mathematical function by the EasyCrypt model.
	3127	operation		
	message_hash	<pre> op message_hash : pkey -> context -> message -> crh = fun pk ctx m => mkseq (fun i => nth 0 (encode_pkey pk ++ ctx ++ m) i) crhbytes.</pre>	<pre> message_hash : 1 -> pkey- > context- > message- > crh</pre>	Defines the random-oracle/hash inter data used by the games.
	3129	operation		
	challenge_source	<pre> op challenge_source (highbits : polyveck) (lowbits : poly) (mu : crh) : byte list = encode_poly lowbits ++ encode_polyveck highbits ++ mu.</pre>	<pre> challenge_source : polyveck × poly × crh -> List(byte)</pre>	Defines a total mathematical function by the EasyCrypt model.

Line	Construct	What was written	Symbolic correspondence
3132	operation		
challenge_from_seed	<pre> op challenge_from_seed (md : mode) (src : byte list) : challenge = mkseq (fun i => if i < mode_tau md then if nth 0 src i %% 2 = 0 then 1 else -1 else 0) n. </pre>	<pre> challenge_from_seed : mode × List(byte) → challenge </pre>	Defines the random-oracle/hash inter- data used by the games.
3138	operation		
challenge_hash	<pre> op challenge_hash : mode -> polyveck -> poly -> crh -> challenge = fun md (_ : polyveck) lowbits mu => challenge_from_seed md (encode_poly lowbits ++ mu). </pre>	<pre> challenge_hash : polyveck -> mode -> polyveck -> poly -> crh -> challenge </pre>	Defines the random-oracle/hash inter- data used by the games.
3141	operation		
commitment_challenge	<pre> op commitment_challenge : mode -> skey -> message -> context -> random_coins -> challenge = fun md sk m ctx coins => challenge_hash md (response_aux_vector md sk m ctx coins) (commitment_lowbits md sk m ctx coins) (message_hash (public_key_of_secret md sk) ctx m). </pre>	<pre> commitment_challenge : 1 -> mode -> skey -> message -> context -> random_coins -> challenge </pre>	Defines a total mathematical function by the CoinsCrypt challenge
3148	operation		
commitment_highbits	<pre> op commitment_highbits : mode -> skey -> message -> context -> random_coins -> polyveck = fun md sk m ctx coins => let pk = public_key_of_secret md sk in let ch = commitment_challenge md sk m ctx coins in reconstructed_highbits md (response_aux_vector md sk m ctx coins) (public_key_challenge_term md pk ch). </pre>	<pre> commitment_highbits : 1 -> mode -> skey -> message -> context -> random_coins -> polyveck </pre>	Defines a total mathematical function by the CoinsCrypt polyveck
3156	operation		

Line	Construct	What was written	Symbolic correspondence
response_vector	<pre> op response_vector : mode -> skey -> message -> context -> random_coins -> polyvecl = fun md sk m ctx coins => deterministic_unit_polyvecl md 17 (response_source sk m ctx coins). </pre>	<pre> response_vector : 1 -> mode -> skey -> message -> context -> </pre>	Defines a total mathematical function on $\text{mode} \times \text{skey} \times \text{message} \times \text{context} \times \text{random_coins}$.
3160	operation		
hint_vector	<pre> op hint_vector : mode -> skey -> message -> context -> random_coins -> hint_t = fun md sk m ctx coins => deterministic_unit_polyveck md 41 (response_source sk m ctx coins). </pre>	<pre> hint_vector : 1 -> mode -> skey -> message -> context -> </pre>	Defines a total mathematical function on $\text{mode} \times \text{skey} \times \text{message} \times \text{context} \times \text{random_coins}$.
3164	operation		
signature_token	<pre> op signature_token : mode -> skey -> message -> context -> random_coins -> sig_token = fun md sk m ctx coins => (response_vector md sk m ctx coins, response_aux_vector md sk m ctx coins, hint_vector md sk m ctx coins). </pre>	<pre> signature_token : 1 -> mode -> skey -> message -> context -> </pre>	Defines a total mathematical function on $\text{mode} \times \text{skey} \times \text{message} \times \text{context} \times \text{random_coins}$.
3171	lemma		
deterministic_poly_wf	<pre> lemma deterministic_poly_wf tag src : poly_wf (deterministic_poly tag src). </pre>	<pre> deterministic_poly_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact. </pre>	Proves a structural correctness fact u justify later reductions.
3175	lemma		
deterministic_polyveck_wf	<pre> lemma deterministic_polyveck_wf md src : polyveck_wf md (deterministic_polyveck md src). </pre>	<pre> deterministic_polyveck_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact. </pre>	Proves a structural correctness fact u justify later reductions.
3185	lemma		
deterministic_unit_poly_wf	<pre> lemma deterministic_unit_poly_wf tag src : poly_wf (deterministic_unit_poly tag src). </pre>	<pre> deterministic_unit_poly_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact. </pre>	Proves a structural correctness fact u justify later reductions.
3189	lemma		

	Line	Construct	What was written	Symbolic correspondence
deterministic_unit_poly_unit		lemma deterministic_unit_poly_unit tag src : poly_unit (deterministic_unit_poly tag src).	deterministic_unit_poly_unit : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	3200	lemma		
deterministic_unit_polyveck_wf		lemma deterministic_unit_polyveck_wf md tag src : polyveck_wf md (deterministic_unit_polyveck md tag src).	deterministic_unit_polyveck_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	3210	lemma		
deterministic_unit_polyveck_unit		lemma deterministic_unit_polyveck_unit md tag src : polyveck_unit md (deterministic_unit_polyveck md tag src).	deterministic_unit_polyveck_unit : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	3220	lemma		
deterministic_unit_polyvecl_wf		lemma deterministic_unit_polyvecl_wf md tag src : polyvecl_wf md (deterministic_unit_polyvecl md tag src).	deterministic_unit_polyvecl_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	3230	lemma		
deterministic_unit_polyvecl_unit		lemma deterministic_unit_polyvecl_unit md tag src : polyvecl_unit md (deterministic_unit_polyvecl md tag src).	deterministic_unit_polyvecl_unit : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	3240	lemma		
commitment_raw_wf		lemma commitment_raw_wf md sk m ctx coins : polyveck_wf md (commitment_raw md sk m ctx coins).	commitment_raw_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	3246	lemma		
commitment_highbits_wf		lemma commitment_highbits_wf md sk m ctx coins : polyveck_wf md (commitment_highbits md sk m ctx coins).	commitment_highbits_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	3253	lemma		

	Line	Construct	What was written	Symbolic correspondence
		<code>commitment_lowbits_wf</code> <code>lemma commitment_lowbits_wf md</code> <code>sk m ctx coins : poly_wf</code> <code>(commitment_lowbits md sk m ctx</code> <code>coins).</code>	<code>commitment_lowbits_wf</code> : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	3260	lemma		
		<code>commitment_lowbits_entropy_token</code> <code>lemma</code> <code>commitment_lowbits_entropy_token</code> <code>md sk m ctx coins : nth 0</code> <code>(commitment_lowbits md sk m ctx</code> <code>coins) 0 =</code> <code>signing_entropy_token_of_coins</code> <code>coins.</code>	<code>commitment_lowbits_entropy_token</code> : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	3270	lemma		
		<code>challenge_from_seed_wf</code> <code>lemma challenge_from_seed_wf md</code> <code>src : challenge_wf</code> <code>(challenge_from_seed md src).</code>	<code>challenge_from_seed_wf</code> : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.
	3282	lemma		
		<code>challenge_from_seed_sparse</code> <code>lemma challenge_from_seed_sparse</code> <code>md src : challenge_sparse md</code> <code>(challenge_from_seed md src).</code>	<code>challenge_from_seed_sparse</code> : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.
	3297	lemma		
		<code>challenge_hash_wf</code> <code>lemma challenge_hash_wf md w1 w0</code> <code>mu : challenge_wf</code> <code>(challenge_hash md w1 w0 mu).</code>	<code>challenge_hash_wf</code> : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.
	3301	lemma		
		<code>challenge_hash_sparse</code> <code>lemma challenge_hash_sparse md</code> <code>w1 w0 mu : challenge_sparse md</code> <code>(challenge_hash md w1 w0 mu).</code>	<code>challenge_hash_sparse</code> : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.
	3305	lemma		
		<code>response_vector_wf</code> <code>lemma response_vector_wf md sk m</code> <code>ctx coins : polyvecl_wf md</code> <code>(response_vector md sk m ctx</code> <code>coins).</code>	<code>response_vector_wf</code> : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	3311	lemma		
		<code>response_vector_unit</code> <code>lemma response_vector_unit md sk</code> <code>m ctx coins : polyvecl_unit md</code> <code>(response_vector md sk m ctx</code> <code>coins).</code>	<code>response_vector_unit</code> : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	3317	lemma		

Line	Construct	What was written	Symbolic correspondence
response_aux_vector_wf	lemma response_aux_vector_wf md sk m ctx coins : polyveck_wf md (response_aux_vector md sk m ctx coins).	response_aux_vector_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
3323	lemma		
response_aux_vector_unit	lemma response_aux_vector_unit md sk m ctx coins : polyveck_unit md (response_aux_vector md sk m ctx coins).	response_aux_vector_unit : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
3329	lemma		
signing_sample_y1_wf	lemma signing_sample_y1_wf md sk m ctx coins : polyvecl_wf md (signing_sample_y1 md sk m ctx coins).	signing_sample_y1_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.
3335	lemma		
signing_sample_y1_unit	lemma signing_sample_y1_unit md sk m ctx coins : polyvecl_unit md (signing_sample_y1 md sk m ctx coins).	signing_sample_y1_unit : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.
3341	lemma		
signing_sample_y2_wf	lemma signing_sample_y2_wf md sk m ctx coins : polyveck_wf md (signing_sample_y2 md sk m ctx coins).	signing_sample_y2_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.
3347	lemma		
signing_sample_y2_unit	lemma signing_sample_y2_unit md sk m ctx coins : polyveck_unit md (signing_sample_y2 md sk m ctx coins).	signing_sample_y2_unit : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.
3353	lemma		
hint_vector_wf	lemma hint_vector_wf md sk m ctx coins : polyveck_wf md (hint_vector md sk m ctx coins).	hint_vector_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
3359	lemma		
hint_vector_unit	lemma hint_vector_unit md sk m ctx coins : polyveck_unit md (hint_vector md sk m ctx coins).	hint_vector_unit : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
3365	lemma		
message_hash_size	lemma message_hash_size pk ctx m : size (message_hash pk ctx m) = crhbytes.	message_hash_size : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose

	Line	Construct	What was written	Symbolic correspondence
	3369	lemma		
challenge_hash_size		lemma challenge_hash_size md w1 w0 mu : size (challenge_hash md w1 w0 mu) = n.	challenge_hash_size : $L = R$ is an equality/rewriting fact.	Proves a bound that contributes to a game-hop or final security inequality.
	3373	operation		
keygen_internal		op keygen_internal (md : mode) (sd : seed) : pkey * skey = let sk = secret_key_of_seed md sd in (public_key_of_secret md sk, sk).	keygen_internal : mode $\times$ seed $\rightarrow$ pkey $\times$ skey	Defines a total mathematical function by the EasyCrypt model.
	3375	operation		
sign_internal		op sign_internal : mode -> skey -> message -> context -> random_coins -> signature = fun (md : mode) (sk : skey) (m : message) (ctx : context) (coins : random_coins) => let pk = public_key_of_secret md sk in let highbits = commitment_highbits md sk m ctx coins in let lowbits = commitment_lowbits md sk m ctx coins in let mu = message_hash pk ctx m in let c...	sign_internal : mode $\times$ skey $\times$ message $\times$ context $\times$ random_coins $\rightarrow$ mode $\rightarrow$ skey $\rightarrow$ message $\rightarrow$ context $\rightarrow$ random_coins $\rightarrow$ signature	Defines a total mathematical function by the EasyCrypt model.
	3386	operation		
verify_challenge_consistent		op verify_challenge_consistent (md : mode) (pk : pkey) (m : message) (ctx : context) (sig : signature) : bool = let mu = message_hash pk ctx m in sig_message_hash sig = mu /\ sig_challenge sig = challenge_hash md (sig_commitment_highbits sig) (sig_commitment_lowbits sig) mu.	verify_challenge_consistent $\subseteq$ mode $\times$ pkey $\times$ message $\times$ context $\times$ signature is a Boolean-valued predicate.	Names a Boolean mathematical cond used in statements and invariants.
	3397	operation		

	Line	Construct	What was written	Symbolic correspondence
verify_reconstructed_highbits		<pre> op verify_reconstructed_highbits (md : mode) (pk : pkey) (sig : signature) : polyveck = let pk_ch = public_key_challenge_term md pk (sig_challenge sig) in reconstructed_highbits md (sig_response_aux sig) pk_ch. </pre>	<pre> verify__reconstructed__highbits : mode × pkey × signature → polyveck </pre>	Defines a total mathematical function by the EasyCrypt model.
	3402	operation		
verify_public_equation		<pre> op verify_public_equation (md : mode) (pk : pkey) (sig : signature) : bool = sig_commitment_highbits sig = verify_reconstructed_highbits md pk sig. </pre>	<pre> verify__public__equation ⊆ mode × pkey × signature is a Boolean-valued predicate. </pre>	Names a Boolean mathematical condition used in statements and invariants.
	3407	lemma		
keygen_internal_valid		<pre> lemma keygen_internal_valid md sd : valid_keypair md (keygen_internal md sd).‘1 (keygen_internal md sd).‘2. </pre>	<pre> keygen__internal__valid : ∀x̄. P(x̄) is a universally quantified fact. </pre>	Proves a structural correctness fact to justify later reductions.
	3411	lemma		
public_key_of_keygen		<pre> lemma public_key_of_keygen md sd : public_key_of_secret md (keygen_internal md sd).‘2 = (keygen_internal md sd).‘1. </pre>	<pre> public__key__of__keygen : L = R is an equality/rewriting fact. </pre>	Records a reusable theorem so later proofs/scripts can rewrite, apply, or compose.
	3416	lemma		
public_key_vector_seed_keygen_equation		<pre> lemma public_key_vector_seed_keygen_equation md sd : public_key_vector_seed md sd = haetae_public_key_vector_from_relation md sd (haetae_keygen_secret_vector md sd) (haetae_keygen_error_vector md sd). </pre>	<pre> public__key__vector__seed__keygen__equation : L = R is an equality/rewriting fact. </pre>	Records a reusable theorem so later proofs/scripts can rewrite, apply, or compose.
	3423	lemma		

	Line	Construct	What was written	Symbolic correspondence
public_key_vector_seed_mode23_keygen_equation		<pre> lemma public_key_vector_seed_mode23_keygen_equation md sd : (md = Mode2 ∨ md = Mode3) => public_key_vector_seed md sd = polyveck_vk_highbits md (polyveck_add md (public_rounding_vector_seed md sd) (polyveck_add md (matrix_vec_mul md (haetae_keygen_a0_matrix md sd) (haetae_keygen_secret_vector md sd)) (haetae_keygen_error_vector md sd))). </pre>	<pre> public_key_vector_seed_mode23_keygen_equation P=Q </pre> is an implication used as a proof obligation.	Equation is a reusable theorem so later p scripts can rewrite, apply, or compose
	3443	lemma		
public_key_vector_seed_mode5_keygen_equation		<pre> lemma public_key_vector_seed_mode5_keygen_equation sd : public_key_vector_seed Mode5 sd = polyveck_neg Mode5 (polyveck_double Mode5 (polyveck_add Mode5 (matrix_vec_mul Mode5 (haetae_keygen_a0_matrix Mode5 sd) (haetae_keygen_secret_vector Mode5 sd)) (haetae_keygen_error_vector Mode5 sd))). </pre>	<pre> public_key_vector_seed_mode5_keygen_equation I=J </pre> is an equality/rewriting fact.	Equation is a reusable theorem so later p scripts can rewrite, apply, or compose
	3459	lemma		
haetae_keygen_raw_public_vector_keygen_equation		<pre> lemma haetae_keygen_raw_public_vector_keygen_equation md sd : haetae_keygen_raw_public_vector md sd = polyveck_add md (matrix_vec_mul md (haetae_keygen_a0_matrix md sd) (haetae_keygen_secret_vector md sd)) (haetae_keygen_error_vector md sd). </pre>	<pre> haetae_keygen_raw_public_vector_keygen_equation I=J </pre> is an equality/rewriting fact.	Equation is a reusable theorem so later p scripts can rewrite, apply, or compose
	3468	lemma		

	Line	Construct	What was written	Symbolic correspondence
haetae_reference_keygen_raw_public_vector_equation		lemma	haetae_reference_keygen_raw_public_vector_equation	Equation reusable theorem so later p
		haetae_reference_keygen_raw_public_vector_equation	$L \leftrightarrow R$ is an equality/rewriting fact.	scripts can rewrite, apply, or compose
		md sd :		
		haetae_reference_keygen_raw_public_vector		
		md sd = polyveck_add md		
		(matrix_vec_mul md		
		(haetae_reference_polymatkm_expand_matA		
		md (haetae_keygen_rhoprim sd))		
		(haetae_reference_keygen_secret_vector		
		md sd))		
		(haetae_reference_keygen_error_vector		
		md sd).		
	3481	lemma		
haetae_reference_keygen_mode23_public_vector_equation		lemma	haetae_reference_keygen_mode23_public_vector_equation	Equation reusable theorem so later p
		haetae_reference_keygen_mode23_public_vector_equation	$P \Rightarrow Q$ is an implication used as a proof obligation.	scripts can rewrite, apply, or compose
		md sd : (md = Mode2 \/\ md =		
		Mode3) =>		
		haetae_reference_keygen_public_vector		
		md sd = polyveck_vk_highbits md		
		(polyveck_add md		
		(haetae_reference_polyveck_expand_vecA		
		md (haetae_keygen_rhoprim sd))		
		(polyveck_add md (matrix_vec_mul		
		md		
		(haetae_reference_polymatkm_expand_matA		
		md (haetae_keygen_rhoprim...		
	3503	lemma		
haetae_reference_keygen_mode23_adjusted_error_equation		lemma	haetae_reference_keygen_mode23_adjusted_error_equation	Equation reusable theorem so later p
		haetae_reference_keygen_mode23_adjusted_error_equation	$P \Rightarrow Q$ is an implication used as a proof obligation.	scripts can rewrite, apply, or compose
		md sd : (md = Mode2 \/\ md =		
		Mode3) =>		
		haetae_reference_keygen_mode23_adjusted_error_vector		
		md sd = polyveck_sub md		
		(haetae_reference_keygen_error_vector		
		md sd) (polyveck_vk_lowbits md		
		(polyveck_add md		
		(haetae_reference_polyveck_expand_vecA		
		md (haetae_keygen_rhoprim sd))		
		(haetae_reference_keygen...		
	3520	lemma		

	Line	Construct	What was written	Symbolic correspondence
haetae_reference_keygen_mode5_public_vector_equation		<pre> lemma haetae_reference_keygen_mode5_public_vector_equation sd : haetae_reference_keygen_public_vector Mode5 sd = polyveck_neg Mode5 (polyveck_double Mode5 (polyveck_add Mode5 (matrix_vec_mul Mode5 (haetae_reference_polymatkm_expand_matA Mode5 (haetae_keygen_rhoprime sd)) (haetae_reference_keygen_secret_vector Mode5 sd)) (haetae_reference_keygen_error_vect...</pre>	<pre> haetae_reference_keygen_mode5_public_vector_equation R is an equality/rewriting fact.</pre>	Records a reusable theorem so later proofs can rewrite, apply, or compose
packed_haetae_reference_public_key_seedE	3536	<pre> lemma packed_haetae_reference_public_key_seedE md sd : (packed_haetae_reference_public_key md sd).‘1 = haetae_keygen_rhoprime sd.</pre>	<pre> packed_haetae_reference_public_key_seedE R is an equality/rewriting fact.</pre>	Records a reusable theorem so later proofs can rewrite, apply, or compose
packed_haetae_reference_public_key_bodyE	3541	<pre> lemma packed_haetae_reference_public_key_bodyE md sd : (packed_haetae_reference_public_key md sd).‘2 = haetae_reference_keygen_public_vector md sd.</pre>	<pre> packed_haetae_reference_public_key_bodyE R is an equality/rewriting fact.</pre>	Records a reusable theorem so later proofs can rewrite, apply, or compose
haetae_reference_keygen_seed_flow_wf_ok	3546	<pre> lemma haetae_reference_keygen_seed_flow_wf_ok md sd : haetae_reference_keygen_seed_flow_wf md sd.</pre>	<pre> haetae_reference_keygen_seed_flow_wf_ok $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.</pre>	Proves a structural correctness fact u justify later reductions.
packed_haetae_public_key_wf	3557	<pre> lemma packed_haetae_public_key_wf md sd : polyveck_wf md (packed_haetae_public_key md sd).‘2.</pre>	<pre> packed_haetae_public_key_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.</pre>	Proves a structural correctness fact u justify later reductions.

	Line	Construct	What was written	Symbolic correspondence
	3564	lemma		
packed_haetae_public_key_polyq_wf		lemma packed_haetae_public_key_polyq_wf md sd : haetae_polyveck_polyq_coeffs_wf md (packed_haetae_public_key md sd). '2.	packed_haetae_public_key_polyq_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
	3571	lemma		
packed_haetae_public_key_unpacked_wf		lemma packed_haetae_public_key_unpacked_wf md sd : size sd = seedbytes => haetae_public_key_unpacked_wf md (packed_haetae_public_key md sd).	packed_haetae_public_key_unpacked_wf : $P \Rightarrow Q$ is an implication used as a proof obligation.	Proves a structural correctness fact u justify later reductions.
	3579	lemma		
haetae_keygen_public_key_roundtrip		lemma haetae_keygen_public_key_roundtrip md sd : size sd = seedbytes => haetae_keygen_public_key_roundtrip_ok md sd.	haetae_keygen_public_key_roundtrip : $P \Rightarrow Q$ is an implication used as a proof obligation.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	3589	lemma		
public_key_of_secret_packed		lemma public_key_of_secret_packed md sk : public_key_of_secret md sk = packed_haetae_public_key md sk. '1.	public_key_of_secret_packed : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	3596	lemma		
keygen_public_key_packed		lemma keygen_public_key_packed md sd : (keygen_internal md sd). '1 = packed_haetae_public_key md sd.	keygen_public_key_packed : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	3603	lemma		
public_verification_matrix_packedE		lemma public_verification_matrix_packedE md pk : public_verification_matrix md pk = packed_haetae_verification_matrix md pk.	public_verification_matrix_packedE : $L =$ R is an equality/rewriting fact.	Records a reusable theorem so later p scripts can rewrite, apply, or compose
	3607	lemma		

	Line	Construct	What was written	Symbolic correspondence
packed_haetae_keygen_verification_matrixE		lemma	packed_haetae_keygen_verification_matrixE	Records a reusable theorem so later p
		packed_haetae_keygen_verification_matrixE	$L \leftrightarrow R$ is an equality/rewriting fact.	scripts can rewrite, apply, or compose
		md sd :		
		packed_haetae_keygen_verification_matrix		
		md sd =		
		haetae_verification_matrix_from_unpacked		
		md sd		
		(haetae_keygen_public_vector md		
		sd).		
	3617	lemma		
honest_public_verification_matrix_eq_packed_keygen		lemma	honest_public_verification_matrix_eq_packed_keygen	Records a reusable theorem so later p
		honest_public_verification_matrix_eq_packed_keygen	$L \leftrightarrow R$ is an equality/rewriting fact.	scripts can rewrite, apply, or compose
		md sd :		
		public_verification_matrix md		
		(keygen_internal md sd). '1 =		
		packed_haetae_keygen_verification_matrix		
		md sd.		
	3627	lemma		
concrete_keygen_public_verification_matrix_correct		lemma	concrete_keygen_public_verification_matrix_correct	Proves a structural correctness fact u
		concrete_keygen_public_verification_matrix_correct	$P \Rightarrow Q$ is an implication used as a proof	justify later reductions.
		md sd : size sd = seedbytes =>	obligation.	
		public_verification_matrix md		
		(keygen_internal md sd). '1 =		
		concrete_haetae_keygen_verification_matrix		
		md sd.		
	3644	lemma		
concrete_keygen_public_verification_matrix_ref_correct		lemma	concrete_keygen_public_verification_matrix_ref_correct	Proves a structural correctness fact u
		concrete_keygen_public_verification_matrix_ref_correct	$P \Rightarrow Q$ is an implication used as a proof	justify later reductions.
		md sd : size sd = seedbytes =>	obligation.	
		public_verification_matrix md		
		(keygen_internal md sd). '1 =		
		concrete_haetae_keygen_verification_matrix_ref		
		md sd.		
	3661	lemma		

	Line	Construct	What was written	Symbolic correspondence
concrete_reference_keygen_public_verification_matrix_ref_correct		lemma	concrete_reference_keygen_public_verification_matrix_ref_correct	Proves structural correctness fact u
		concrete_reference_keygen_public_verification_matrix_ref_correct	$L \leftrightarrow R$ is an equality/rewriting fact.	justify later reductions.
		md sd :		
		packed_haetae_verification_matrix		
		md		
		(packed_haetae_reference_public_key		
		md sd) =		
		concrete_haetae_reference_keygen_verification_matrix_ref		
		md sd.		
	3673	lemma	concrete_keygen_public_verification_matrix_size	Records a reusable theorem so later p
concrete_keygen_public_verification_matrix_size		lemma	concrete_keygen_public_verification_matrix_size	$L \leftrightarrow R$ is an equality/rewriting fact.
		concrete_keygen_public_verification_matrix_size		scripts can rewrite, apply, or compose
		md sd : size		
		(concrete_haetae_keygen_verification_matrix		
		md sd) = mode_k md.		
	3681	lemma	concrete_keygen_public_verification_matrix_ref_size	Records a reusable theorem so later p
concrete_keygen_public_verification_matrix_ref_size		lemma	concrete_keygen_public_verification_matrix_ref_size	$L \leftrightarrow R$ is an equality/rewriting fact.
		concrete_keygen_public_verification_matrix_ref_size		scripts can rewrite, apply, or compose
		md sd : size		
		(concrete_haetae_keygen_verification_matrix_ref		
		md sd) = mode_k md.		
	3689	lemma	concrete_reference_keygen_public_verification_matrix_ref_size	Records a reusable theorem so later p
concrete_reference_keygen_public_verification_matrix_ref_size		lemma	concrete_reference_keygen_public_verification_matrix_ref_size	$L \leftrightarrow R$ is an equality/rewriting fact.
		concrete_reference_keygen_public_verification_matrix_ref_size		scripts can rewrite, apply, or compose
		md sd : size		
		(concrete_haetae_reference_keygen_verification_matrix_ref		
		md sd) = mode_k md.		
	3698	lemma	concrete_keygen_public_verification_matrix_rows_wf	Proves wf structural correctness fact u
concrete_keygen_public_verification_matrix_rows_wf		lemma	concrete_keygen_public_verification_matrix_rows_wf	$\forall \vec{x}. R(\vec{x})$ is a universally quantified fact.
		concrete_keygen_public_verification_matrix_rows_wf		justify later reductions.
		md sd : all (polyvecl_wf md)		
		(concrete_haetae_keygen_verification_matrix		
		md sd).		
	3718	lemma	concrete_keygen_public_verification_matrix_ref_rows_wf	Proves wf structural correctness fact u
concrete_keygen_public_verification_matrix_ref_rows_wf		lemma	concrete_keygen_public_verification_matrix_ref_rows_wf	$\forall \vec{x}. R(\vec{x})$ is a universally quantified fact.
		concrete_keygen_public_verification_matrix_ref_rows_wf		justify later reductions.
		md sd : all (polyvecl_wf md)		
		(concrete_haetae_keygen_verification_matrix_ref		
		md sd).		
	3739	lemma		

	Line	Construct	What was written	Symbolic correspondence
concrete_reference_keygen_public_verification_matrix_ref_rows_wf		lemma concrete_reference_keygen_public_verification_matrix_ref_rows_wf md sd : all (polyvecl_wf md) (concrete_haetae_reference_keygen_verification_matrix_ref md sd).	concrete_reference_keygen_public_verification_matrix_ref_rows_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	From matrix structural correctness fact u justify later reductions.
valid_signature_sign_internal	3760	lemma valid_signature_sign_internal md sk m ctx coins : valid_signature md (sign_internal md sk m ctx coins).	valid_signature_sign_internal : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact u justify later reductions.
verify_challenge_consistent_sign_internal	3769	lemma verify_challenge_consistent_sign_internal md sk m ctx coins : verify_challenge_consistent md (public_key_of_secret md sk) m ctx (sign_internal md sk m ctx coins).	verify_challenge_consistent_sign_internal : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a bound that contributes to a game-hop or final security inequality.
coeff_norm_sq	3777	operation op coeff_norm_sq (x : coeff) : int = x * x.	coeff_norm_sq : $\text{coeff} \rightarrow \mathbb{Z}$	Defines a total mathematical function by the EasyCrypt model.
poly_norm_sq	3778	operation op poly_norm_sq (p : poly) : int = if poly_unit p then size p else if p = poly_zero then 0 else sumz (map coeff_norm_sq p).	poly_norm_sq : $\text{poly} \rightarrow \mathbb{Z}$	Defines a total mathematical function by the EasyCrypt model.
polyvecl_norm_sq	3782	operation op polyvecl_norm_sq (md : mode) (xs : polyvecl) : int = if polyvecl_unit md xs then mode_l md * n else if xs = polyvecl_zero md then 0 else sumz (map poly_norm_sq xs).	polyvecl_norm_sq : $\text{mode} \times \text{polyvecl} \rightarrow \mathbb{Z}$	Defines a total mathematical function by the EasyCrypt model.
polyveck_norm_sq	3786	operation op polyveck_norm_sq (md : mode) (xs : polyveck) : int = if polyveck_unit md xs then mode_k md * n else if xs = polyveck_zero md then 0 else sumz (map poly_norm_sq xs).	polyveck_norm_sq : $\text{mode} \times \text{polyveck} \rightarrow \mathbb{Z}$	Defines a total mathematical function by the EasyCrypt model.

	Line	Construct	What was written	Symbolic correspondence
	3790	operation		
signature_response_norm_sq		<pre> op signature_response_norm_sq (md : mode) (sig : signature) : int = polyvecl_norm_sq md (sig_response sig) + polyveck_norm_sq md (sig_response_aux sig). </pre>	<pre> signature_response_norm_sq : mode × signature → ℤ </pre>	Defines a total mathematical function by the EasyCrypt model.
	3793	operation		
signature_verify_norm_sq		<pre> op signature_verify_norm_sq (md : mode) (sig : signature) : int = signature_response_norm_sq md sig + polyveck_norm_sq md (sig_hint sig). </pre>	<pre> signature_verify_norm_sq : mode × signature → ℤ </pre>	Defines a total mathematical function by the EasyCrypt model.
	3797	operation		
secret_norm_sq		<pre> op secret_norm_sq (sk : skey) : int = sk.'2. </pre>	<pre> secret_norm_sq : skey → ℤ </pre>	Defines a total mathematical function by the EasyCrypt model.
	3798	operation		
response_norm_sq		<pre> op response_norm_sq (md : mode) (sig : signature) : int = signature_response_norm_sq md sig. </pre>	<pre> response_norm_sq : mode × signature → ℤ </pre>	Defines a total mathematical function by the EasyCrypt model.
	3800	operation		
verify_norm_sq		<pre> op verify_norm_sq (md : mode) (sig : signature) : int = signature_verify_norm_sq md sig. </pre>	<pre> verify_norm_sq : mode × signature → ℤ </pre>	Defines a total mathematical function by the EasyCrypt model.
	3803	operation		
secret_norm_ok		<pre> op secret_norm_ok (md : mode) (sk : skey) : bool = 0 <= secret_norm_sq sk /\ secret_norm_sq sk <= mode_b0sq md. </pre>	<pre> secret_norm_ok ⊆ mode × skey is a Boolean-valued predicate. </pre>	Names a Boolean mathematical condition used in statements and invariants.
	3805	operation		
response_norm_ok		<pre> op response_norm_ok (md : mode) (sig : signature) : bool = 0 <= response_norm_sq md sig /\ response_norm_sq md sig <= mode_b1sq md. </pre>	<pre> response_norm_ok ⊆ mode × signature is a Boolean-valued predicate. </pre>	Names a Boolean mathematical condition used in statements and invariants.
	3807	operation		

Line	Construct	What was written	Symbolic correspondence
verify_norm_ok	<pre> op verify_norm_ok (md : mode) (sig : signature) : bool = 0 <= verify_norm_sq md sig /\ verify_norm_sq md sig <= mode_b2sq md. </pre>	verify_norm_ok $\subseteq$ mode $\times$ signature is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.
3810	operation		
verify_internal	<pre> op verify_internal : mode -> pkey -> message -> context -> signature -> bool = fun (md : mode) (pk : pkey) (m : message) (ctx : context) (sig : signature) => valid_signature md sig /\ verify_challenge_consistent md pk m ctx sig /\ verify_public_equation md pk sig /\ verify_norm_ok md sig. </pre>	verify_internal : mode $\times$ pkey $\times$ message $\times$ context $\times$ signature $\rightarrow$ mode- $\times$ pkey- $\times$ message- $\times$ context- $\times$ signature- $\rightarrow$ bool	Defines a total mathematical function by the EasyCrypt model.
3819	lemma		
secret_norm_ok_current	<pre> lemma secret_norm_ok_current md sd : secret_norm_ok md (secret_key_of_seed md sd). </pre>	secret_norm_ok_current : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later proofs can rewrite, apply, or compose
3823	lemma		
response_norm_ok_current	<pre> lemma response_norm_ok_current md sk m ctx coins : response_norm_ok md (sign_internal md sk m ctx coins). </pre>	response_norm_ok_current : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later proofs can rewrite, apply, or compose
3843	lemma		
verify_norm_ok_current	<pre> lemma verify_norm_ok_current md sk m ctx coins : verify_norm_ok md (sign_internal md sk m ctx coins). </pre>	verify_norm_ok_current : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later proofs can rewrite, apply, or compose
3868	lemma		
verify_internal_sign_internal	<pre> lemma verify_internal_sign_internal md sk m ctx coins : verify_internal md (public_key_of_secret md sk) m ctx (sign_internal md sk m ctx coins). </pre>	verify_internal_sign_internal : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later proofs can rewrite, apply, or compose

16 HAETAE_Params.ec

mode-dependent parameter constants

Line	Construct	What was written	Symbolic correspondence	Why it is written th
5	operation			
seedbytes	op seedbytes : int = 32.	seedbytes : $1 \rightarrow \mathbb{Z}$	Defines a total mathematical function used by the EasyCrypt model.	
6	operation			
crhbytes	op crhbytes : int = 64.	crhbytes : $1 \rightarrow \mathbb{Z}$	Defines a total mathematical function used by the EasyCrypt model.	
7	operation			
n	op n : int = 256.	n : $1 \rightarrow \mathbb{Z}$	Defines a total mathematical function used by the EasyCrypt model.	
8	operation			
q	op q : int = 64513.	q : $1 \rightarrow \mathbb{Z}$	Defines a total mathematical function used by the EasyCrypt model.	
9	operation			
dq	op dq : int = 2 * q.	dq : $1 \rightarrow \mathbb{Z}$	Defines a sampler/distribution used by game procedures and probability bounds.	
11	type			
mode	type mode = [Mode2 Mode3 Mode5].	mode $\equiv$ [Mode2 Mode3 Mode5]	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.	
13	operation			
mode_k	op mode_k (md : mode) : int = with md = Mode2 => 2 with md = Mode3 => 3 with md = Mode5 => 4.	mode_k : mode $\rightarrow \mathbb{Z}$	Defines a total mathematical function used by the EasyCrypt model.	
18	operation			
mode_l	op mode_l (md : mode) : int = with md = Mode2 => 4 with md = Mode3 => 6 with md = Mode5 => 7.	mode_l : mode $\rightarrow \mathbb{Z}$	Defines a total mathematical function used by the EasyCrypt model.	
23	operation			
mode_tau	op mode_tau (md : mode) : int = with md = Mode2 => 58 with md = Mode3 => 80 with md = Mode5 => 128.	mode_tau : mode $\rightarrow \mathbb{Z}$	Defines a total mathematical function used by the EasyCrypt model.	
28	operation			
mode_b0sq	op mode_b0sq (md : mode) : int = with md = Mode2 => 96944109 with md = Mode3 => 335438492 with md = Mode5 => 499239142.	mode_b0sq : mode $\rightarrow \mathbb{Z}$	Defines a total mathematical function used by the EasyCrypt model.	
33	operation			

Line	Construct	What was written	Symbolic correspondence	Why it is written th
mode_b1sq	<pre> op mode_b1sq (md : mode) : int = with md = Mode2 => 96805527 with md = Mode3 => 335171879 with md = Mode5 => 498849991. </pre>	$\text{mode_b1sq} : \text{mode} \rightarrow \mathbb{Z}$	Defines a total mathematical function used by the EasyCrypt model.	
38	operation			
mode_b2sq	<pre> op mode_b2sq (md : mode) : int = with md = Mode2 => 163265017 with md = Mode3 => 479901314 with md = Mode5 => 597386433. </pre>	$\text{mode_b2sq} : \text{mode} \rightarrow \mathbb{Z}$	Defines a total mathematical function used by the EasyCrypt model.	
43	operation			
mode_ln	<pre> op mode_ln (_ : mode) : int = 8192. </pre>	$\text{mode_ln} : \text{mode} \rightarrow \mathbb{Z}$	Defines a total mathematical function used by the EasyCrypt model.	
45	operation			
mode_alpha_hint	<pre> op mode_alpha_hint (md : mode) : int = with md = Mode2 => 512 with md = Mode3 => 512 with md = Mode5 => 256. </pre>	$\text{mode_alpha_hint} : \text{mode} \rightarrow \mathbb{Z}$	Defines a total mathematical function used by the EasyCrypt model.	
50	operation			
mode_m	<pre> op mode_m (md : mode) : int = mode_l md - 1. </pre>	$\text{mode_m} : \text{mode} \rightarrow \mathbb{Z}$	Defines a total mathematical function used by the EasyCrypt model.	
52	operation			
mode_crypto_bytes	<pre> op mode_crypto_bytes (md : mode) : int = with md = Mode2 => 1474 with md = Mode3 => 2349 with md = Mode5 => 2948. </pre>	$\text{mode_crypto_bytes} : \text{mode} \rightarrow \mathbb{Z}$	Defines a total mathematical function used by the EasyCrypt model.	
57	operation			
mode_polyq_packedbytes	<pre> op mode_polyq_packedbytes (md : mode) : int = with md = Mode2 => 480 with md = Mode3 => 480 with md = Mode5 => 512. </pre>	$\text{mode_polyq_packedbytes} : \text{mode} \rightarrow \mathbb{Z}$	Defines a total mathematical function used by the EasyCrypt model.	
62	operation			
mode_publickeybytes	<pre> op mode_publickeybytes (md : mode) : int = seedbytes + mode_k md * mode_polyq_packedbytes md. </pre>	$\text{mode_publickeybytes} : \text{mode} \rightarrow \mathbb{Z}$	Defines a total mathematical function used by the EasyCrypt model.	
65	lemma			
seedbytes_gt0	<pre> lemma seedbytes_gt0 : 0 < seedbytes by rewrite /seedbytes. </pre>	$\text{seedbytes_gt0} : \forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
66	lemma			
crhbytes_gt0	<pre> lemma crhbytes_gt0 : 0 < crhbytes by rewrite /crhbytes. </pre>	$\text{crhbytes_gt0} : \forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	

Line	Construct	What was written	Symbolic correspondence	Why it is written th
67	lemma			
n_gt0	lemma n_gt0 : 0 < n by rewrite /n.	n_gt0 : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
68	lemma			
q_gt0	lemma q_gt0 : 0 < q by rewrite /q.	q_gt0 : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
69	lemma			
dqE	lemma dqE : dq = 129026 by rewrite /dq /q.	dqE : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
71	lemma			
mode_k_gt0	lemma mode_k_gt0 md : 0 < mode_k md.	mode_k_gt0 : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
74	lemma			
mode_l_gt0	lemma mode_l_gt0 md : 0 < mode_l md.	mode_l_gt0 : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
77	lemma			
mode_tau_gt0	lemma mode_tau_gt0 md : 0 < mode_tau md.	mode_tau_gt0 : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
80	lemma			
mode_tau_le_n	lemma mode_tau_le_n md : mode_tau md <= n.	mode_tau_le_n : $X \leq Y$ is an arithmetic or probability upper bound.	Proves a bound that contributes to a game-hop or final security inequality.	
83	lemma			
mode_b0sq_gt0	lemma mode_b0sq_gt0 md : 0 < mode_b0sq md.	mode_b0sq_gt0 : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
86	lemma			
mode_b1sq_gt0	lemma mode_b1sq_gt0 md : 0 < mode_b1sq md.	mode_b1sq_gt0 : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
89	lemma			
mode_b2sq_gt0	lemma mode_b2sq_gt0 md : 0 < mode_b2sq md.	mode_b2sq_gt0 : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
92	lemma			
mode_ln_gt0	lemma mode_ln_gt0 md : 0 < mode_ln md.	mode_ln_gt0 : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
95	lemma			
mode_alpha_hint_gt0	lemma mode_alpha_hint_gt0 md : 0 < mode_alpha_hint md.	mode_alpha_hint_gt0 : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
98	lemma			
mode_m_gt0	lemma mode_m_gt0 md : 0 < mode_m md.	mode_m_gt0 : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
101	lemma			
mode_crypto_bytes_gt0	lemma mode_crypto_bytes_gt0 md : 0 < mode_crypto_bytes md.	mode_crypto_bytes_gt0 : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	

Line	Construct	What was written	Symbolic correspondence	Why it is written th
104	lemma mode_polyq_packedbytes_gt0 md : 0 < mode_polyq_packedbytes md.	mode_polyq_packedbytes_gt0 : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
107	lemma n_le_mode_polyq_packedbytes n_le_mode_polyq_packedbytes md : n <= mode_polyq_packedbytes md.	n_le_mode_polyq_packedbytes : $X \leq Y$ is an arithmetic or probability upper bound.	Proves a bound that contributes to a game-hop or final security inequality.	
110	lemma mode_publickeybytes_gt0 md : 0 < mode_publickeybytes md.	mode_publickeybytes_gt0 : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	

17 HAETAЕ_FIPS202.ec

abstracted FIPS202/SHAKE support

Line	Construct	What was written	Symbolic correspondence	Why it is written
8	type			
fips_byte	type fips_byte = int.	$\text{fips_byte} \equiv \mathbb{Z}$	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.	
9	type			
fips202_state	type fips202_state = fips_byte list.	$\text{fips202_state} \equiv \text{List}(\text{fips_byte})$	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.	
11	operation			
fips202_state_bytes	op fips202_state_bytes : int = 200.	$\text{fips202_state_bytes} : 1 \rightarrow \mathbb{Z}$	Defines a total mathematical function used by the EasyCrypt model.	
12	operation			
shake256_rate_bytes	op shake256_rate_bytes : int = 136.	$\text{shake256_rate_bytes} : 1 \rightarrow \mathbb{Z}$	Defines a total mathematical function used by the EasyCrypt model.	
13	operation			
shake256_capacity_bytes	op shake256_capacity_bytes : int = 64.	$\text{shake256_capacity_bytes} : 1 \rightarrow \mathbb{Z}$	Defines a total mathematical function used by the EasyCrypt model.	
14	operation			
shake256_domain_separator	op shake256_domain_separator : fips_byte = 31.	$\text{shake256_domain_separator} : 1 \rightarrow \text{fips_byte}$	Defines a total mathematical function used by the EasyCrypt model.	
15	operation			
fips202_final_padding_byte	op fips202_final_padding_byte : fips_byte = 128.	$\text{fips202_final_padding_byte} : 1 \rightarrow \text{fips_byte}$	Defines a total mathematical function used by the EasyCrypt model.	
17	operation			
fips202_keccak_f1600	op fips202_keccak_f1600 (st : fips202_state) : fips202_state = keccak_f1600_bytes st.	$\text{fips202_keccak_f1600} : \text{fips202_state} \rightarrow \text{fips202_state}$	Defines a total mathematical function used by the EasyCrypt model.	
20	operation			
shake256_absorb_once_short_domain	op shake256_absorb_once_short_domain (_ : fips_byte list) (inlen : int) : bool = 0 <= inlen /\ inlen < shake256_rate_bytes.	$\text{shake256_absorb_once_short_domain} \subseteq \text{List}(\text{fips_byte}) \times \mathbb{Z}$ is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.	
24	operation			

Line	Construct	What was written	Symbolic correspondence	Why it is written
shake256_absorb_once_short_block_byte	<pre> op shake256_absorb_once_short_block_byte : (input : fips_byte list) (inlen i : int) : fips_byte = if 0 <= i /\ i < inlen then nth 0 input i else if i = inlen /\ i = shake256_rate_bytes - 1 then shake256_domain_separator + fips202_final_padding_byte else if i = inlen then shake256_domain_separator else if i = shake256_rate_bytes - 1 then fips202_final_paddin... </pre>	<pre> shake256_absorb_once_short_block_byte : List(fips_byte) $\times \mathbb{Z} \rightarrow$ fips_byte </pre>	Defines a total mathematical function used by the EasyCrypt model.	
33	operation			
shake256_absorb_once_short_state	<pre> op shake256_absorb_once_short_state : (input : fips_byte list) (inlen : int) : fips202_state = mkseq (fun i => if i < shake256_rate_bytes then shake256_absorb_once_short_block_byte input inlen i else 0) fips202_state_bytes. </pre>	<pre> shake256_absorb_once_short_state : List(fips_byte) $\times \mathbb{Z} \rightarrow$ fips202_state </pre>	Defines a total mathematical function used by the EasyCrypt model.	
42	operation			
shake256_absorb_once	<pre> op shake256_absorb_once (input : fips_byte list) (inlen : int) : fips202_state = fips202_keccak_f1600 (shake256_absorb_once_short_state input inlen). </pre>	<pre> shake256_absorb_once : List(fips_byte) $\times \mathbb{Z} \rightarrow$ fips202_state </pre>	Defines a total mathematical function used by the EasyCrypt model.	
47	operation			
shake256_squeeze	<pre> op shake256_squeeze (st : fips202_state) (outlen : int) : fips_byte list = mkseq (fun i => nth 0 st i) outlen. </pre>	<pre> shake256_squeeze : fips202_state $\times \mathbb{Z} \rightarrow$ List(fips_byte) </pre>	Defines a total mathematical function used by the EasyCrypt model.	
50	operation			
shake256	<pre> op shake256 (input : fips_byte list) (inlen outlen : int) : fips_byte list = shake256_squeeze (shake256_absorb_once input inlen) outlen. </pre>	<pre> shake256 : List(fips_byte) $\times \mathbb{Z} \times \mathbb{Z} \rightarrow$ List(fips_byte) </pre>	Defines a total mathematical function used by the EasyCrypt model.	

Line	Construct	What was written	Symbolic correspondence	Why it is written
54	lemma fips202_state_bytesE	lemma fips202_state_bytesE : fips202_state_bytes = 200.	fips202_state_bytesE : $L = R$ is an equality/rewriting fact.	Proves an invariant preservation fact for an oracle, adversary call, or game main procedure.
58	lemma shake256_rate_bytesE	lemma shake256_rate_bytesE : shake256_rate_bytes = 136.	shake256_rate_bytesE : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.
62	lemma shake256_capacity_bytesE	lemma shake256_capacity_bytesE : shake256_capacity_bytes = 64.	shake256_capacity_bytesE : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.
66	lemma shake256_domain_separatorE	lemma shake256_domain_separatorE : shake256_domain_separator = 31.	shake256_domain_separatorE : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.
70	lemma fips202_final_padding_byteE	lemma fips202_final_padding_byteE : fips202_final_padding_byte = 128.	fips202_final_padding_byteE : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.
74	lemma shake256_rate_capacity_state_bytesE	lemma shake256_rate_capacity_state_bytesE : shake256_rate_bytes + shake256_capacity_bytes = fips202_state_bytes.	shake256_rate_capacity_state_bytesE : $L = R$ is an equality/rewriting fact.	Proves an invariant preservation fact for an oracle, adversary call, or game main procedure.
81	lemma shake256_absorb_once_short_state_size	lemma shake256_absorb_once_short_state_size input inlen : size (shake256_absorb_once_short_state input inlen) = fips202_state_bytes.	shake256_absorb_once_short_state_size : $L = R$ is an equality/rewriting fact.	Proves an invariant preservation fact for an oracle, adversary call, or game main procedure.
88	lemma shake256_squeeze_size	lemma shake256_squeeze_size st outlen : 0 <= outlen => size (shake256_squeeze st outlen) = outlen.	shake256_squeeze_size : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.
97	lemma			

Line	Construct	What was written	Symbolic correspondence	Why it is written
shake256_size	lemma shake256_size input inlen outlen : 0 <= outlen => size (shake256 input inlen outlen) = outlen.	shake256_size : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	

18 HAETAE_Keccak1600.ec

Keccak state-level support used by the hash model

Line	Construct	What was written	Symbolic correspondence	Why it is written
5	type keccak_byte type keccak_byte = int.	$\text{keccak_byte} \equiv \mathbb{Z}$	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.	
6	type keccak_lane type keccak_lane = bool list.	$\text{keccak_lane} \equiv \text{List}(\text{Bool})$	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.	
7	type keccak_state type keccak_state = keccak_lane list.	$\text{keccak_state} \equiv \text{List}(\text{keccak_lane})$	Fixes the mathematical carrier used by later declarations without committing to an implementation representation.	
9	operation keccak_lane_bits op keccak_lane_bits : int = 64.	$\text{keccak_lane_bits} : 1 \rightarrow \mathbb{Z}$	Defines a total mathematical function used by the EasyCrypt model.	
10	operation keccak_state_lanes op keccak_state_lanes : int = 25.	$\text{keccak_state_lanes} : 1 \rightarrow \mathbb{Z}$	Defines a total mathematical function used by the EasyCrypt model.	
11	operation keccak_state_bytes op keccak_state_bytes : int = 200.	$\text{keccak_state_bytes} : 1 \rightarrow \mathbb{Z}$	Defines a total mathematical function used by the EasyCrypt model.	
13	operation keccak_byte_norm op keccak_byte_norm (b : keccak_byte) : keccak_byte = b %% 256.	$\text{keccak_byte_norm} : \text{keccak_byte} \rightarrow \text{keccak_byte}$	Defines a total mathematical function used by the EasyCrypt model.	
14	operation keccak_byte_bit op keccak_byte_bit (b : keccak_byte) (i : int) : bool = ((keccak_byte_norm b) % (2 ^ i)) %% 2 <> 0.	$\text{keccak_byte_bit} \subseteq \text{keccak_byte} \times \mathbb{Z}$ is a Boolean-valued predicate.	Names a Boolean mathematical condition used in statements and invariants.	
16	operation keccak_word_norm op keccak_word_norm (w : int) : int = w %% (2 ^ keccak_lane_bits).	$\text{keccak_word_norm} : \mathbb{Z} \rightarrow \mathbb{Z}$	Defines a total mathematical function used by the EasyCrypt model.	
17	operation			

Line	Construct	What was written	Symbolic correspondence	Why it is written
	<pre> keccak_word_bit op keccak_word_bit (w : int) (i : int) : bool = ((keccak_word_norm w) %/ (2 ^ i)) %% 2 <> 0. </pre>	<pre> keccak_word_bit $\subseteq \mathbb{Z} \times \mathbb{Z}$ is a Boolean-valued predicate. </pre>	Names a Boolean mathematical condition used in statements and invariants.	
20	operation			
	<pre> keccak_lane_zero op keccak_lane_zero : keccak_lane = nseq keccak_lane_bits false. </pre>	<pre> keccak_lane_zero : $1 \rightarrow \text{keccak_lane}$ </pre>	Defines a total mathematical function used by the EasyCrypt model.	
21	operation			
	<pre> keccak_lane_bit op keccak_lane_bit (lane : keccak_lane) (i : int) : bool = nth false lane i. </pre>	<pre> keccak_lane_bit $\subseteq \text{keccak_lane} \times \mathbb{Z}$ is a Boolean-valued predicate. </pre>	Names a Boolean mathematical condition used in statements and invariants.	
23	operation			
	<pre> keccak_lane_wf op keccak_lane_wf (lane : keccak_lane) : bool = size lane = keccak_lane_bits. </pre>	<pre> keccak_lane_wf $\subseteq \text{keccak_lane}$ is a Boolean-valued predicate. </pre>	Names a well-formedness or support condition so it can be reused in lemmas.	
26	operation			
	<pre> keccak_lane_xor op keccak_lane_xor (a b : keccak_lane) : keccak_lane = mkseq (fun i => keccak_lane_bit a i <> keccak_lane_bit b i) keccak_lane_bits. </pre>	<pre> keccak_lane_xor : keccak_lane $\times$ keccak_lane $\rightarrow$ keccak_lane </pre>	Defines a total mathematical function used by the EasyCrypt model.	
30	operation			
	<pre> keccak_lane_and op keccak_lane_and (a b : keccak_lane) : keccak_lane = mkseq (fun i => keccak_lane_bit a i /\ keccak_lane_bit b i) keccak_lane_bits. </pre>	<pre> keccak_lane_and : keccak_lane $\times$ keccak_lane $\rightarrow$ keccak_lane </pre>	Defines a total mathematical function used by the EasyCrypt model.	
34	operation			
	<pre> keccak_lane_not op keccak_lane_not (a : keccak_lane) : keccak_lane = mkseq (fun i => if keccak_lane_bit a i then false else true) keccak_lane_bits. </pre>	<pre> keccak_lane_not : keccak_lane $\rightarrow$ keccak_lane </pre>	Defines a total mathematical function used by the EasyCrypt model.	
38	operation			
	<pre> keccak_lane_rotl op keccak_lane_rotl (a : keccak_lane) (offset : int) : keccak_lane = mkseq (fun i => keccak_lane_bit a ((i - offset) %% keccak_lane_bits)) keccak_lane_bits. </pre>	<pre> keccak_lane_rotl : keccak_lane $\times \mathbb{Z} \rightarrow$ keccak_lane </pre>	Defines a total mathematical function used by the EasyCrypt model.	

Line	Construct	What was written	Symbolic correspondence	Why it is written
43	operation			
keccak_lane_of_int	op keccak_lane_of_int (x : int) : keccak_lane = mkseq (fun i => keccak_word_bit x i) keccak_lane_bits.	keccak_lane_of_int : $\mathbb{Z} \rightarrow \text{keccak_lane}$	Defines a total mathematical function used by the EasyCrypt model.	
46	operation			
keccak_lane_to_byte	op keccak_lane_to_byte (lane : keccak_lane) (byte_index : int) : keccak_byte = sumz (map (fun j => if keccak_lane_bit lane (8 * byte_index + j) then 2 ^ j else 0) (range 0 8)).	keccak_lane_to_byte : $\text{keccak_lane} \times \mathbb{Z} \rightarrow \text{keccak_byte}$	Defines a total mathematical function used by the EasyCrypt model.	
55	operation			
keccak_bytes_to_lane	op keccak_bytes_to_lane (bs : keccak_byte list) (lane_index : int) : keccak_lane = mkseq (fun bit => keccak_byte_bit (nth 0 bs (8 * lane_index + bit % 8)) (bit % 8)) keccak_lane_bits.	keccak_bytes_to_lane : $\text{List}(\text{keccak_byte}) \times \mathbb{Z} \rightarrow \text{keccak_lane}$	Defines a total mathematical function used by the EasyCrypt model.	
64	operation			
keccak_lanes_of_bytes	op keccak_lanes_of_bytes (bs : keccak_byte list) : keccak_state = mkseq (fun lane => keccak_bytes_to_lane bs lane) keccak_state_lanes.	keccak_lanes_of_bytes : $\text{List}(\text{keccak_byte}) \rightarrow \text{keccak_state}$	Defines a total mathematical function used by the EasyCrypt model.	
67	operation			
keccak_bytes_of_lanes	op keccak_bytes_of_lanes (st : keccak_state) : keccak_byte list = mkseq (fun i => keccak_lane_to_byte (nth keccak_lane_zero st (i % 8)) (i % 8)) keccak_state_bytes.	keccak_bytes_of_lanes : $\text{keccak_state} \rightarrow \text{List}(\text{keccak_byte})$	Defines a total mathematical function used by the EasyCrypt model.	
75	operation			
keccak_lane_index	op keccak_lane_index (x y : int) : int = (x % 5) + 5 * (y % 5).	keccak_lane_index : $\mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{Z}$	Defines a total mathematical function used by the EasyCrypt model.	
77	operation			
keccak_state_lane	op keccak_state_lane (st : keccak_state) (x y : int) : keccak_lane = nth keccak_lane_zero st (keccak_lane_index x y).	keccak_state_lane : $\text{keccak_state} \times \mathbb{Z} \times \mathbb{Z} \rightarrow \text{keccak_lane}$	Defines a total mathematical function used by the EasyCrypt model.	

Line	Construct	What was written	Symbolic correspondence	Why it is written
80	operation			
keccak_rho_offsets	<pre> op keccak_rho_offsets : int list = 0 :: 1 :: 62 :: 28 :: 27 :: 36 :: 44 :: 6 :: 55 :: 20 :: 3 :: 10 :: 43 :: 25 :: 39 :: 41 :: 45 :: 15 :: 21 :: 8 :: 18 :: 2 :: 61 :: 56 :: 14 :: []. </pre>	$\text{keccak_rho_offsets} : 1 \rightarrow \text{List}(\mathbb{Z})$	Defines a total mathematical function used by the EasyCrypt model.	
87	operation			
keccak_round_constants	<pre> op keccak_round_constants : int list = 1 :: 32898 :: 9223372036854808714 :: 9223372039002292224 :: 32907 :: 2147483649 :: 9223372039002292353 :: 9223372036854808585 :: 138 :: 136 :: 2147516425 :: 2147483658 :: 2147516555 :: 9223372036854775947 :: 9223372036854808713 :: 9223372036854808579 :: 9223372036854808578 :: 9223372036854775936 :: 32778 :: 922337203... </pre>	$\text{keccak_round_constants} : 1 \rightarrow \text{List}(\mathbb{Z})$	Defines a total mathematical function used by the EasyCrypt model.	
113	operation			
keccak_theta_c	<pre> op keccak_theta_c (st : keccak_state) (x : int) : keccak_lane = keccak_lane_xor (keccak_lane_xor (keccak_lane_xor (keccak_lane_xor (keccak_state_lane st x 0) (keccak_state_lane st x 1)) (keccak_state_lane st x 2)) (keccak_state_lane st x 3)) (keccak_state_lane st x 4). </pre>	$\text{keccak_theta_c} : \text{keccak_state} \times \mathbb{Z} \rightarrow \text{keccak_lane}$	Defines a total mathematical function used by the EasyCrypt model.	
124	operation			

Line	Construct	What was written	Symbolic correspondence	Why it is written
keccak_theta_d	<pre> op keccak_theta_d (st : keccak_state) (x : int) : keccak_lane = keccak_lane_xor (keccak_theta_c st (x + 4)) (keccak_lane_rotl (keccak_theta_c st (x + 1)) 1).</pre>	<pre> keccak_theta_d : keccak_state $\times \mathbb{Z} \rightarrow$ keccak_lane</pre>	Defines a total mathematical function used by the EasyCrypt model.	
129	operation			
keccak_theta	<pre> op keccak_theta (st : keccak_state) : keccak_state = mkseq (fun i => let x = i %% 5 in let y = i %/ 5 in keccak_lane_xor (keccak_state_lane st x y) (keccak_theta_d st x)) keccak_state_lanes.</pre>	<pre> keccak_theta : keccak_state $\rightarrow$ keccak_state</pre>	Defines a total mathematical function used by the EasyCrypt model.	
139	operation			
keccak_rho_pi	<pre> op keccak_rho_pi (st : keccak_state) : keccak_state = mkseq (fun i => let x = i %% 5 in let y = i %/ 5 in let sx = (x + 3 * y) %% 5 in let sy = x in keccak_lane_rotl (keccak_state_lane st sx sy) (nth 0 keccak_rho_offsets (keccak_lane_index sx sy))) keccak_state_lanes.</pre>	<pre> keccak_rho_pi : keccak_state $\rightarrow$ keccak_state</pre>	Defines a total mathematical function used by the EasyCrypt model.	
151	operation			
keccak_chi	<pre> op keccak_chi (st : keccak_state) : keccak_state = mkseq (fun i => let x = i %% 5 in let y = i %/ 5 in keccak_lane_xor (keccak_state_lane st x y) (keccak_lane_and (keccak_lane_not (keccak_state_lane st (x + 1) y)) (keccak_state_lane st (x + 2) y))) keccak_state_lanes.</pre>	<pre> keccak_chi : keccak_state $\rightarrow$ keccak_state</pre>	Defines a total mathematical function used by the EasyCrypt model.	
163	operation			

Line	Construct	What was written	Symbolic correspondence	Why it is written
keccak_iota	<pre> op keccak_iota (st : keccak_state) (rc : keccak_lane) : keccak_state = mkseq (fun i => if i = 0 then keccak_lane_xor (nth keccak_lane_zero st 0) rc else nth keccak_lane_zero st i) keccak_state_lanes. </pre>	$\text{keccak_iota} : \text{keccak_state} \times \text{keccak_lane} \rightarrow \text{keccak_state}$	Defines a total mathematical function used by the EasyCrypt model.	
171	operation			
keccak_round	<pre> op keccak_round (st : keccak_state) (rc : keccak_lane) : keccak_state = keccak_iota (keccak_chi (keccak_rho_pi (keccak_theta st))) rc. </pre>	$\text{keccak_round} : \text{keccak_state} \times \text{keccak_lane} \rightarrow \text{keccak_state}$	Defines a total mathematical function used by the EasyCrypt model.	
174	operation			
keccak_f1600_lanes	<pre> op keccak_f1600_lanes (st : keccak_state) : keccak_state = foldl (fun acc rc => keccak_round acc (keccak_lane_of_int rc)) st keccak_round_constants. </pre>	$\text{keccak_f1600_lanes} : \text{keccak_state} \rightarrow \text{keccak_state}$	Defines a total mathematical function used by the EasyCrypt model.	
180	operation			
keccak_f1600_bytes	<pre> op keccak_f1600_bytes (st : keccak_byte list) : keccak_byte list = keccak_bytes_of_lanes (keccak_f1600_lanes (keccak_lanes_of_bytes st)). </pre>	$\text{keccak_f1600_bytes} : \text{List(keccak_byte)} \rightarrow \text{List(keccak_byte)}$	Defines a total mathematical function used by the EasyCrypt model.	
183	lemma			
keccak_lane_bitsE	<pre> lemma keccak_lane_bitsE : keccak_lane_bits = 64. </pre>	$\text{keccak_lane_bitsE} : L = R$ is an equality/rewriting fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
187	lemma			
keccak_state_lanesE	<pre> lemma keccak_state_lanesE : keccak_state_lanes = 25. </pre>	$\text{keccak_state_lanesE} : L = R$ is an equality/rewriting fact.	Proves an invariant preservation fact for an oracle, adversary call, or game main procedure.	
191	lemma			
keccak_state_bytesE	<pre> lemma keccak_state_bytesE : keccak_state_bytes = 200. </pre>	$\text{keccak_state_bytesE} : L = R$ is an equality/rewriting fact.	Proves an invariant preservation fact for an oracle, adversary call, or game main procedure.	
195	lemma			
keccak_lane_zero_wf	<pre> lemma keccak_lane_zero_wf : keccak_lane_wf keccak_lane_zero. </pre>	$\text{keccak_lane_zero_wf} : \forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact used to justify later reductions.	

Line	Construct	What was written	Symbolic correspondence	Why it is written
199	lemma			
keccak_lane_xor_wf	lemma keccak_lane_xor_wf a b : keccak_lane_wf (keccak_lane_xor a b).	keccak_lane_xor_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact used to justify later reductions.	
203	lemma			
keccak_lane_and_wf	lemma keccak_lane_and_wf a b : keccak_lane_wf (keccak_lane_and a b).	keccak_lane_and_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact used to justify later reductions.	
207	lemma			
keccak_lane_not_wf	lemma keccak_lane_not_wf a : keccak_lane_wf (keccak_lane_not a).	keccak_lane_not_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact used to justify later reductions.	
211	lemma			
keccak_lane_rotl_wf	lemma keccak_lane_rotl_wf a offset : keccak_lane_wf (keccak_lane_rotl a offset).	keccak_lane_rotl_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact used to justify later reductions.	
215	lemma			
keccak_lane_of_int_wf	lemma keccak_lane_of_int_wf x : keccak_lane_wf (keccak_lane_of_int x).	keccak_lane_of_int_wf : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Proves a structural correctness fact used to justify later reductions.	
219	lemma			
keccak_lane_mod_range	lemma keccak_lane_mod_range x : 0 <= x %% keccak_lane_bits < keccak_lane_bits.	keccak_lane_mod_range : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
223	lemma			
keccak_word_modulus_pos	lemma keccak_word_modulus_pos : 0 < 2 ^ keccak_lane_bits.	keccak_word_modulus_pos : $\forall \vec{x}. P(\vec{x})$ is a universally quantified fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
227	lemma			
keccak_word_norm_small	lemma keccak_word_norm_small w : 0 <= w < 2 ^ keccak_lane_bits => keccak_word_norm w = w.	keccak_word_norm_small : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
235	lemma			
keccak_pow2_0E	lemma keccak_pow2_0E : 2 ^ 0 = 1.	keccak_pow2_0E : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
239	lemma			
keccak_pow2_1E	lemma keccak_pow2_1E : 2 ^ 1 = 2.	keccak_pow2_1E : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
243	lemma			
keccak_pow2_2E	lemma keccak_pow2_2E : 2 ^ 2 = 4.	keccak_pow2_2E : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
247	lemma			

Line	Construct	What was written	Symbolic correspondence	Why it is written
254	keccak_pow2_3E lemma <code>lemma keccak_pow2_3E : 2 ^ 3 = 8.</code>	<code>keccak_pow2_3E : L = R</code> is an equality/rewriting fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
261	keccak_pow2_4E lemma <code>lemma keccak_pow2_4E : 2 ^ 4 = 16.</code>	<code>keccak_pow2_4E : L = R</code> is an equality/rewriting fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
268	keccak_pow2_5E lemma <code>lemma keccak_pow2_5E : 2 ^ 5 = 32.</code>	<code>keccak_pow2_5E : L = R</code> is an equality/rewriting fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
275	keccak_pow2_6E lemma <code>lemma keccak_pow2_6E : 2 ^ 6 = 64.</code>	<code>keccak_pow2_6E : L = R</code> is an equality/rewriting fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
282	keccak_pow2_7E lemma <code>lemma keccak_pow2_7E : 2 ^ 7 = 128.</code>	<code>keccak_pow2_7E : L = R</code> is an equality/rewriting fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
286	keccak_bytes_to_lane_wf lemma <code>lemma keccak_bytes_to_lane_wf bs lane_index : keccak_lane_wf (keccak_bytes_to_lane bs lane_index).</code>	<code>keccak_bytes_to_lane_wf : $\forall \vec{x}. P(\vec{x})$</code> is a universally quantified fact.	Proves a structural correctness fact used to justify later reductions.	
290	keccak_rho_offsets_size lemma <code>lemma keccak_rho_offsets_size : size keccak_rho_offsets = keccak_state_lanes.</code>	<code>keccak_rho_offsets_size : L = R</code> is an equality/rewriting fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
294	keccak_round_constants_size lemma <code>lemma keccak_round_constants_size : size keccak_round_constants = 24.</code>	<code>keccak_round_constants_size : L = R</code> is an equality/rewriting fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
298	keccak_lanes_of_bytes_size lemma <code>lemma keccak_lanes_of_bytes_size bs : size (keccak_lanes_of_bytes bs) = keccak_state_lanes.</code>	<code>keccak_lanes_of_bytes_size : L = R</code> is an equality/rewriting fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
	keccak_lanes_of_bytes_lane_wf lemma <code>lemma keccak_lanes_of_bytes_lane_wf bs lane : 0 <= lane < keccak_state_lanes => keccak_lane_wf (nth keccak_lane_zero (keccak_lanes_of_bytes bs) lane).</code>	<code>keccak_lanes_of_bytes_lane_wf : $X \leq Y$</code> is an arithmetic or probability upper bound.	Proves a structural correctness fact used to justify later reductions.	

Line	Construct	What was written	Symbolic correspondence	Why it is written
308	lemma			
keccak_lanes_of_bytes_bitE	<pre>lemma keccak_lanes_of_bytes_bitE bs lane bit : 0 <= lane < keccak_state_lanes => 0 <= bit < keccak_lane_bits => keccak_lane_bit (nth keccak_lane_zero (keccak_lanes_of_bytes bs) lane) bit = keccak_byte_bit (nth 0 bs (8 * lane + bit %/ 8)) (bit %/ 8).</pre>	keccak_lanes_of_bytes_bitE : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
324	lemma			
keccak_lane_xor_bitE	<pre>lemma keccak_lane_xor_bitE a b bit : 0 <= bit < keccak_lane_bits => keccak_lane_bit (keccak_lane_xor a b) bit = (keccak_lane_bit a bit <> keccak_lane_bit b bit).</pre>	keccak_lane_xor_bitE : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
334	lemma			
keccak_lane_and_bitE	<pre>lemma keccak_lane_and_bitE a b bit : 0 <= bit < keccak_lane_bits => keccak_lane_bit (keccak_lane_and a b) bit = (keccak_lane_bit a bit /\ keccak_lane_bit b bit).</pre>	keccak_lane_and_bitE : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
344	lemma			
keccak_lane_not_bitE	<pre>lemma keccak_lane_not_bitE a bit : 0 <= bit < keccak_lane_bits => keccak_lane_bit (keccak_lane_not a) bit = (if keccak_lane_bit a bit then false else true).</pre>	keccak_lane_not_bitE : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
354	lemma			
keccak_lane_rotl_bitE	<pre>lemma keccak_lane_rotl_bitE a offset bit : 0 <= bit < keccak_lane_bits => keccak_lane_bit (keccak_lane_rotl a offset) bit = keccak_lane_bit a ((bit - offset) %% keccak_lane_bits).</pre>	keccak_lane_rotl_bitE : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
364	lemma			

Line	Construct	What was written	Symbolic correspondence	Why it is written
keccak_theta_c_bitE	<pre> lemma keccak_theta_c_bitE st x bit : 0 <= bit < keccak_lane_bits => keccak_lane_bit (keccak_theta_c st x) bit = (((keccak_lane_bit (keccak_state_lane st x 0) bit <> keccak_lane_bit (keccak_state_lane st x 1) bit) <> keccak_lane_bit (keccak_state_lane st x 2) bit) <> keccak_lane_bit (keccak_state_lane st x 3) bit) <> keccak_lane_bit (keccak_state_lane st...</pre>	keccak_theta_c_bitE : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
415	lemma			
keccak_theta_c_bit_indexE	<pre> lemma keccak_theta_c_bit_indexE st x bit : 0 <= bit < keccak_lane_bits => keccak_lane_bit (keccak_theta_c st x) bit = keccak_theta_c_bit_index_value st x bit.</pre>	keccak_theta_c_bit_indexE : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
425	lemma			
keccak_theta_d_bitE	<pre> lemma keccak_theta_d_bitE st x bit : 0 <= bit < keccak_lane_bits => keccak_lane_bit (keccak_theta_d st x) bit = (keccak_lane_bit (keccak_theta_c st (x + 4)) bit <> keccak_lane_bit (keccak_theta_c st (x + 1)) ((bit - 1) %% keccak_lane_bits)).</pre>	keccak_theta_d_bitE : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
439	lemma			
keccak_theta_d_bit_indexE	<pre> lemma keccak_theta_d_bit_indexE st x bit : 0 <= bit < keccak_lane_bits => keccak_lane_bit (keccak_theta_d st x) bit = keccak_theta_d_bit_index_value st x bit.</pre>	keccak_theta_d_bit_indexE : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	

Line	Construct	What was written	Symbolic correspondence	Why it is written
452	lemma			
keccak_theta_bitE	<pre>lemma keccak_theta_bitE st lane bit : 0 <= lane < keccak_state_lanes => 0 <= bit < keccak_lane_bits => keccak_lane_bit (nth keccak_lane_zero (keccak_theta st) lane) bit = (let x = lane %% 5 in let y = lane %% 5 in keccak_lane_bit (keccak_state_lane st x y) bit <> keccak_lane_bit (keccak_theta_d st x) bit).</pre>	keccak_theta_bitE : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
469	lemma			
keccak_theta_bit_indexE	<pre>lemma keccak_theta_bit_indexE st lane bit : 0 <= lane < keccak_state_lanes => 0 <= bit < keccak_lane_bits => keccak_lane_bit (nth keccak_lane_zero (keccak_theta st) lane) bit = (let x = lane %% 5 in let y = lane %% 5 in keccak_lane_bit (nth keccak_lane_zero st (keccak_lane_index x y)) bit <> keccak_lane_bit (keccak_theta_d st x) bit).</pre>	keccak_theta_bit_indexE : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
484	lemma			
keccak_rho_pi_bitE	<pre>lemma keccak_rho_pi_bitE st lane bit : 0 <= lane < keccak_state_lanes => 0 <= bit < keccak_lane_bits => keccak_lane_bit (nth keccak_lane_zero (keccak_rho_pi st) lane) bit = (let x = lane %% 5 in let y = lane %% 5 in let sx = (x + 3 * y) %% 5 in let sy = x in keccak_lane_bit (keccak_state_lane st sx sy) ((bit - nth 0 keccak_rho_offsets (keccak_lane_index s...</pre>	keccak_rho_pi_bitE : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	

Line	Construct	What was written	Symbolic correspondence	Why it is written
505	lemma			
keccak_rho_pi_bit_indexE	<pre> lemma keccak_rho_pi_bit_indexE st lane bit : 0 <= lane < keccak_state_lanes => 0 <= bit < keccak_lane_bits => keccak_lane_bit (nth keccak_lane_zero (keccak_rho_pi st) lane) bit = (let x = lane %% 5 in let y = lane %/ 5 in let sx = (x + 3 * y) %% 5 in let sy = x in keccak_lane_bit (nth keccak_lane_zero st (keccak_lane_index sx sy)) ((bit - nth 0 keccak_rho...</pre>	keccak_rho_pi_bit_indexE : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
524	lemma			
keccak_chi_bitE	<pre> lemma keccak_chi_bitE st lane bit : 0 <= lane < keccak_state_lanes => 0 <= bit < keccak_lane_bits => keccak_lane_bit (nth keccak_lane_zero (keccak_chi st) lane) bit = (let x = lane %% 5 in let y = lane %/ 5 in keccak_lane_bit (keccak_state_lane st x y) bit <> ((if keccak_lane_bit (keccak_state_lane st (x + 1) y) bit then false else true) /\ keccak_lane_bi...</pre>	keccak_chi_bitE : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
545	lemma			

Line	Construct	What was written	Symbolic correspondence	Why it is written
keccak_chi_bit_indexE	<pre> lemma keccak_chi_bit_indexE st lane bit : 0 <= lane < keccak_state_lanes => 0 <= bit < keccak_lane_bits => keccak_lane_bit (nth keccak_lane_zero (keccak_chi st) lane) bit = (let x = lane %% 5 in let y = lane %/ 5 in keccak_lane_bit (nth keccak_lane_zero st (keccak_lane_index x y)) bit <> ((if keccak_lane_bit (nth keccak_lane_zero st (keccak_lane_index (x...</pre>	keccak_chi_bit_indexE : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
566	lemma			
keccak_iota_bitE	<pre> lemma keccak_iota_bitE st rc lane bit : 0 <= lane < keccak_state_lanes => 0 <= bit < keccak_lane_bits => keccak_lane_bit (nth keccak_lane_zero (keccak_iota st rc) lane) bit = if lane = 0 then keccak_lane_bit (nth keccak_lane_zero st 0) bit <> keccak_lane_bit rc bit else keccak_lane_bit (nth keccak_lane_zero st lane) bit.</pre>	keccak_iota_bitE : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
585	lemma			
keccak_round_bitE	<pre> lemma keccak_round_bitE st rc lane bit : 0 <= lane < keccak_state_lanes => 0 <= bit < keccak_lane_bits => keccak_lane_bit (nth keccak_lane_zero (keccak_round st rc) lane) bit = if lane = 0 then keccak_lane_bit (nth keccak_lane_zero (keccak_chi (keccak_rho_pi (keccak_theta st))) 0) bit <> keccak_lane_bit rc bit else keccak_lane_bit (nth keccak_lane_zero (k...</pre>	keccak_round_bitE : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	

Line	Construct	What was written	Symbolic correspondence	Why it is written
611	lemma			
keccak_bytes_of_lanes_size	lemma keccak_bytes_of_lanes_size st : size (keccak_bytes_of_lanes st) = keccak_state_bytes.	keccak_bytes_of_lanes_size : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
615	lemma			
keccak_bytes_of_lanes_byteE	lemma keccak_bytes_of_lanes_byteE st i : 0 <= i < keccak_state_bytes => nth 0 (keccak_bytes_of_lanes st) i = keccak_lane_to_byte (nth keccak_lane_zero st (i % 8)) (i % 8).	keccak_bytes_of_lanes_byteE : $X \leq Y$ is an arithmetic or probability upper bound.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
626	lemma			
keccak_lane_to_byte_bitsE	lemma keccak_lane_to_byte_bitsE lane byte_index : keccak_lane_to_byte lane byte_index = (if keccak_lane_bit lane (8 * byte_index) then 2 ^ 0 else 0) + ((if keccak_lane_bit lane (8 * byte_index + 1) then 2 ^ 1 else 0) + ((if keccak_lane_bit lane (8 * byte_index + 2) then 2 ^ 2 else 0) + ((if keccak_lane_bit lane (8 * byte_index + 3) then 2 ^ 3 else 0) + (...)	keccak_lane_to_byte_bitsE : $L = R$ is an equality/rewriting fact.	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
650	lemma			

Line	Construct	What was written	Symbolic correspondence	Why it is written
keccak_lane_to_byte_from_bitsE	<pre> lemma keccak_lane_to_byte_from_bitsE lane byte_index b0 b1 b2 b3 b4 b5 b6 b7 : keccak_lane_bit lane (8 * byte_index) = b0 => keccak_lane_bit lane (8 * byte_index + 1) = b1 => keccak_lane_bit lane (8 * byte_index + 2) = b2 => keccak_lane_bit lane (8 * byte_index + 3) = b3 => keccak_lane_bit lane (8 * byte_index + 4) = b4 => keccak_lane_bit lane (8 * byte_i...</pre>	<pre> keccak_lane_to_byte_from_bitsE : P => Q is an implication used as a proof obligation.</pre>	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
676	<pre> lemma keccak_lane_to_byte_70E lane byte_index : keccak_lane_bit lane (8 * byte_index) = false => keccak_lane_bit lane (8 * byte_index + 1) = true => keccak_lane_bit lane (8 * byte_index + 2) = true => keccak_lane_bit lane (8 * byte_index + 3) = false => keccak_lane_bit lane (8 * byte_index + 4) = false => keccak_lane_bit lane (8 * byte_index + 5) = false...</pre>	<pre> keccak_lane_to_byte_70E : P => Q is an implication used as a proof obligation.</pre>	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	
693	<pre> lemma keccak_f1600_bytes_size : size (keccak_f1600_bytes st) = keccak_state_bytes.</pre>	<pre> keccak_f1600_bytes_size : L = R is an equality/rewriting fact.</pre>	Records a reusable theorem so later proof scripts can rewrite, apply, or compose it.	

19 Regenerating This Guide

From haetae-security, run:

```
python3 guide/generate-symbolic-correspondence-guide.py
pdflatex -interaction=nonstopmode -halt-on-error guide/haetae-symbolic-correspondence-guide.tex
```

```
pdflatex -interaction=nonstopmode -halt-on-error guide/haetae-symbolic-correspondence-guide.tex
```